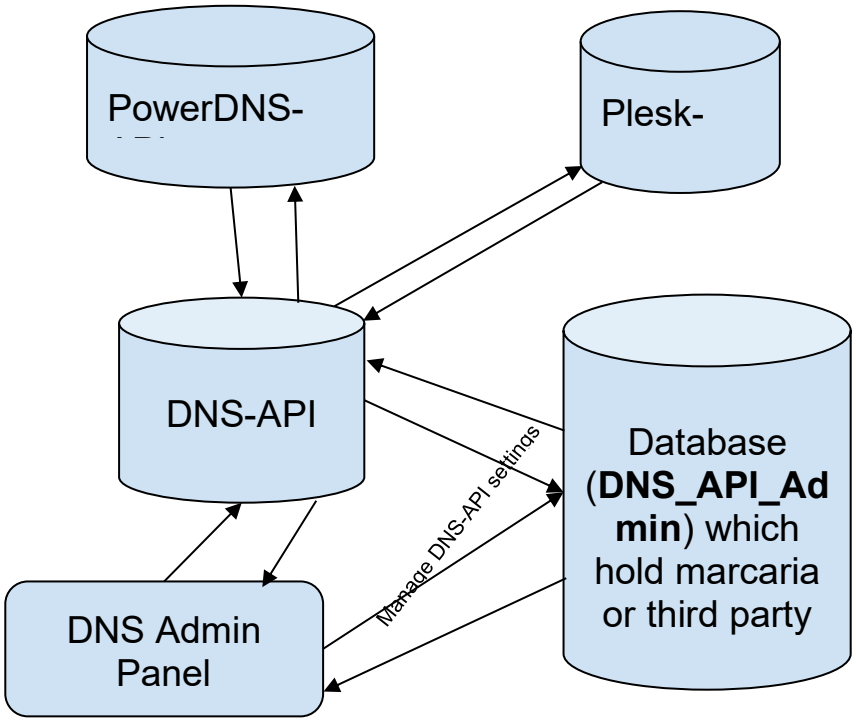
https://docs.google.com/document/d/1J9ECa-4y3qtU4Byk_HXiyIzgqUDgx0gFLXLIB1oGDBo/edit

Marcaria DNS Application Guide

These are the following list of applications used by Marcaria to manage DNS of hosted domains.

1. http://admin.dns-api.marcaria.com (Asp.net application)
2. https://dns-api.marcaria.com (WebAPI application)
3. https://plesk-l01.marcaria-it.me:8443/(Plesk API)

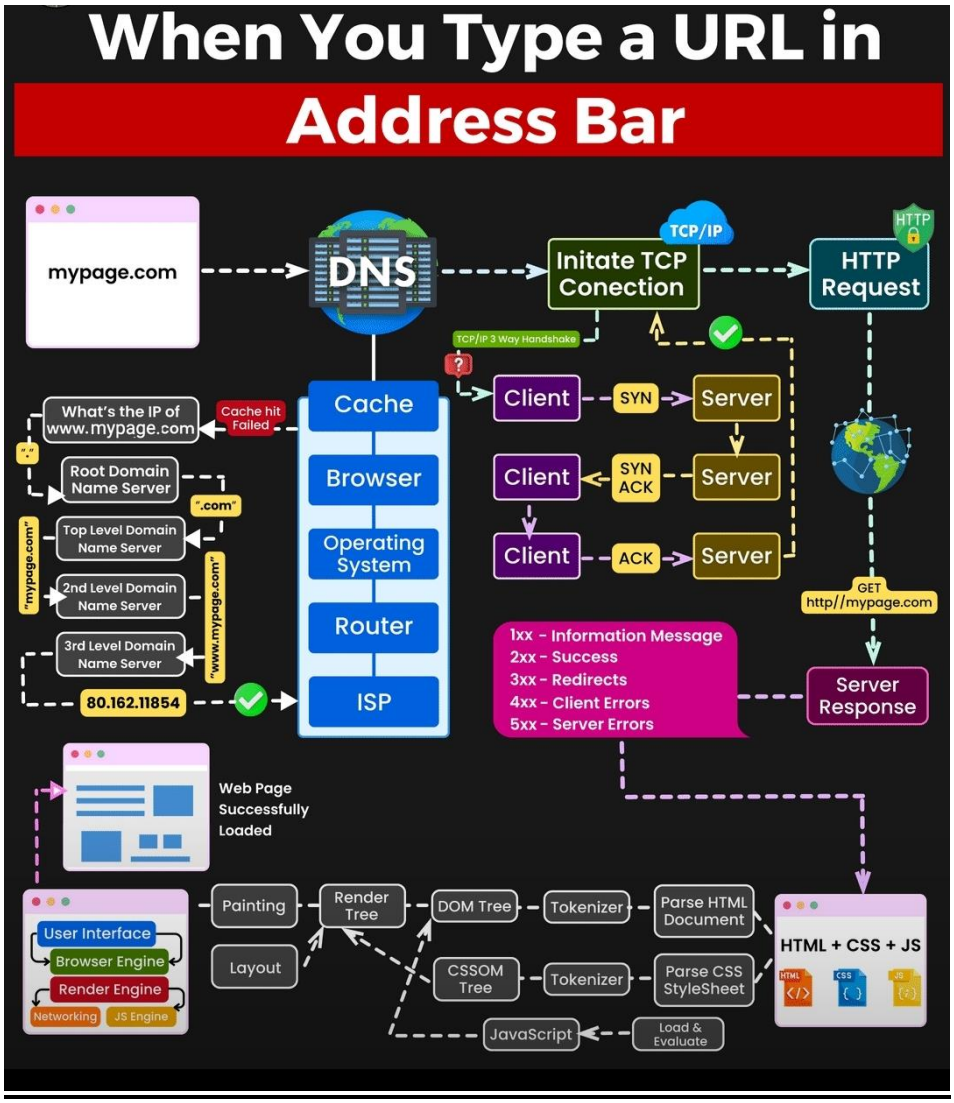


==========

INTRODUCTION : <u>Marcaria DNS Application.</u>

==========

- Marcaria is in the business of domain registration like GoDaddy.com. Once a user registers a domain for his personal or business purpose the immediate need is to set up DNS records.
- When someone browses any website, DNS records are the one which provide details like where the application is hosted i.e. what is the server IP, where the emails are configured etc.



What is DNS :

- DNS stands for **D**omain **N**ame **S**ystem.
- Mainly Used is convert human friendly domain name (www.google .com) into IP address.
- DNS is a directory service that provides a mapping between the name of a host on the network and its numerical address.
- User can know in depth about DNS by using below link : https://www.javatpoint.com/computer-network-dns

★ How DNS Works :-

- DNS servers convert URLs and domain names into IP addresses that computers can understand and use. They translate what a user types into a browser into something the machine can use to find a webpage. This process of translation and lookup is called *DNS resolution*.

- The process of DNS resolution involves converting a hostname (such as www.example.com) into a computer-friendly IP address (such as 192.168.1.1). An IP address is given to each device on the Internet, and that address is necessary to find the appropriate Internet device.

- User can know in depth about how to DNS work by using below link : https://www.techtarget.com/searchnetworking/definition/domain-name-system

★ Use Of Admin Panel :-

- **The main use of the admin panel is for web forwarding and email forwarding.** For example: If the user wants to forward a website to another website, then the user can use web forwarding and if the user wants to forward an email to another email ID. So he/she can use email forwarding.
- To use web and email forwarding, the user has to register a domain, create records like A, AAAA, MX records and **all this process will be done using the admin panel.**

★ Use Of API's :-

- Here, the user API is used to create, update and delete the domain, creating records within the domain.
- Users can run all these APIs through swagger.

★ Who will use the API's? :-

- **As Admin :-** Admin will be able to **use all API's by going through swagger.**
- **As API User :-** API user will not be able to **use all API's by going through swagger.**
- **As Normal Agent :-** Here a normal agent cannot use **direct** swagger from the admin panel to run API.
  - **A normal agent can use Swagger by opening Swagger's URL directly in a new tab to run the API.**

★ Who will use the admin panel? :-

- **As Admin :-** Admin will be able to access the **entire admin panel.**
- **As API User :-** API user will not be able to access the **entire admin panel.**
- **As a Normal Agent :-** Here, a normal agent will not be able to access the entire admin panel, **It has access to only 6 modules..**
  1. Domain Search
  2. API Search Log
  3. Error Log
  4. API Search Chart
  5. Domain Count
  6. Domain Count Chart

★ What Is Cloudflare :-

- **Cloudflare** is a content delivery network (CDN) and cloud security platform that provides website optimization, security, and performance services. It acts as a mediator between a website's server and its visitors, improving the speed and reliability of the website while also protecting it from online threats.
- User can know in depth about DNS by using below link : https://www.geeksforgeeks.org/what-is-cloudflare

★ How Cloudflare Works :-

- To use Cloudflare, website owners need to sign up for an account and add their website to the Cloudflare dashboard. Cloudflare will scan the website for DNS records and configure its CDN and security features automatically. Once configured, website owners can monitor and adjust Cloudflare's settings from the dashboard.
- User can know in depth about DNS by using below link : https://www.geeksforgeeks.org/what-is-cloudflare

➔ QA Site for accessing Marcaria DNS application :

- https://admin-dns-api-qa.nominus.com

- This site is accessible only for the QA and Development team.

➔ **Production Site for accessing Marcaria DNS application :**

- https://admin-dns-api.nominus.com
- This is the production site URL.
- **So do not add any data or any changes in this URL.**

==========

Index : Marcaria DNS Application.

==========

❖ **Users**

❖ **Power DNS**

- ➢ Domain Search6 .
- ➢ API Search Log
- ➢ Error Log
- ➢ API Search Chart
- ➢ Domain Count
- ➢ Domain Count Chart
- ➢ DNS Profile
- ➢ PowerDNS Setting

❖ **Cloudflare**

- ➢ Domain List
- ➢ API Search Log
- ➢ Error Log
- ➢ API Search Chart
- ➢ Domain Count

- ➢ Domain Count Chart
- ➢ DNS Profile
- ➢ Cloudflare Setting

❖ **General**

- ➢ DNS Profile
- ➢ Extension Settings
- ➢ Blocked Domain
- ➢ Customer DNS Profile

❖ **Forwarding**

- ➢ Forwarding Settings
- ➢ Forwarding Server
- ➢ Blocked Domain

❖ **Swagger**

DNS Admin Panel – User Management

1. User Roles

The system shall support the following user types:

- Super User
- Admin
- Normal Agent
- API User

Each user type shall have different permissions based on their assigned role.

2. User Listing

The system shall display users according to the logged-in user's permissions. The user list shall always be sorted by the most recently updated user in descending order.

Super User

The Super User shall be able to:

- View all users.
- View both Active and Disabled users.
- Manage every user type.

User Enable / Disable functionality

Only the Super User shall have access to Enable/Disable functionality.
The Admin shall not:

- View the toggle
- Enable users
- Disable users

The Super User shall not disable their own account.

Super User Protection

The system shall always maintain at least one Active Super User.
If only one Active Super User exists:

- The role shall not be changed.
- The role shall not be downgraded.
- The system shall display:

"At least one active SuperUser must be present."
The existing role shall remain unchanged.

Admin

The Admin shall only be able to:

- View Admin users.
- View Normal Agent users.
- View API users.
- View only Active users.

3. Role-Based Access

Super User

The Super User shall have unrestricted access to:

- DNS Admin Panel
- Swagger API
- PowerDNS
- Cloudflare Native Zone
- Cloudflare Secondary Zone

Admin

The Admin shall have full access to:

- DNS Admin Panel
- Swagger API

The Admin shall not have Super User administrative privileges to view other super user or disabled user.

Normal Agent

The Normal Agent shall have access only to:

- PowerDNS module
- PowerDNS API

The Normal Agent shall not access administrative modules.

API User

The API User shall only be permitted to access the PowerDNS API using a valid API token.
The API User shall not access the Admin Panel.

4. User Creation

Super User

The Super User shall be allowed to create:

- Super User
- Admin
- Normal Agent
- API User

Admin

The Admin shall be allowed to create:

- Admin
- Normal Agent
- API User

The system shall prevent creation of duplicate usernames.
If a duplicate username is entered, the system shall display:
"User already exists."

5. User Update

The system shall allow editing user information according to role permissions.

Super User

The Super User may update:

- First Name
- Last Name
- Username
- User Type
- Password
- Comments

For API Users, password modification shall not be available.

Admin

The Admin may update:

- First Name
- Last Name
- Username
- Comments

The Admin shall not modify:

- User Type
- Password

The system shall validate username uniqueness during updates.
If another user already owns the username, the system shall display:
"UserName you try to update is already exist."

6. Token Management

Only the Super User shall have permission to:

- View generated tokens
- Generate new tokens
- Regenerate existing tokens

Admin users shall only be able to view the default token assigned during user creation.
Admin users shall not regenerate tokens.

7.Disabled User Behavior

Disabled users shall not authenticate successfully.

Normal Agent

A disabled Normal Agent:

- Cannot log in.
- Cannot access PowerDNS.

- Cannot use PowerDNS API.
- Shall receive HTTP 401 Unauthorized for API requests.

API User

A disabled API User:

- Cannot access PowerDNS API.
- Shall receive HTTP 401 Unauthorized.

Admin

A disabled Admin:

- Cannot log in.
- Cannot access APIs.
- Shall receive HTTP 401 Unauthorized.

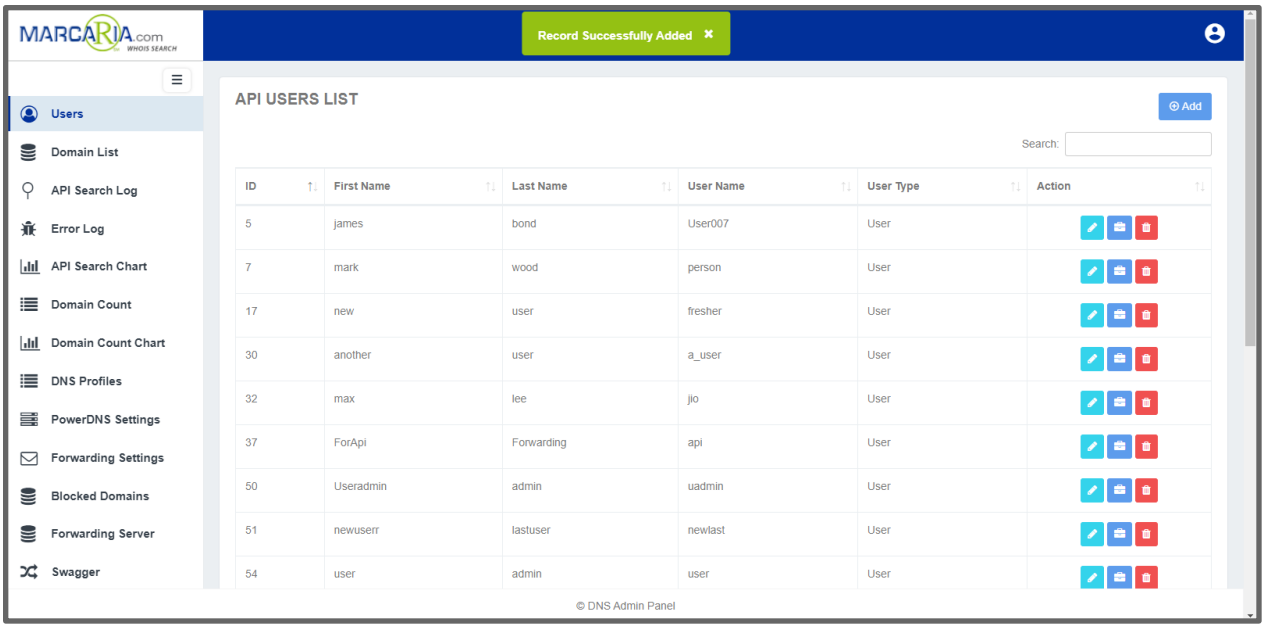
Super User

A disabled Super User:

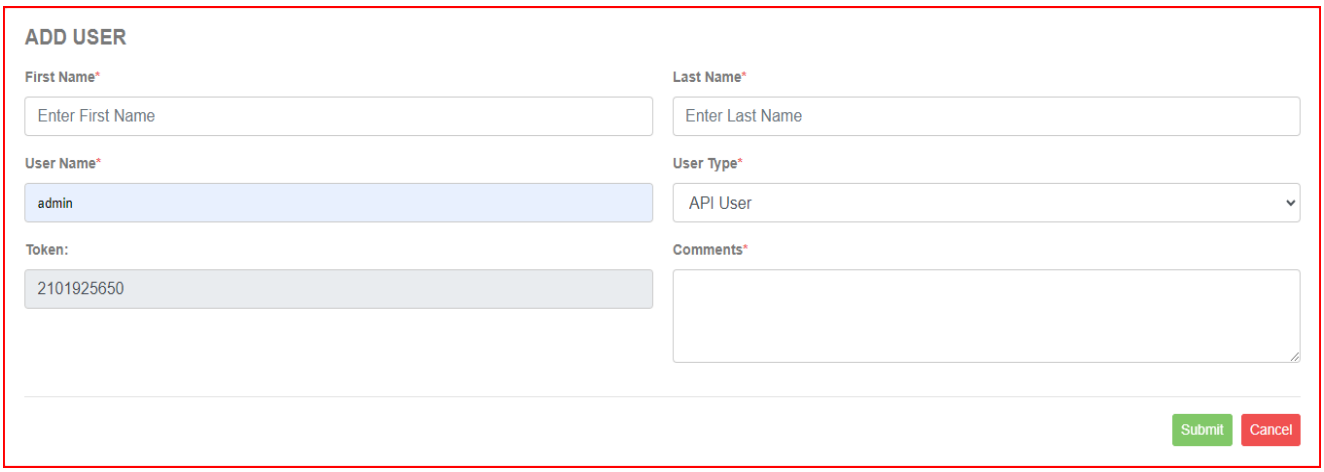
- Cannot log in.
- Cannot access APIs.
- Shall receive HTTP 401 Unauthorized.

USERS.

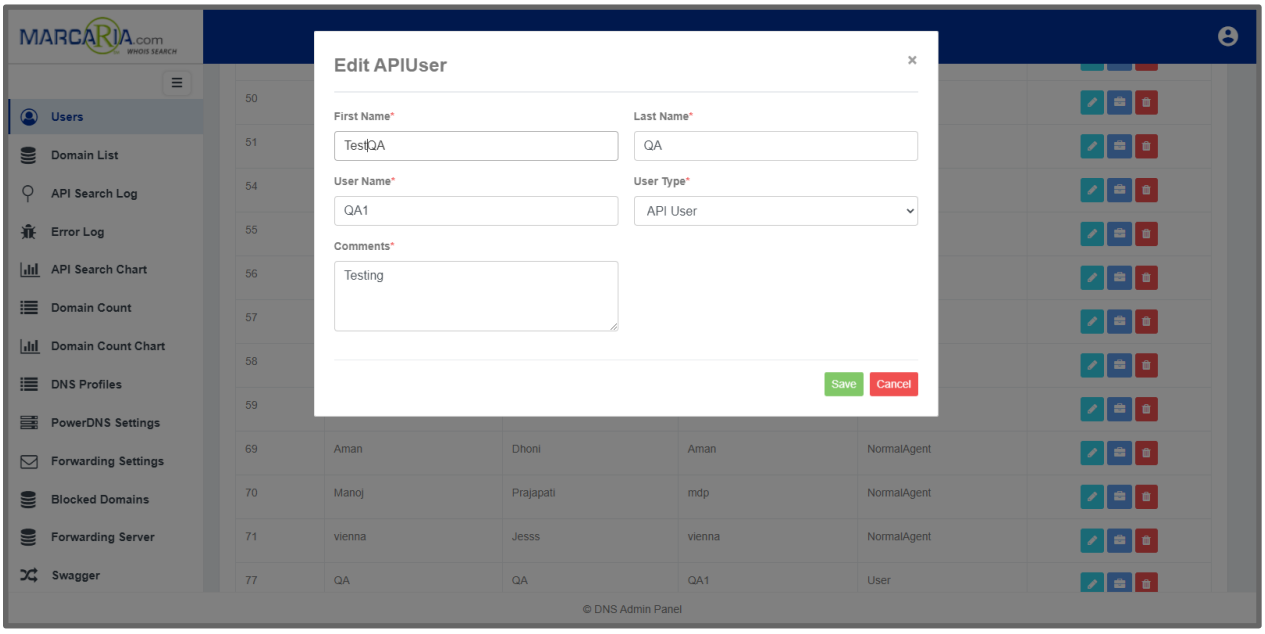
- Click on the Users option form the side menu bar.
- Users are able to view a list of API user tables with fields like ID, First Name, Last Name, User Name, User Type, Action, Search Box, Add button, sorting and pagination functionality.
- Users can search any user using the search box.



- **<u>Add Button:</u>**
  - Click on Add button it will open the window for Add User.
  - It has the following options..
    - **First Name:** Add First Name of the User
    - **Last Name:** Add Last Name of the User
    - **User Name:** Add User name which you need to set as a User Name.
    - **User Type:** Select User Type from drop down i.e. API User, Normal Agent, Admin
    - **Token:** Auto generated token and should be in read only mode
    - **Comments:** Add comment
    - **Submit button:** Click on the submit button it will generate the profile with the given details.
    - **Cancel button:** It will cancel the process and add user pop will be closed.

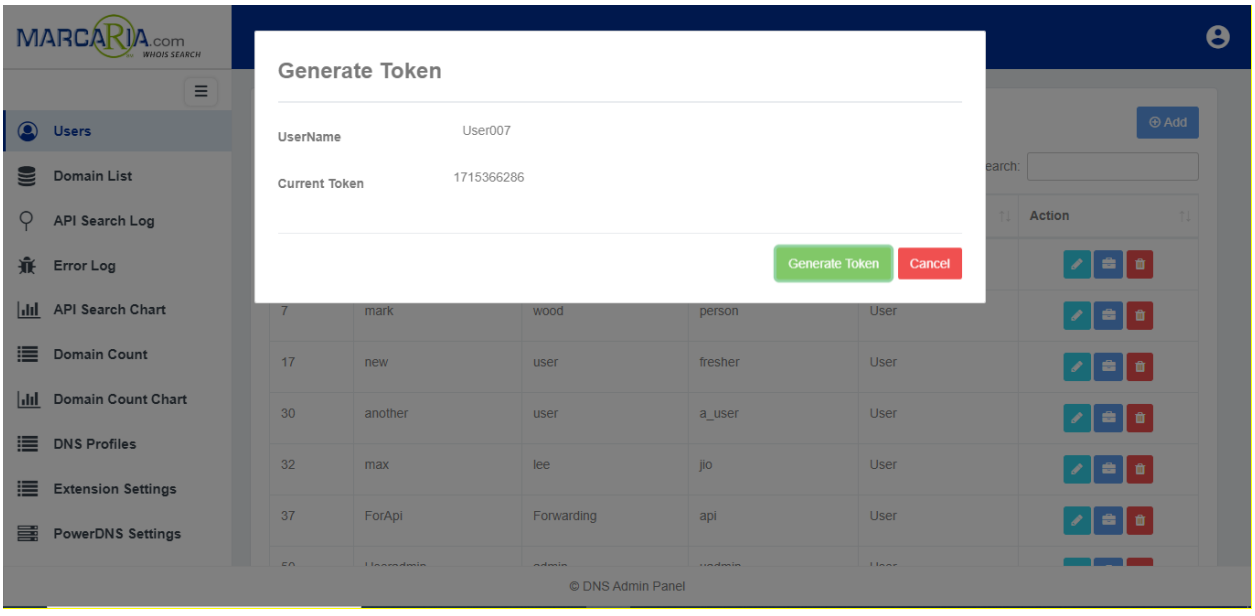


- **Edit Icon:**
  - Click on the Edit icon it will open the popup for Edit API User.
  - It allows us to edit First Name, Last Name, User Name, User Type, Comments and Save & Cancel button.
  - Here, there are three type of the user type
    - Admin.
    - Users.
    - Normal Agent.

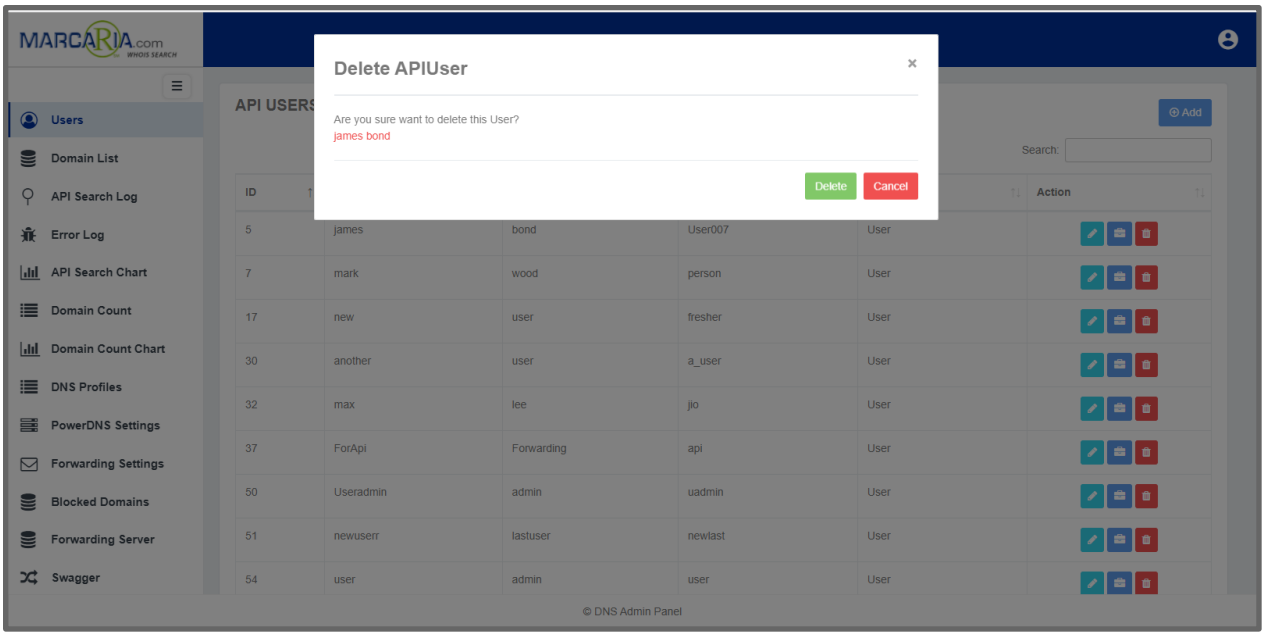


  - After successfully updation user will get the notification message like - “Record Successfully Updated”

- **Generate Token Icon:**
    - Click on this icon it will open the popup of Generate Token.
    - The popup has information like User Name, Current Token, Generate Token button & Cancel button.
    - Once a user clicks on the Generate Token button it will generate the new token and **this token is used in Swagger API.**



- **Delete Icon:**
    - Click on the delete icon and it will delete the user.

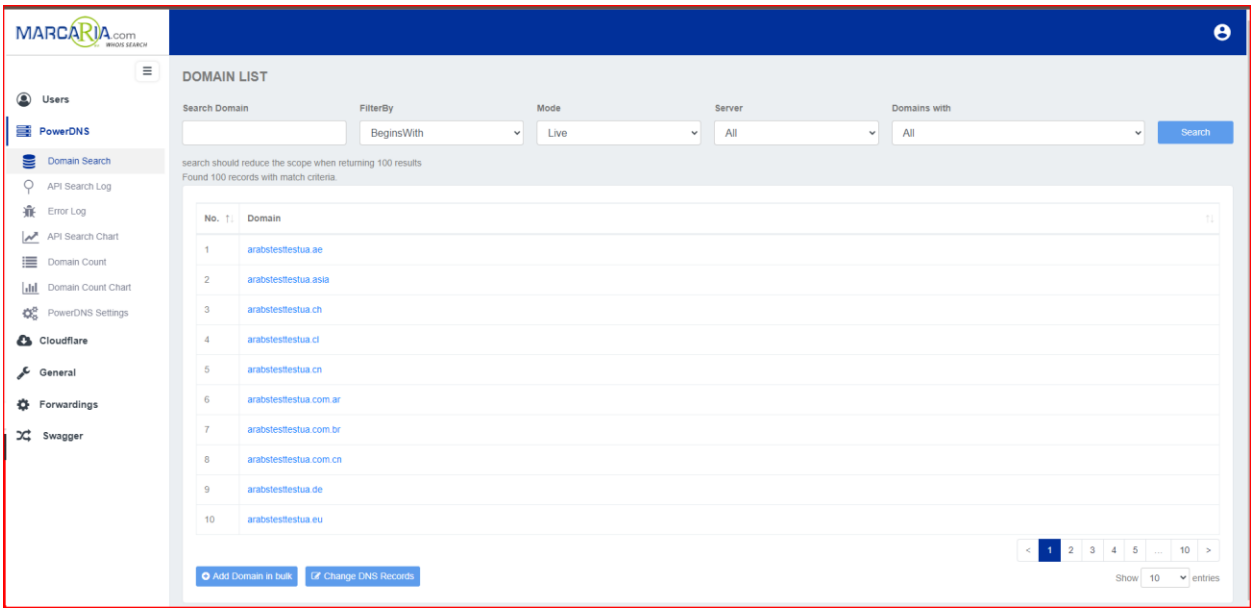


Power DNS.

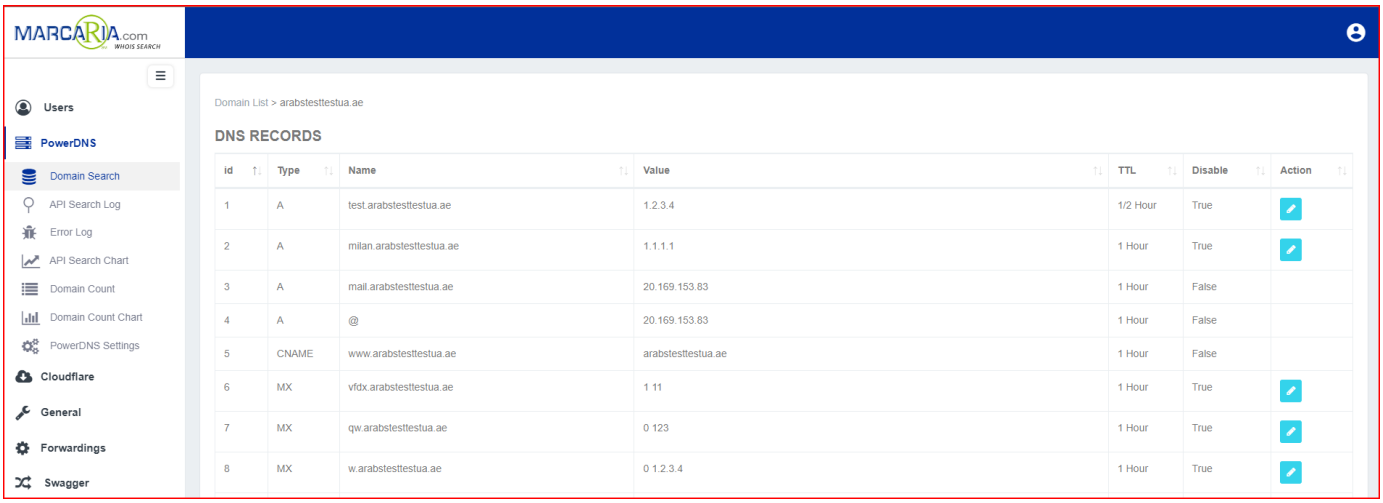
- Power DNS has 7 submodules.
  1. Domain Search.
  2. API Search Log.
  3. Error Log.
  4. API Search Chart.
  5. Domain Count.
  6. Domain Count Chart.
  7. Power DNS Settings.

- **Domain Search :**

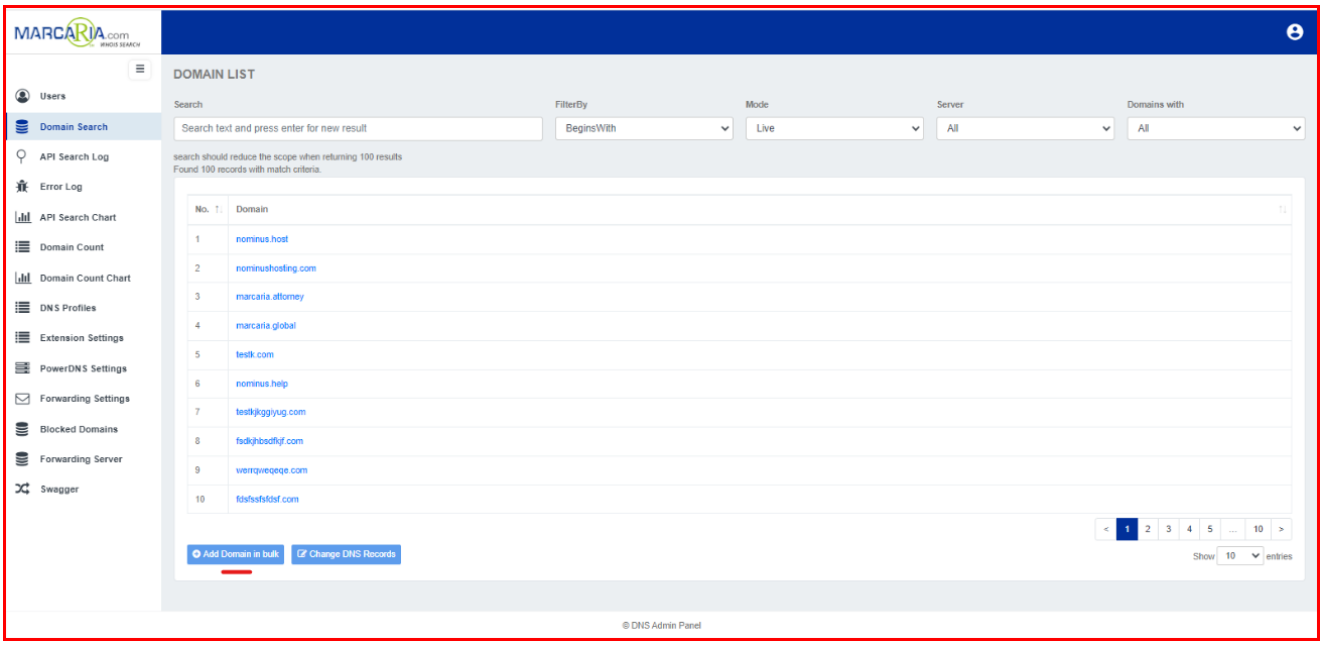
- Here, users can see the search domain search box, filter by, mode,server and domains with dropdowns, search button, add domain in bulk and change DNS records.
- Users can add/create domains from the Domain list menu.
- By clicking on the Domain from the side menu panel the user is able to view a list of domains which are already added.



- When the user clicks on the domain name, the user can see the domain records.

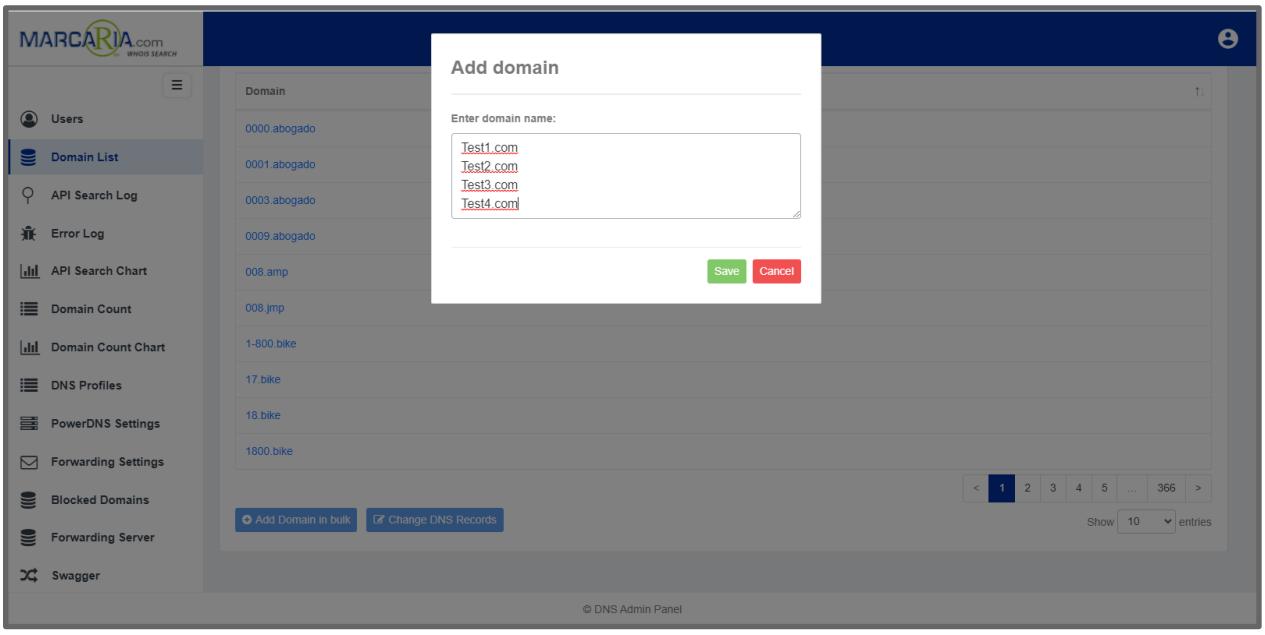


- **Add Domain in Bulk :**
    - When the user clicks on the Add domain in bulk button it will open the popup with textarea ,Save and Cancel button.
    - Here, users can add multiple domains in the text area separated each by entering a domain. i.e. every domain must be in a new line

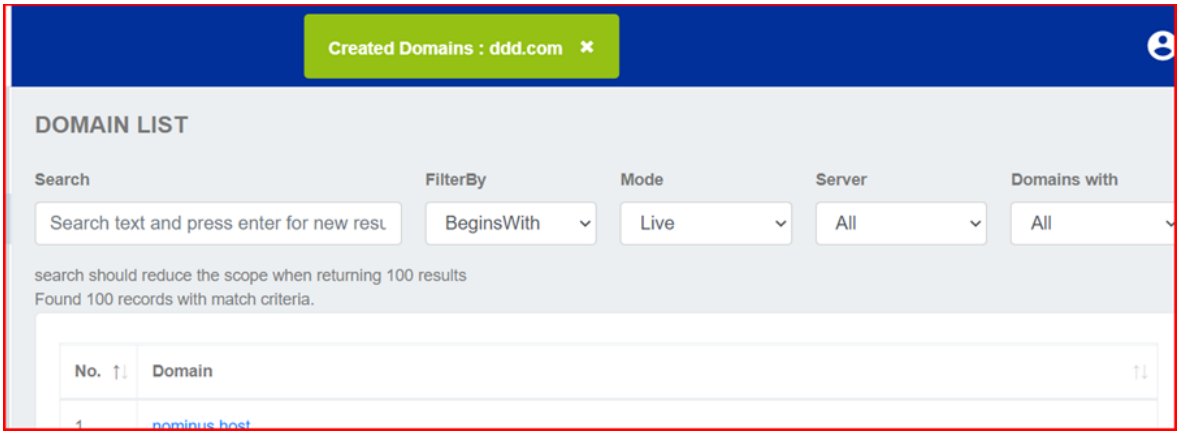


- **Add Domain in bulk button:**
    - When the user clicks on the Add domain in bulk button it will open the popup with textarea and Save, Cancel button.

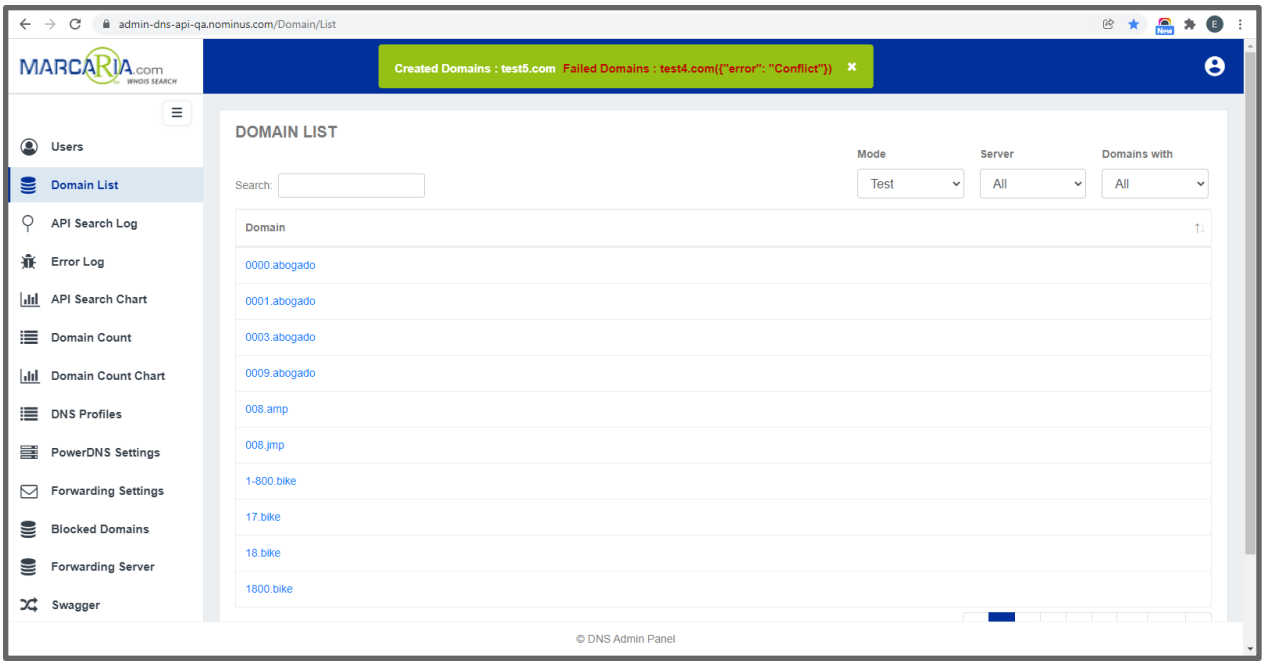
- Add multiple domains in the text area separated each by entering a domain name. i.e. every domain must be in a new line.



- Click on save button it will add the domain in the list.

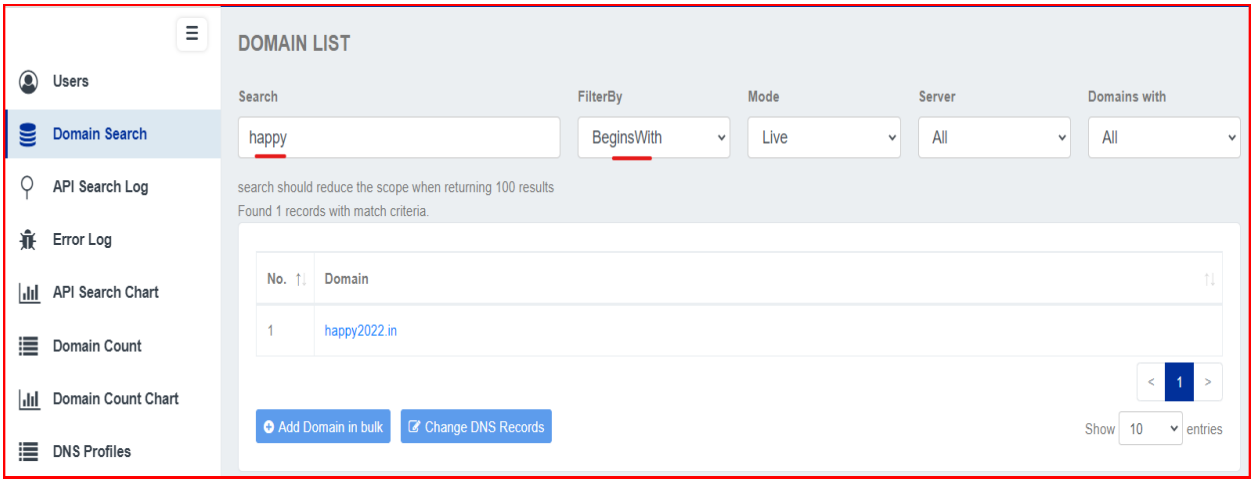


- **NOTE:** Entered domain name must be unique. If a user entered name which is already available in the list it will give validation of the conflict domain name.

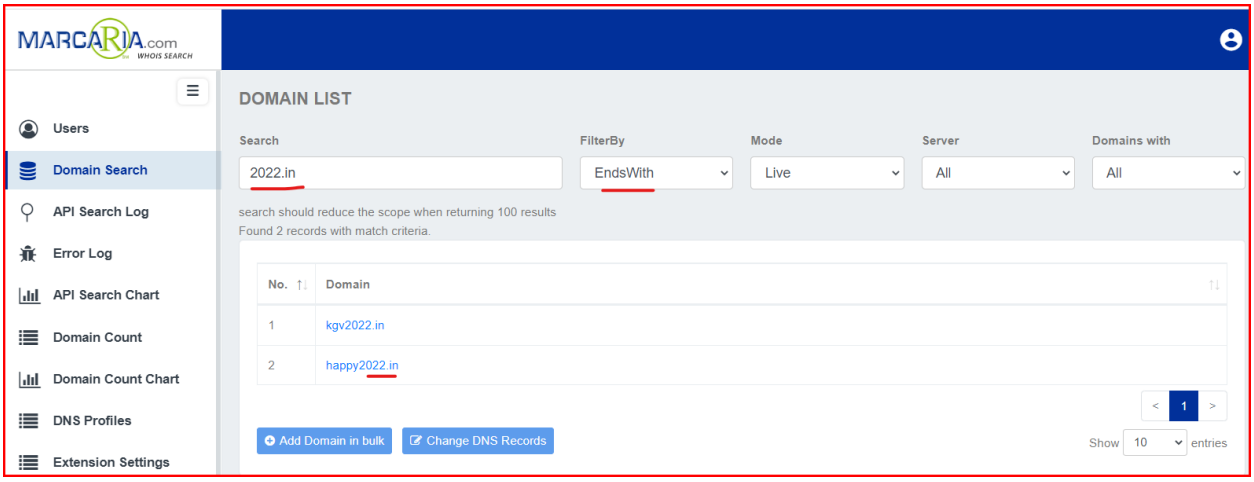


- **FilterBy Functionality :**

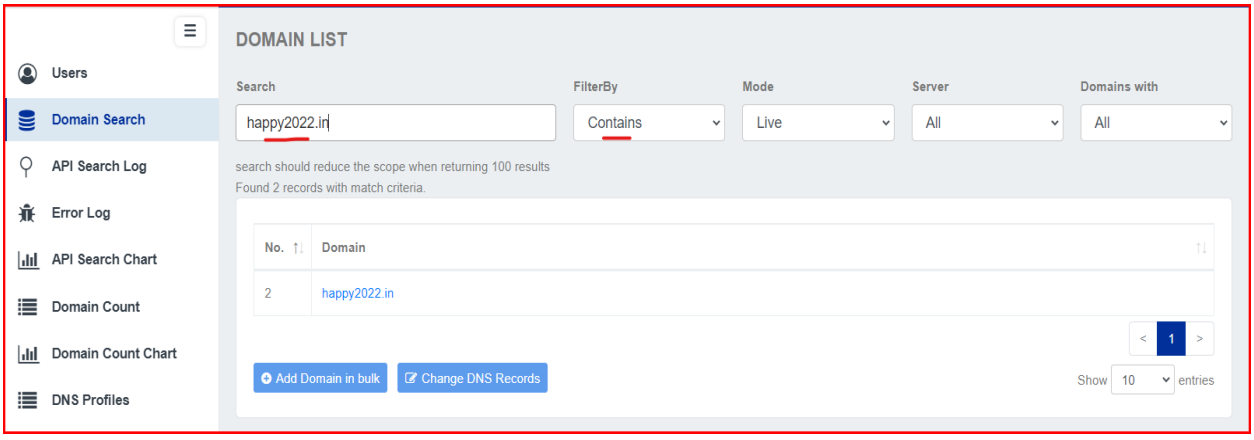
1. **Begins with :** Enter the initial letter of the domain and press enter.



2. **Ends with :** Enter the end letter of the domain and press enter.



3. **Contains :** Enter the Domain and press enter.

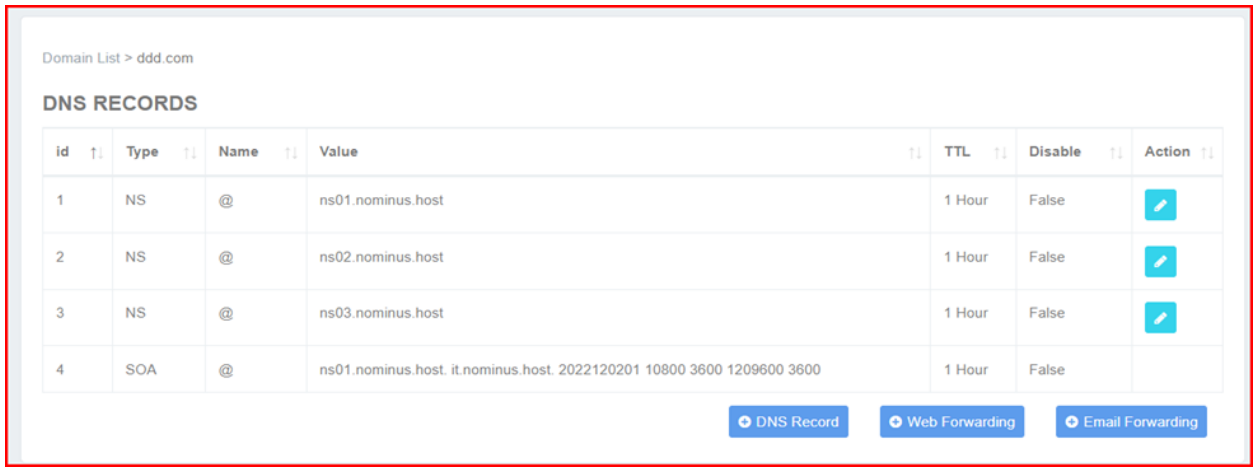


- **Server Dropdown Functionality :**

  - As many IDs as you create in the forwarding server will be displayed in the server dropdown.

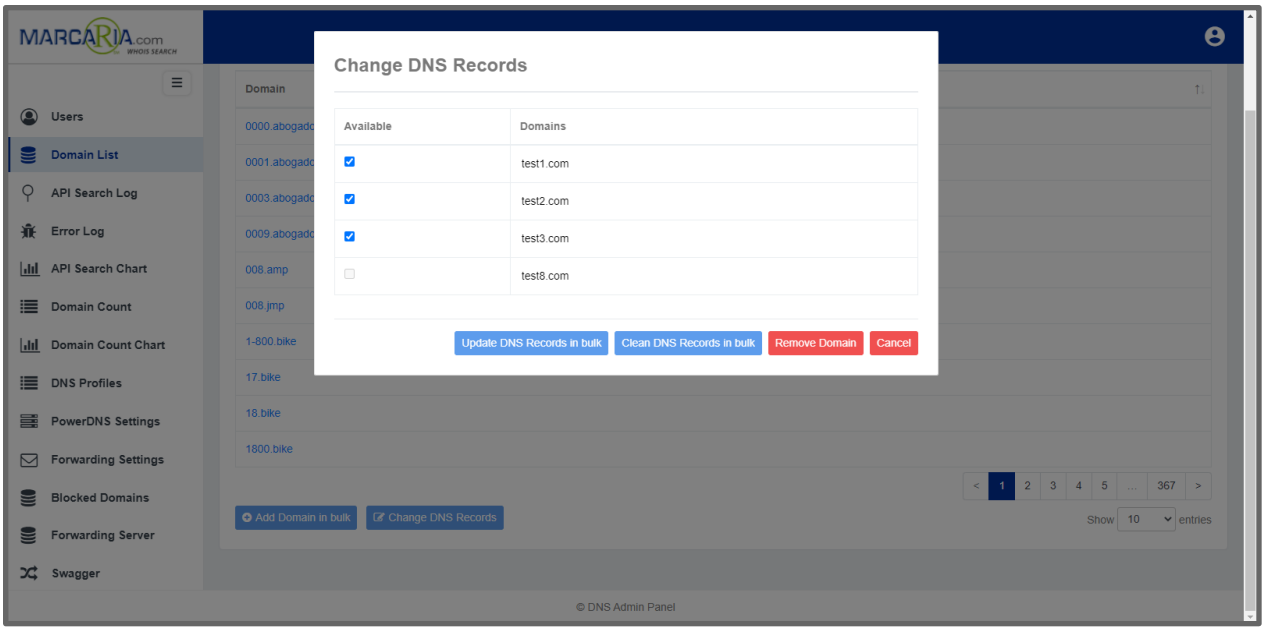
  - Here users see Default = Yes in front of the profile which is Active. When creating a new profile at that time the newly added profile will have Default = Yes. It means the newly added profile will be considered as Active.

- Click on Created Domain then open domain Record :



- **Change DNS Records Button:**
    - Click on the change DNS record button it will open the popup with text area and Continue & Cancel button.
    - Add domain name in the text area. Users can add multiple domain names by separating it with the enter key. It means every domain name must be in the new line.
    - Here, users can update records or clean dns records or remove (delete) domain.

- **Continue button:** Click on the continue button it opens the popup of the Change DNS Records with following information.



- Users can see tables with the fields Available and Domains.

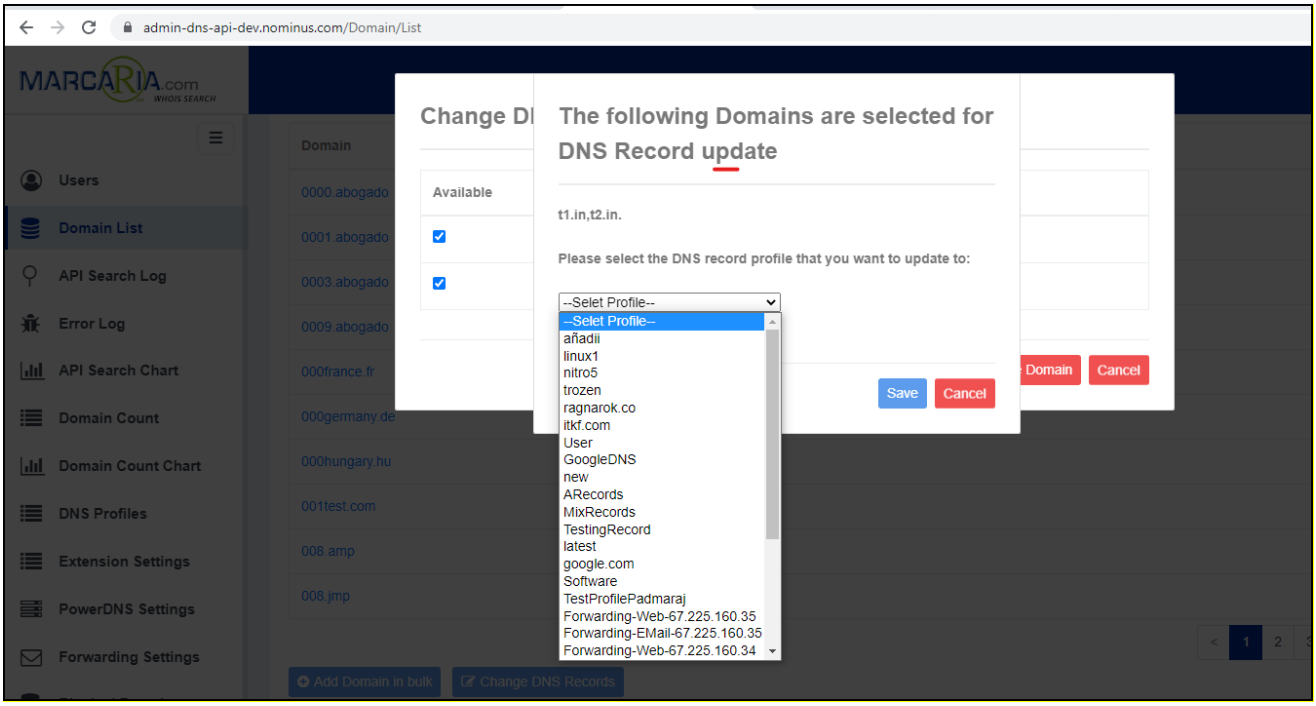
  - **Available field:**
    - Here in available fields if the domain is already available in the list it will have checked the checkbox.
    - Here the user is able to check/uncheck any domain as per their requirements.
    - If domain is not available in the list check box of it is in disable mode.

  - **Domains:**
    - It will display the name of the domains which the user entered for the change.

- **Update DNS Records in bulk button:**

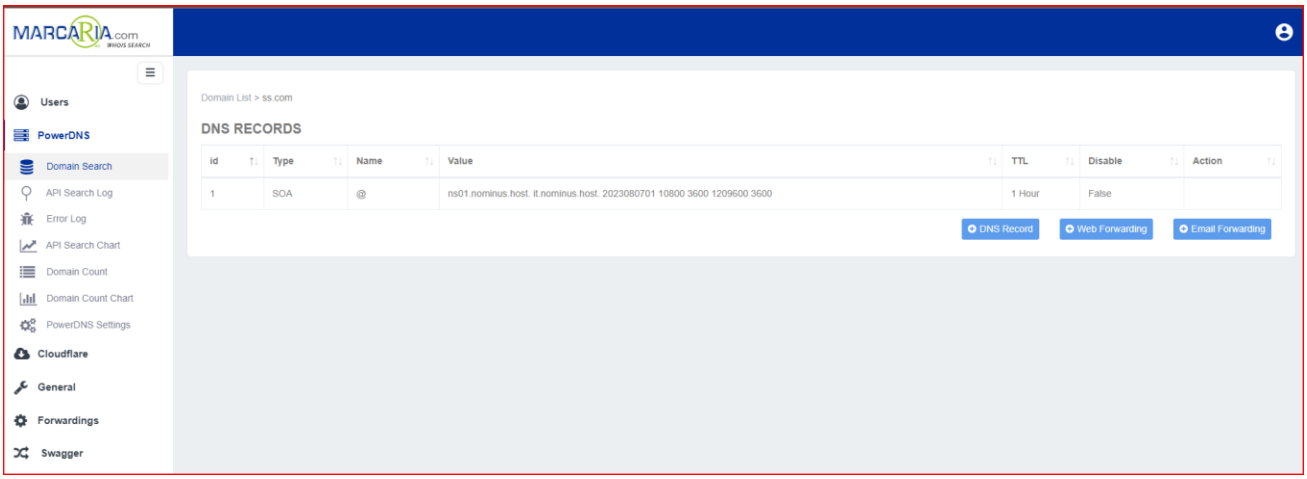
  - When the user clicks on this button it will open the popup of - “ The following Domains are selected for DNS Record update ”.
  - Here users can get all names of the domain which they are updated.
  - Users can get one drop down for the selection of the DNS record profile that they want to update.
  - In the drop down box users can view the profiles which are created under the **DNS profile and customer DNS Profile menu**.
  - Click on the **Cancel button** and it will close the popup.
  - Click on the Save **button and the user** is able to replace old DNS records with the selected profile records and get the notification message - “Profile Updated Successfully.

DNS Record Update : Select the DNS Record Profile and Customer Dns profile that you Want to Update to.



- **Clean DNS records in bulk button:**

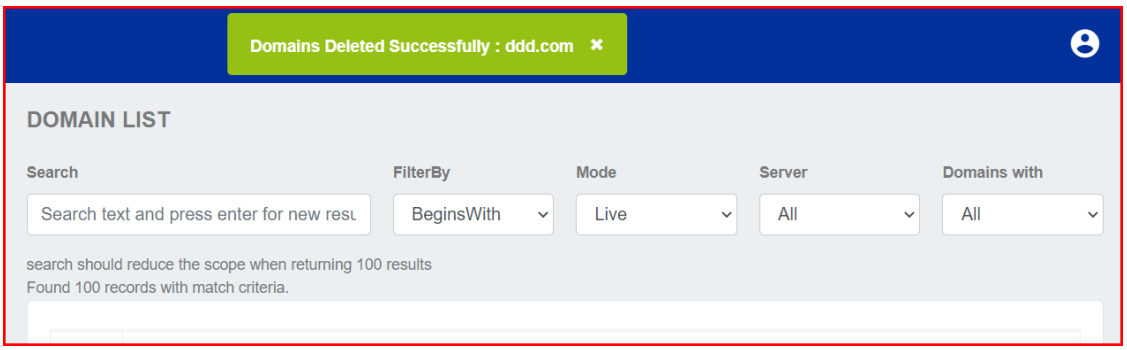
  - Here users are able to clean bulk domains records.
  - Users can check/uncheck the domain as per the requirements which are available in the domain list.
  - By clicking on this button the user is able to clean all the records **except SOA** which are available in the selected domain.



- **Remove Domain button:**

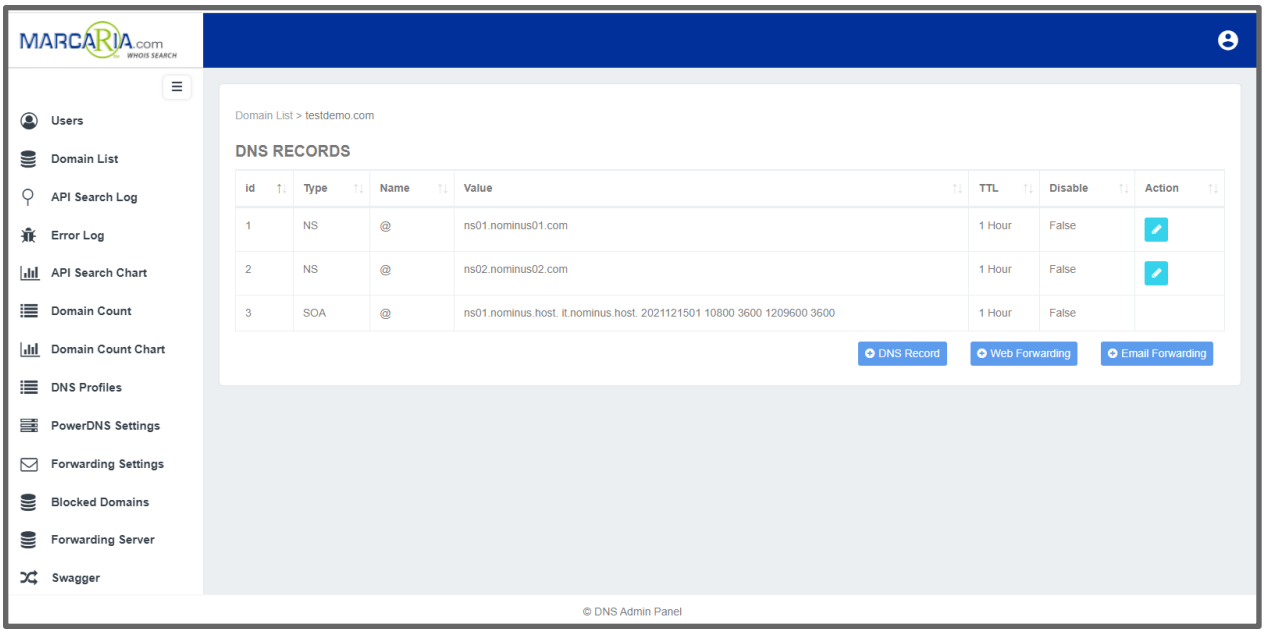
  - Here users are able to remove multiple/bulk domains.
  - Users can check/uncheck the domain as per the requirements which are available in the domain list.
  - After checking the checkbox, click on the remove button and the user can able to remove the selected domain from the domain list.

-



- **Cancel button:**
  - By click on the cancel button “Change DNS record popup will be close”

- **DNS Record List :**
  - By click on any domain name user can able to view DNS Records
  - In this list users are able to view default records like 2 NS records and 1 NS record which are automatically added at the time of the creation of the domain.
  - Here users can see ID, type name , value, TTL, disable and cancel column.
  - Users are able to see DNS records , web forwarding and email forwarding buttons.



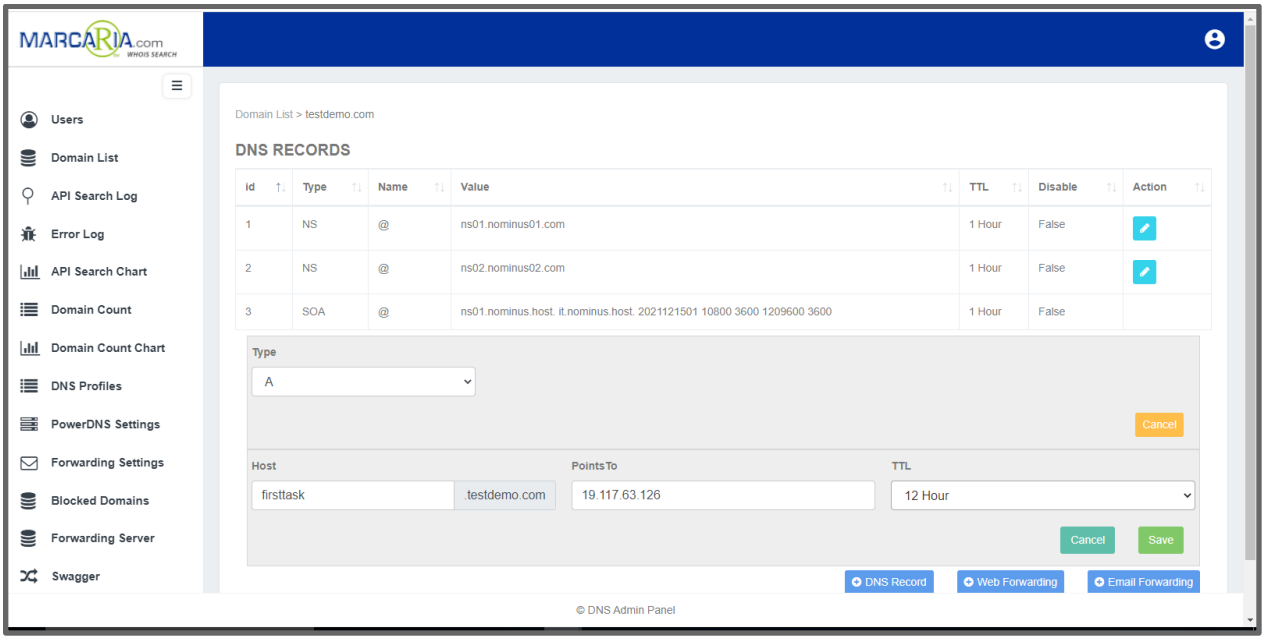
- Here users are also able to add new records by clicking on the DNS Record button.
- When the user clicks on the DNS record button it will open the section for selection of type of record from the drop down.
- Type of records are :
    1. **A** Record
    2. **AAAA** Record
    3. **MX** Record
    4. **TXT** Record
    5. **SRV** Record
    6. **CAA** Record
    7. **CNAME** Record
    8. **NS** Record

- Following are the available types.

    - **A type record :**

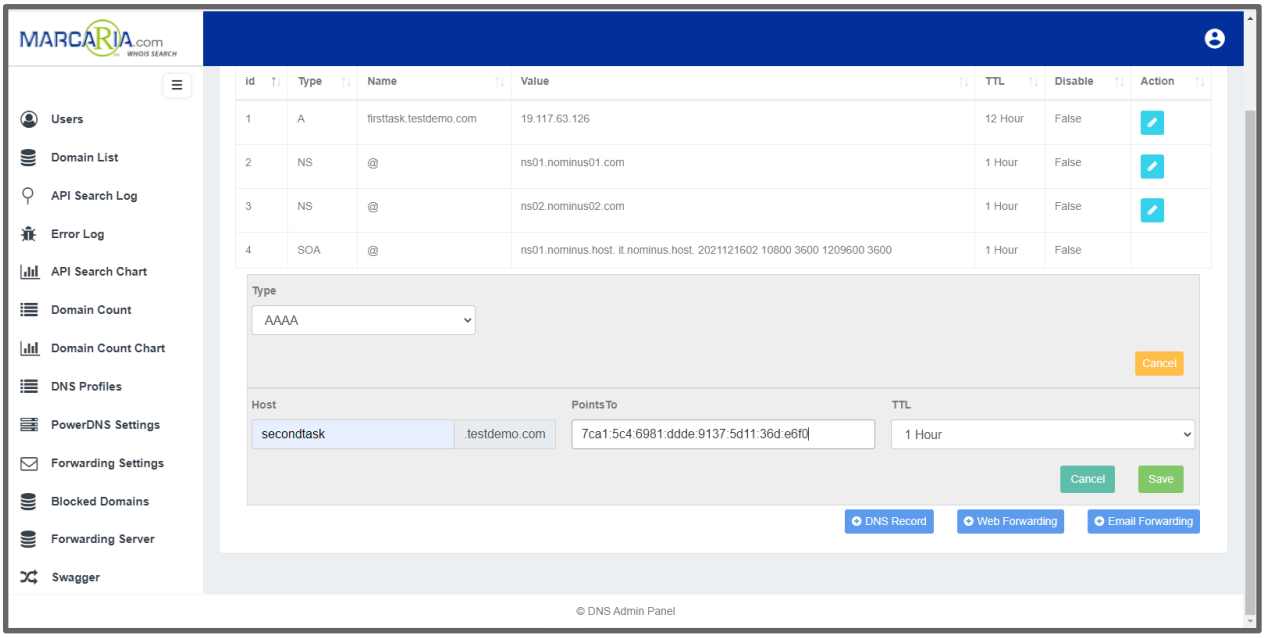
- Firstly, address (A) records are the most common record type by far. In brief, A records map domain names to IPv4 addresses.
- IPv4 notation is : **192.0.2.146**.
- When a user selects to add A type of records it will have the following options…

    - **Host :** Provide host name.
    - **Points To :** Add IPV4 address. Here if user not enter the valid IPV4 it will generate validation message like ..”Entered value is not valid IP”
    - **TTL :** Select time duration from drop down
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.



- **AAAA Type Record :**

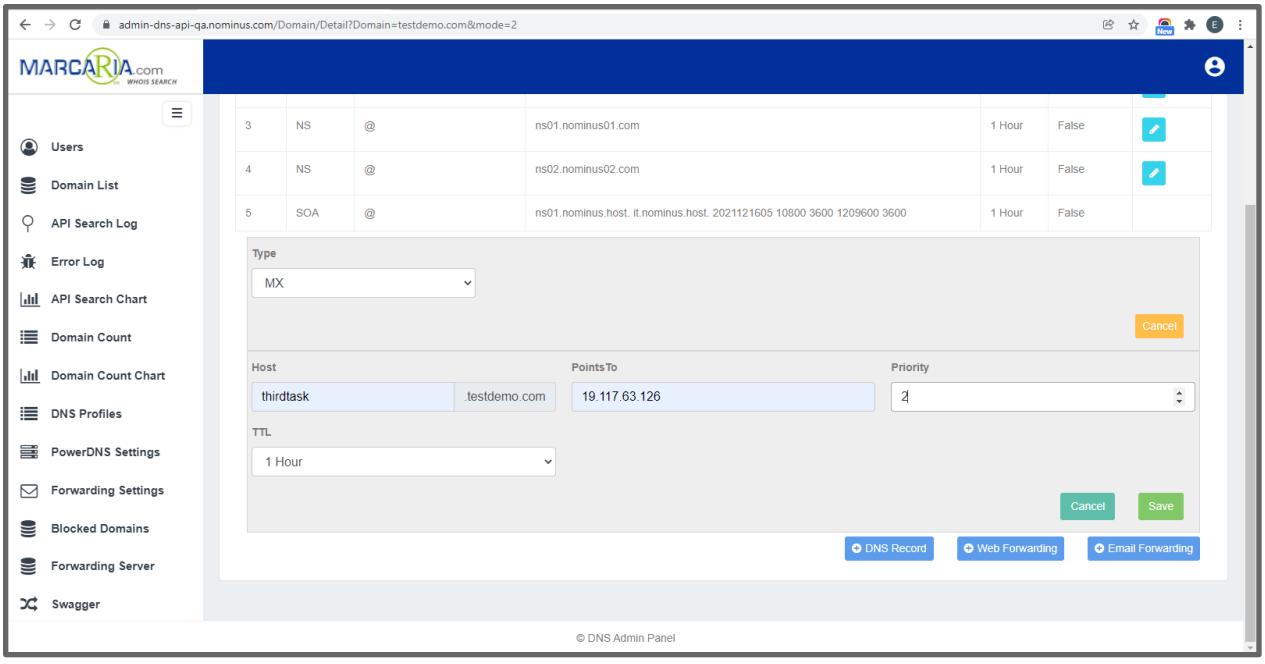
- As the internet gradually makes the transition to IPv6, there are AAAA records (spoken as “quad A”). An AAAA record is the IPv6 equivalent of an A record, responsible for mapping a domain name to an IPv6 address.
- IPv6 notation is :2001:db8:3333:4444:5555:6666:7777.8888
- When a user selects to add AAAA type of records it will have the following options…
    - **Host :** Provide host name.
    - **Points To :** Add IPV6 address. Here if user not enter the valid IPV6 it will generate validation message like ”Enter value in valid IPV6 format”
    - **TTL :** Select time duration from drop down
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.



- **MX Type Record** :

- A mail exchanger (MX) record stores the domain names of mail servers responsible for receiving emails on behalf of a domain.
- These records can be helpful to identify authorized mail servers. An MX record is sometimes called an MX entry, particularly in configuring mail servers.
    - **Host :** Provide host name.
    - **Points To :** Value is Required.
    - **Priority :** Set the priority. The priority you set when you add an MX record set specifies the sequence that an email server receives emails. A smaller value indicates a higher priority.
    - **TTL :** Select time duration from drop down
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.

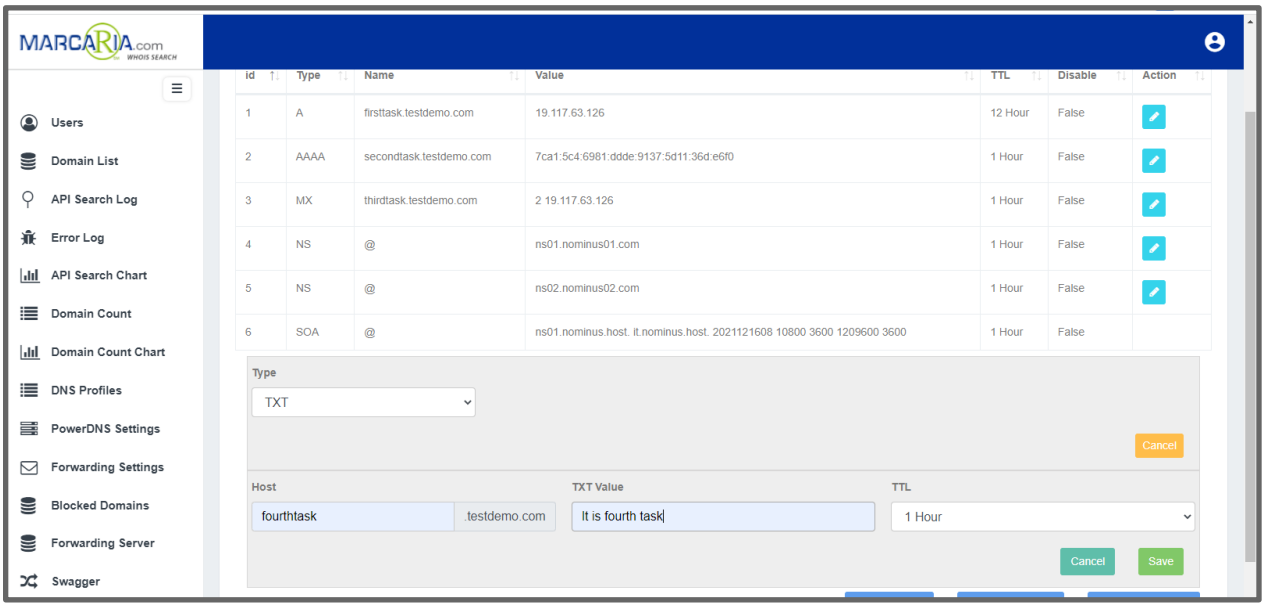
web fs



- **TXT Type Record** :
    - A text (TXT) record can store any type of descriptive information in text format. For example, network antivirus

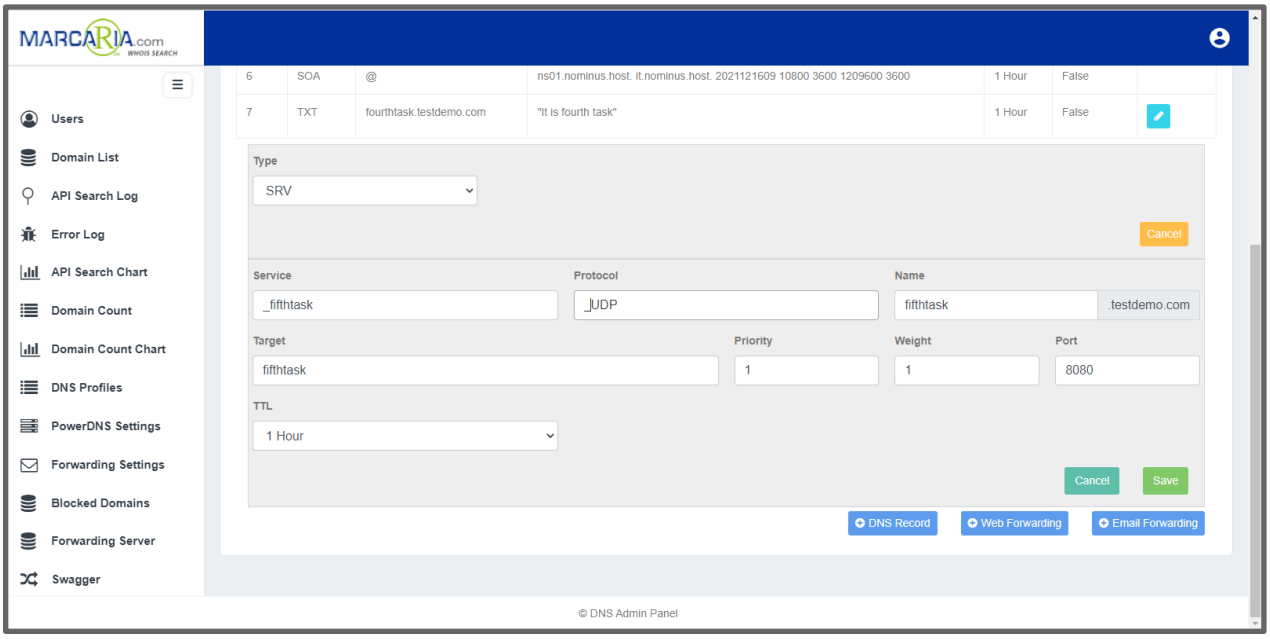
providers can use TXT records to send and receive information.

- **Host :** Provide host name.
- **TXT value :** Provides text value for the record.
- **TTL :** Select time duration from drop down
- **Save button :** Click on save button it will save the record.
- **Cancel button :** Click on cancel button it will stop the process.



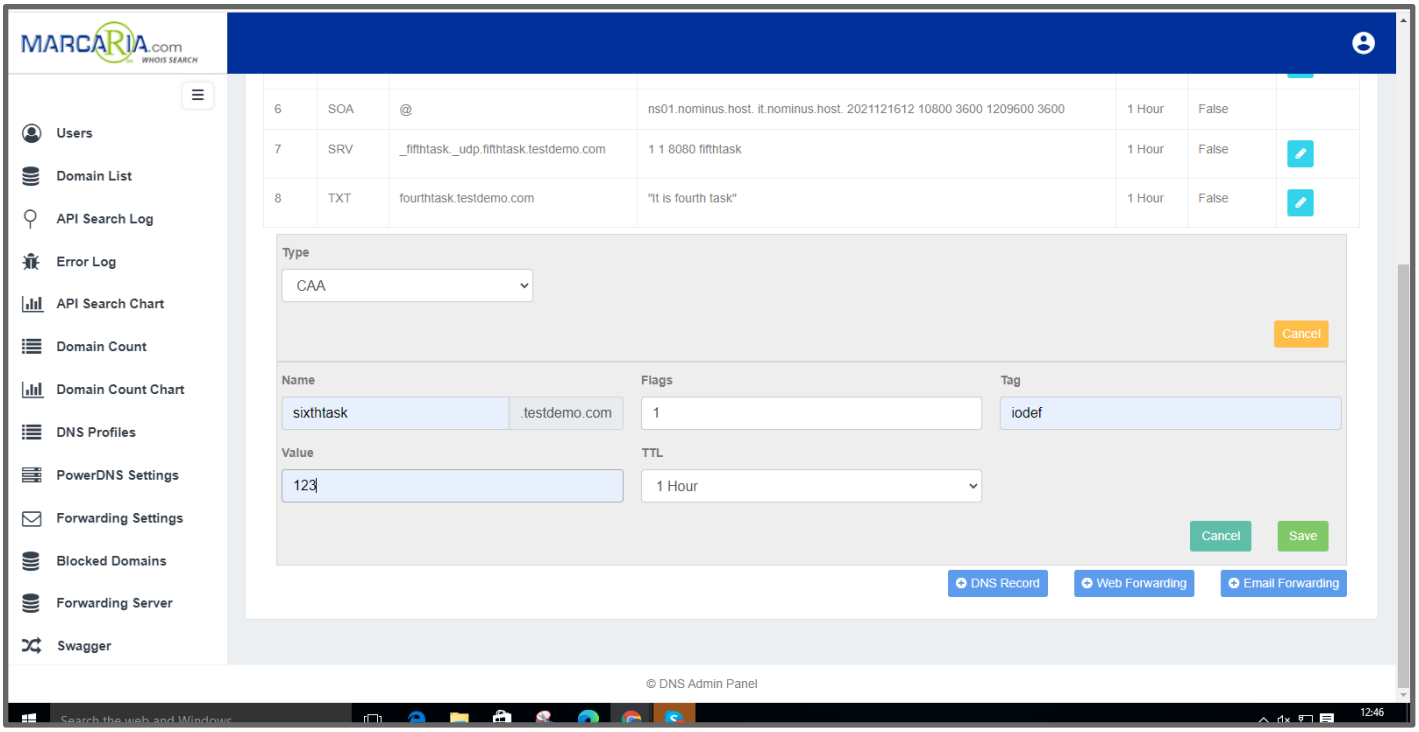
- **SRV Type Record :**
  - The SRV record is a Domain Name System (DNS) resource record. It's used to identify computers hosting specific services. SRV resource records are used to locate domain controllers for Active Directory.
    - **Service :** Symbolic name for a service. Here value must be start with underscore (_)and alphanumeric
    - **Protocol :** Specifies if the service is using TCP or UDP protocols. Here value must start with underscore (_) and alphanumeric.

- **Name :** Stores a domain name.
- **Target :** The canonical hostname for the machine providing the service.
- **Priority :** Determines which server is looked at first, with lower values giving higher priority.
- **Weight :** Determines which server is looked at first if more than one has the same priority value. Higher values give more priority.
- **Port :** The TCP or UDP port the service is running on. {Transport Layer Protocol : 1. TCP (Transmission Control Protocol) , 2. UDP(User Datagram Protocol)}
- **TTL :** Select time duration from drop down
- **Save button :** Click on save button it will save the record.
- **Cancel button :** Click on cancel button it will stop the process.



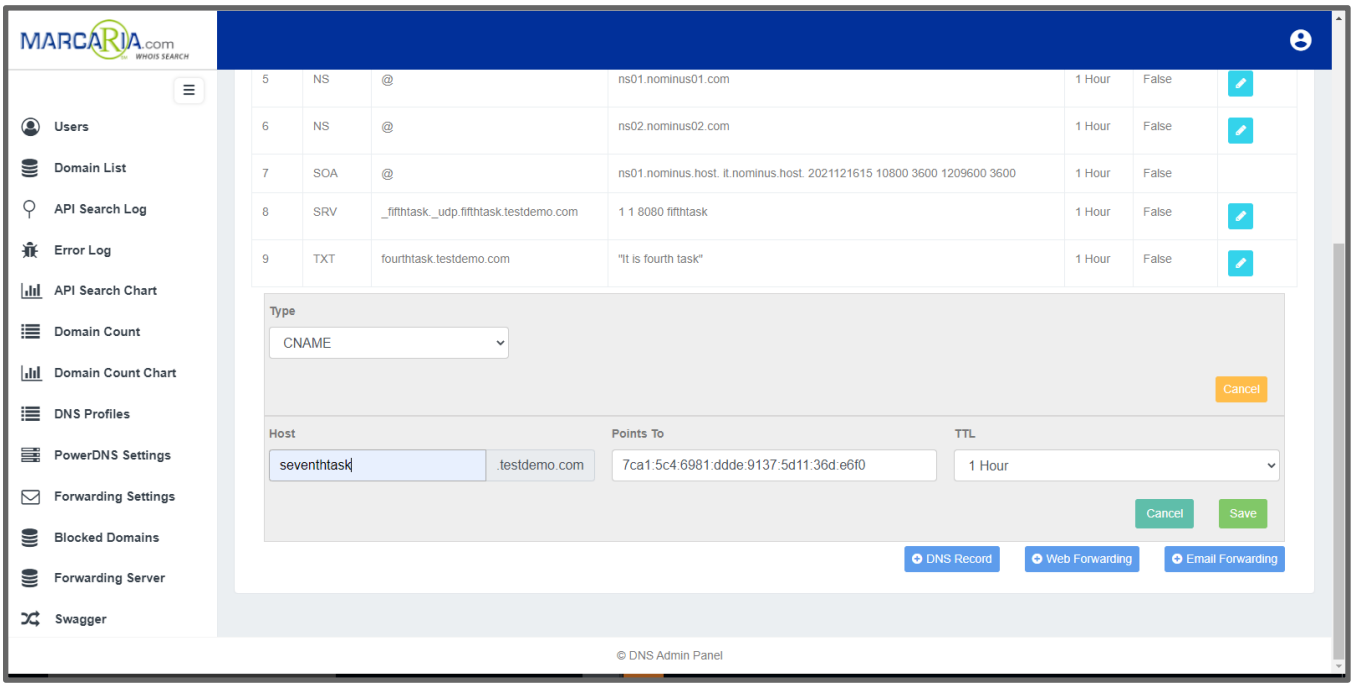
- **CAA Type Record**:

- CAA is a type of DNS record that allows site owners to specify which Certificate Authorities (CAs) are allowed to issue certificates containing their domain names.
    - **Name :** The name of the domain that is being certified.
    - **Flags :** Can be either 1 (critical) or 0 (non-critical). A critical flag means the certificate authority cannot use the CAA record if it doesn't understand the property. A non-critical flag means it can use the CAA record regardless of whether it understands the property.
    - **Tag :** The three tag options are issue (authorized to issue a single certificate), issuewild (authorized to issue a wildcard certificate), and iodef (specifies a URL for reporting policy violations).
    - **Value :** User can enter any value
    - **TTL :** Select time duration from drop down
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.

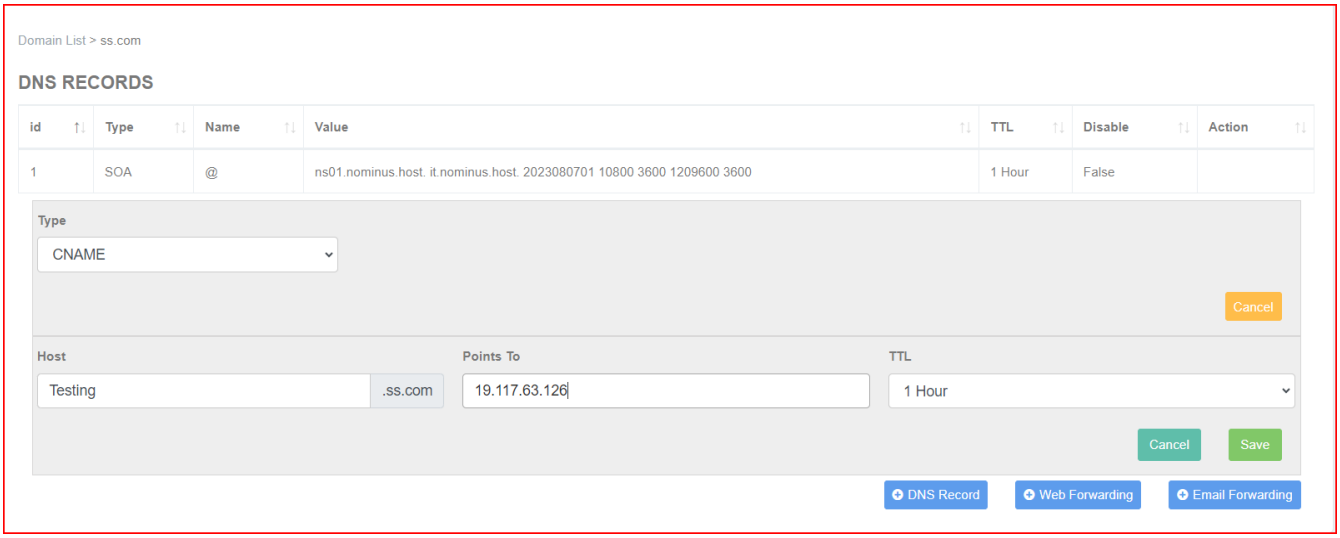


- **CNAME Type Record :**

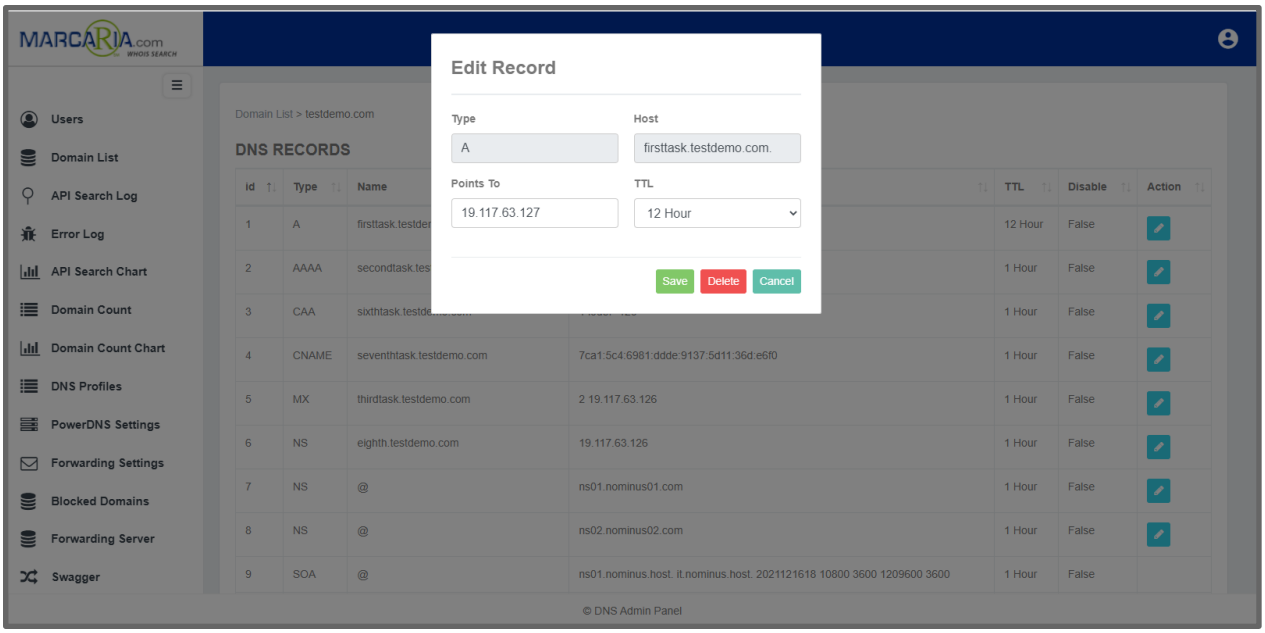
- Redirects a domain to a different domain. CNAME records are typically used to map a subdomain such as www or mail to the domain hosting that subdomain's content.
    - **Host :** Provide host name.
    - **Points To :** Add IPV6 address.
    - **TTL :** Select time duration from drop down.
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.



- **NS Type Record :**
    - Provides a list of the authoritative name servers responsible for the domain. it's important to note that NS records point to the domain of the name server—not IP addresses.
        - **Host :** Provide host name.
        - **Points To :** Add IPV4 address.
        - **TTL :** Select time duration from drop down.
        - **Save button :** Click on save button it will save the record.
        - **Cancel button :** Click on cancel button it will stop the process.



- **Edit Icon :**
  - Click on the edit icon from the Action field of the DNS records table.
  - Users can get the popup with following details..
    - Fields of the record which are generated by the user.
    - Save button
    - Delete button
    - Cancel button
  - **Type** and **hostname** are disabled at the time of edit and all other fields users can modify.
  - **NOTE:** For the SRV record type - Type, Service, Protocol and Name fields are disabled.



- **Save Button:** Click on the save button after doing modification it will update old data with the updated data.

- **Delete Button:** click on the delete button it will delete the selected type record from the list.

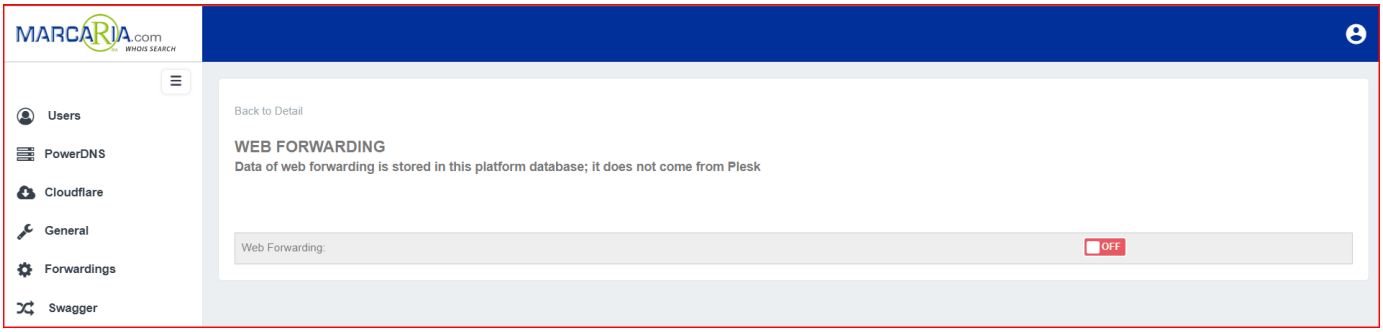
- **Cancel Button:** click on the cancel button it will close the popup.

- **NOTE :** Edit option is not available for SOA record.
- **NOTE :** Users can add @ if we want domain name as name.

- **Web Forwarding Button:**

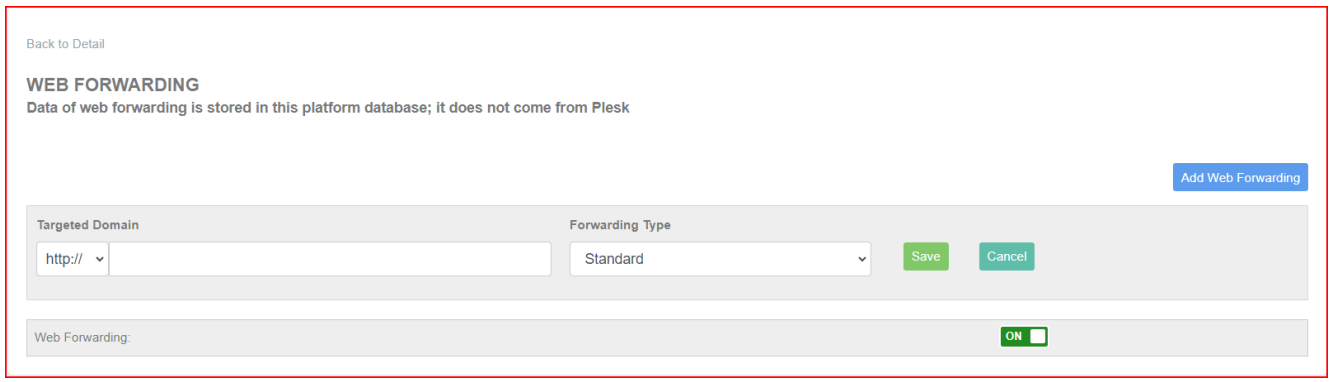
  - When clicking on the web forwarding button it will open the Web forwarding page.
  - Here users are able to get the switch for ON/OFF the web forwarding for the selected domain.
  - When user clicks ON (add/update TXT record for identify forwarding server as well webfwd/email fwd ON/OFF, create domain in plesk and add dns records in powerDns from profile, reverse for OFF, If both will be off then remove all.)

- Here users can ON the switch to enable Web Forwarding.
- And users can OFF the switch to disable the Web Forwarding.
- Add web forwarding option will be shown when user switches on

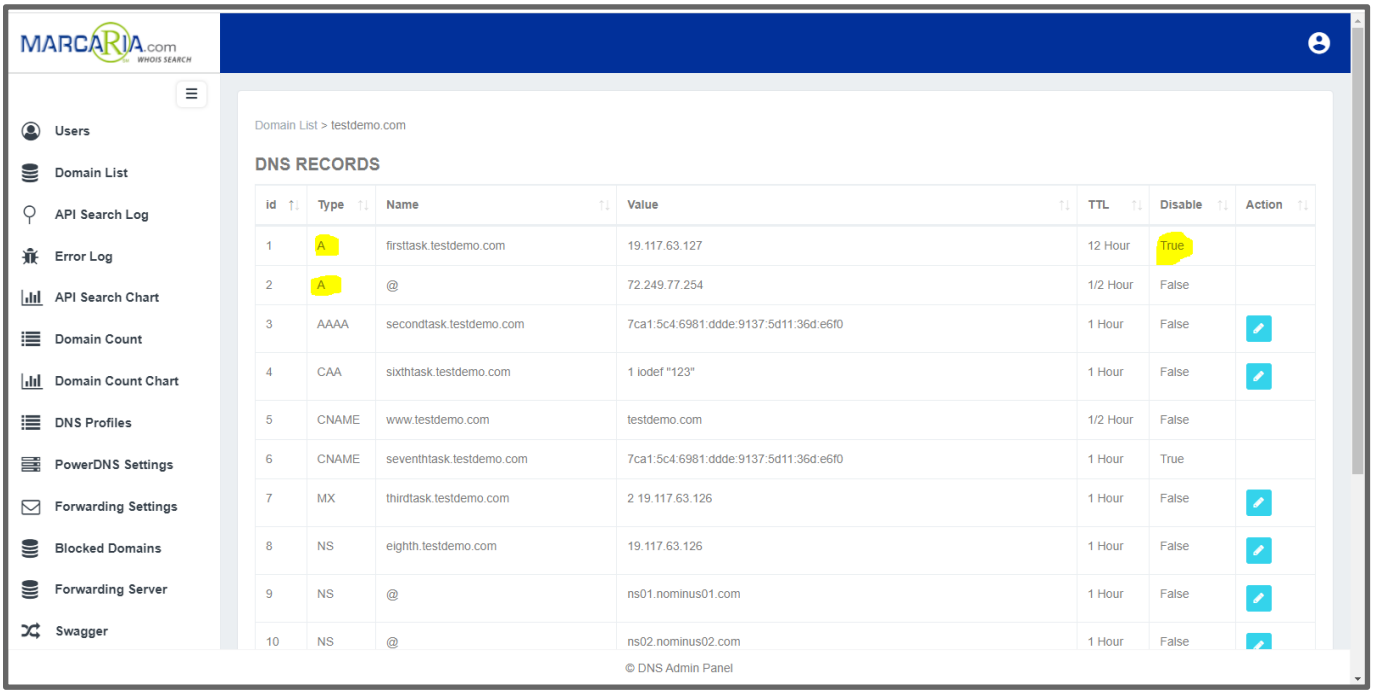


- **Add Web forwarding Button:**

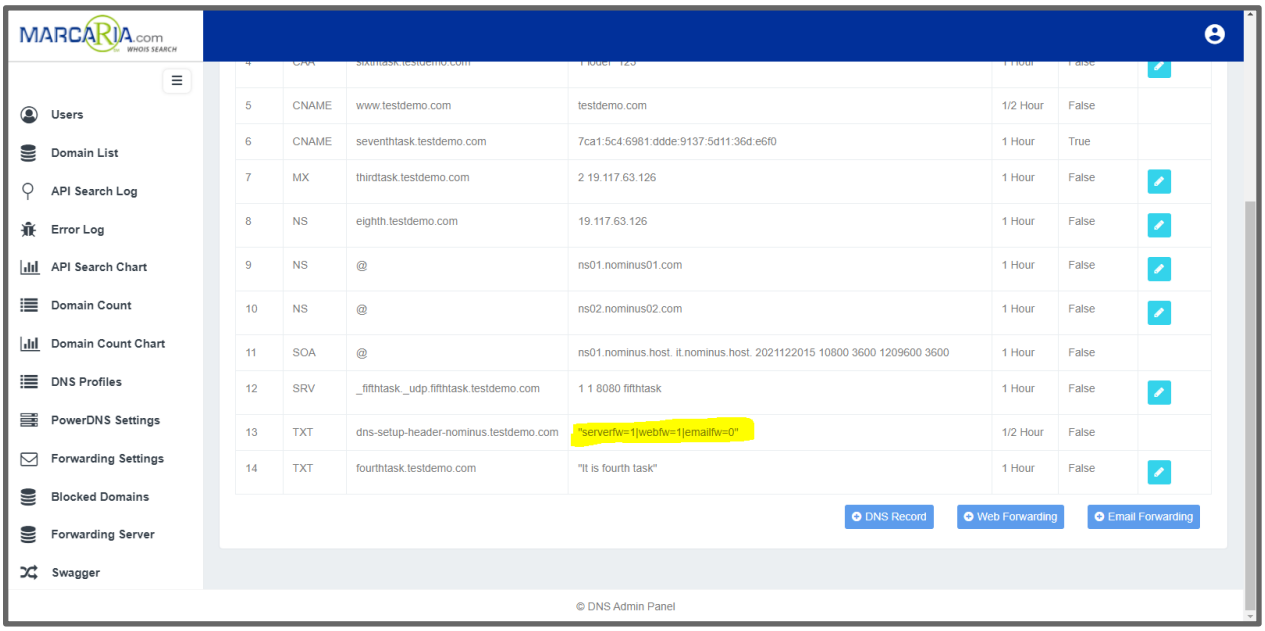
  - Click on Add Web forwarding Button it will show the section for entering the information about Target domain and Forwarding Type.
  - Added records also create on plesk as well database same for email forwarding
  - **Target Domain:**
    - Here target domain have to protocols : 1) http:// 2) https://
    - Enter the Website URL in which you want forwarding.
  - **Forwarding Type:**
    - Here users get three options such as
    - **Standard:** With the Standard forwarding, users who have been redirected to another URL can see the destination URL in the browser address bar.
    - **Frame:** With the Frame forwarding, when visitors are redirected to another site, the address bar of their browsers continues to show the source URL. Thus, visitors remain unaware of the redirection.
    - **None:** It means not redirecting.
  - Click on save button to save the data.



- After successfully Save the data Move to your domain i.e. **Domain List > Click on your domain name.**
- Here on the DNS records page, users need to check the following things..
    - If there is the same type of record already available in the list then it will make disable = true to the new record and consider the newly added record.
    - **NOTE:** Here, new records are imported from the Forwarding Server > Server name which is active i.e. Default = YES

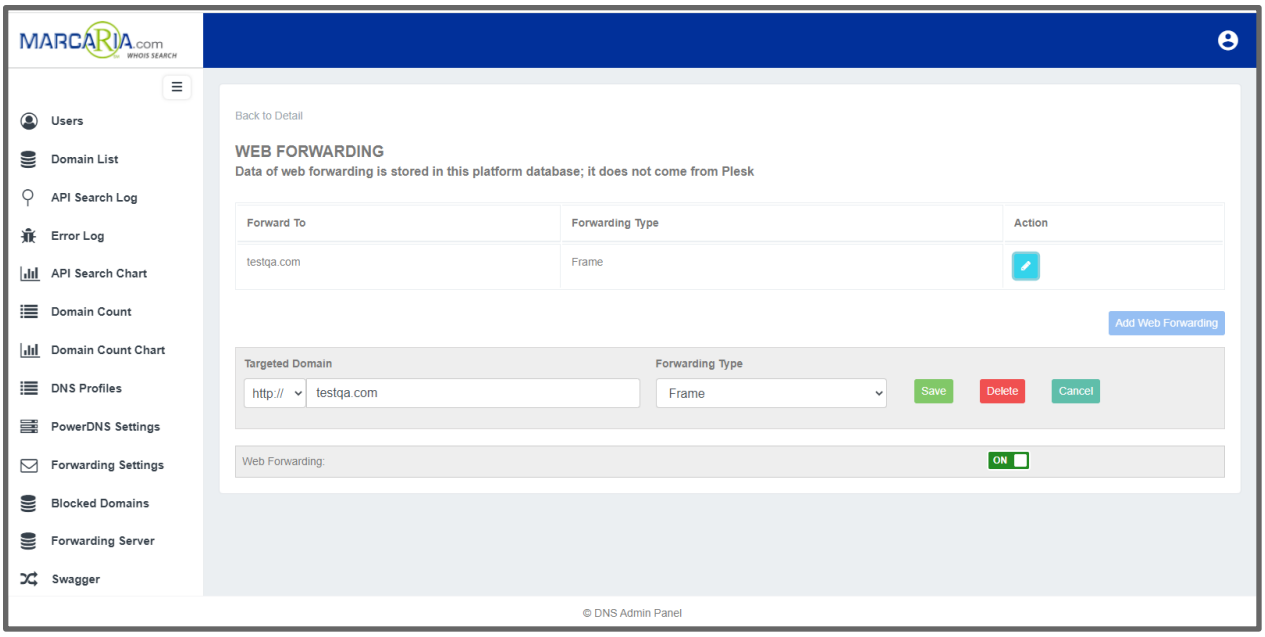


- TXT Record is by default added and it carries information like ..
  - ServerFW=’1’ means “1” is Server ID
  - WebFW=’1’ means “1” for Web forwarding ON and “0” for OFF
  - EmailFW=’0’ means email forwarding is OFF and “1” for ON.



- **Edit Web forwarding Icon:**

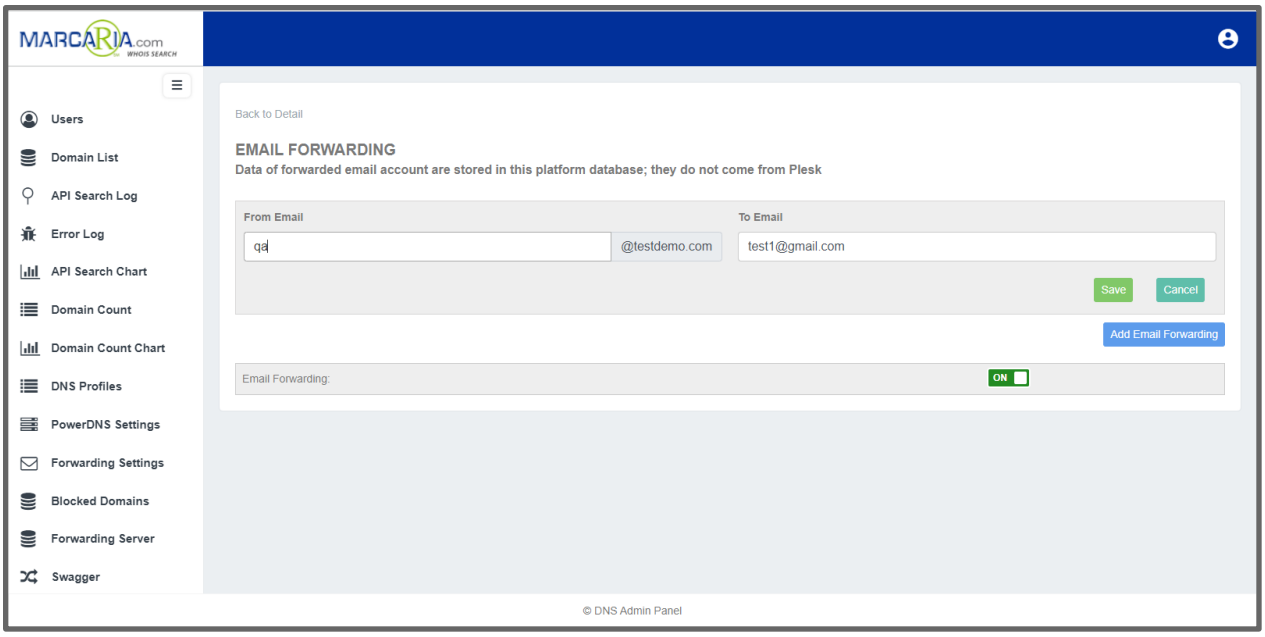
  - Once domain forwarding is created it will create a table with fields like Forward To, Forwarding Type, Action - Edit Icon.
  - **Forward To:** It will display the name of the domain on which you want to redirect.
  - **Forwarding Type:** It will display the selected type for the domain.
  - **Edit Icon:** Click on the Edit icon it will open the section which allows us to edit the data.



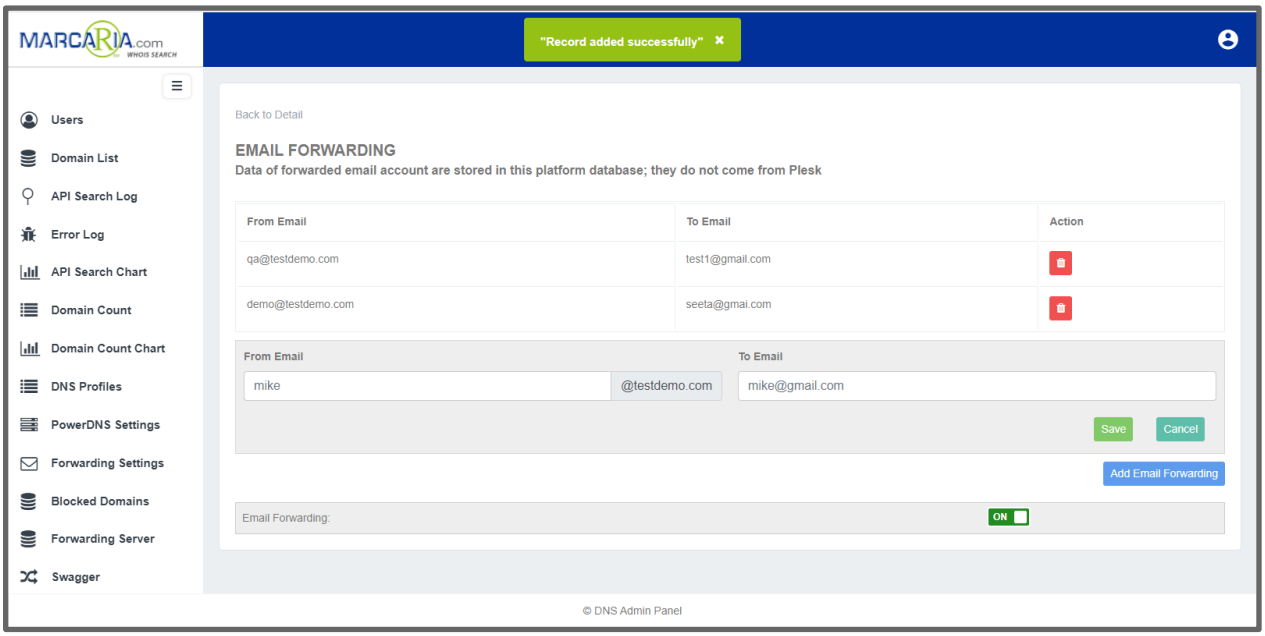
- **Save Button:** Click on the Save button it will save the modification.
- **Delete Button:** Click on the Delete button it will delete the record.
- **Cancel Button:** It will Cancel the process

- **Email Forwarding Button:**

- Click on the Email Forwarding button it will open the email forwarding page.
- Here users are able to get the switch for ON/OFF the email forwarding for the selected domain.
- Here users can ON the switch to enable email Forwarding.
- And users can OFF the switch to disable the email Forwarding.
- By doing ON email forwarding it will open the section for the entering details of From Email and To Email.
- Here, users can add multiple records.
- **From Email :** Here users need to add a source email address from which you want to forward the emails.
- **To Email :** Here users need to add a destination email address in which they want the email address.



- **Save:** Click on the Save button record is added and display it on the screen.
- **Cancel:** Click on the Cancel button, the process will stop.

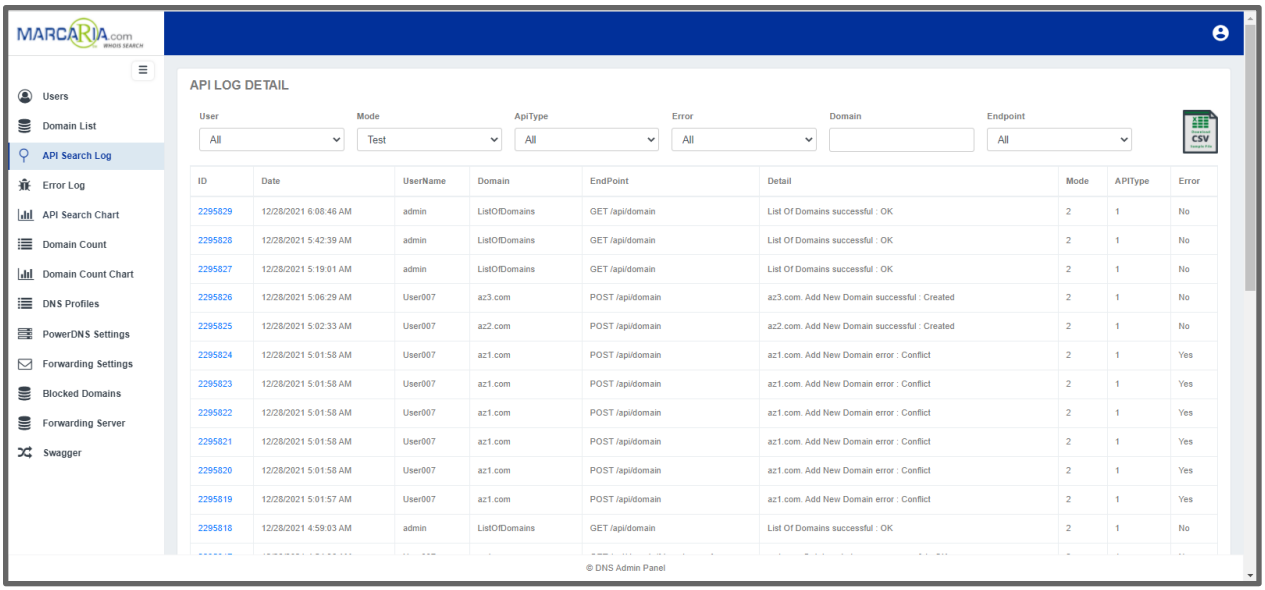


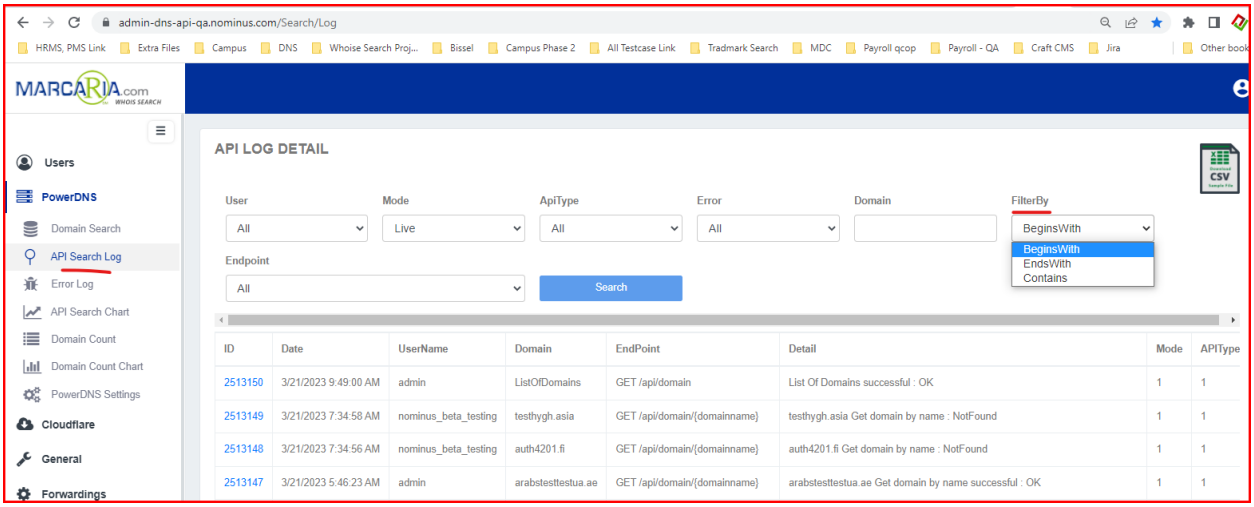
- **Delete Icon:** Users are able to view the delete icon at the table of the list of email addresses. Click on it and the user will be able to delete the record.

- **API Search Log:**

    - Here users are able to view API log details.
    - Users are able to download the CSV file of the API log.
    - Users are able to see different filter options such as…
        - **User:** Can filter according to the Users.
        - **Mode:** Can filter according to the Mode.
        - **API Type:** Can filter according to the API Type.
        - **Error:** Can filter according to the Error Type.
        - **Domain:** Search by entering the domain name.
        - **FilterBy:** Can filter according to the beginning with, end with and contains.
        - **Endpoint:** Can filter according to the Endpoint.

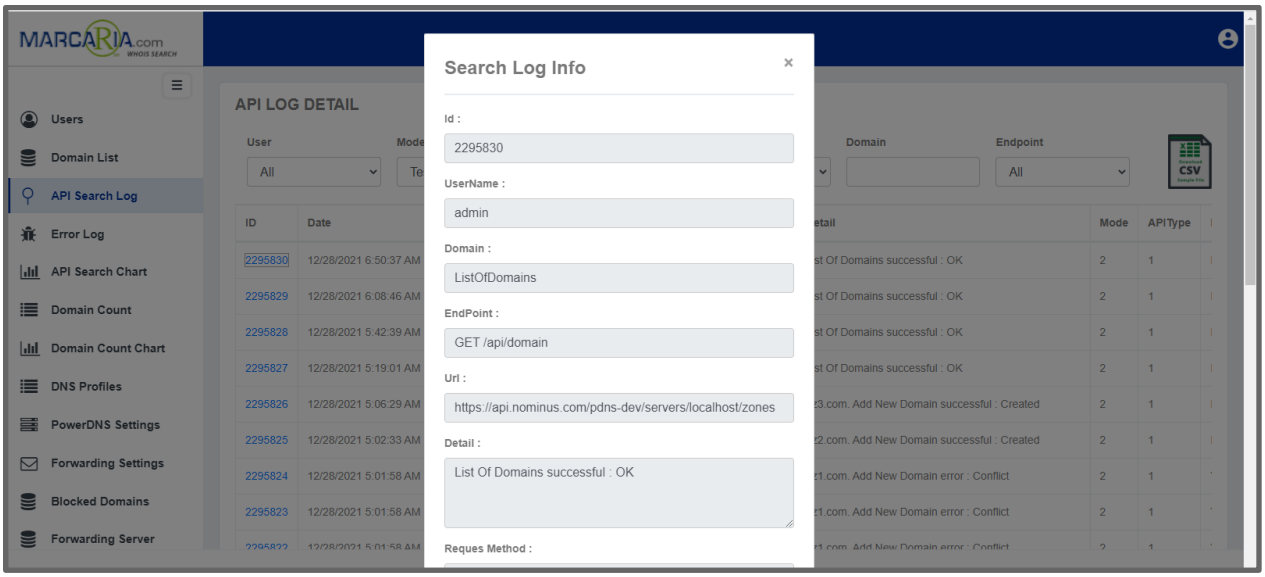
    - Users are able to view information like….
        - **ID:** Unique identification number.
        - **Date:** Timestamp of the error log.
        - **User Name:** Display user name. (Login with that name - admin)
        - **Domain:** Display domain name
        - **End Point:** Display API endpoint.
        - **Detail:** Provides information.
        - **Mode:** Display information of mode.
        - **APIType:** Display type of API.
        - **Error:** Display status if error occurs then Yes else No.
        - **FilterBy :** Display Dropdown with three option : Beginswith, Endswith, Contains **(New Implementation)**





- Click on the unique identification number (**ID**) and the user will get a popup with the information about the Search Log.
- All search log information will be displayed as disabled mode.
- In search Log Info popup users will get the following information.
    - **Id:** Unique identification number.
    - **User Name:** Information about User Name.
    - **Domain:** Information about domain.
    - **End Point:** Information about End Point.
    - **URL:** Information about URL.
    - **Detail:** Provides Information.
    - **Request Methods:** Information about method name.
    - **Request Header:** Information about requested header.
    - **Requested Response:** Information about requested response.

- **Request Body:** Information about request body.
- **API Type:** Information about API type.
- **Mode:** Information about Mode.
- **Date:** Information about time stamp.



● Error Log:

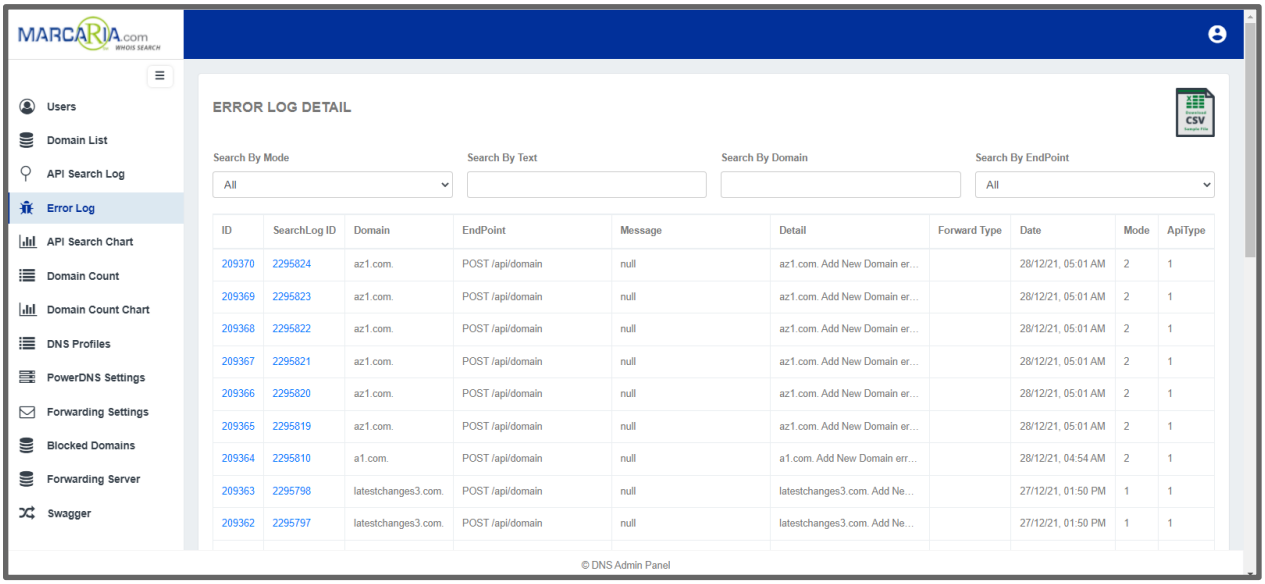
- Here users are able to view the error log if any errors occurred.
- Users can have the option to apply filters based on..
  - **Search By Mode:** It have three option Live, Test, All
  - **Search By Text:** Enter any text in the textbox and the user will get equivalent results.
  - **Search By Domain:** Enter the domain name which you want to search.
  - **Search By Endpoint:** Select the endpoint from the list.
  - **FilterBy:** Display Dropdown with three options : Beginswith, Endswith, Contains.

- Users are able to view the following information..
  - **ID:** Unique Identification number.
  - **SearchLog ID:** It is display the unique ID of the Search Log i.e. API Search Log.

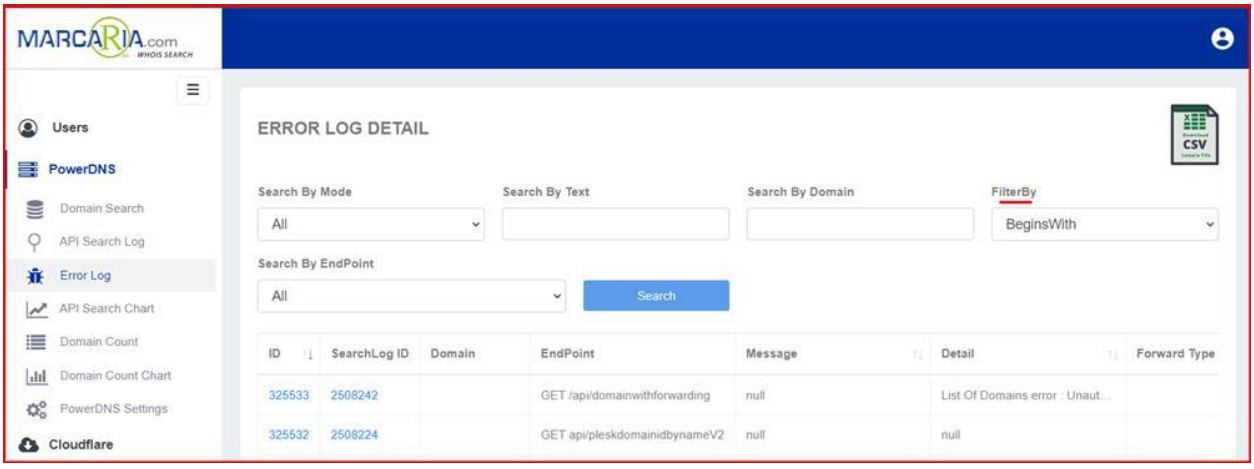
- **Domain:** Name of the domain in which error occurred.
- **EndPoint:** Name of the End Point.
- **Message:** It will print the exception message which has occurred.
- **Detail:** It will display the errors in details
- **Forward Type:** Display the forward type.
- **Date:** Timestamp is displayed. (Display Date & Time)
- **Mode:** Display information about Mode. (Live :1 , Test : 2)
- **API Type:** Display information about API type.
- **FilterBy :** Display Dropdown with three option : Beginswith, Endswith, Contains

○ Here users are able to download CSV files of the error log.

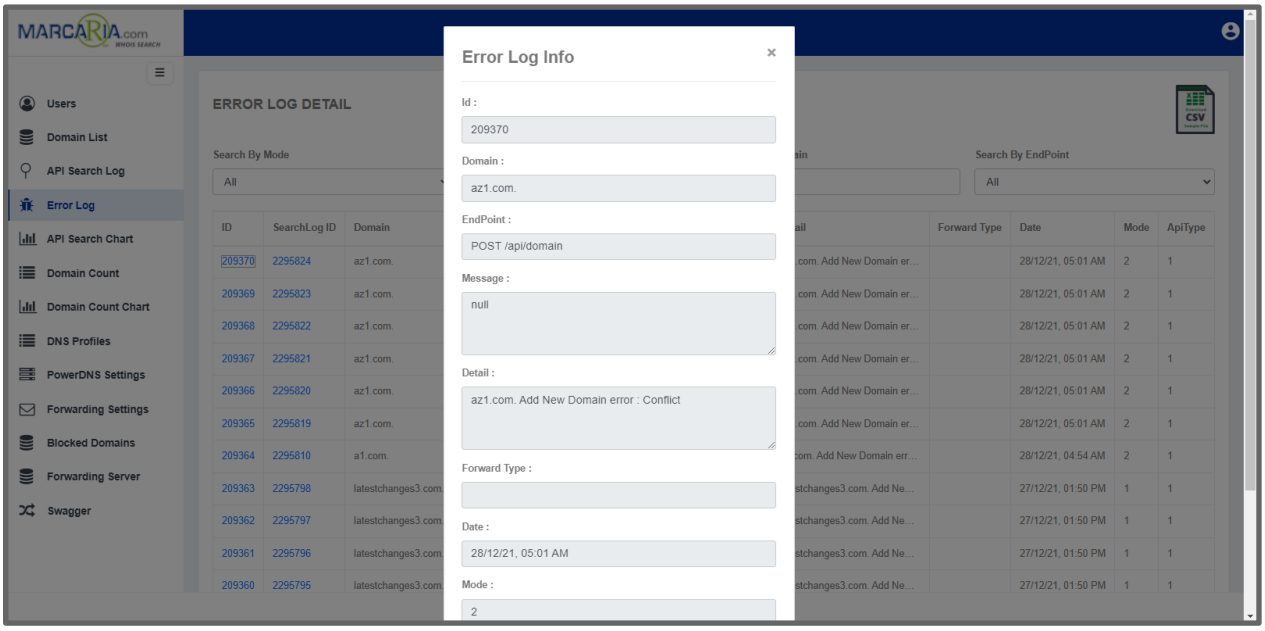
Old Snap :



New Changes Snap : Added Filterby option in Error Log Page.



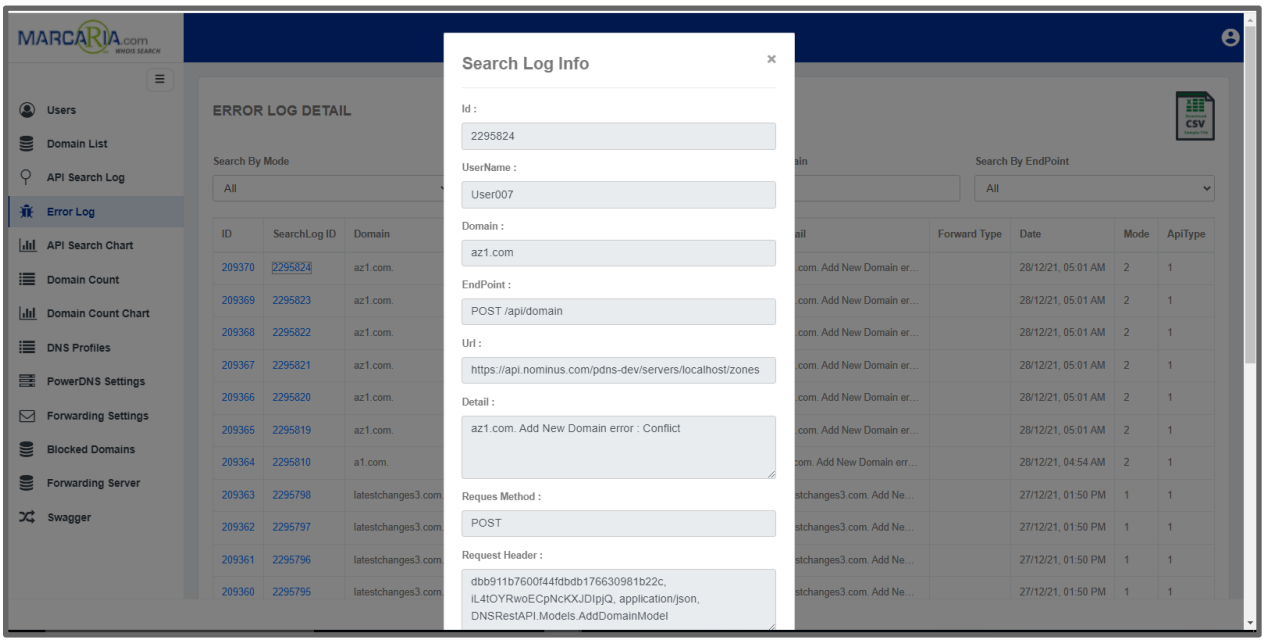
- Click on the ID user will be able to view the Error Log Info popup with following details..
  - **ID:** Unique identification number.
  - **Domain:** Display domain name.
  - **End Point:** Display End Point name.
  - **Message:** It will print the exception message which has occurred.
  - **Detail:** Display details about error.
  - **Forward Type:** Display the forward type.
  - **Date:** Display timestamp.
  - **Mode:** Display mode information.
  - **API Type:** Details of API type.
- All error log information will be displayed as disabled mode.



- Click on the **Search Log ID** user will be able to view Search Log Info popup with following information…
  - **Id:** Unique identification number.
  - **User Name:** Information about User Name.
  - **Domain:** Information about domain.
  - **End Point:** Information about End Point.
  - **URL:** Information about URL.
  - **Detail:** Provides Information.
  - **Request Methods:** Information about method name.
  - **Request Header:** Information about requested header.

- **Requested Response:** Information about requested response.
- **Request Body:** Information about request body.
- **API Type:** Information about API type.
- **Mode:** Information about Mode.
- **Date:** Information about time stamp.

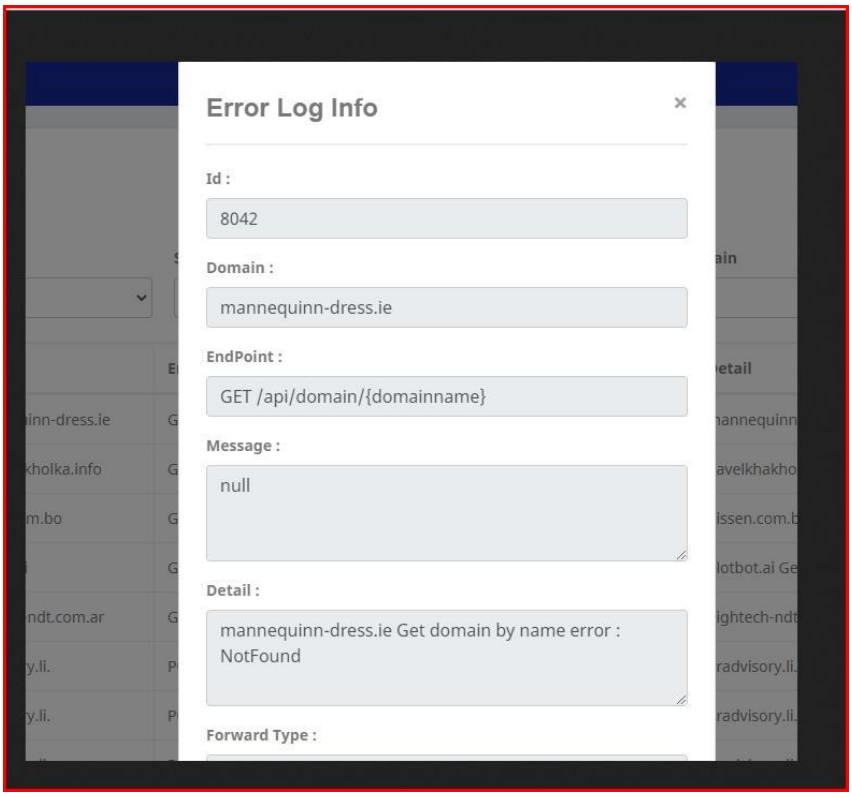
  - All search log information will be displayed as disabled mode.



New Changes Error Log :

Currently when checking if a domain is in the DNS we use
GET /api/domain/{domainname}
if the domain is not there, we are marking it as an error
for example https://i.imgur.com/B1E5sPN.png
[
In truth this is not an error, it means that the domain is just not created. Please do not mark them as error so they do not appear in https://admin-dns-api.nominus.com/ErrorLog/Index

-In https://admin-dns-api.nominus.com/Search/Log
https://admin-dns-api.nominus.com/ErrorLog/Index
New Implementation: please add the same filter **"Filterby"** for domains https://i.imgur.com/EicFQAT.png



TestingDocumentLink: https://docs.google.com/document/d/1cnYv158LLzsq75EuY4D-ZHDQp6SYIQzg8UtCM3nS4BE/edit

Error Log Changes :

1. Api search log and error log page: Added dropdown
2. If the domain is not created, you do not get the error in the error log and the error should not be displayed.
3. If the domain is not created, you will get a **"Not Found error"** display in the **search log.**

New Changes API Search Log Page: Endpoint Column

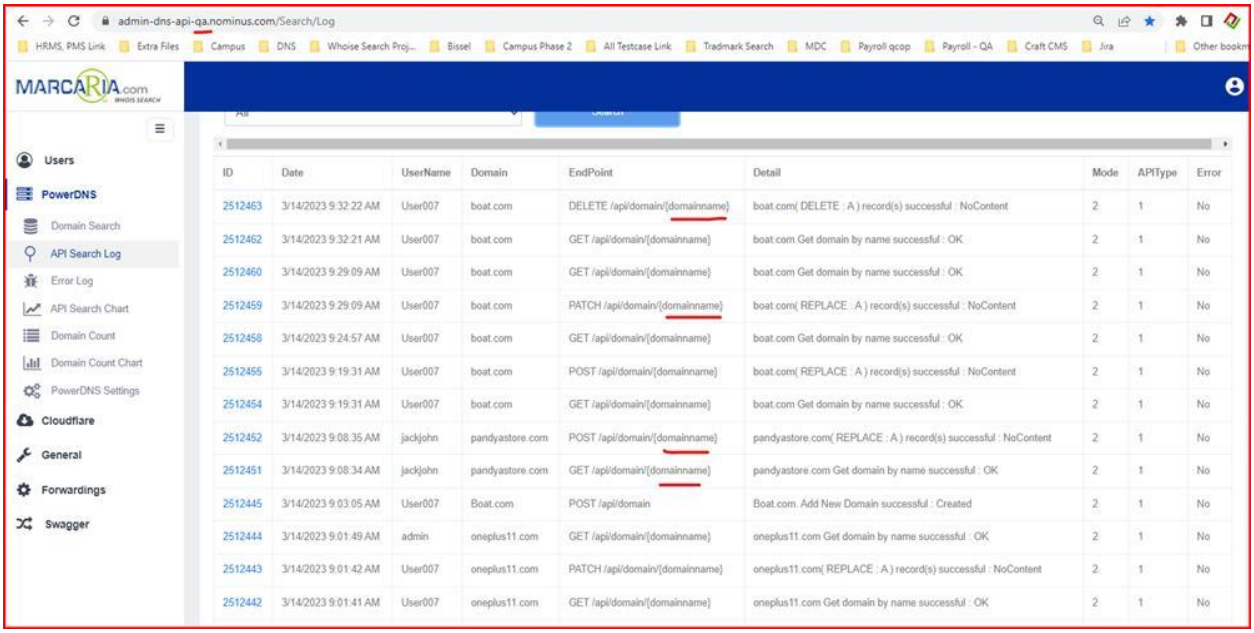
The last change done to improve the texts related to DNS operations introduced a confusing endpoint text for Domain Records operations.

Please, we need you to change the text shown in that column since can be confusing when following an issue:

In the column endpoint, we need you to fix the new way to show the data to the old one and it should be like:
POST /api/domain/{domainname}
PATCH /api/domain/{domainname}

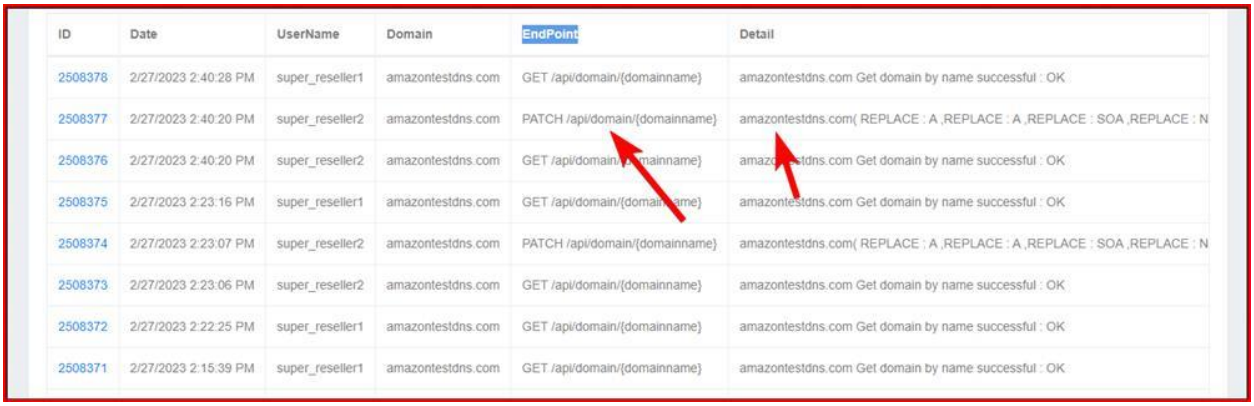
DELETE /api/domain/{domainname}

This comment is related to the Endpoint column only.



Update A Record Changes :

- The name of the Other record should not be displayed in the detail column when the A record is updated. (Ex : SOA , NS Record)



Detail Column : amazontestdns.com(REPLACE : A ,REPLACE : A ,REPLACE : SOA ,REPLACE : NS ,REPLACE : A) record(s) successful : NoContent

Would it be possible to reflect in the log the correct endpoint that was queried and also the exact change it was made, in this case " **we updated only A record and not SOA or NS ".**

A similar comment for endpoint for adding and deleting domains:
https://dns-api-qa.nominus.com/swagger/ui/index#!/DomainRecords/DomainRecords_Post
https://dns-api-qa.nominus.com/swagger/ui/index#!/DomainRecords/DomainRecords_Delete

Note Error Log :

- **Error Log :**

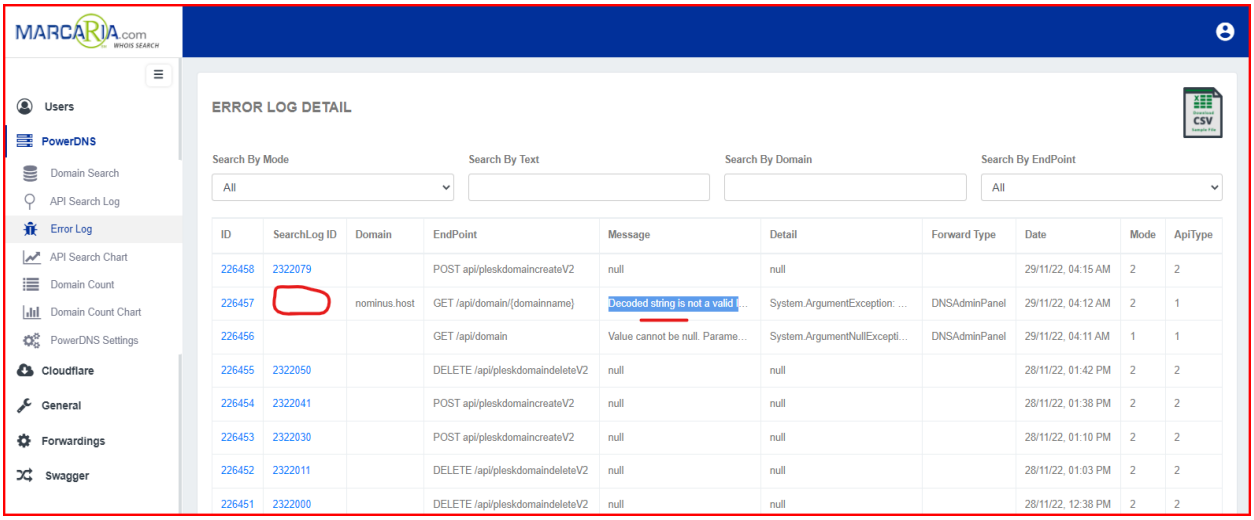
1. **Message :** NULL

- If there is an error in the coding, the message will be Displayed in the Message Column. (**Message : Decoded String is not a valid**)
- Id will not be displayed in Search Log Id. (Blank Display)

2. **Detail :**

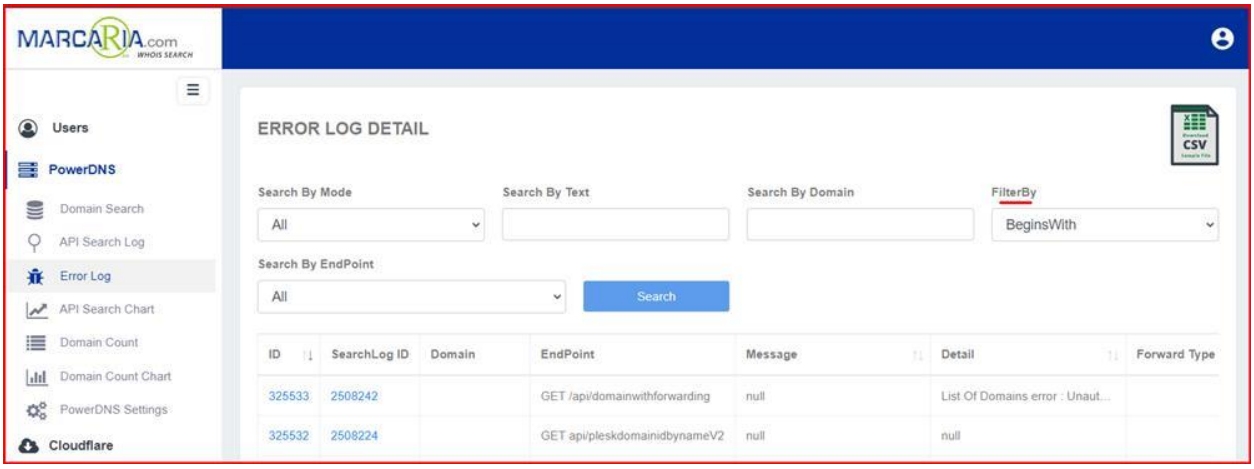
- If there is any error in the Api then the message will be displayed in the Detail column. (**Message :** System.ArgumentException)
- The ID will be displayed in the search log ID.

Old Snap :



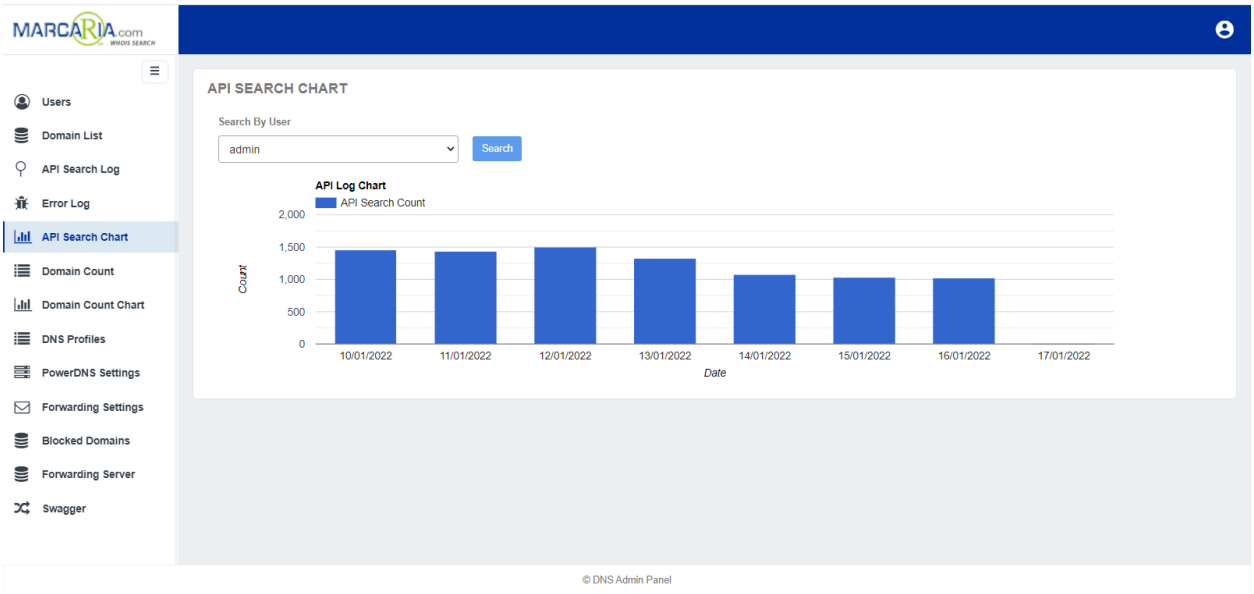
New Change Snap :

- **Error log Page :** Added Filter by Dropdown.

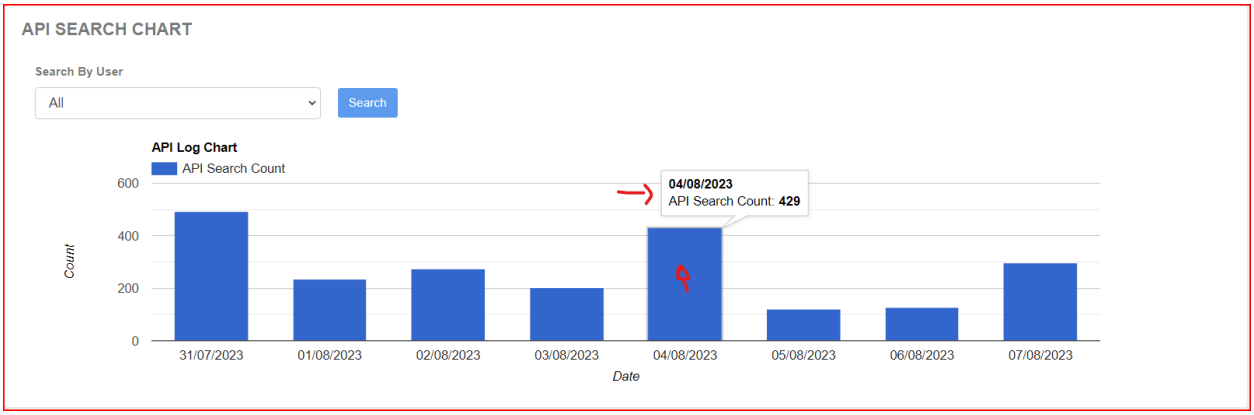


- **API Search Chart:**

  - It will display the total of API searches in the chart form.
  - It will display date on X axis and count on Y axis
  - Here users also can see the data according to the user by selecting the user available from the drop down.

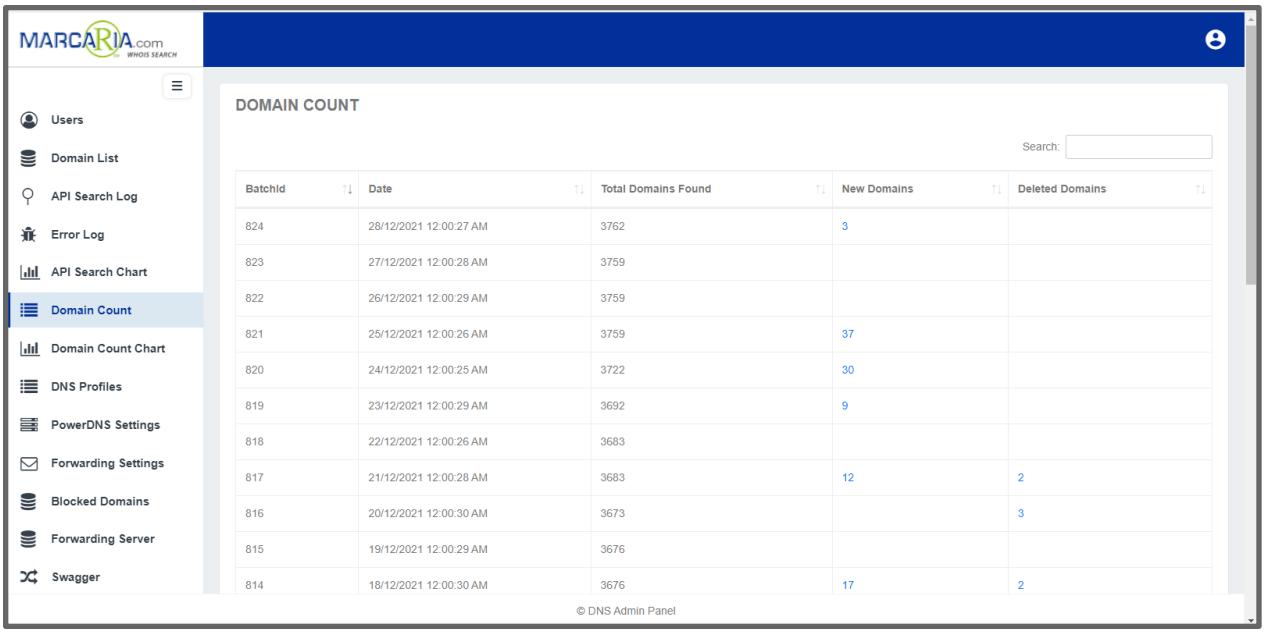


- When the user hovers over the graph, the date and API search counts will appear.



- **Domain Count :**

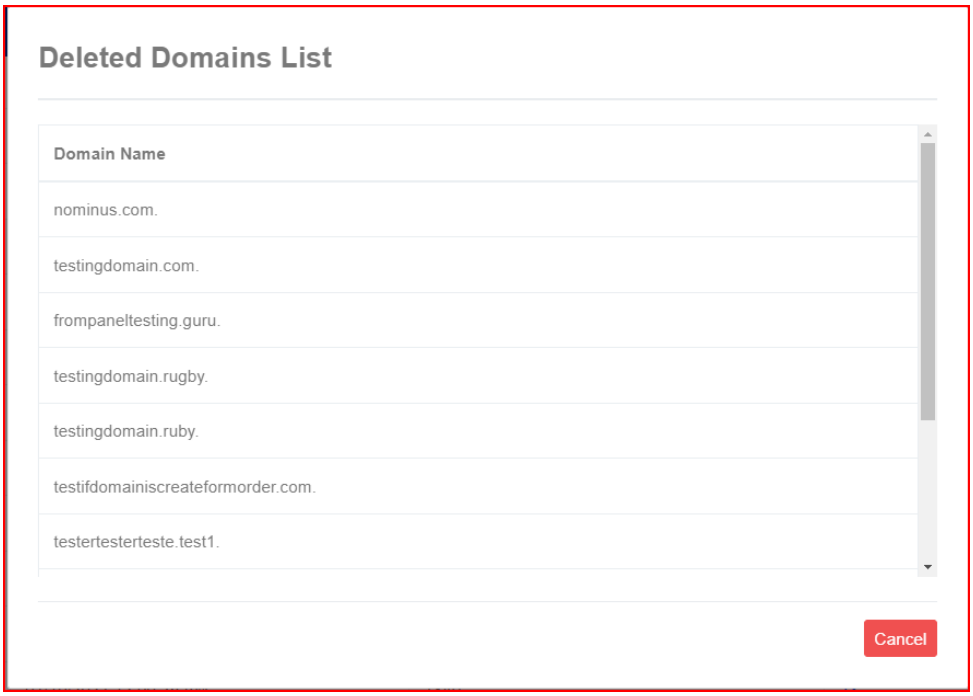
  - Here users can check the log of the created domain according to the date.
  - The Latest created domain will be displayed **Tomorrow**. (Domain Count)
  - Users are able to find the table with the following fields..
    - **Batch ID:** It is a unique identification number.
    - **Date:** The timestamp of the domain creation.
    - **Total Domain Found:** It will provide information about the total domain creation till date.
    - **New Domain:** It will provide information about the newly created domain of the specific date.
    - **Deleted Domain:** It will provide information about the deleted domain for the specific date.
  - Using a search box, users are able to search the records.
  - Using sorting functionality users are able to sort the data.



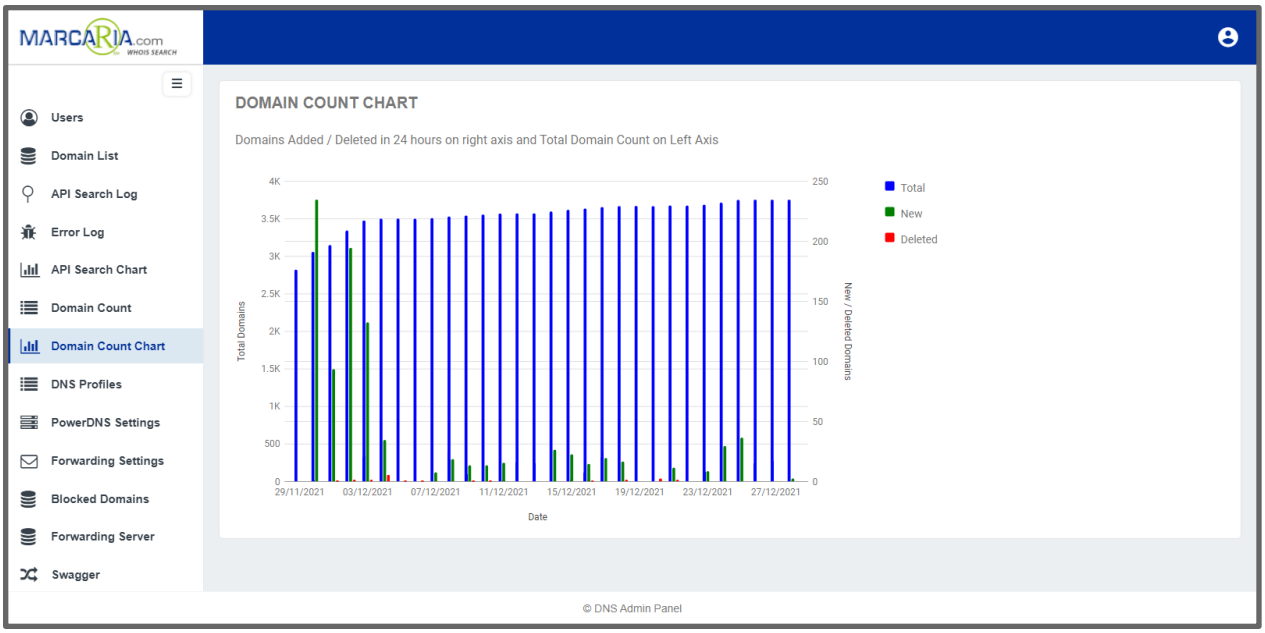
- When the user clicks on the **new domain count**, the list of newly created domains will be shown



- When the user clicks on the **deleted domain count**, the list of deleted domains will be shown.



- **Domain Count Chart:**
    - Here user can view the data in the chart form.
    - Here users can see Date on X-axis and Total Domains on Y-axis.
    - Here blue represents Total Domain, green represents Newly created domain, and red represents Deleted domain.
    - When the user hovers over the blue line, the date and total counts of the domains will appear.
    - When the user hovers over the green line, the date and new domain counts will appear.
    - When the user hovers over the red line, the date and deleted domain count will appear.

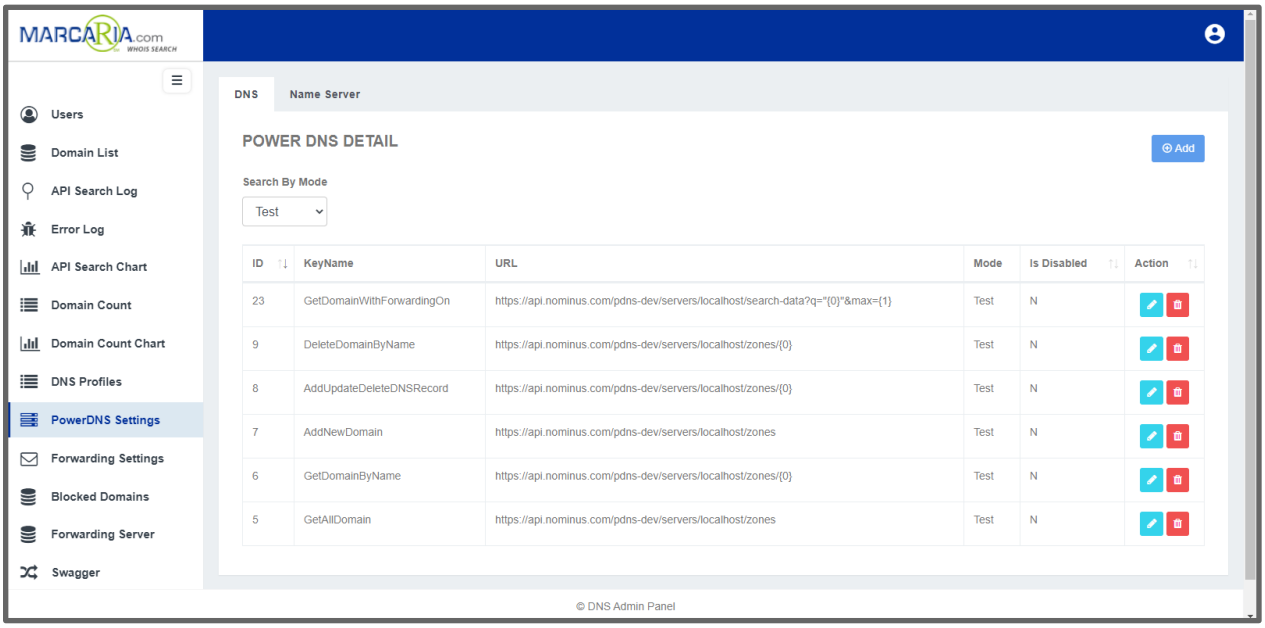


- **<u>PowerDNS Settings:</u>**
  - Power DNS details have two modules.
    1. DNS.
    2. Name Server.

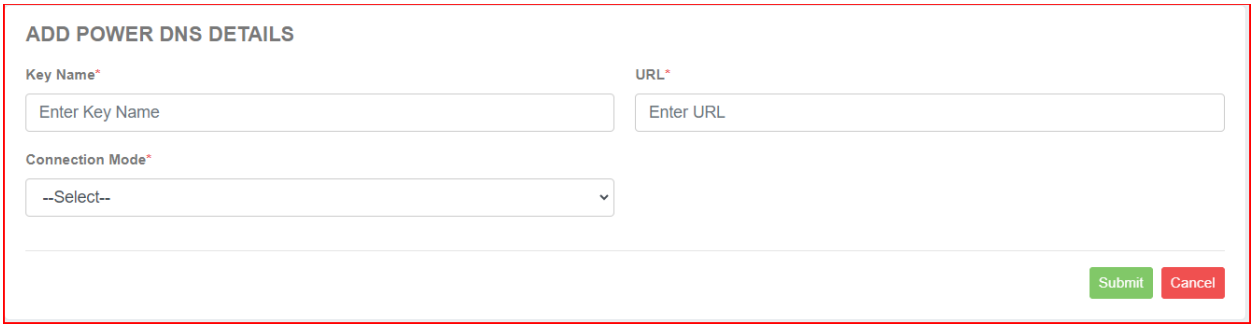
1) <u>DNS:</u>

- Users can map the base (PowerDNS) API with the key name and this key name is used in coding to identify the Base (PowerDNS) API.
- Here users are able to see search by mode dropdown with live and test filters.
- Here users can add power DNS details using the add button.
- Here users are able to view details like.
  - **ID:** Unique Identification number.
  - **Key Name:** It is the name which is used for creating reference of the API in the coding part.
  - **URL:** Display the URL which is assigned to the key. I.e. when a key is used in code it will indirectly call the URL.
  - **Mode:** Display the Mode information.
  - **Is Disabled:** Here if the user does not check the checkbox its value is N otherwise its value is Y. Here if users don't want to use the API longer at that time they can do Is Disabled checkbox checked instead of hard delete.
  - **Action:** User is able to Edit the information or delete it.

- Using sorting functionality users are able to sort the data.

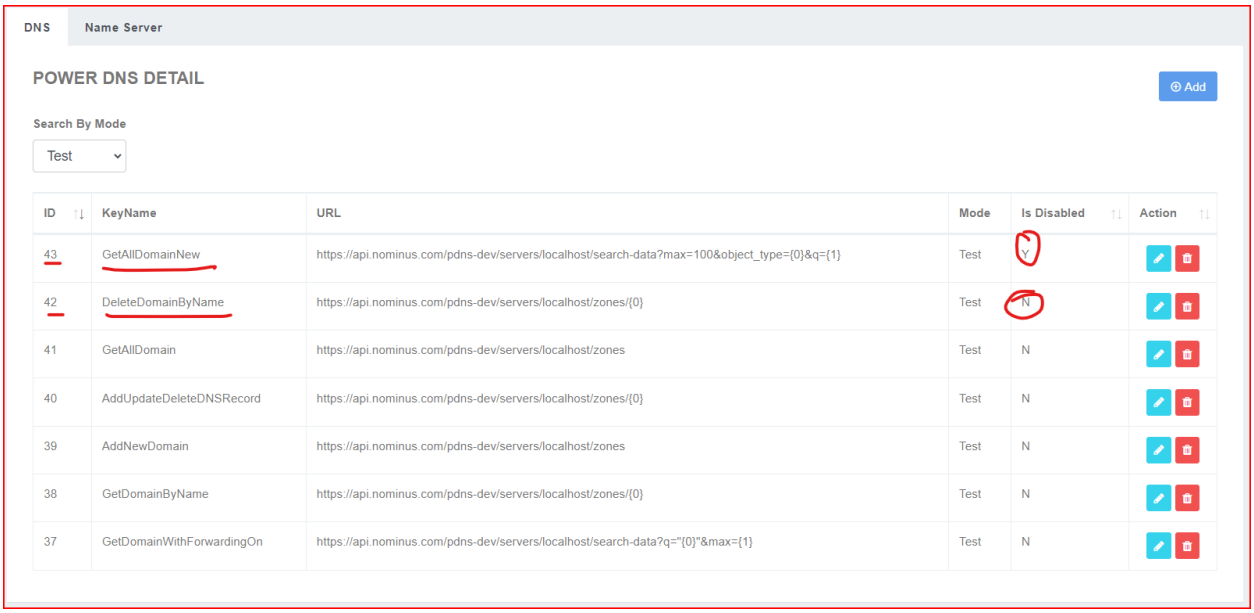


- **<u>Add Button:</u>**
  - Once the user clicks on the add button it will show key name and URL text area and connection mode.
  - **Submit Button:** Click on Submit button it will add DNS detail
  - **Cancel Button:** Click on Cancel button pop will closed



- **<u>Edit Button:</u>**
  - Click on the Edit icon it will open the key name , URL, connection mode and IsDisabled checkbox which allows us to edit the data.
  - If the user checks the **Is Disabled checkbox** and saves the data, **Y** (Yes) will be displayed in the **Is Disabled** column of the power DNS details page.

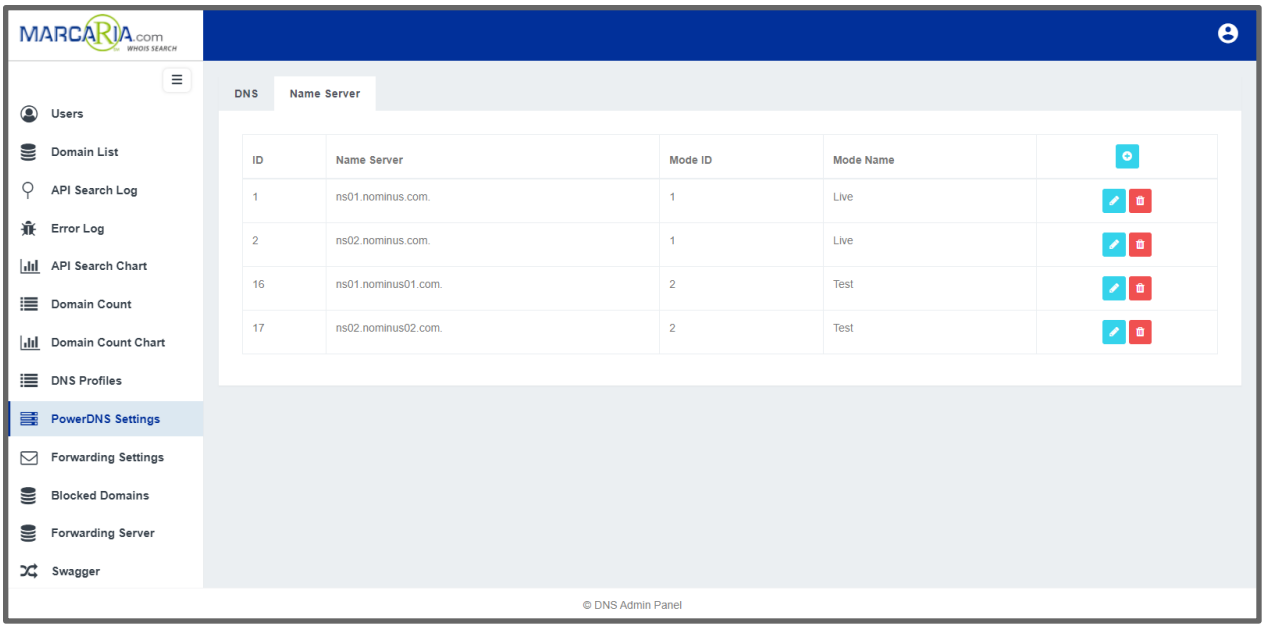
- If the user unchecks the **Is Disabled checkbox** and saves the data, **N** (No) will be displayed in the **Is Disabled** column of the power DNS details page.



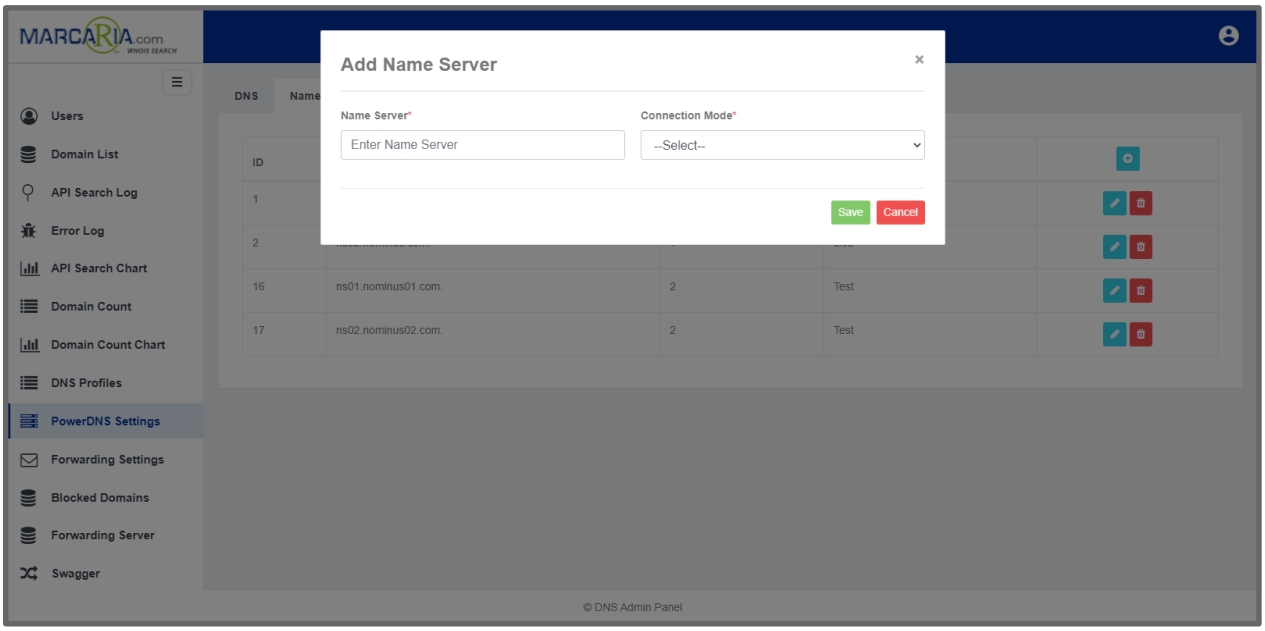
- **Delete Button:**
  - Click on the Delete Button user is able to delete the Power DNS Detail and get the message like - **“Power DNS Record Successfully Deleted”.**

2) Name Server :

- When a user creates a new domain at that time the records which are added by default will be managed from the PowerDNS settings -> Name Server (Tab).
- Here, users are able to view information like..
  - **ID:** Unique IDentification number.
  - **Name Server:** Information about Name Server.
  - **Mode ID:** Information about mode id.
  - **Mode Name:** Information about mode name.
  - **Action field:** Provide Edit and Delete option.

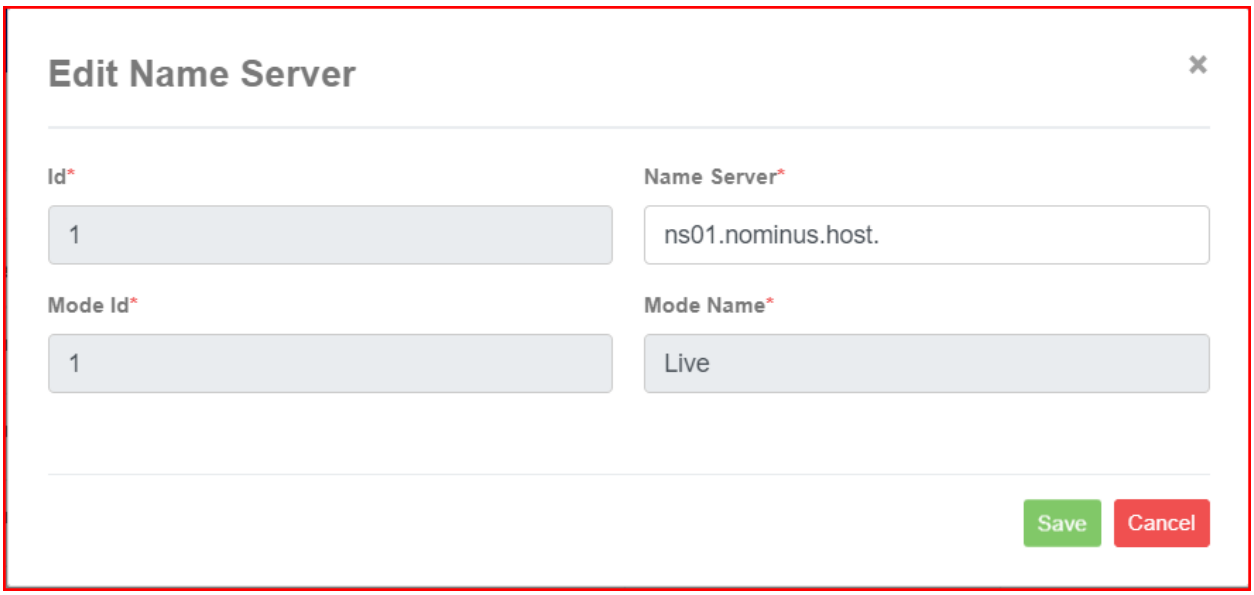


- **<u>Add Name Server Icon:</u>**
    - Click on the Add Name Server icon it will open the popup and the user can see the name server text area and connection mode dropdown for the adding data.



    - **Save Button:** Click on save button it will add DNS server
    - **Cancel Button:** Click on Cancel button pop will closed

- **Edit Button:**
    - Click on the Edit icon pop will open like.. ID, mode ID,Name server and Mode name.
    - Here users can **edit only the server name**. Id , Model Id and Mode name will be in a disabled.

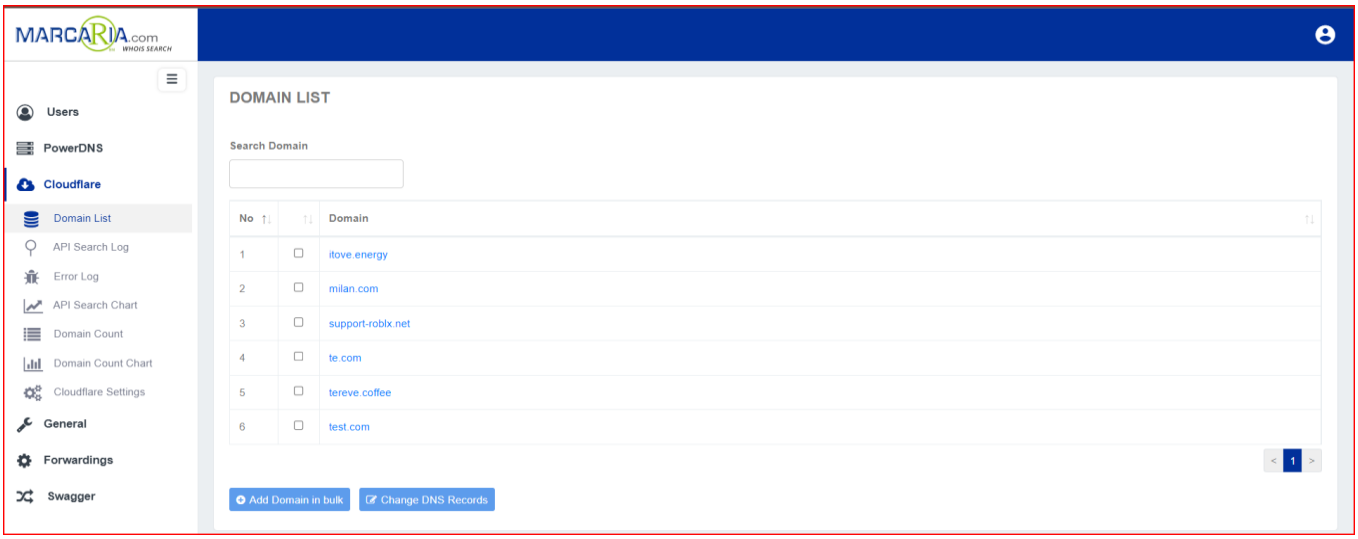


- **Delete Button:**
    - Click on the Delete Button user is able to delete the name server and get the message like - **“Name Server Successfully Deleted”.**

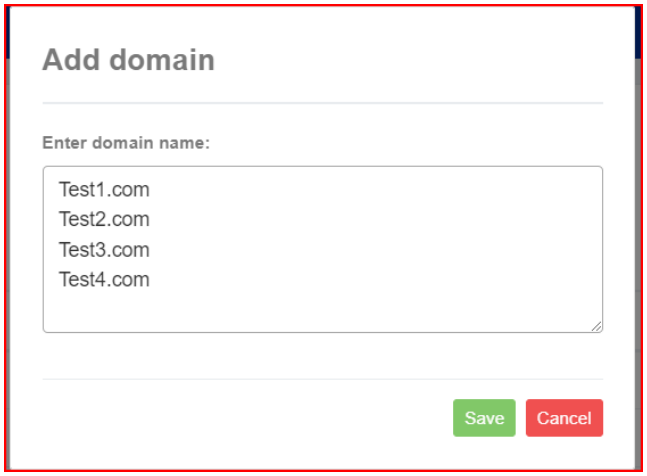
CLOUDFLARE

- Cloudflare has 7 submodules.
    1. Domain List.
    2. API Search Log.
    3. Error Log.
    4. API Search Chart.
    5. Domain Count.
    6. Domain Count Chart.
    7. Cloudflare Settings.

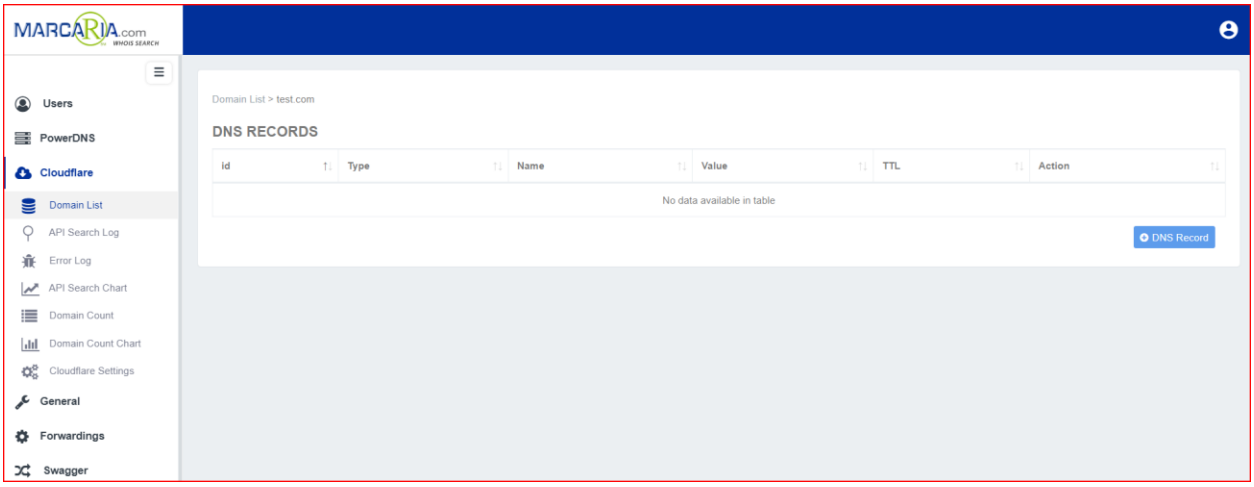
- **Domain List :**
    - Here, users can see the domain list, add domain in bulk button,change DNS record button and search domain search box.
    - Here, users are able to see no, checkbox functionality and domain name.
    - Users can add/create domains from the Domain list menu.
    - By clicking on the Domain from the side menu panel the user is able to view a list of domains which are already added.



- **Add Domain in bulk button:**
    - When the user clicks on the Add domain in bulk button it will open the popup with textarea and Save, Cancel button.
    - Add multiple domains in the text area separated each by entering a domain name. i.e. every domain must be in a new line.
    - **NOTE:** Entered domain name must be unique. If a user entered name which is already available in the list it will give validation of the conflict domain name.



- **DNS Record List :**
  - On first time, click on newly added domain user can't able to view DNS Records
  - Here users can see ID, type,name,value, TTL, action column with edit and delete functionality.



  - Here users are also able to add new records by clicking on the DNS Record button.
  - When the user clicks on the DNS record button it will open the section for selection of type of record from the drop down.
  - Type of records are :

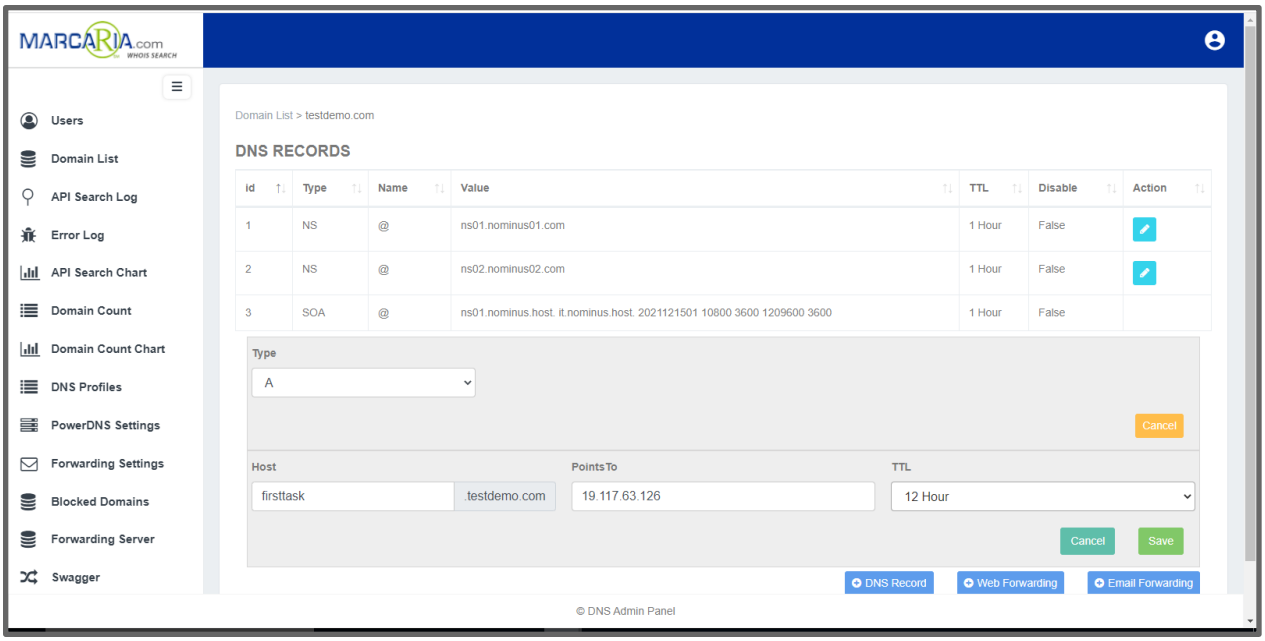
1. **A** Record

2. **AAAA** Record
3. **MX** Record
4. **TXT** Record
5. **SRV** Record
6. **CAA** Record
7. **CNAME** Record
8. **NS** Record.

- Following are the available types.

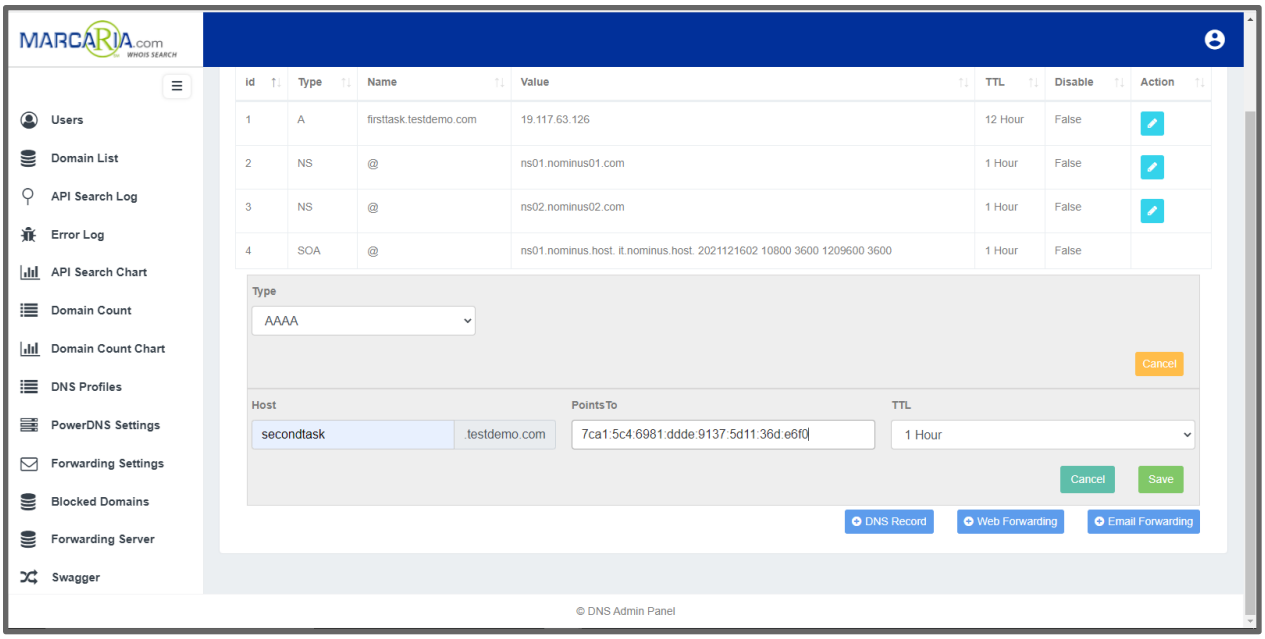
  - **A type record :**

    - Firstly, address (A) records are the most common record type by far. In brief, A records map domain names to IPv4 addresses.
    - IPv4 notation is : **192.0.2.146**.
    - When a user selects to add A type of records it will have the following options…
      - **Host :** Provide host name.
      - **Points To :** Add IPV4 address. Here if user not enter the valid IPV4 it will generate validation message like ..”Entered value is not valid IP”
      - **TTL :** Select time duration from drop down
      - **Save button :** Click on save button it will save the record.
      - **Cancel button :** Click on cancel button it will stop the process.



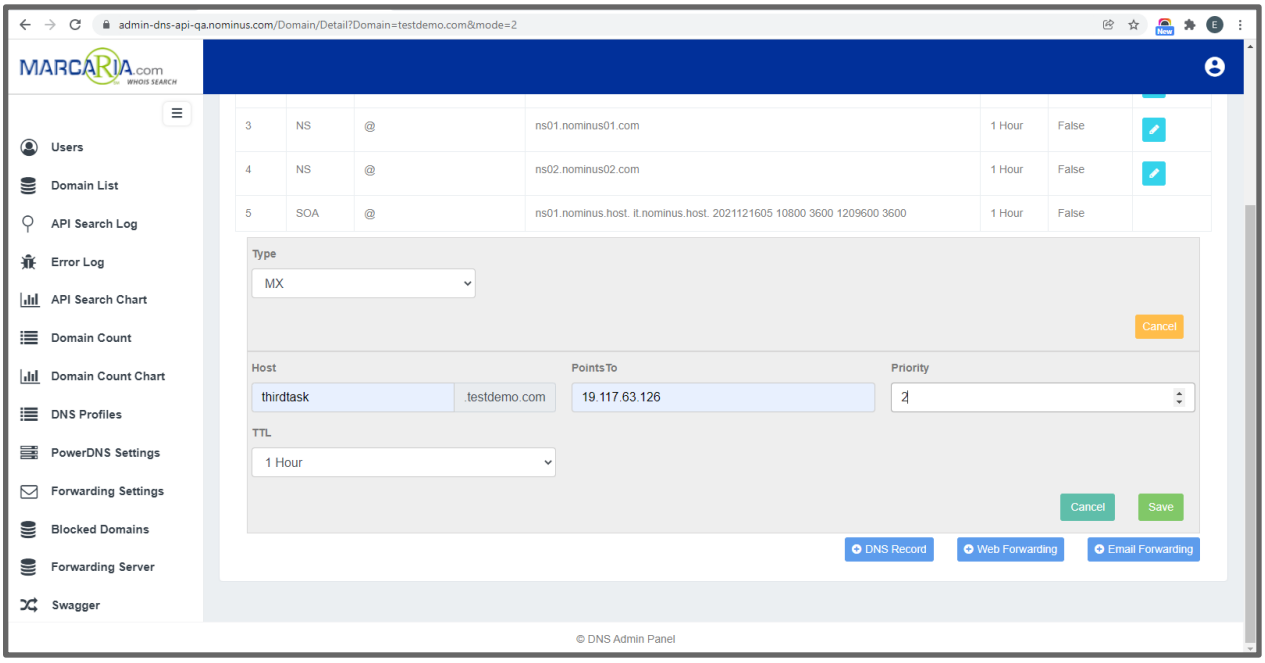
- **AAAA Type Record :**

  - As the internet gradually makes the transition to IPv6, there are AAAA records (spoken as “quad A”). An AAAA record is the IPv6 equivalent of an A record, responsible for mapping a domain name to an IPv6 address.
  - IPv6 notation is :2001:db8:3333:4444:5555:6666:7777.8888
  - When a user selects to add AAAA type of records it will have the following options…
    - **Host :** Provide host name.
    - **Points To :** Add IPV6 address. Here if user not enter the valid IPV6 it will generate validation message like ”Enter value in valid IPV6 format”
    - **TTL :** Select time duration from drop down
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.

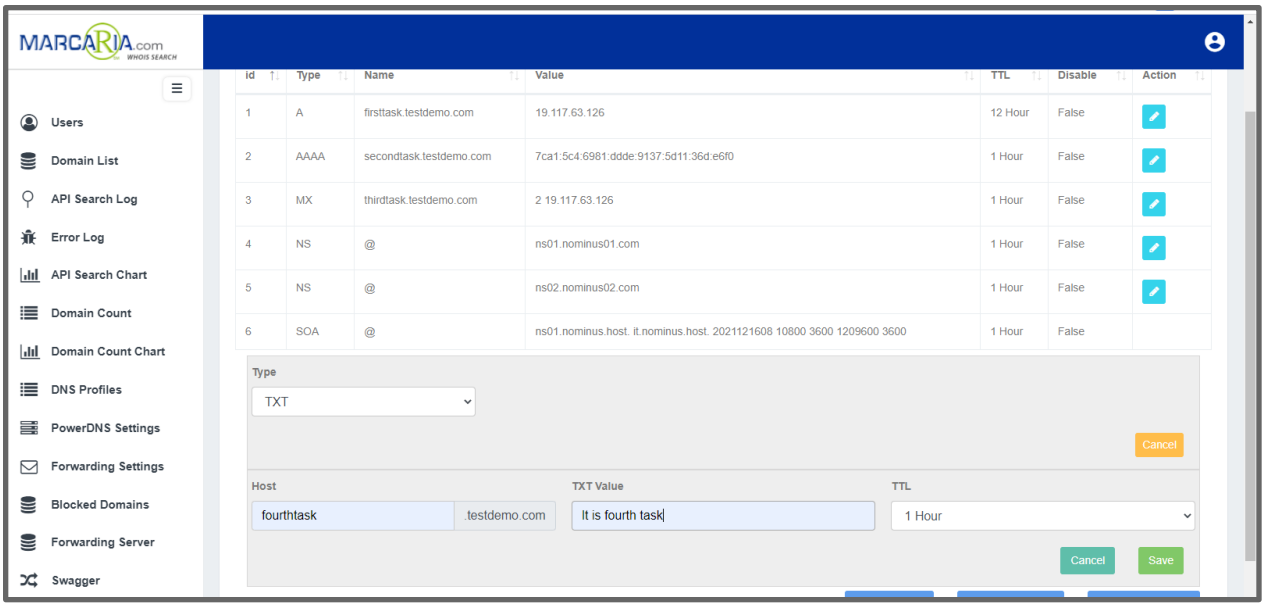


- **MX Type Record** :
  - A mail exchanger (MX) record stores the domain names of mail servers responsible for receiving emails on behalf of a domain.
  - These records can be helpful to identify authorized mail servers. An MX record is sometimes called an MX entry, particularly in configuring mail servers.
    - **Host :** Provide host name.
    - **Points To :** Value is Required.
    - **Priority :** Set the priority. The priority you set when you add an MX record set specifies the sequence that an email server receives emails. A smaller value indicates a higher priority.
    - **TTL :** Select time duration from drop down
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.

web fs



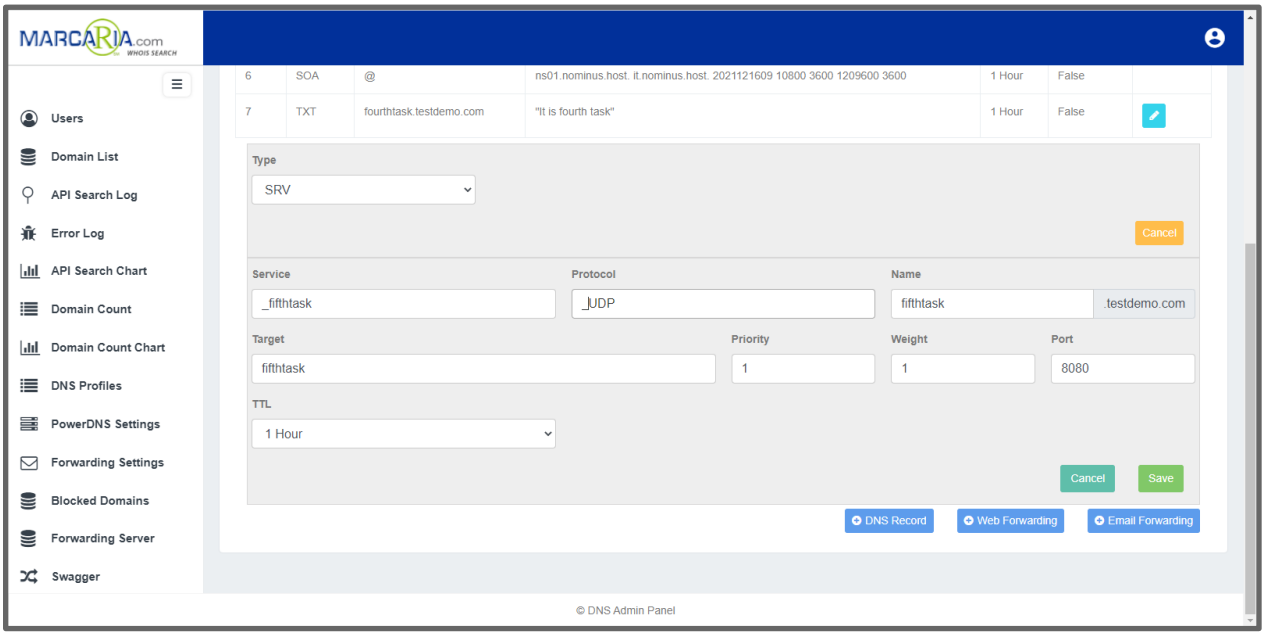
- **TXT Type Record** :
  - A text (TXT) record can store any type of descriptive information in text format. For example, network antivirus providers can use TXT records to send and receive information.
    - **Host :** Provide host name.
    - **TXT value :** Provides text value for the record.
    - **TTL :** Select time duration from drop down
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.



- **SRV Type Record :**
  - The SRV record is a Domain Name System (DNS) resource record. It's used to identify computers hosting specific services. SRV resource records are used to locate domain controllers for Active Directory.

    - **Service :** Symbolic name for a service. Here value must be start with underscore (_)and alphanumeric
    - **Protocol :** Specifies if the service is using TCP or UDP protocols. Here value must start with underscore (_) and alphanumeric.
    - **Name :** Stores a domain name.
    - **Target :** The canonical hostname for the machine providing the service.
    - **Priority :** Determines which server is looked at first, with lower values giving higher priority.
    - **Weight :** Determines which server is looked at first if more than one has the same priority value. Higher values give more priority.
    - **Port :** The TCP or UDP port the service is running on. {Transport Layer Protocol : 1. TCP (Transmission Control Protocol) , 2. UDP(User Datagram Protocol)}
    - **TTL :** Select time duration from drop down

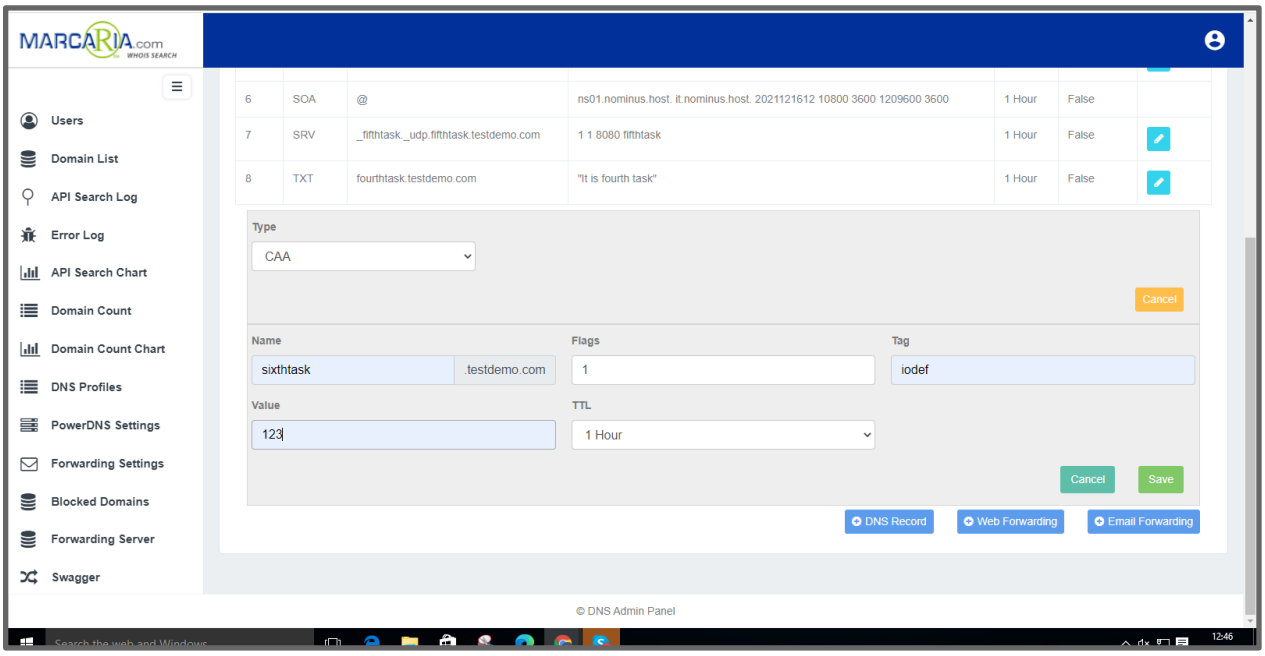
- **Save button :** Click on save button it will save the record.
- **Cancel button :** Click on cancel button it will stop the process.



- **CAA Type Record**:

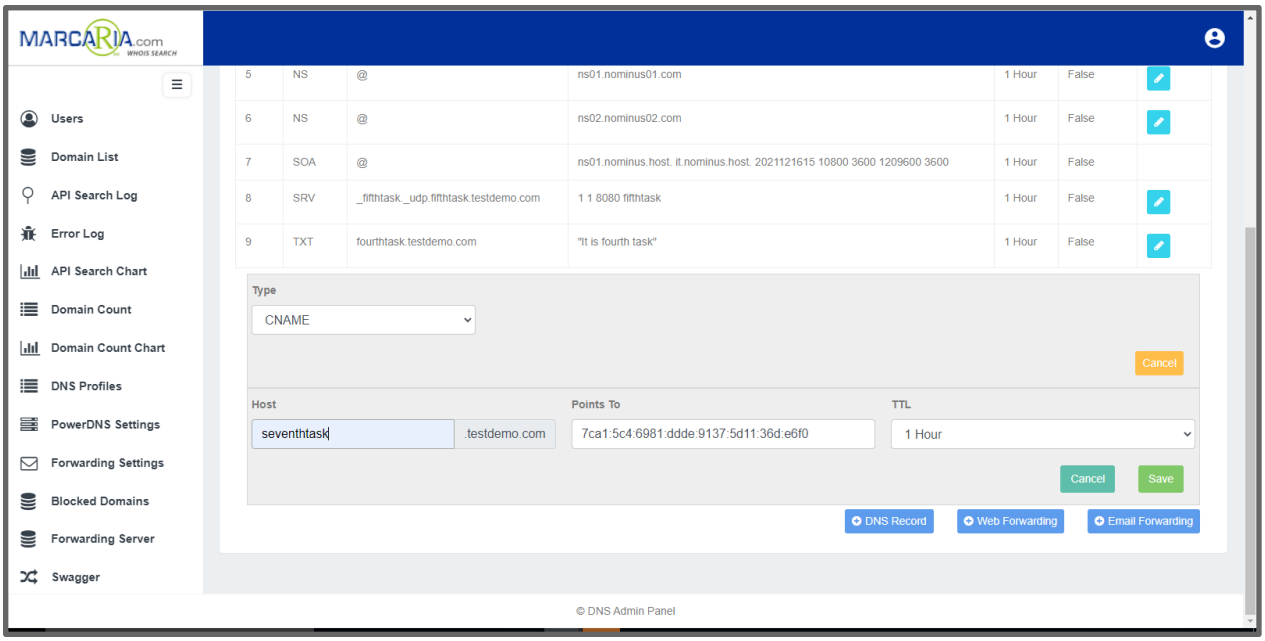
- CAA is a type of DNS record that allows site owners to specify which Certificate Authorities (CAs) are allowed to issue certificates containing their domain names.
  - **Name :** The name of the domain that is being certified.
  - **Flags :** Can be either 1 (critical) or 0 (non-critical). A critical flag means the certificate authority cannot use the CAA record if it doesn't understand the property. A non-critical flag means it can use the CAA record regardless of whether it understands the property.
  - **Tag :** The three tag options are issue (authorized to issue a single certificate), issuewild (authorized to issue a wildcard certificate), and iodef (specifies a URL for reporting policy violations).
  - **Value :** User can enter any value
  - **TTL :** Select time duration from drop down

- **Save button :** Click on save button it will save the record.
- **Cancel button :** Click on cancel button it will stop the process.

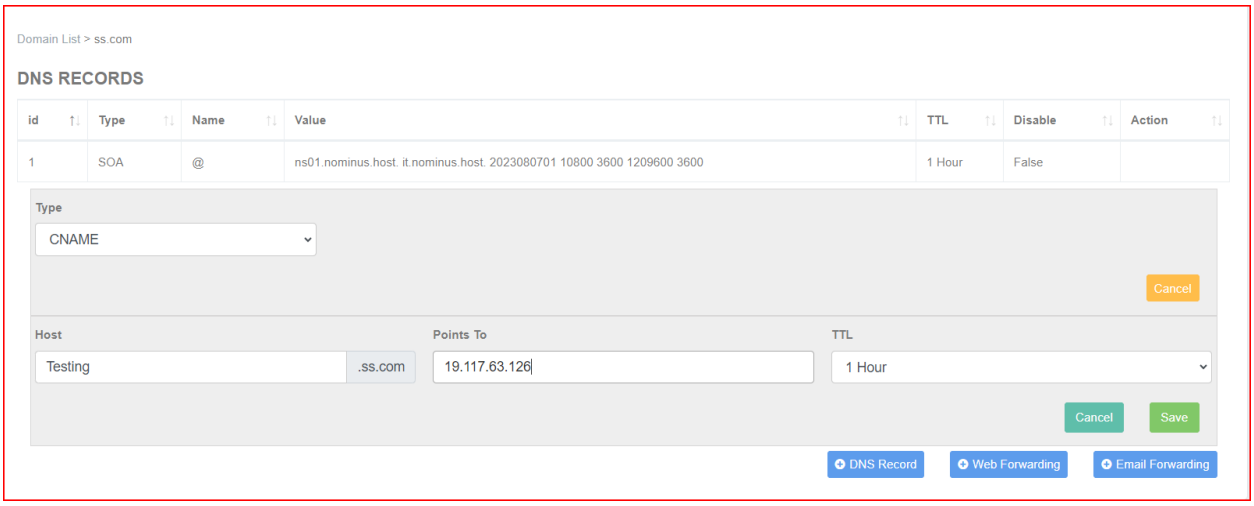


■ **CNAME Type Record :**

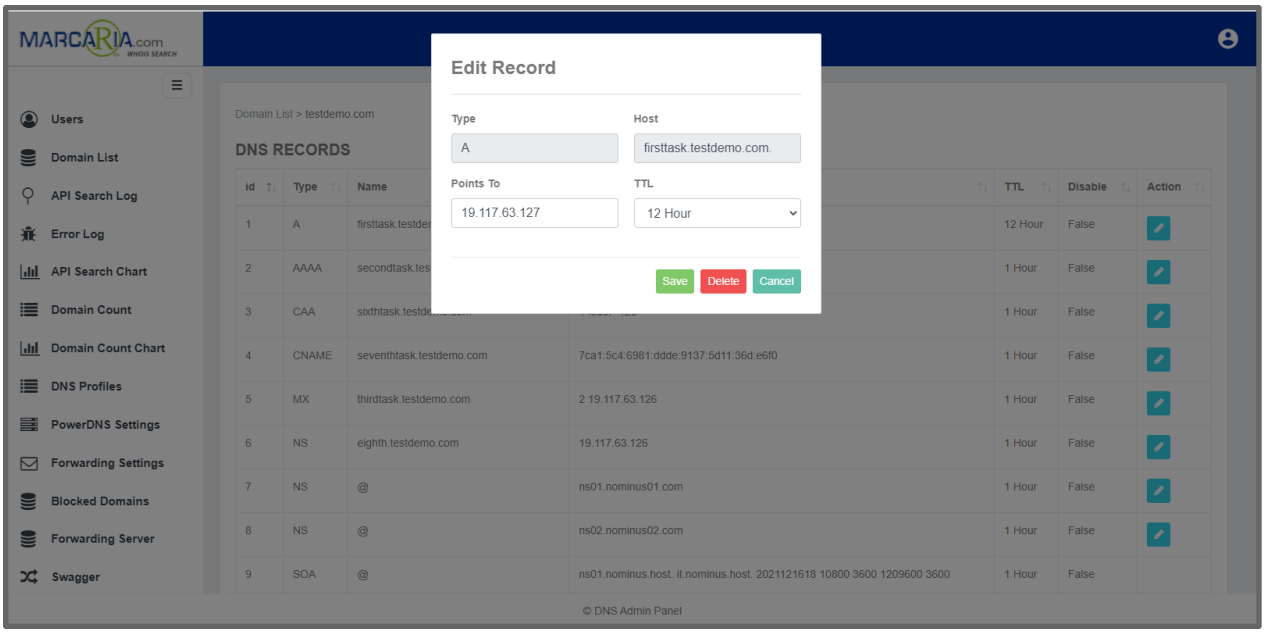
- Redirects a domain to a different domain. CNAME records are typically used to map a subdomain such as www or mail to the domain hosting that subdomain's content.
  - **Host :** Provide host name.
  - **Points To :** Add IPV6 address.
  - **TTL :** Select time duration from drop down.
  - **Save button :** Click on save button it will save the record.
  - **Cancel button :** Click on cancel button it will stop the process.



- **NS Type Record :**
  - Provides a list of the authoritative name servers responsible for the domain. it's important to note that NS records point to the domain of the name server—not IP addresses.
    - **Host :** Provide host name.
    - **Points To :** Add IPV4 address.
    - **TTL :** Select time duration from drop down.
    - **Save button :** Click on save button it will save the record.
    - **Cancel button :** Click on cancel button it will stop the process.



- **Edit Icon :**
    - Click on the edit icon from the Action field of the DNS records table.
    - Users can get the popup with following details..
        - Fields of the record which are generated by the user.
        - Save button
        - Delete button
        - Cancel button
    - **Type** and **hostname** are disabled at the time of edit and all other fields users can modify.
    - **NOTE:** For the SRV record type - Type, Service, Protocol and Name fields are disabled.

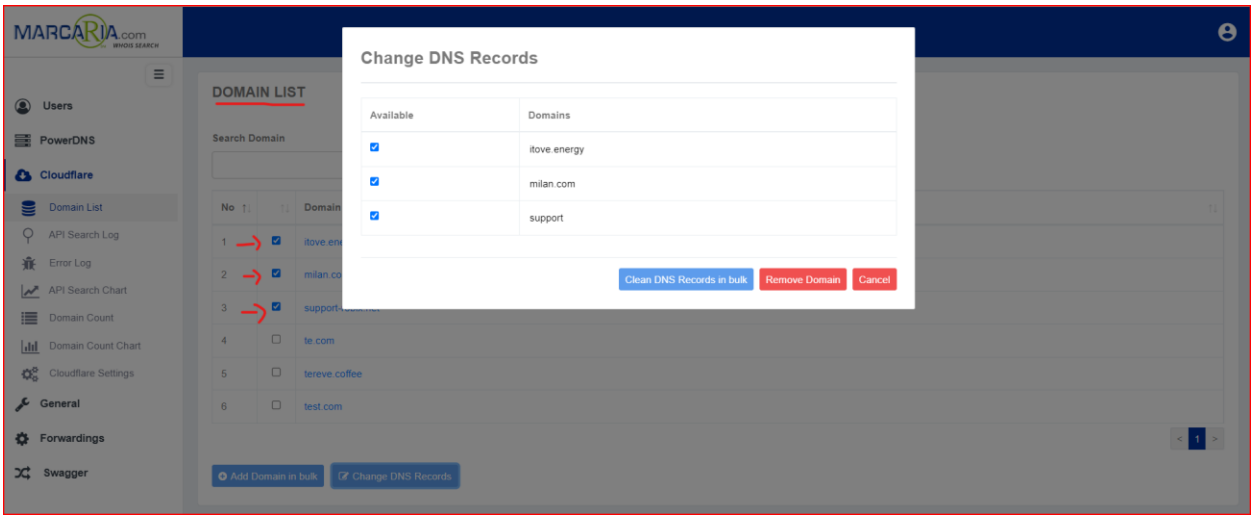


- **Save Button:** Click on the save button after doing modification it will update old data with the updated data.
- **Cancel Button:** click on the cancel button it will close the popup.

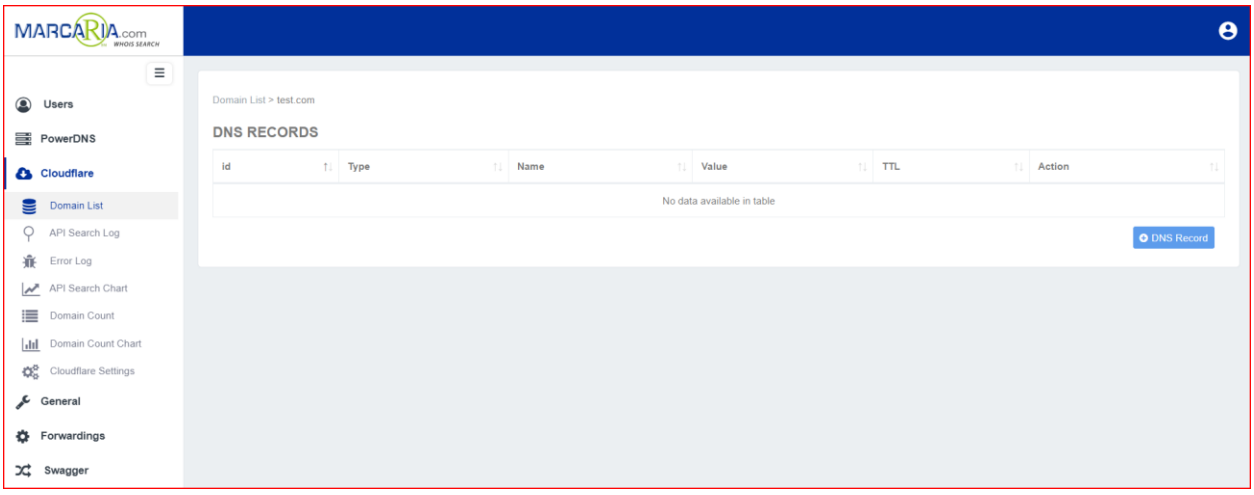
- **NOTE :** Edit option is not available for SOA record.
- **NOTE :** Users can add @ if we want domain name as name.

- **Delete button :**
  - Click on the delete icon from the Action field of the DNS records table.
  - Click on the delete button it will delete the selected type record from the list.
- **Cancel button :**
  - By click on the cancel button “Change DNS record popup will be close”

- **Change DNS Records Button:**
    - When the user clicks the Change DNS Records button for the first time users will be shown an empty list and the clean DNS records in bulk button and remove domain buttons will be disabled.
    - **Users can change DNS records only when users go to the domain list and check the particular domain using the checkbox functionality.**

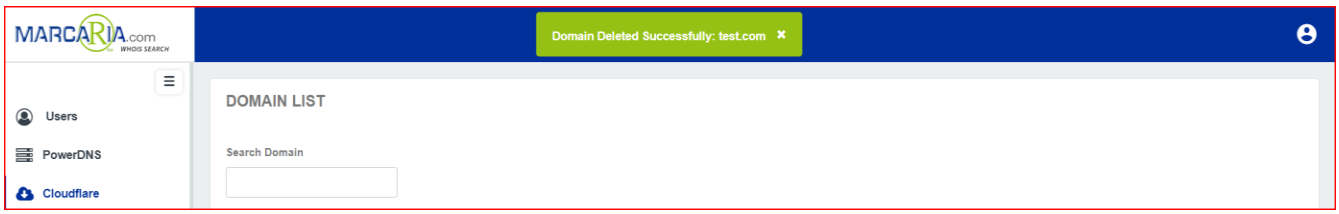


    - **Clean DNS records in bulk button:**
        - Here users are able to clean bulk domains records.
        - Users can check/uncheck the domain as per the requirements which are available in the domain list.



- **Remove Domain button:**

    - Here users are able to remove multiple/bulk domains.
    - Users can check/uncheck the domain as per the requirements which are available in the domain list.
    - After checking the checkbox, click on the remove button and the users are able to remove the selected domain from the domain list.



- **Cancel button:**

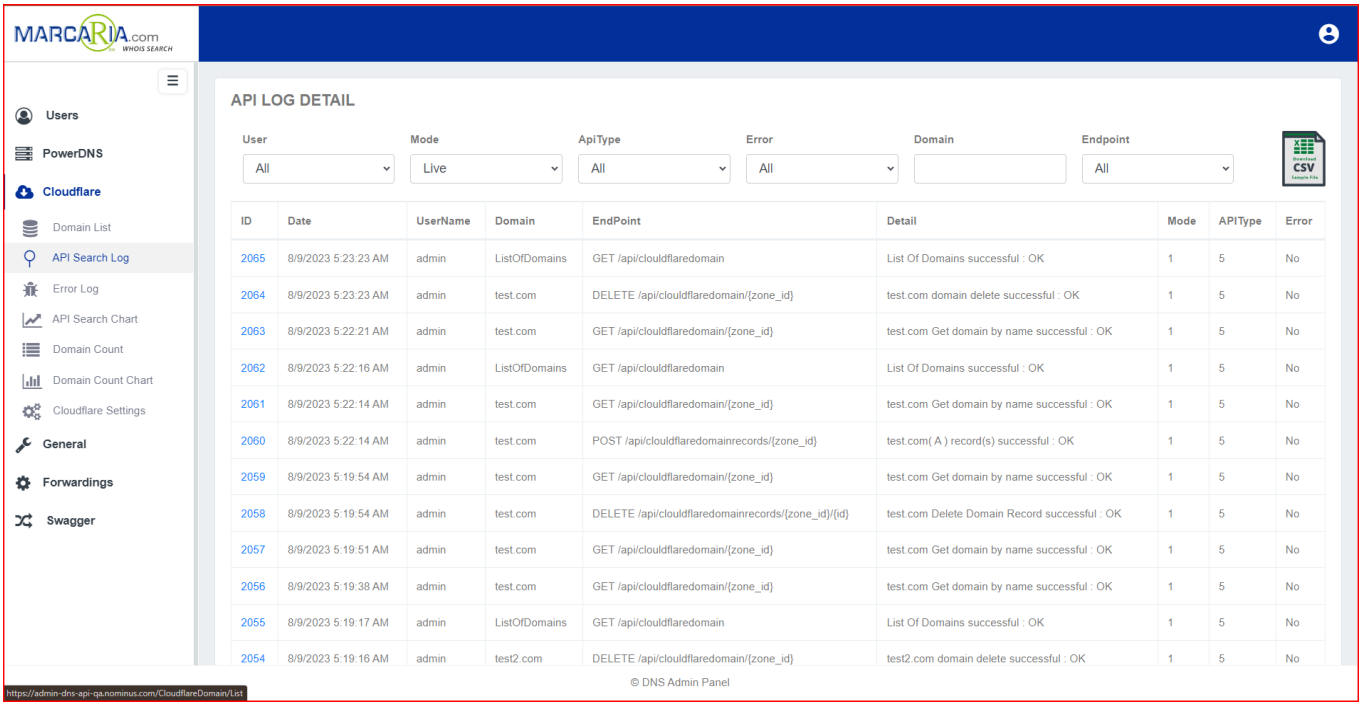
    - By click on the cancel button “Change DNS record popup will be close”

• API Search Log:

- Here users are able to view API log details.
- Users are able to download the CSV file of the API log.
- Users are able to see different filter options such as…
    - **User:** Can filter according to the Users.
    - **Mode:** Can filter according to the Mode.
    - **API Type Dropdown:** Can filter according to the API Type.
    - **Error Dropdown:** Can filter according to the Error Type.
    - **Domain:** Search by entering the domain name.
    - **Endpoint Dropdown:** Can filter according to the Endpoint.

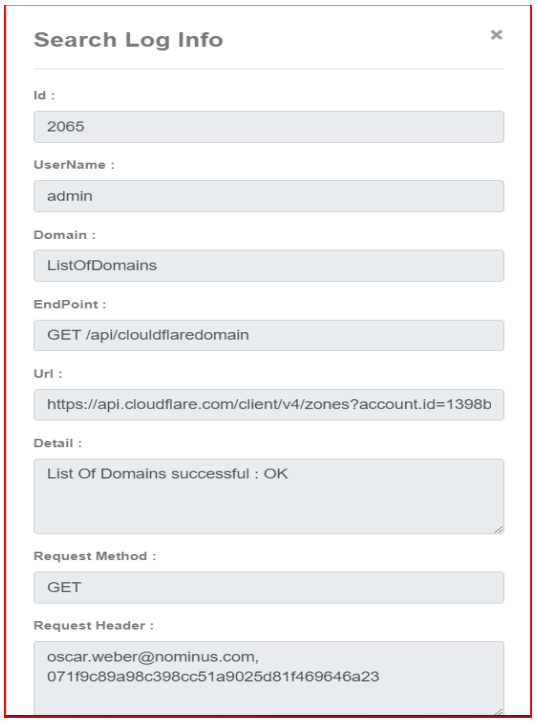
- Users are able to view information like….
    - **ID:** Unique identification number.

- **Date:** Timestamp of the error log.
- **User Name:** Display user name. (Login with that name - admin)
- **Domain:** Display domain name
- **End Point:** Display API endpoint.
- **Detail:** Provides information.
- **Mode:** Display information of mode.
- **APIType:** Display type of API.
- **Error:** Display status if error occurs then Yes else No.



- Once clicked on the unique identification number (**ID**) and the user will get a popup with the information about the Search Log.
- All search log information will be displayed as disabled mode.
- In search Log Info popup users will get the following information.
  - **Id:** Unique identification number.
  - **User Name:** Information about User Name.
  - **Domain:** Information about domain.
  - **End Point:** Information about End Point.
  - **URL:** Information about URL.
  - **Detail:** Provides Information.
  - **Request Methods:** Information about method name.
  - **Request Header:** Information about requested header.
  - **Requested Response:** Information about requested response.

- **Request Body:** Information about request body.
- **API Type:** Information about API type.
- **Mode:** Information about Mode.
- **Date:** Information about time stamp.



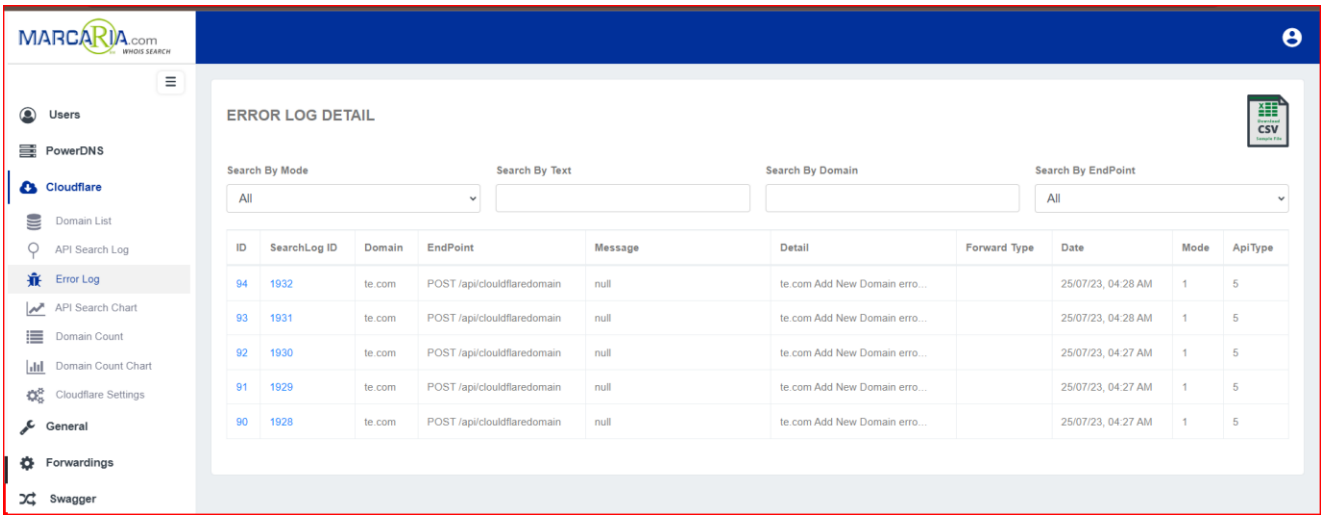
- **Error Log:**

  - Here users are able to view the error log if any errors occurred.
  - Users can have the option to apply filters based on..
    - **Search By Mode Dropdown:** It have three option Live, Test, All
    - **Search By Text:** Enter any text in the textbox and the user will get equivalent results.
    - **Search By Domain:** Enter the domain name which you want to search.
    - **Search By Endpoint Dropdown:** Select the endpoint from the list.

  - Users are able to view the following information..
    - **ID:** Unique Identification number.
    - **SearchLog ID:** It is display the unique ID of the Search Log i.e. API Search Log.

- **Domain:** Name of the domain in which error occurred.
- **EndPoint:** Name of the End Point.
- **Message:** It will print the exception message which has occurred.
- **Detail:** It will display the errors in details
- **Forward Type:** Display the forward type.
- **Date:** Timestamp is displayed. (Display Date & Time)
- **Mode:** Display information about Mode. (Live :1 , Test : 2)
- **API Type:** Display information about API type.

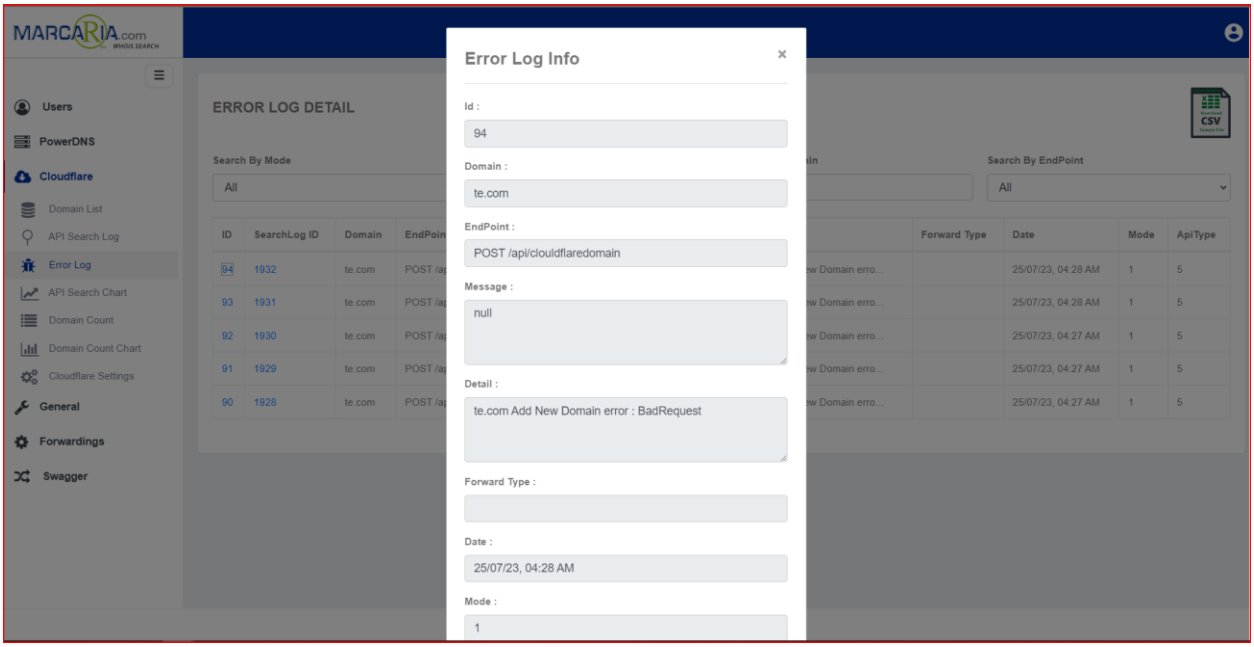
○ Here users are able to download CSV files of the error log.



○ Click on the ID user will be able to view the Error Log Info popup with following details..

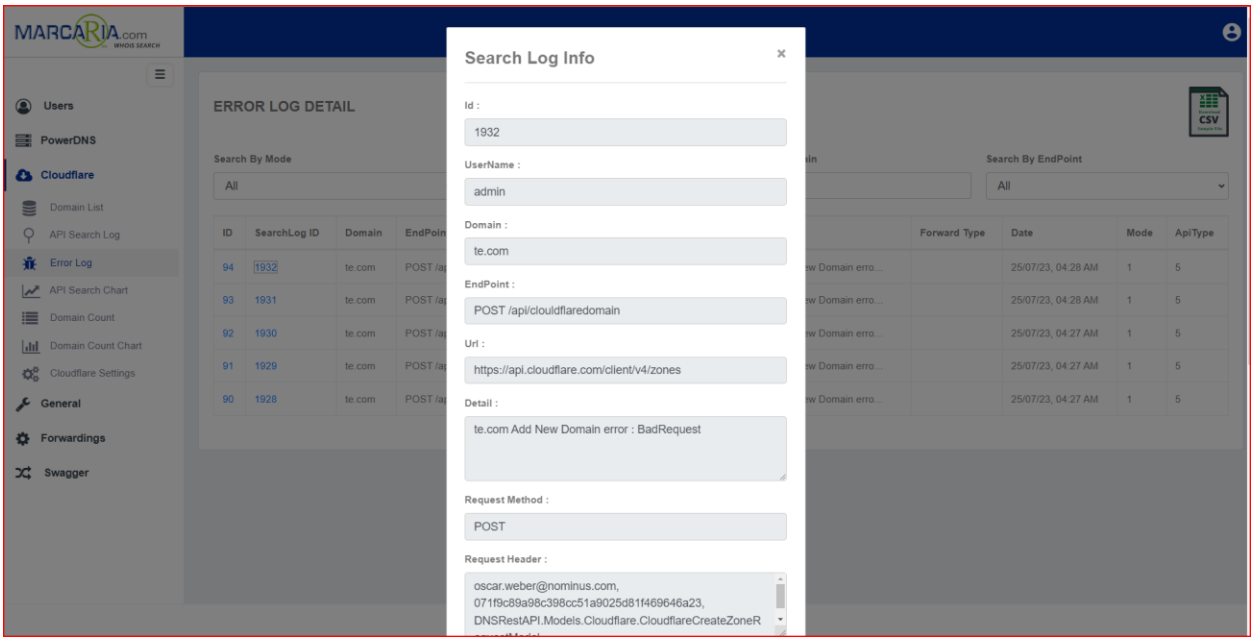
- **ID:** Unique identification number.
- **Domain:** Display domain name.
- **End Point:** Display End Point name.
- **Message:** It will print the exception message which has occurred.
- **Detail:** Display details about error.
- **Forward Type:** Display the forward type.
- **Date:** Display timestamp.
- **Mode:** Display mode information.
- **API Type:** Details of API type.

○ All error log information will be displayed as disabled mode.



- Click on the **Search Log ID** user will be able to view Search Log Info popup with following information…

    - **Id:** Unique identification number.
    - **User Name:** Information about User Name.
    - **Domain:** Information about domain.
    - **End Point:** Information about End Point.
    - **URL:** Information about URL.
    - **Detail:** Provides Information.
    - **Request Methods:** Information about method name.
    - **Request Header:** Information about requested header.
    - **Requested Response:** Information about requested response.
    - **Request Body:** Information about request body.
    - **API Type:** Information about API type.
    - **Mode:** Information about Mode.
    - **Date:** Information about time stamp.
- All search log information will be displayed as disabled mode.



Note Error Log :

- **Error Log :**

1. **Message :** NULL

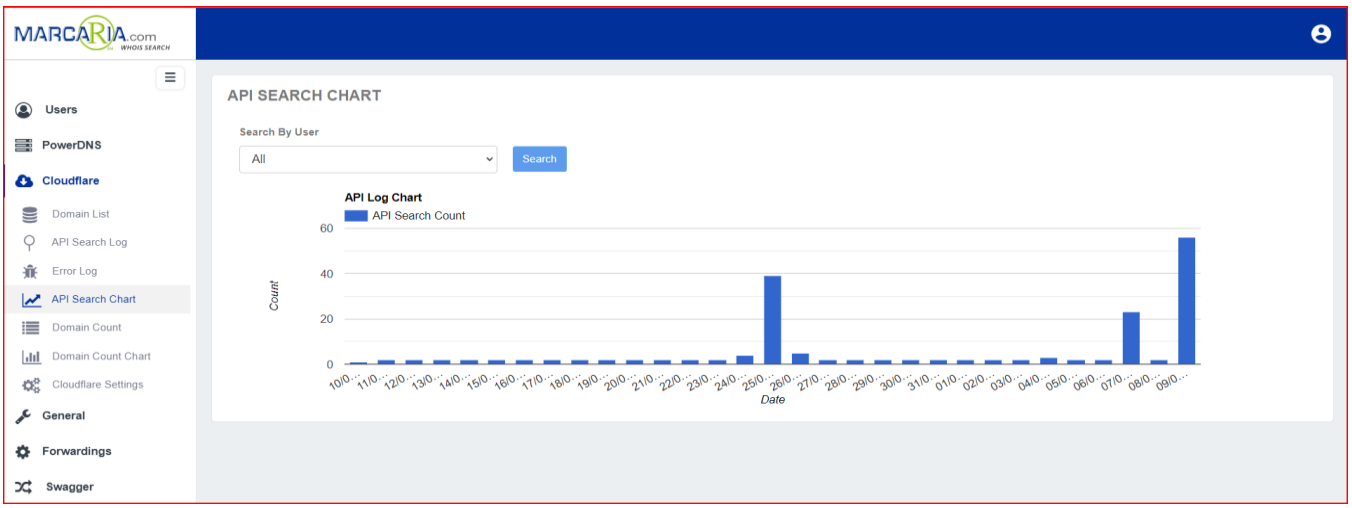
- If there is an error in the coding, the message will be Displayed in the Message Column. (**Message :** Decoded String is not a valid)
- Id will not be displayed in Search Log Id. (Blank Display)

2. **Detail :**

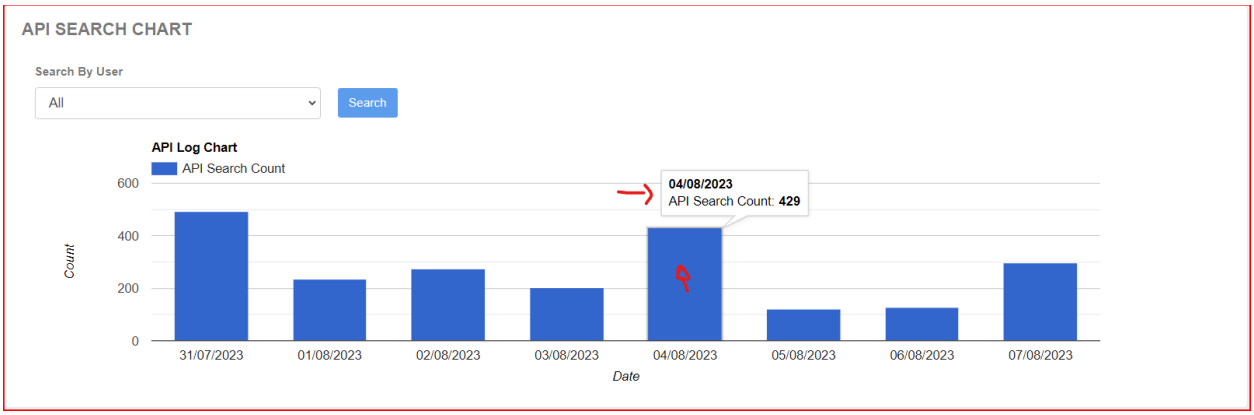
- If there is any error in the Api then the message will be displayed in the Detail column. (**Message :** System.ArgumentException)
- The ID will be displayed in the search log ID.

- **API Search Chart:**

  - It will display the total of API searches in the chart form.
  - It will display date on X axis and count on Y axis
  - Here users also can see the data according to the user by selecting the user available from the drop down.



- When the user hovers over the graph, the date and API search counts will appear.

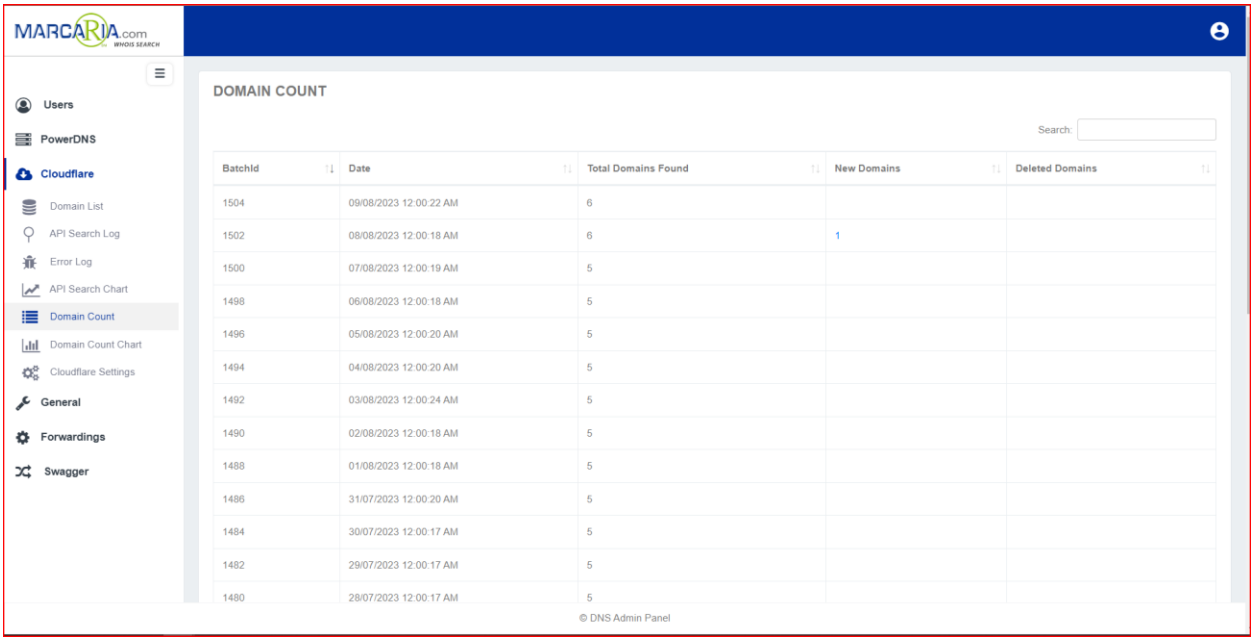


- **<u>Domain Count :</u>**

  - Here users can check the log of the created domain according to the date.
  - The Latest created domain will be displayed **Tomorrow**. (Domain Count)
  - Users are able to find the table with the following fields..
    - **Batch ID:** It is a unique identification number.
    - **Date:** The timestamp of the domain creation.

- **Total Domain Found:** It will provide information about the total domain creation till date.
- **New Domain:** It will provide information about the newly created domain of the specific date.
- **Deleted Domain:** It will provide information about the deleted domain for the specific date.

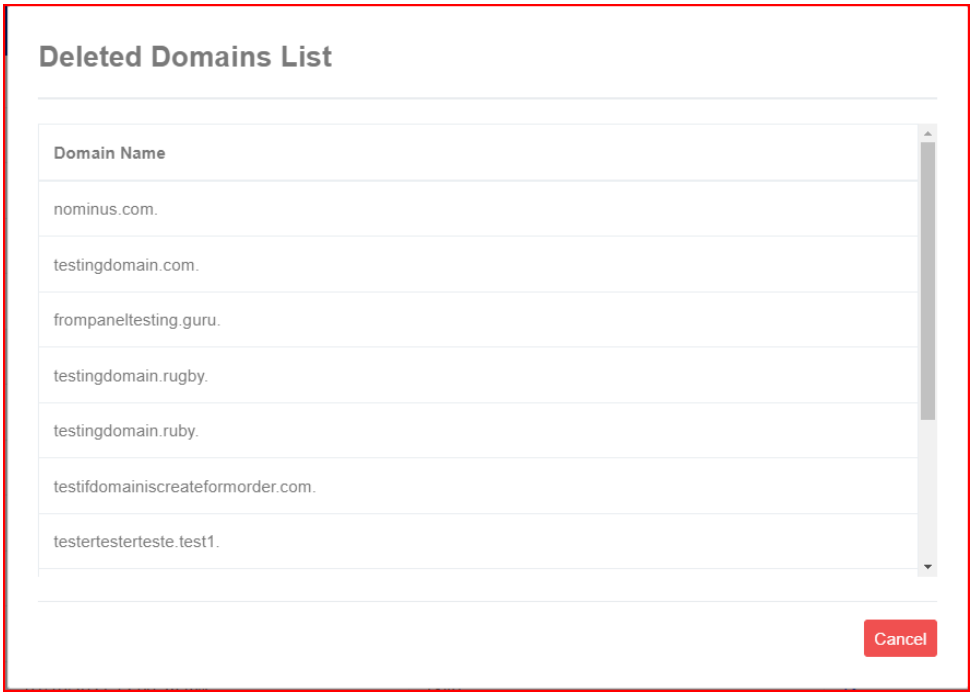
- Using a search box, users are able to search the records.
- Using sorting functionality users are able to sort the data.



- When the user clicks on the **new domain count**, the list of newly created domains will be shown

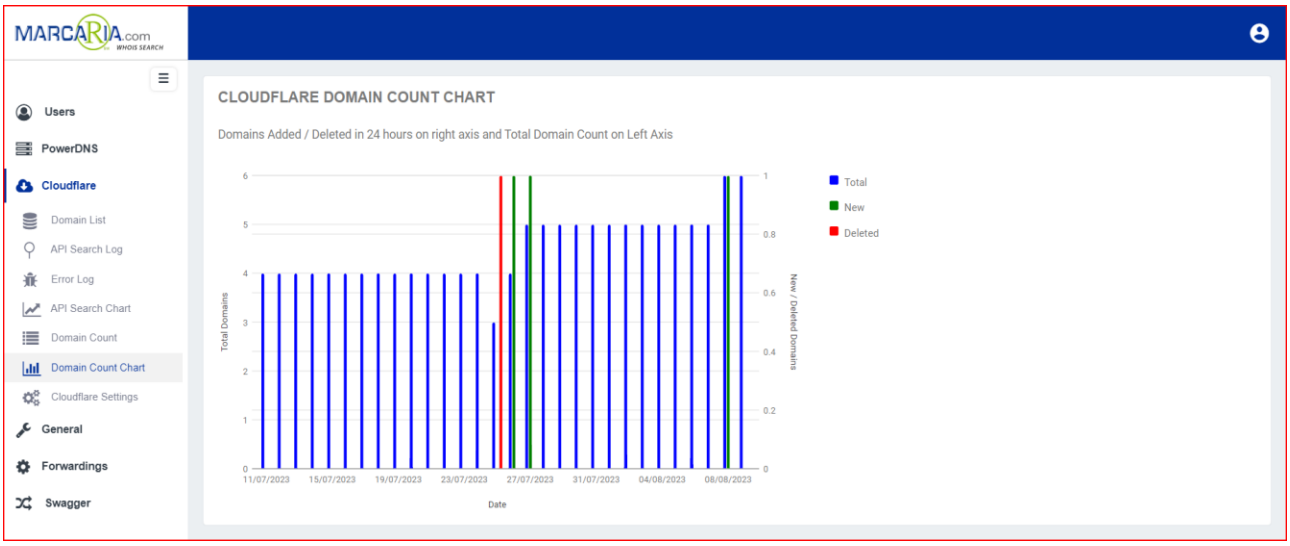


- When the user clicks on the **deleted domain count**, the list of deleted domains will be shown.



- **Domain Count Chart:**
  - Here user can view the data in the chart form.
  - Here users can see Date on X-axis and Total Domains on Y-axis.
  - Here blue represents Total Domain, green represents Newly created domain, and red represents Deleted domain.
  - When the user hovers over the blue line, the date and total counts of the domains will appear.
  - When the user hovers over the green line, the date and new domain counts will appear.

- When the user hovers over the red line, the date and deleted domain count will appear.



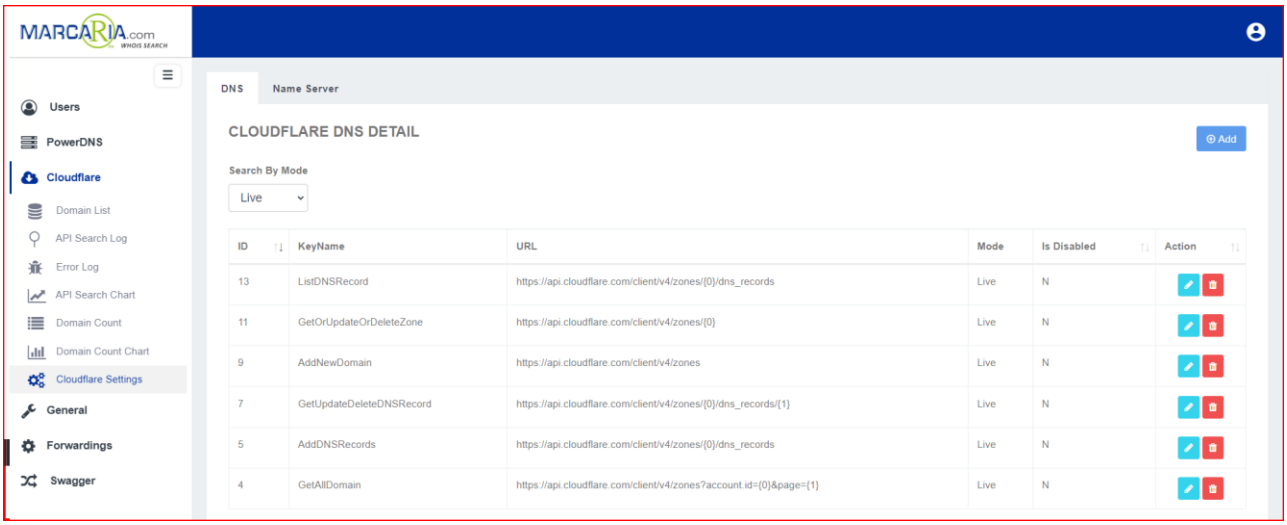
● Cloudflare Settings:

- Cloudflare settings details have two modules.
    1. **DNS.**
    2. **Name Server.**

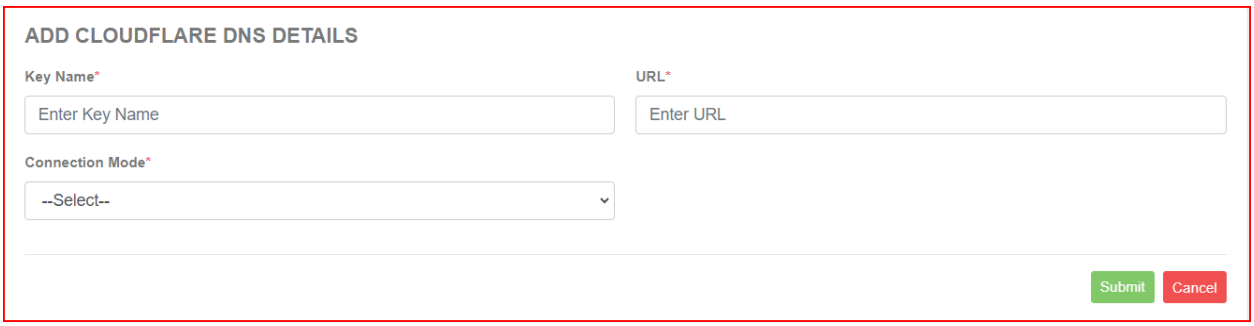
1. DNS:

- Users can map the base (PowerDNS) API with the key name and this key name is used in coding to identify the Base (PowerDNS) API.
- Here users are able to see search by mode dropdown with live and test filters.
- Here users can add power DNS details using the add button.

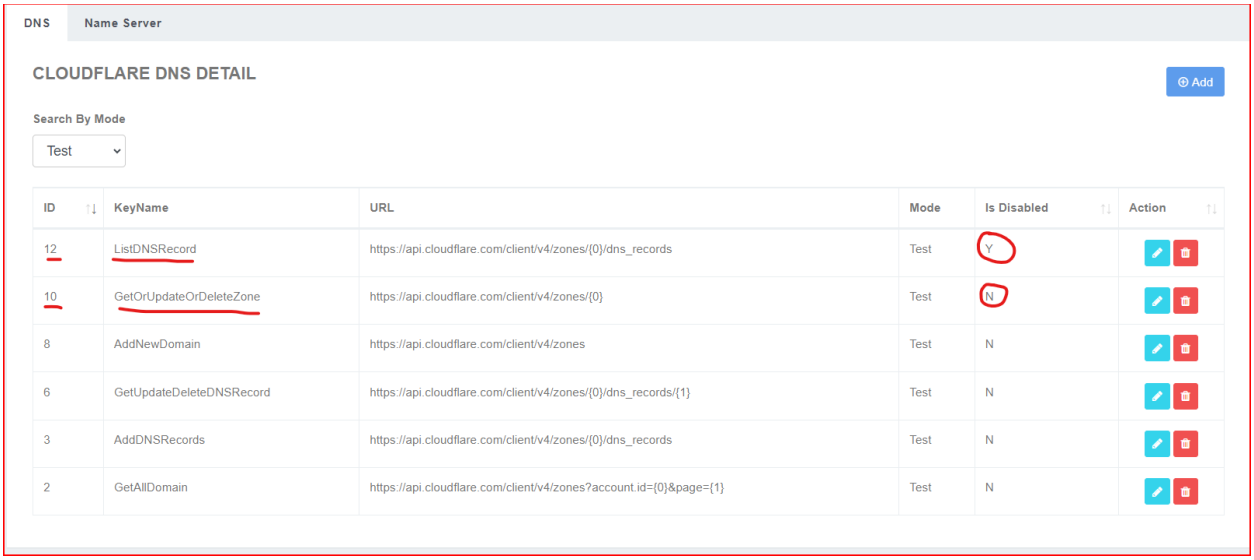
- Here users are able to view details like.
  - **ID:** Unique Identification number.
  - **Key Name:** It is the name which is used for creating reference of the API in the coding part.
  - **URL:** Display the URL which is assigned to the key. I.e. when a key is used in code it will indirectly call the URL.
  - **Mode:** Display the Mode information.
  - **Is Disabled:** Here if the user does not check the checkbox its value is N otherwise its value is Y. Here if users don't want to use the API longer at that time they can do Is Disabled checkbox checked instead of hard delete.
  - **Action:** User is able to Edit the information or delete it.
- Using sorting functionality users are able to sort the data.



- **<u>Add Button:</u>**
  - Once the user clicks on the add button it will show key name and URL text area and connection mode.
  - **Submit Button:** Click on Submit button it will add DNS detail
  - **Cancel Button:** Click on Cancel button pop will closed



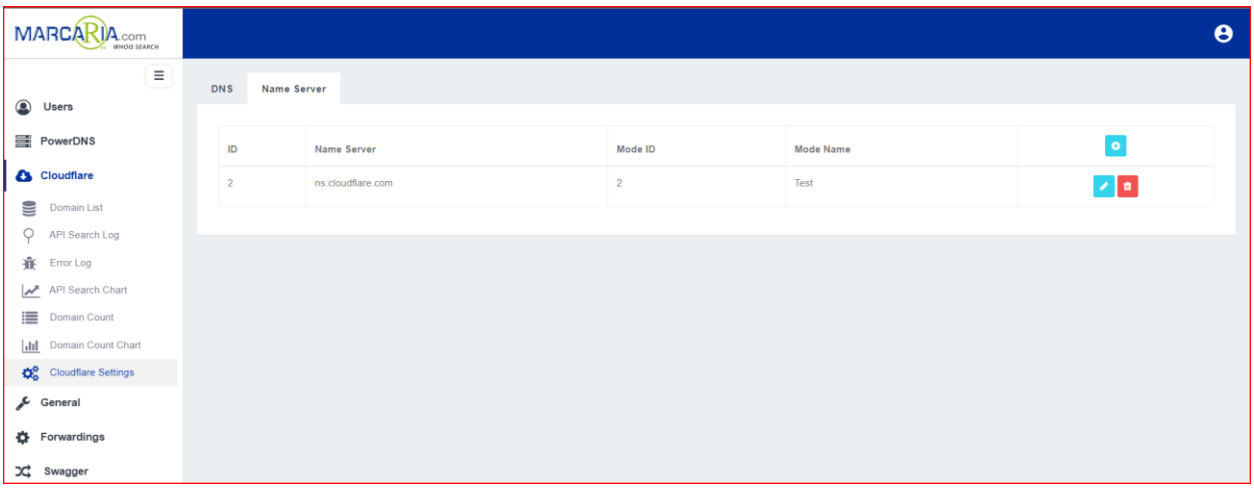
- **Edit Button:**
    - Click on the Edit icon it will open the key name , URL, connection mode and IsDisabled checkbox which allows us to edit the data.
    - If the user checks the **Is Disabled checkbox** and saves the data, **Y** (Yes) will be displayed in the **Is Disabled** column of the power DNS details page.
    - If the user unchecks the **Is Disabled checkbox** and saves the data, **N** (No) will be displayed in the **Is Disabled** column of the power DNS details page.



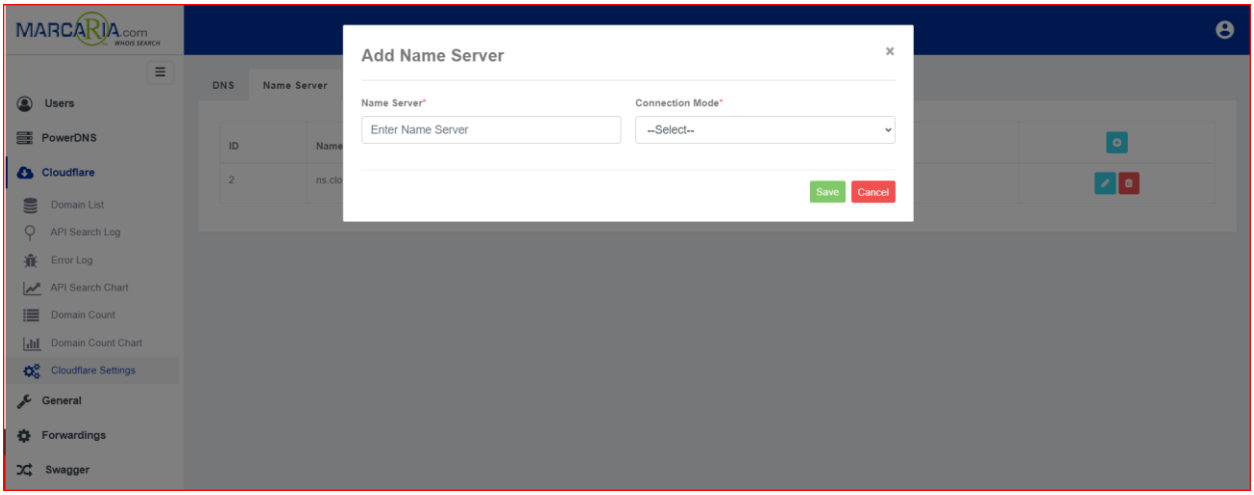
- **Delete Button:**
    - Click on the Delete Button user is able to delete the Power DNS Detail and get the message like - **“Cloudflare Record Successfully Deleted”.**

2. Name Server :

- When a user creates a new domain at that time the records which are added by default will be managed from the Cloudflare settings -> Name Server (Tab).
- Here, users are able to view information like..
  - **ID:** Unique IDentification number.
  - **Name Server:** Information about Name Server.
  - **Mode ID:** Information about mode id.
  - **Mode Name:** Information about mode name.
  - **Action field:** Provide Edit and Delete option.



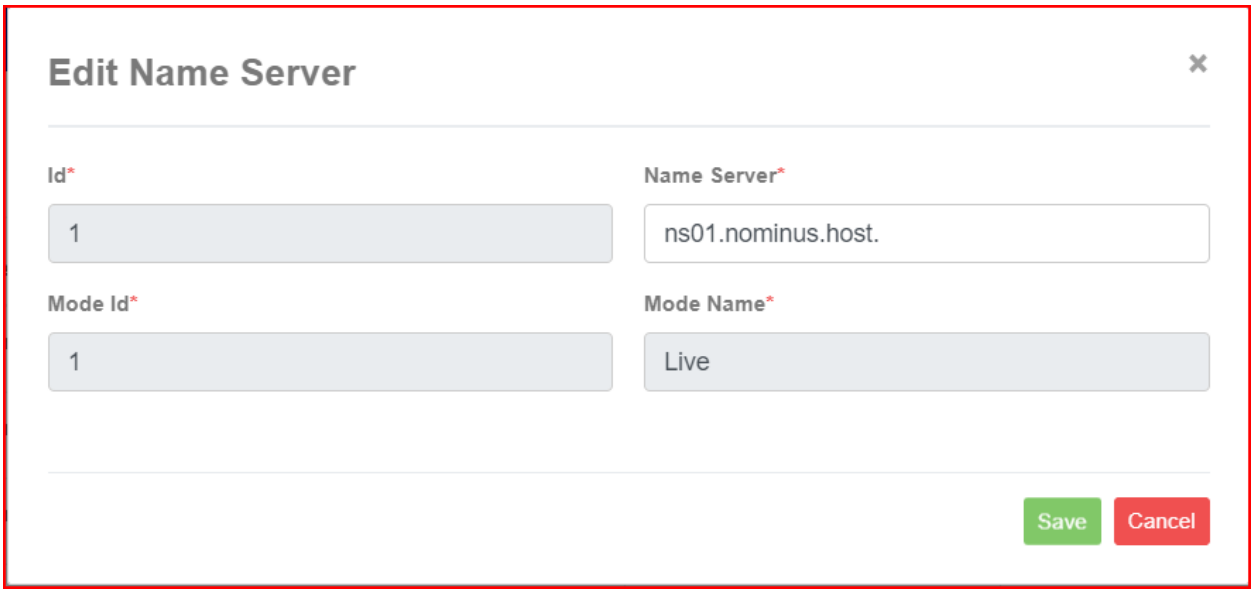
- **Add Name Server Icon:**
  - Click on the Add Name Server icon it will open the popup and the user can see the name server text area and connection mode dropdown for the adding data.



- **Save Button:** Click on save button it will add DNS server
- **Cancel Button:** Click on Cancel button pop will closed

- **Edit Button:**

- Click on the Edit icon pop will open like.. ID, mode ID,Name server and Mode name.
- Here users can **edit only the server name**. Id , Model Id and Mode name will be in a disabled.



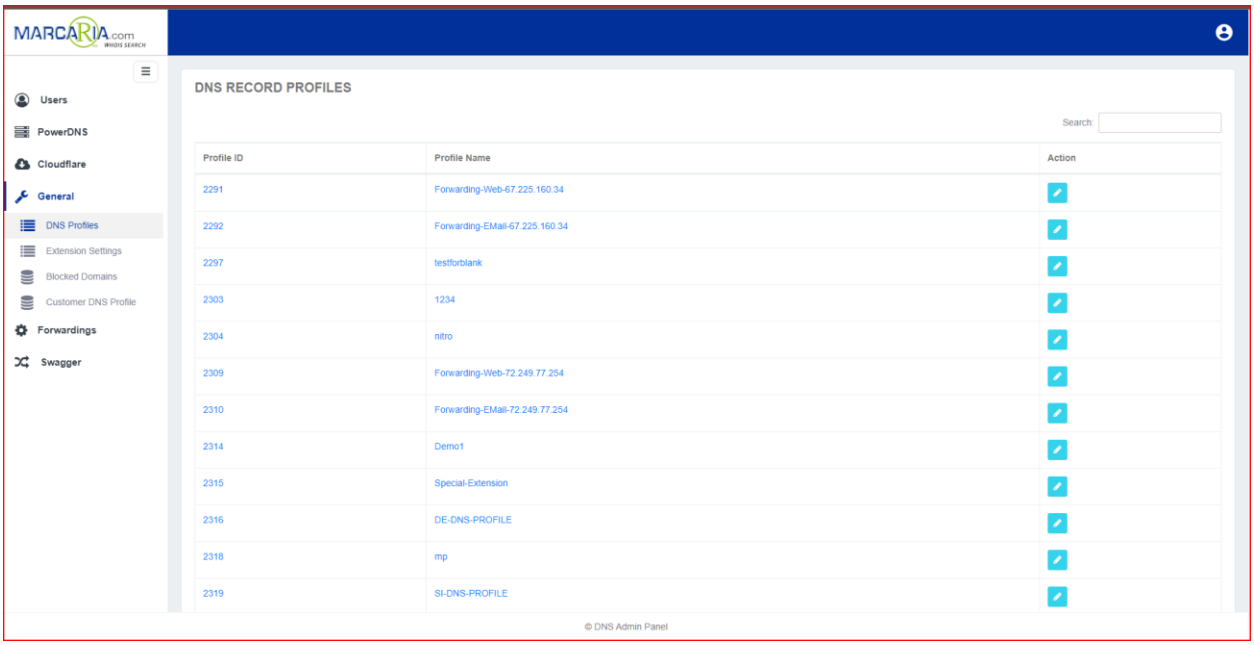
- **Delete Button:**
    - Click on the Delete Button user is able to delete the name server and get the message like - **“Name Server Successfully Deleted”.**

GENERAL.

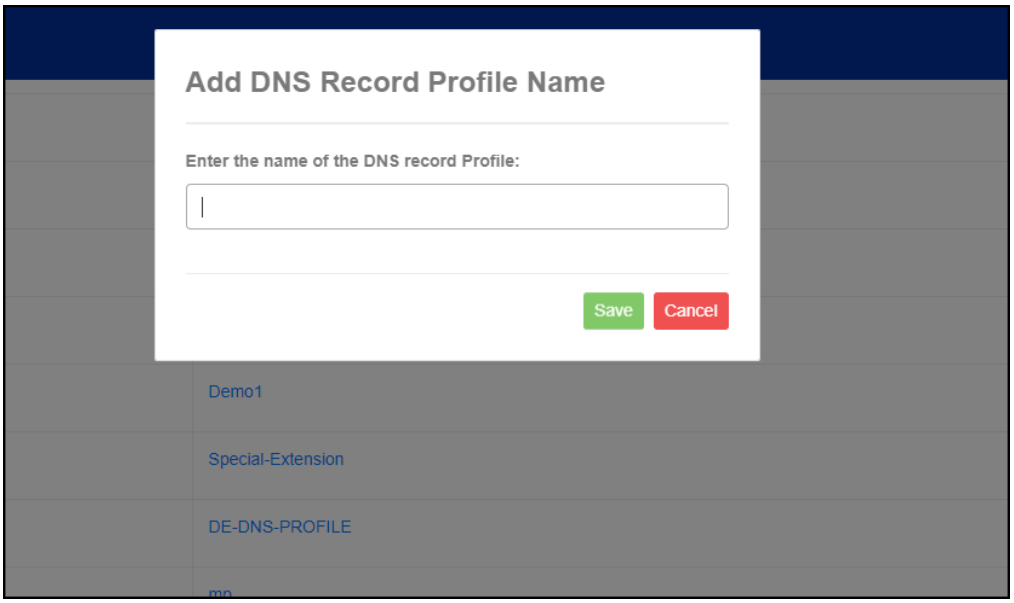
- General has 4 submodules.
    1. DNS Profile.
    2. Extension Settings.
    3. Blocked Domains.

4. Customer DNS Profile.

- **DNS Profiles:**
    - Users can find out the DNS Profile menu option from the side menu panel.
    - Click on it and the user will redirect to the DNS Record Profile page.
    - On this page users are able to view tables with Profile ID, Profile Name, Action field with edit functionality, Add Profile Button, Search box, Pagination.

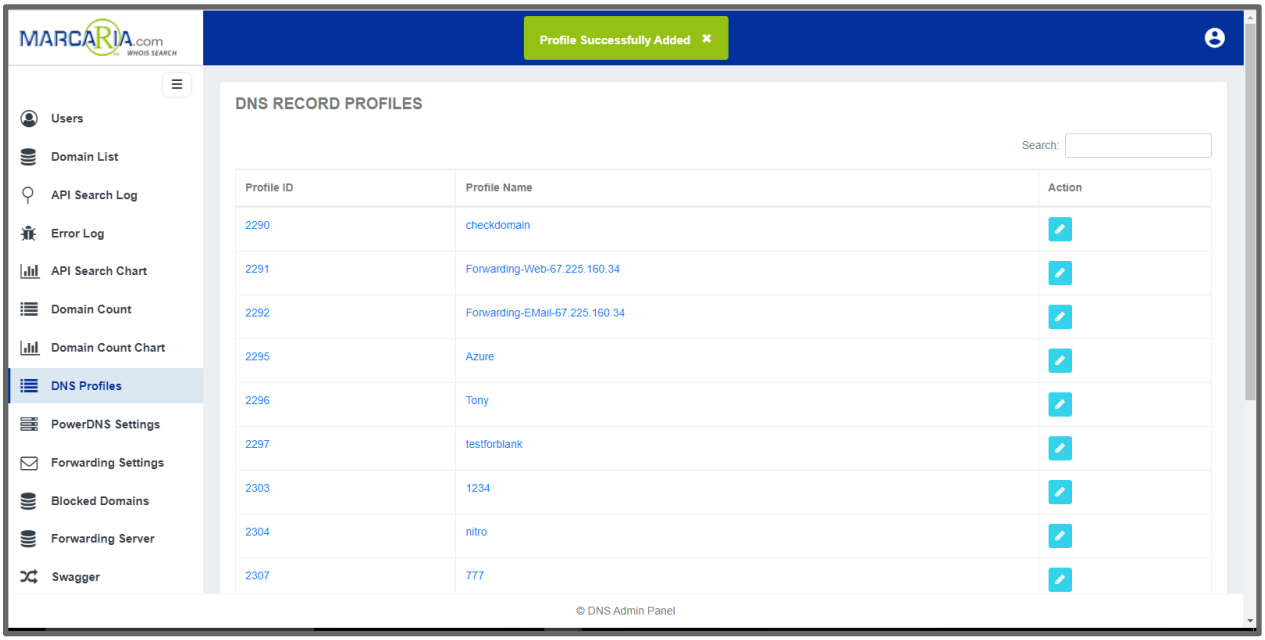


    - **Add Profile Button:**
        - Users can click on the Add Profile button and it will open the popup of Add DNS Record Profile Name.
        - On this popup user is able to view Textbox for entering the profile name, Save & Cancel button.

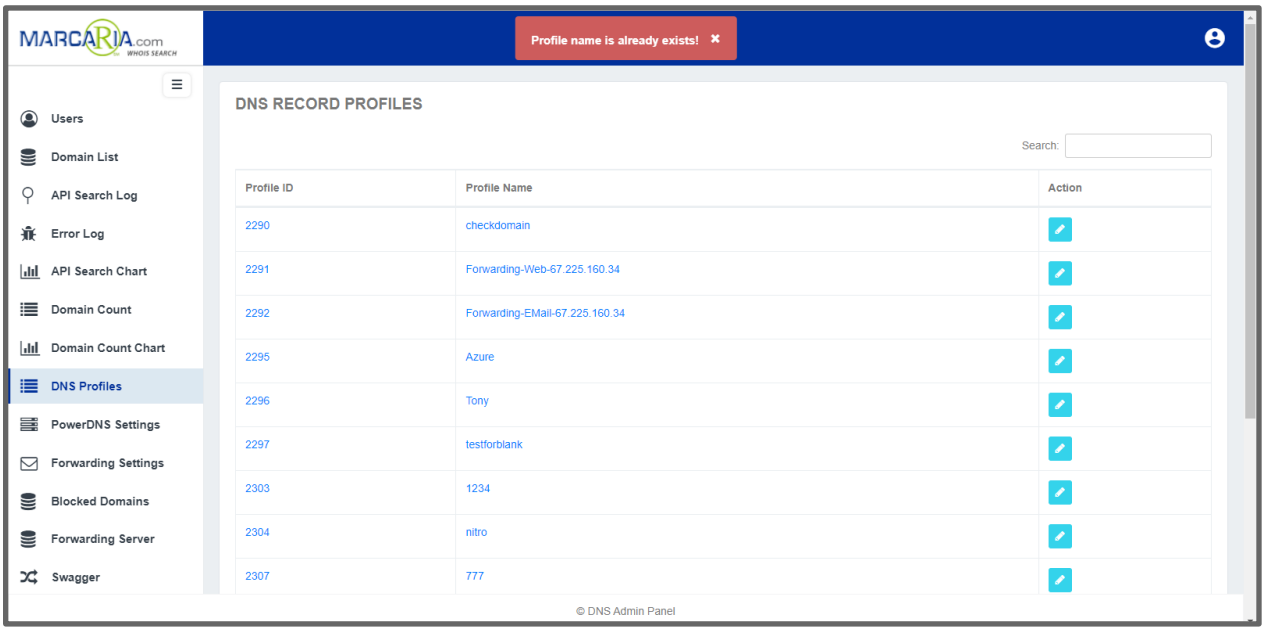


- **Save Button:**
  - Click on the Save button after entering it will create a profile with the same name which is entered and get a message like **“Profile Successfully Added”.**

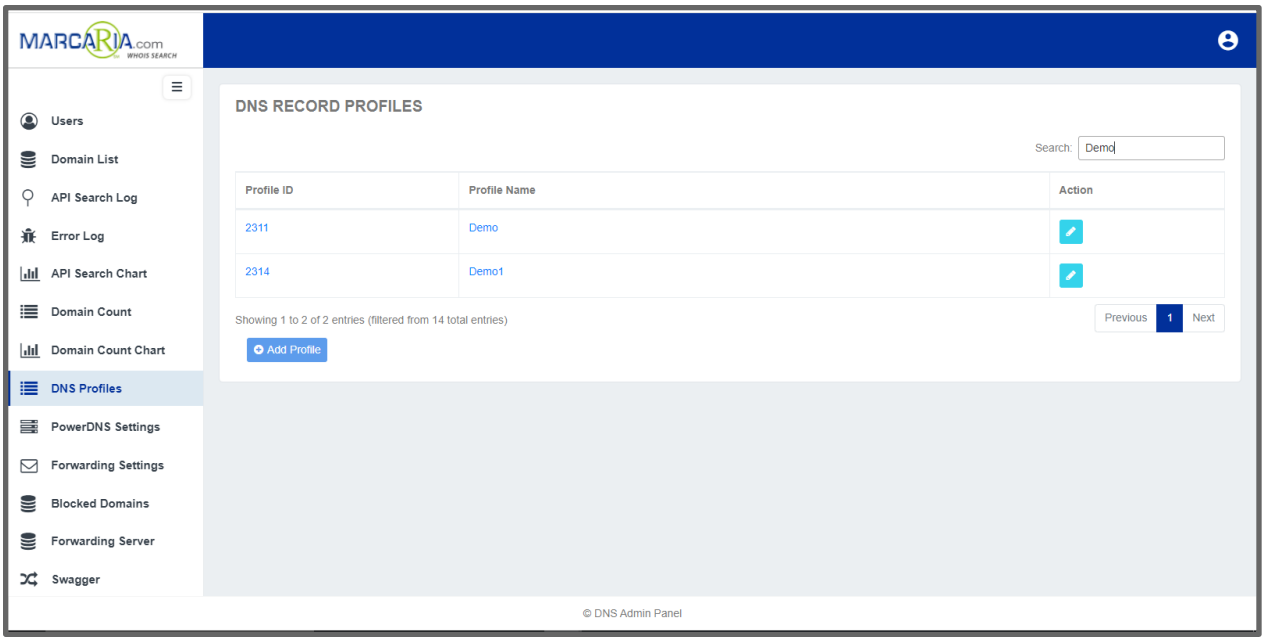
- **Cancel Button:**
  - Click on the cancel button and it will close the popup.



- If the entered profile name is already available in the list it will give a validation message like “**Profile name already exists!**”

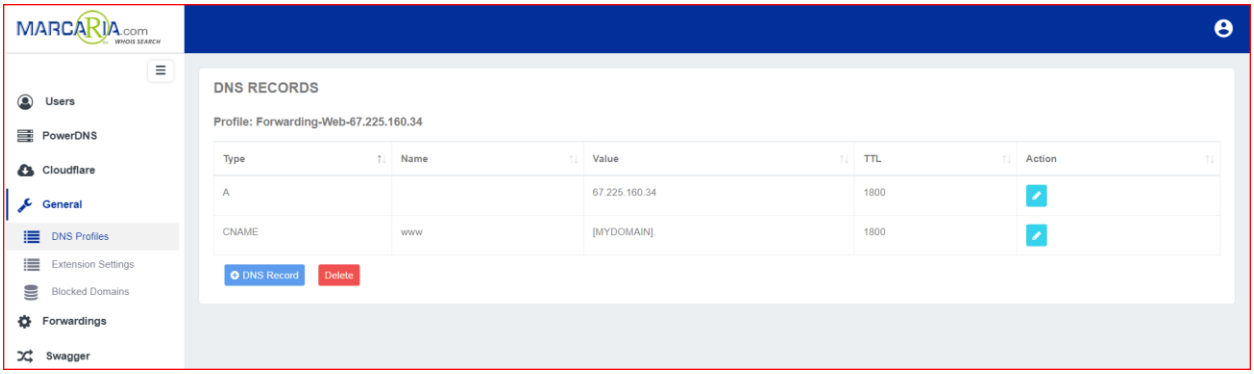


- **Search Box:**
    - Users can enter the name in the search box and get the relevant result.

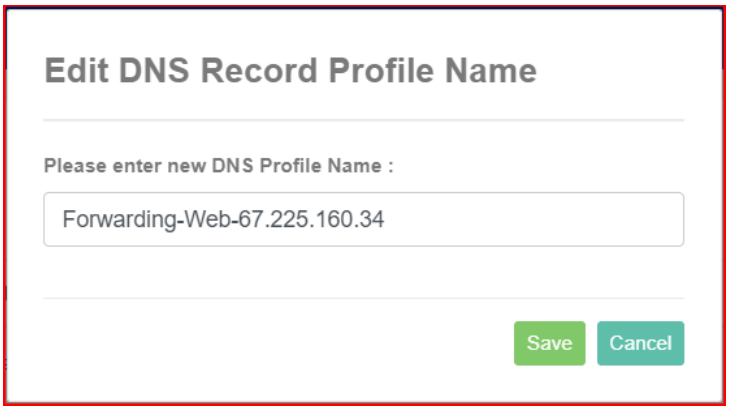


- If search record is not found user get message like - **“No matching records found”.**

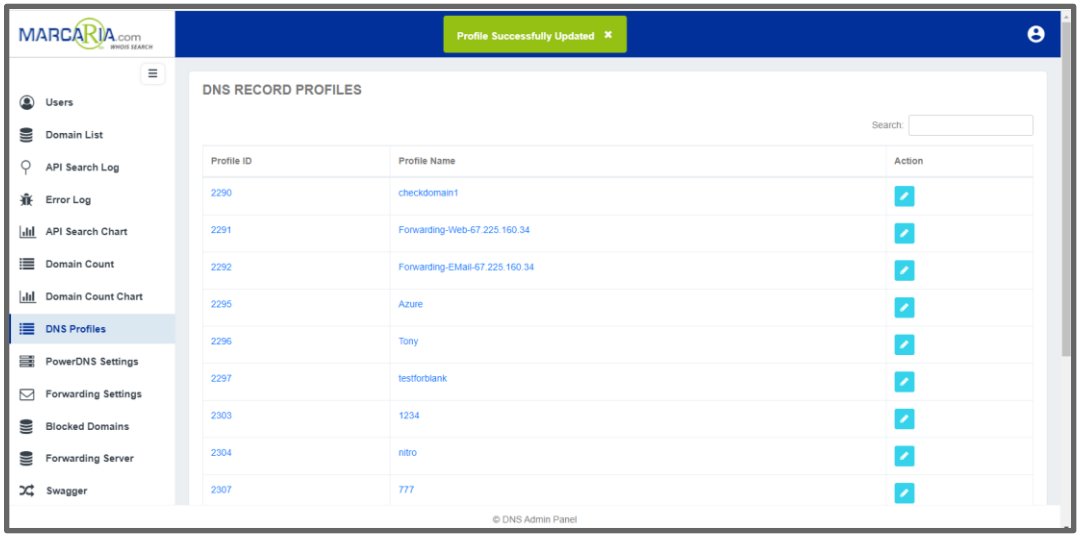
- **<u>Add Records in the created profile:</u>**
  - When user click on the profile ID OR profile name it will redirect to the DNS Record Page
  - From here users are able to add the record for the profile.
  - **NOTE: This created profile is used in the Update DNS Record in bulk , Extension settings & Forwarding server All the validation must be followed here also.**



- **Action Column (Edit) :**
    - Click on Edit Icon it will open the popup of Edit DNS Record Profile Name.

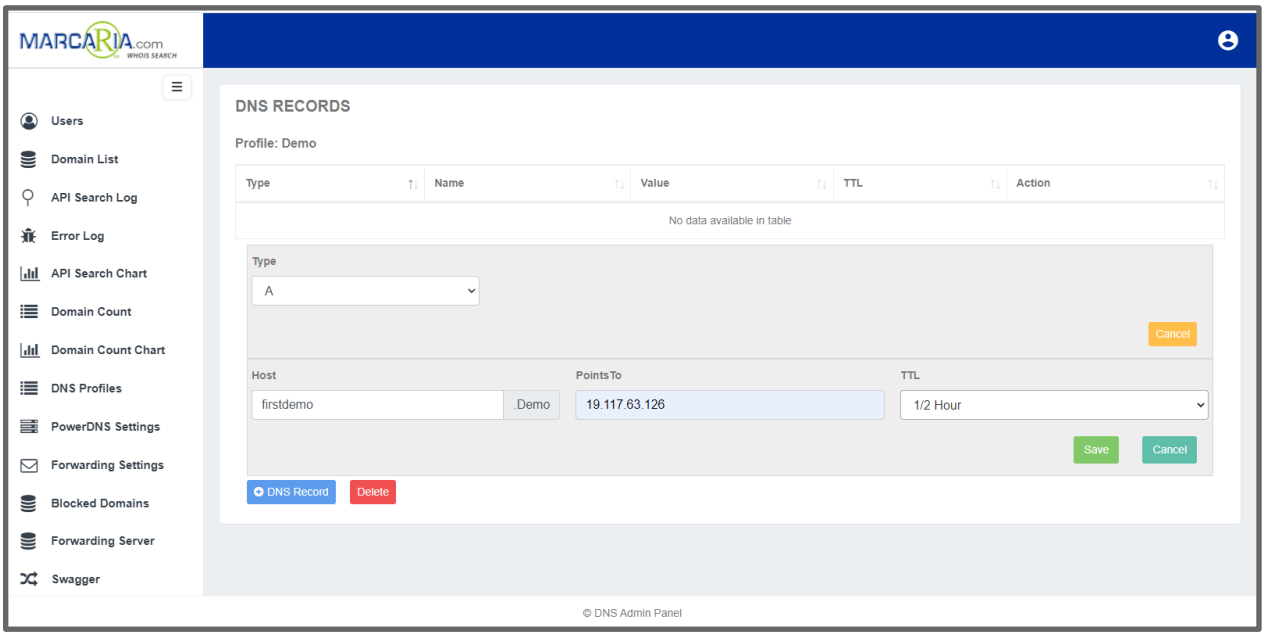


    - Here after modifying the name click on **Save** button user can able to view new name in place of old name and also get the message like - **“Profile successfully updated”.**

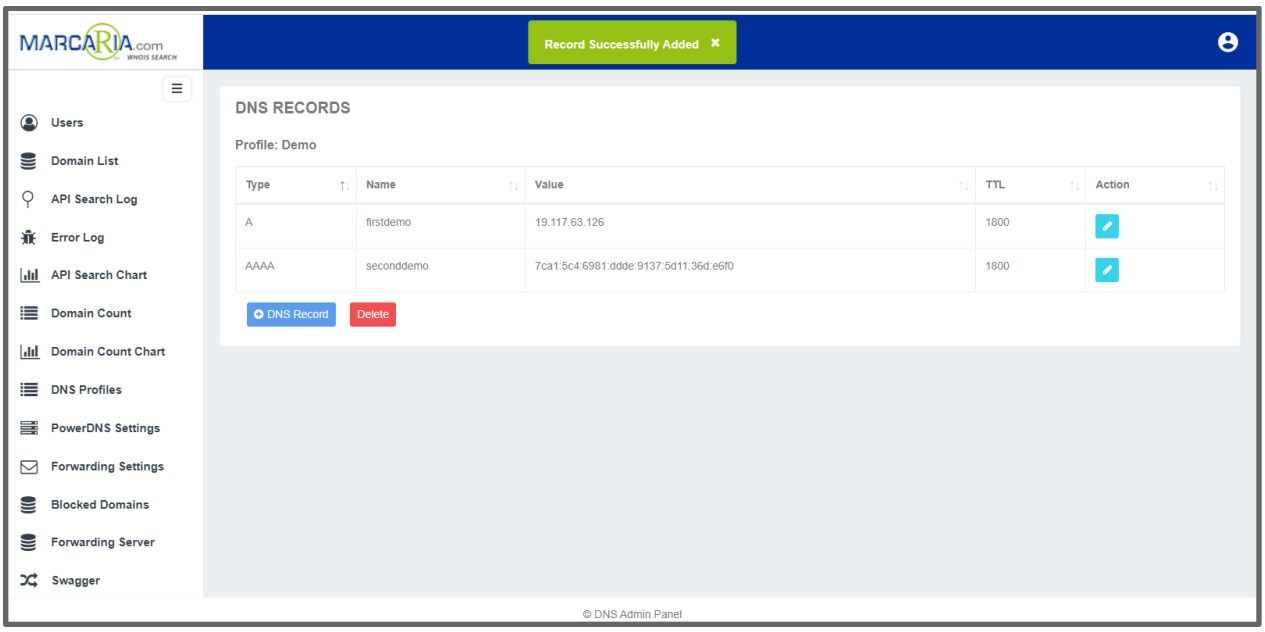


    - **Cancel Button :** If you Click on cancel button, then the edit popup will close.

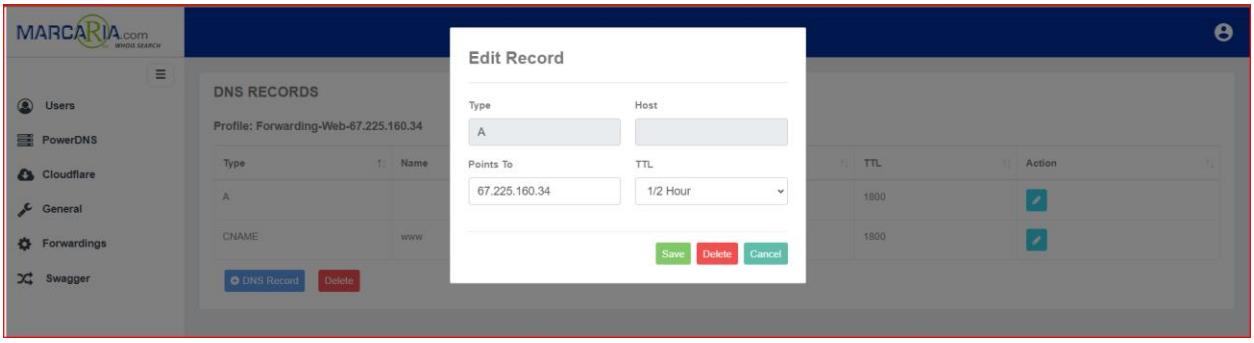
- **DNS Record Button:**
    - Click on the DNS Record button it allows the user to add records just similar to the DNS Record available in the Domain List.



- Click on Save button Record is added to the profile and get a validation message like - **“Record Successfully Added “.**

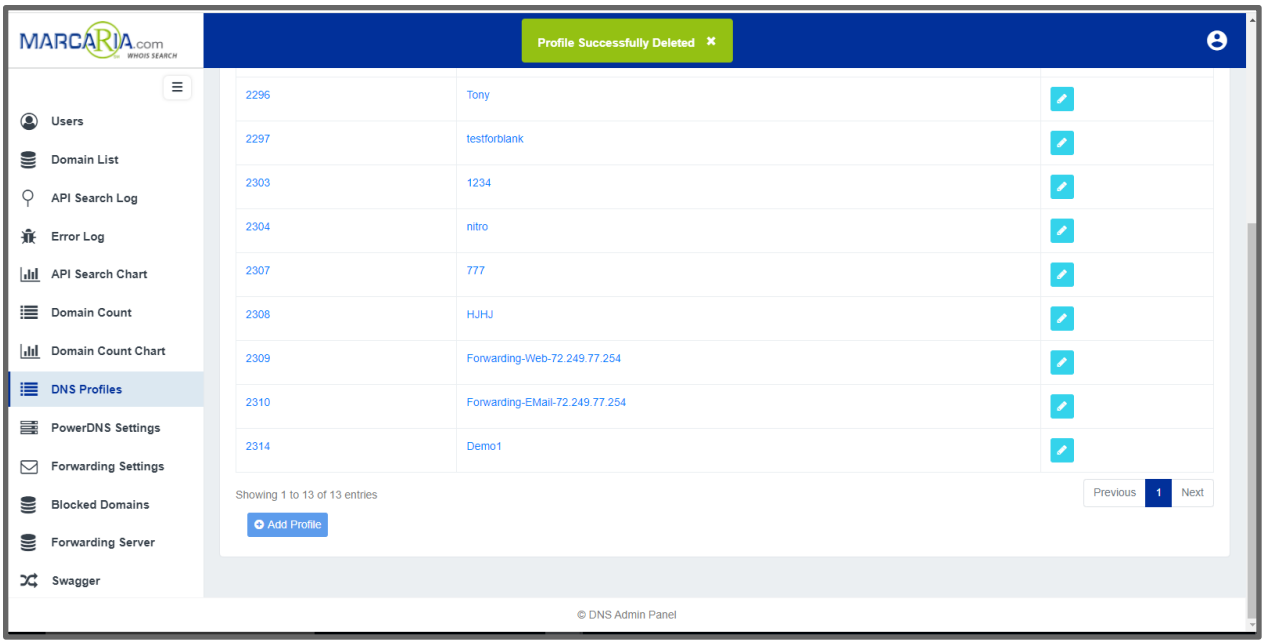


- **Edit Icon :**
    - Click on the edit icon from the Action field of the DNS records table.
    - Users can get the popup with following details..
        - Fields of the record which are generated by the user.
        - Save button
        - Delete button
        - Cancel button
    - **Type** and **hostname** are disabled at the time of edit and all other fields users can modify.

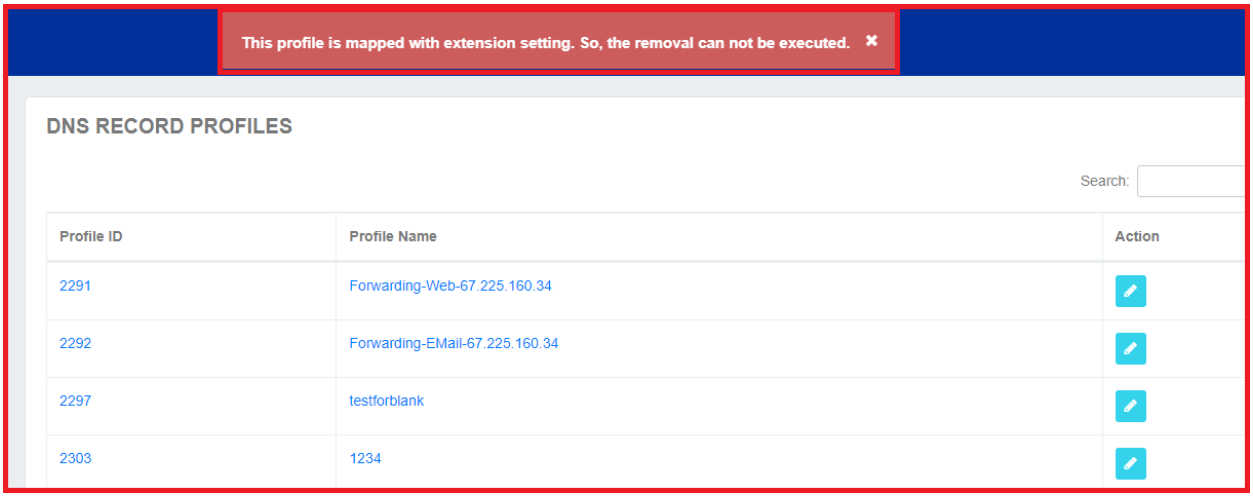


    - **Save Button:**
        - Click on the Save button after editing it will edit the record and get a message like - **“ Profile Successfully updated”.**
    - **Delete Button (For delete record):**
        - When the user clicks on the delete button, the record will be deleted
    - **Cancel Button:**
        - Click on the cancel button and it will close the popup.

- **Delete Button (For delete DNS Profile) :**
    - Once a user opens the profile it is able to see the delete button besides the DNS record button.
    - Click on the Delete Button user is able to delete the profile and get the message like - **“Profile Successfully Deleted”.**



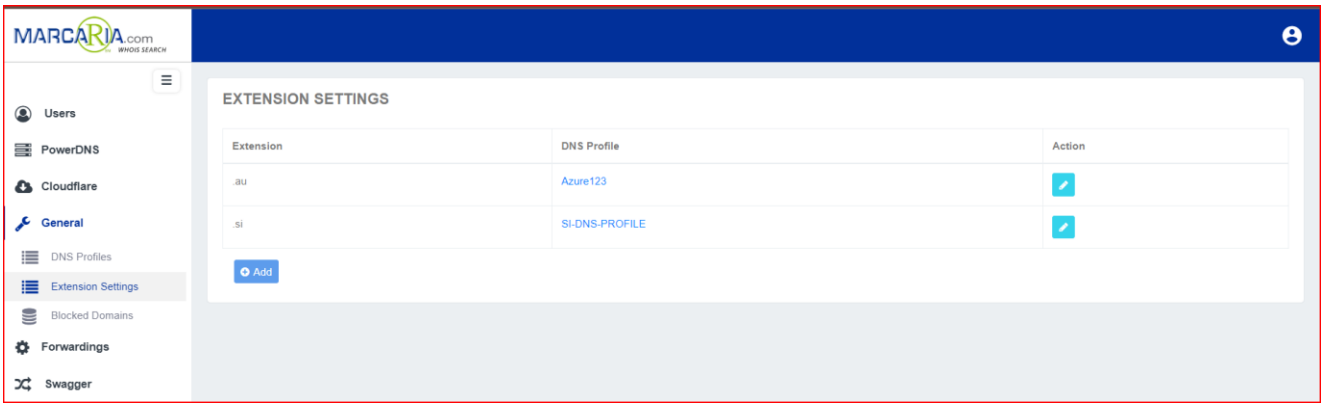
- **NOTE: A DNS profile cannot be deleted unless the extension record is deleted in the extension settings Because it is mapped to extension settings.**



- **Extension Settings :-**

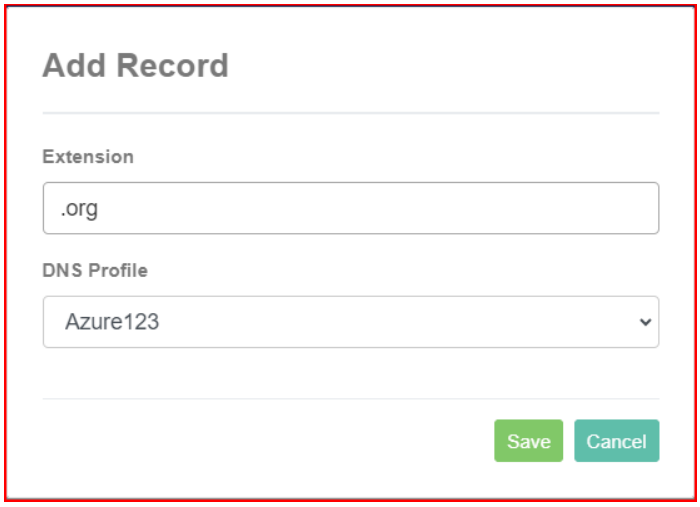
  - Users can find out the extension settings menu option from the side menu panel.

- When users click on the DNS profile it will redirect to the DNS record page.
- On this page users are able to view tables with extension, DNS profile and action with edit functionality and add button.

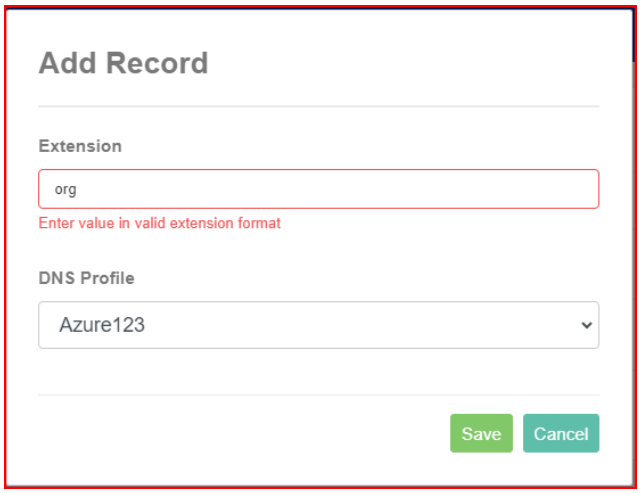


- **Add Button:**

- Once the user clicks on the add button it will show extension text area and DNS profile dropdown.
- Users can add any extension by starting from **. (Dot).** For example: **.org**

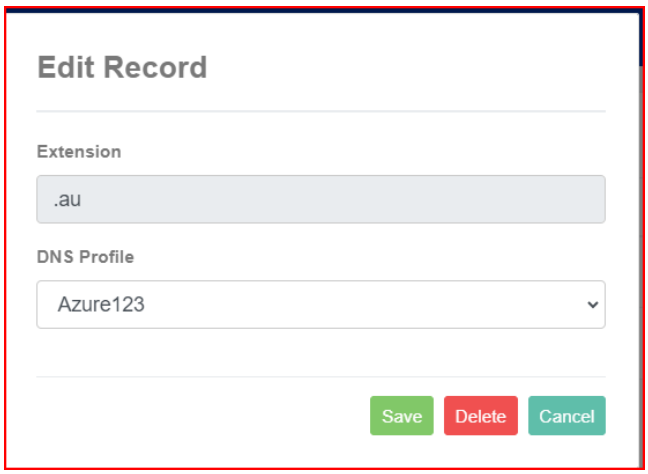


- If a user adds extension without entering dot then validation message will be shown : **“ Enter value in valid extension format”**.



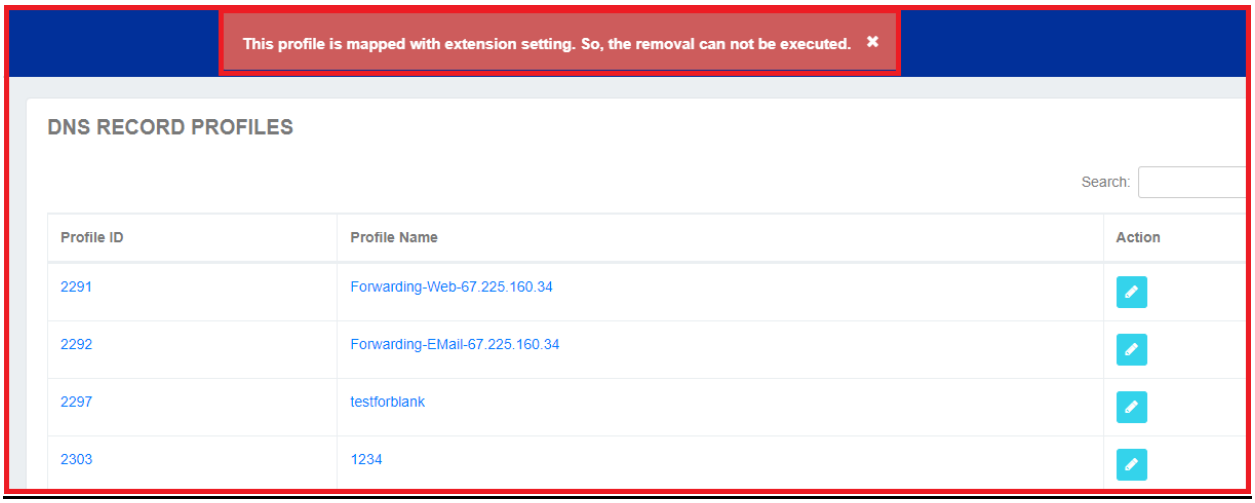
- **Save Button:** Click on save button it will add DNS record
- **Cancel Button:** Click on Cancel button pop will closed

- **Edit Icon :**
  - Click on the edit icon from the Action field of the extension setting page.
  - Users can get the popup with following details..
    - Fields of the record which are generated by the user.
    - Save button
    - Delete button
    - Cancel button
  - **Extension** is disabled at the time of edit and DNS profile users can modify.



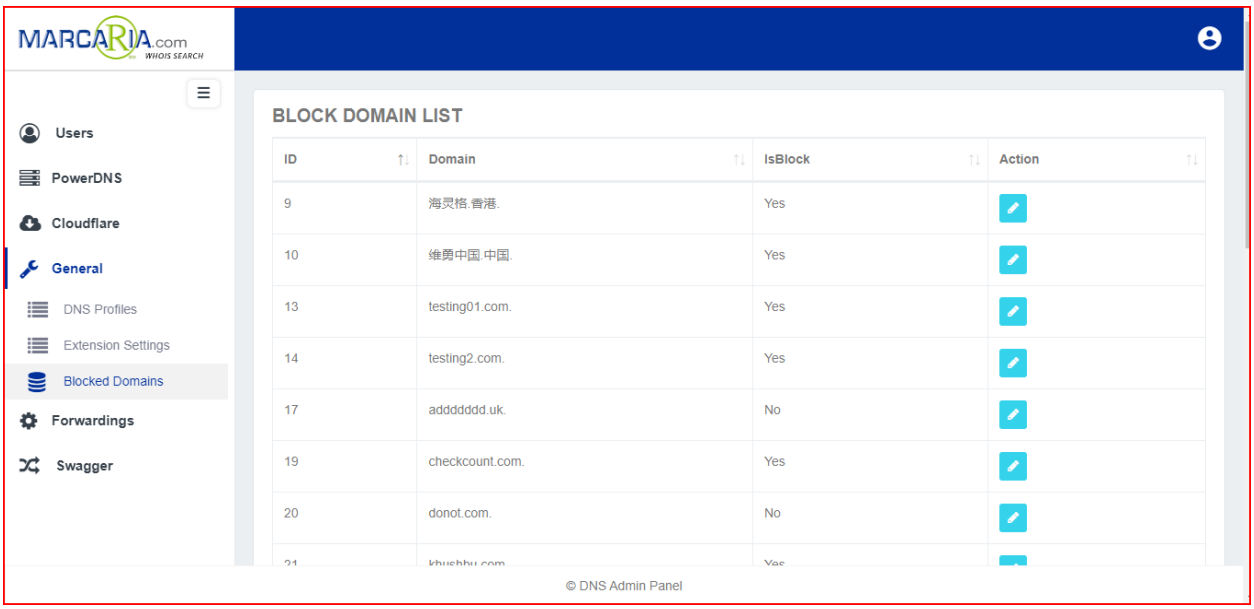
- **Save Button:** Click on save button record will update in DNS record
- **Cancel Button:** Click on Cancel button pop will closed
- **Delete Button:** Click on the Delete Button user is able to delete the profile and get the message like - **“Extension Successfully Deleted”.**

- **NOTE: An extension settings record in Extension Settings cannot be deleted until the DNS profile is deleted. Because it is mapped to a DNS profile.**

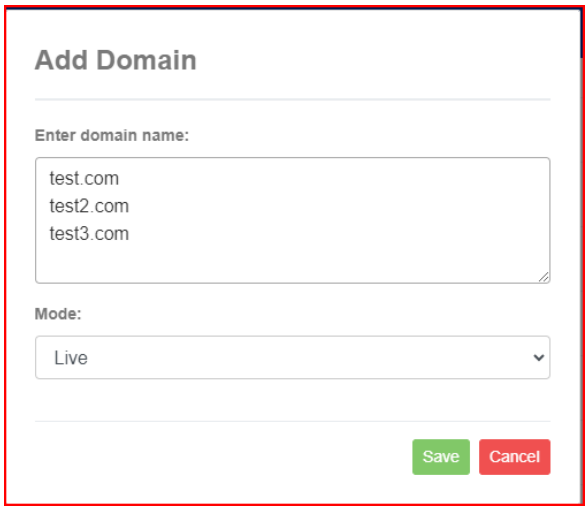


- **Blocked Domains:**

  - On this page users are able to view tables with ID, domain, Is blocked and action with edit functionality and add button.
  - Blocked domains Record will be displayed in the domain list.
  - Users can not add new records in the domain which are blocked.
  - Using sorting functionality users are able to sort the data.

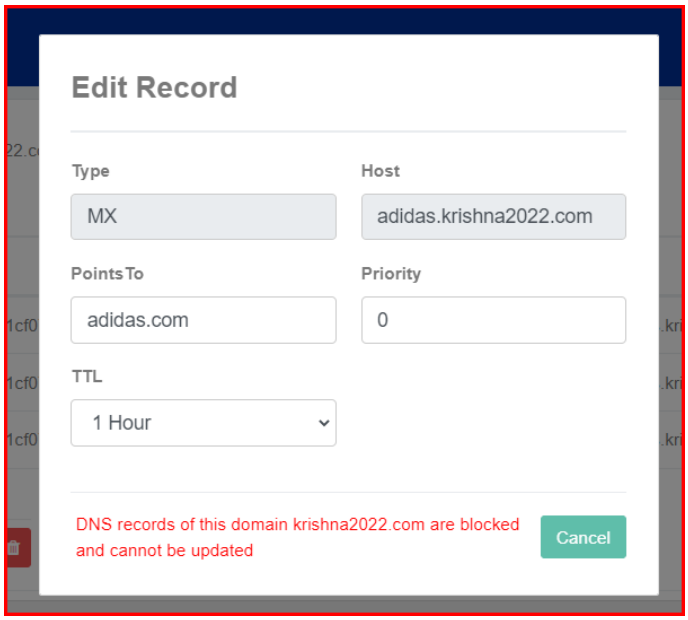


- **Add Button:**
  - Once the user clicks on the add button it will show domain name text area and mode dropdown with live and test option.
  - Here, users are able to block multiple domains in the text area separated each by entering a domain name. i.e. every domain must be in a new line.

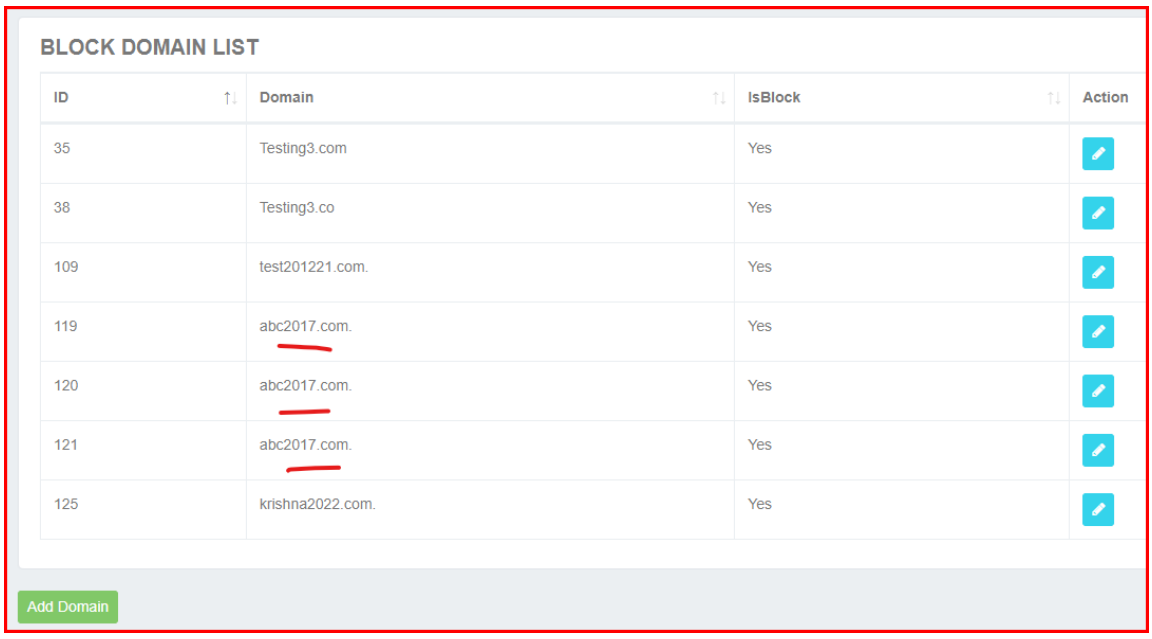


  - **Save Button:** Click on save button it will block the domain.
  - **Cancel Button:** Click on Cancel button pop will be closed.

- **Edit Icon :**
    - Users are able to click the edit icon from the Action field of the blocked domain page.
    - Users can get the popup with following details..
        - Domain name text area field
        - Mode dropdown
        - Is block dropdown
        - Save button
        - Delete button
        - Cancel button
    - The domain is **unblocked** when the user selects the **No option** from the **Is Blocked dropdown menu.**
    - **Users can add records to the domain only when the domain is unlocked**



    - When a domain gets blocked the first time and then blocks the domain again with the same name, a message should be displayed: “**abc2017 is already in the blocked list”.**



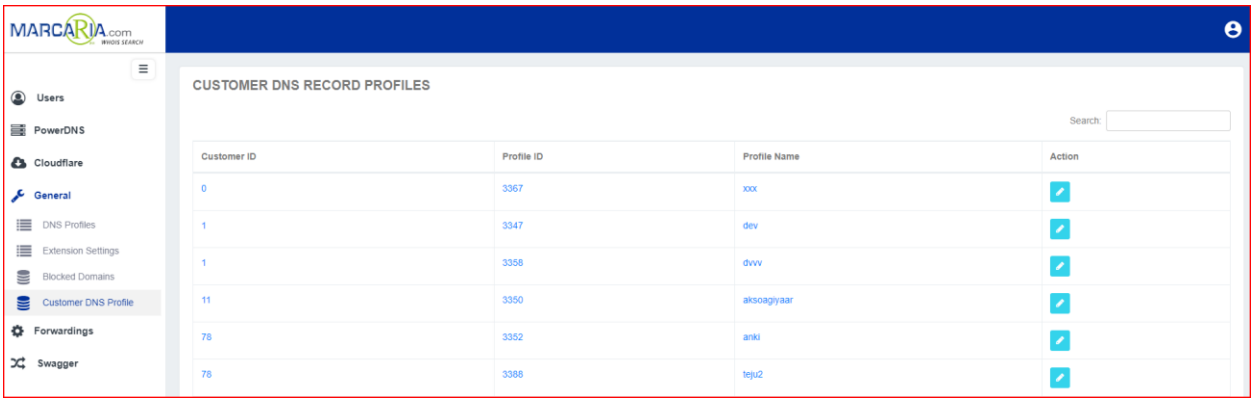
- Email and Web forwarding is allowed in blocked domains.
- **NOTE:** Users need to add (.) dot after the domain name when they are adding it in the blocked domain list.

Client Confirmation regarding the who domain works:

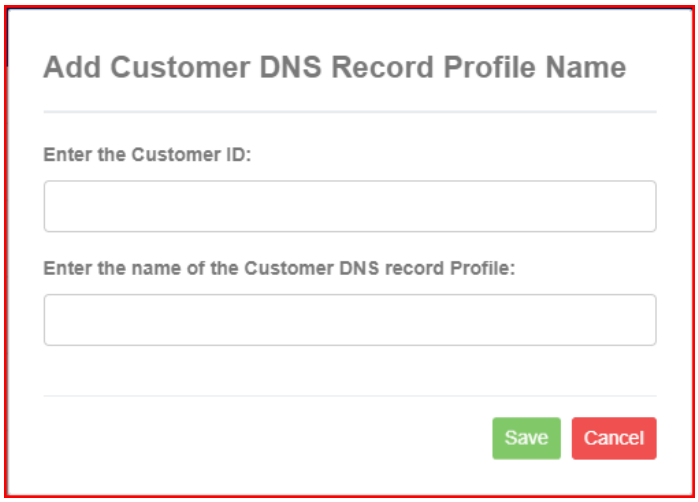
1. Blocked domain behavior (your question from May 1 — apologies for the delay). Proposed behavior when a domain is marked as blocked in the DNS-Admin Panel:

- **Restricted:** all mutating operations — add/edit/delete DNS records, web/email forwarding changes, applying DNS profiles, Setup Secondary Zone, and domain deletion. This applies to NormalAgent users and to API calls (the API should return an explicit "domain is blocked" error, not a generic failure).
- **Still allowed:** read-only operations — viewing the domain and its records, and zone export.
- **Unblocking:** only Admin/Superuser can unblock a domain; the block/unblock action should be logged.

- **Customer DNS Profile :**
    - Users can find out the customer DNS Profile menu option from the side menu panel.
    - Click on it and the user will redirect to the customer DNS Record Profile page.
    - On this page users are able to view tables with Customer ID, Profile ID, Profile Name, Action field with edit functionality, Add Customer Profile Button, Search box, Pagination.

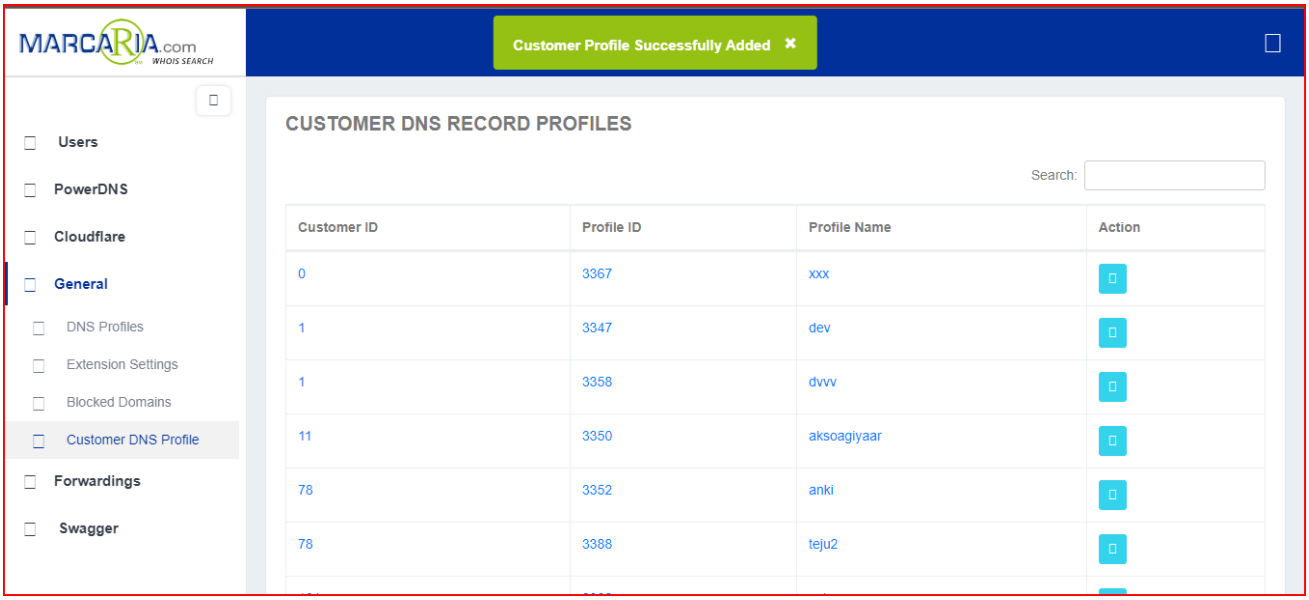


    - **Add Customer Profile Button:**
        - Users can click on the Add customer Profile button and it will open the popup of Add customer DNS record profile name.
        - On this popup user is able to view Textbox for entering the customer ID,customer DNS record profile name, Save & Cancel button.



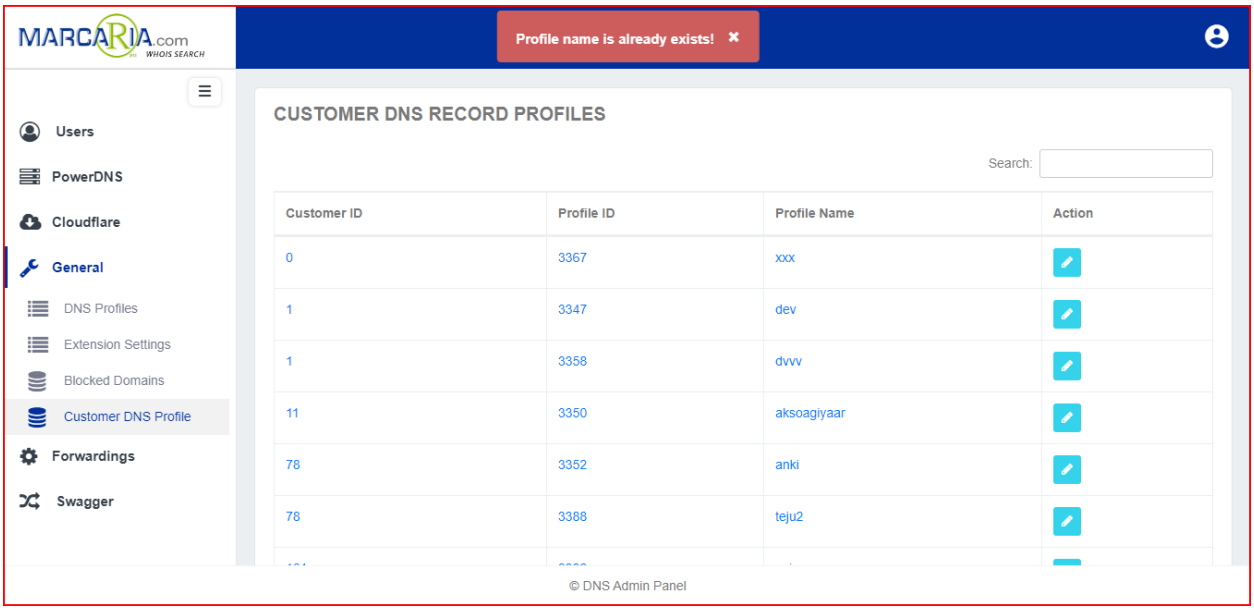
- **Save Button:**
  - Click on the Save button after entering it will create a profile with the same name which is entered and get a message like **“Customer Profile Successfully Added”.**

- **Cancel Button:**
  - Click on the cancel button and it will close the popup.

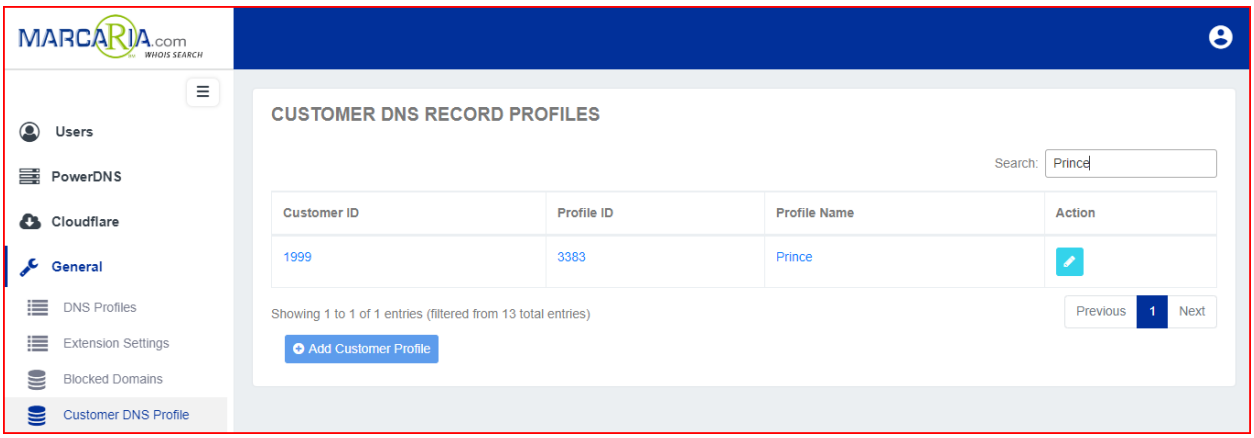


  - **Here, users will be able to create multiple DNS profiles with the same customer ID**.

- If the user enters the customer ID and profile name that is available in the list, it will give a validation message like “**Profile name already exists!**”

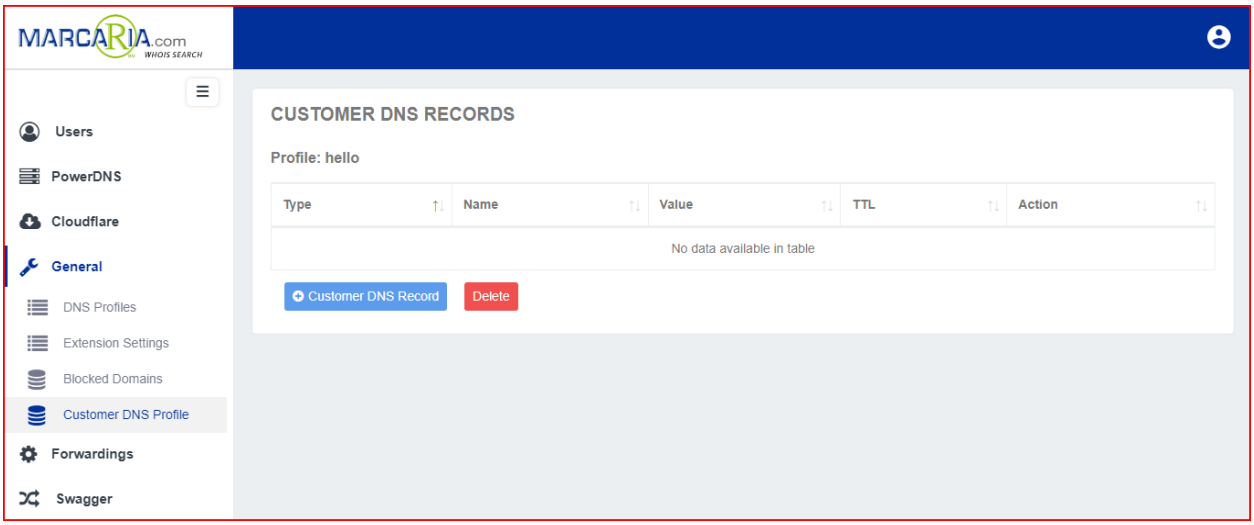


- **Search Box:**
  - Users can enter the name in the search box and get the relevant result.

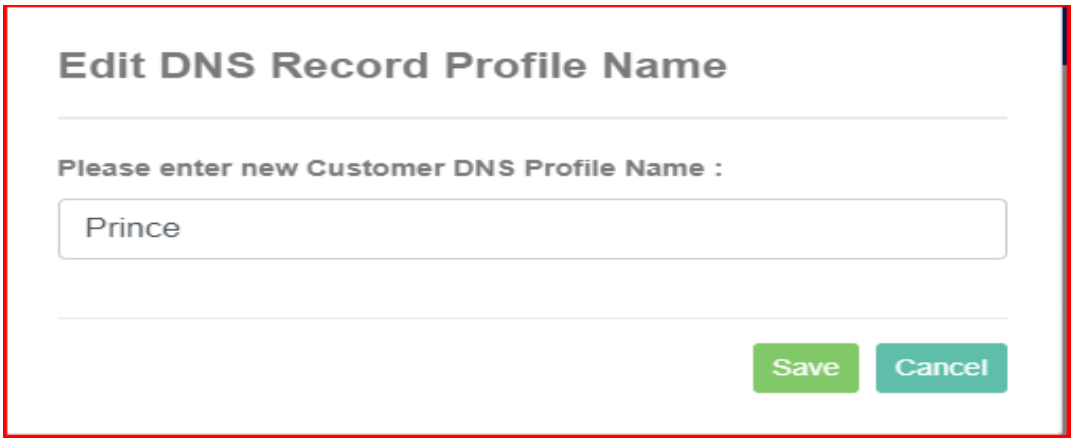


  - If search record is not found user will get message like - **“No matching records found”.**

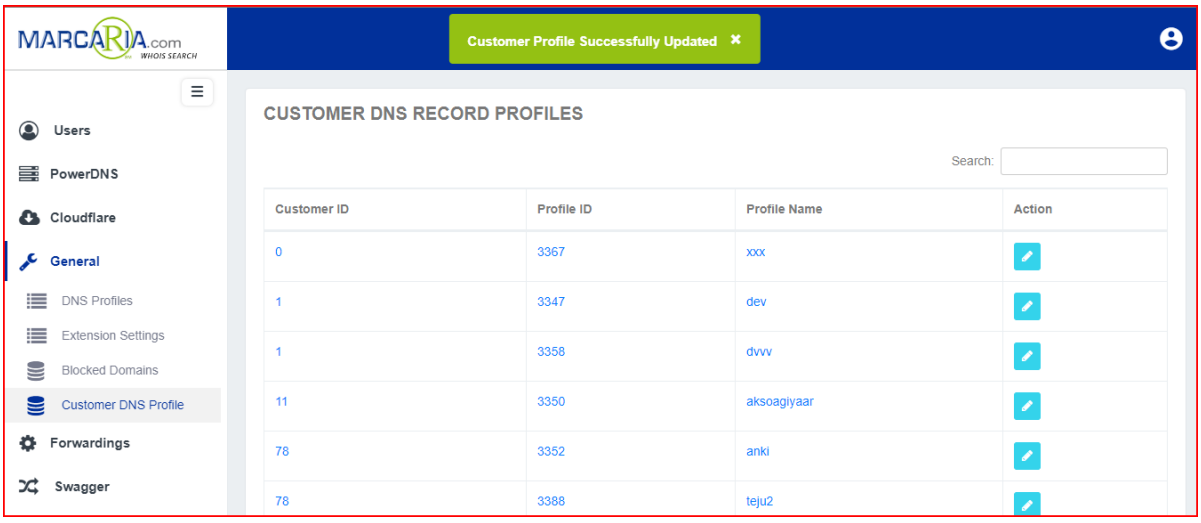
- **Add Records in the Created Customer Profile:**
    - When a user clicks on the **customer ID** OR **profile ID** OR **profile name** it will redirect to the customer DNS record page.
    - From here users are able to add the record for the profile.



- **Action Column (Edit) :**
    - Click on Edit Icon it will open the popup of edit customer DNS Record Profile Name with save and cancel button.



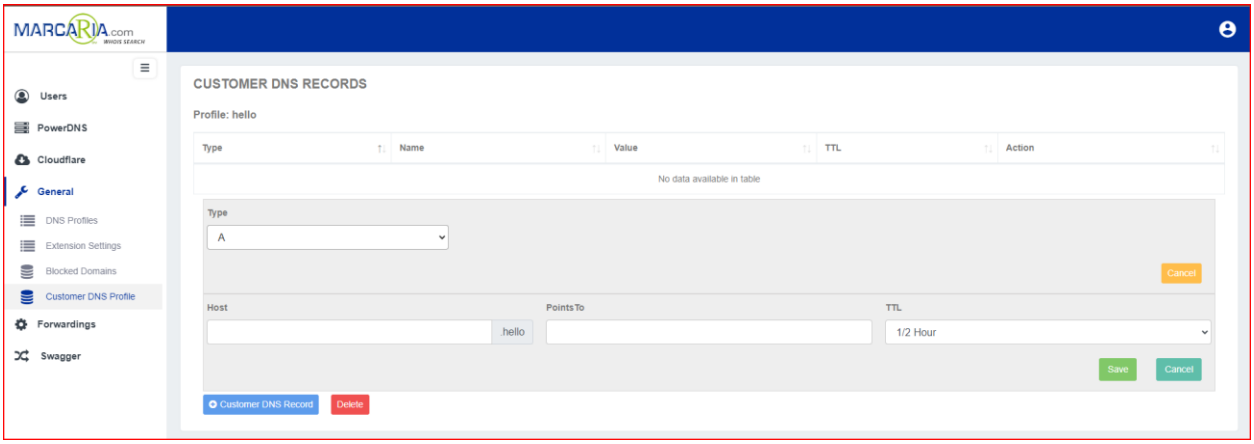
- **Save Button :** Here after modifying the name click on **Save** button user is able to view the new name in place of the old name and also get the message like **“Profile successfully updated”.**



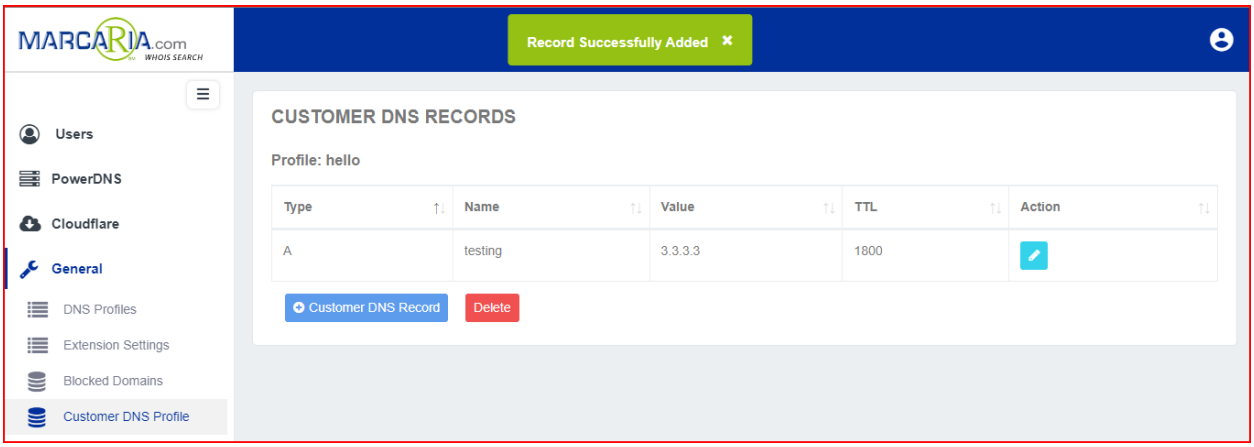
- **Cancel Button :** If you Click on cancel button, then the edit popup will close.

- **<u>Customer DNS Record Button:</u>**

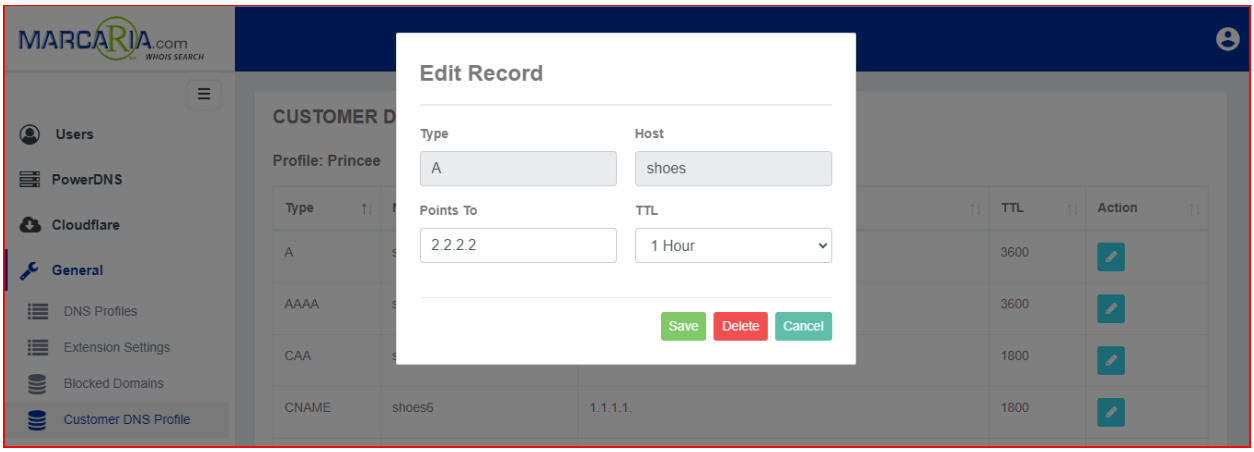
- Click on the customer DNS Record button it allows the user to add records just similar to the DNS Record available in the Domain List.



- Click on Save button Record is added to the profile and get a validation message like - **“Record Successfully Added”.**

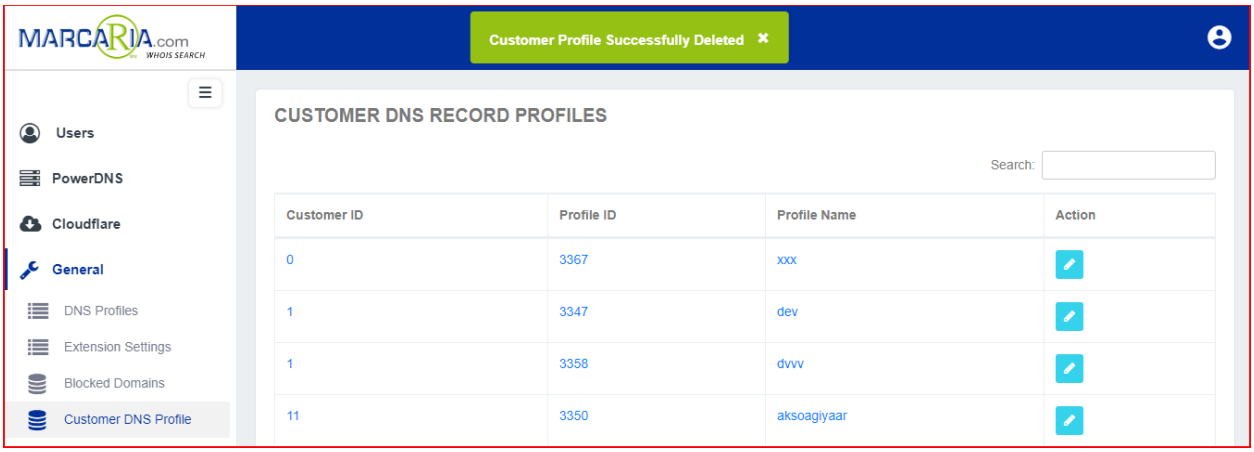


- **<u>Edit Record Icon :</u>**
  - Click on the edit icon from the Action field of the customer DNS records table.
  - Users can get the popup with following details..
    - Fields of the record which are generated by the user.
    - Save button
    - Delete button
    - Cancel button
  - **Type** and **hostname** are disabled at the time of edit and all other fields users can modify.



- **Save Button:**
  - Click on the Save button after editing it will edit the record and get a message like - **“ Customer Profile Successfully updated”.**
- **Delete Button (For delete record):**
  - When the user clicks on the delete button, the record will be deleted.
- **Cancel Button:**
  - Click on the cancel button and it will close the popup.

- **Delete Button (For delete Customer DNS Profile) :**
  - Once a user opens the profile it is able to see the delete button besides the customer DNS record button.
  - Click on the Delete Button user is able to delete the profile and get the message like - **“ Customer Profile Successfully Deleted”.**

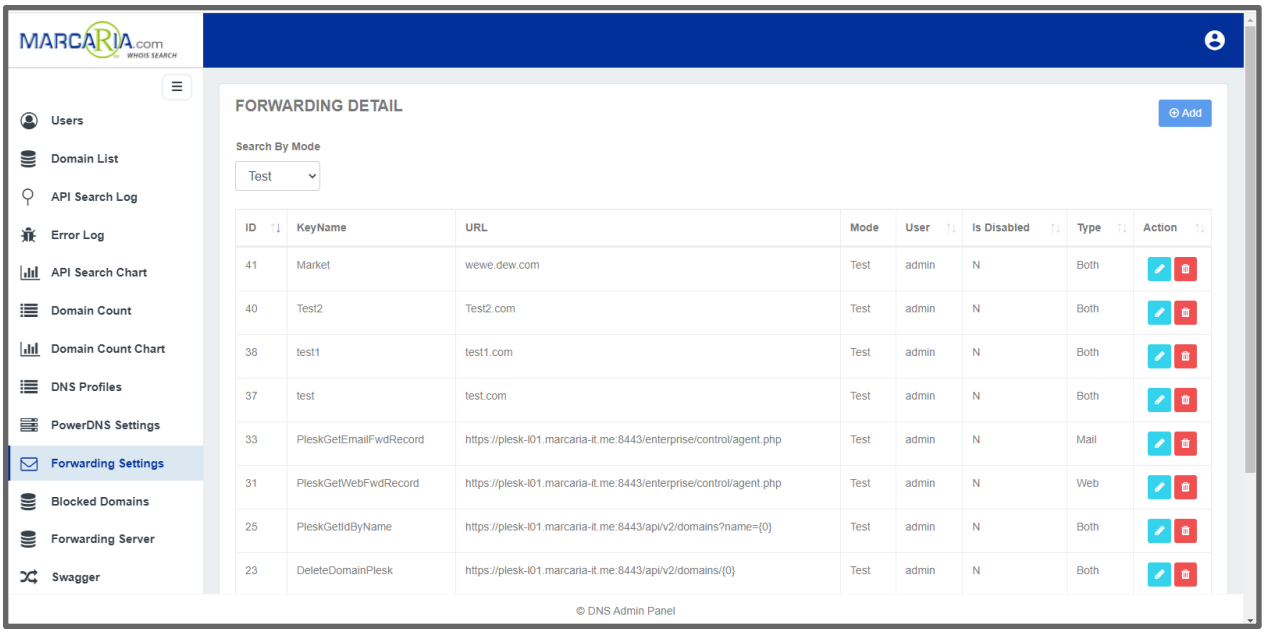


FORWARDING.

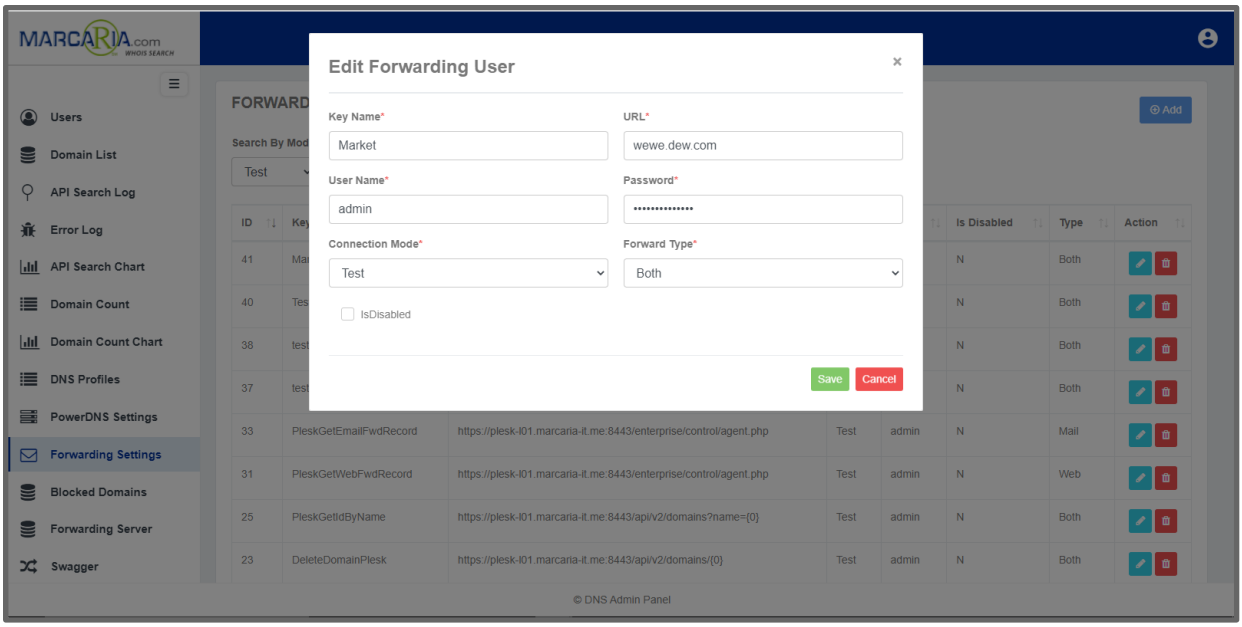
- Forwarding has 2 submodules.
    1. Forwarding Settings.
    2. Forwarding Server.

● Forwarding Settings:

- It is used for mapping the plesk API.
- Here users are able to view the following details with search by mode dropdown and add button.
- Using sorting functionality users are able to sort the data.
- **ID:** Unique Identification number.
- **Key Name:** Key name which is used to map the plesk URL.
- **URL:** A plesk URL
- **Mode:** Users can set the mode either Live or Test.
- **User:** User name
- **Is Disabled:** Here if the user does not check the checkbox its value is N otherwise its value is Y. Here if users don't want to use the API longer at that time they can do Is Disabled checkbox checked instead of hard delete.
- **Type:** Select the type of forwarding i.e. Email Forwarding, Web Forwarding or both.
- **Action:** Users can edit or delete the records.

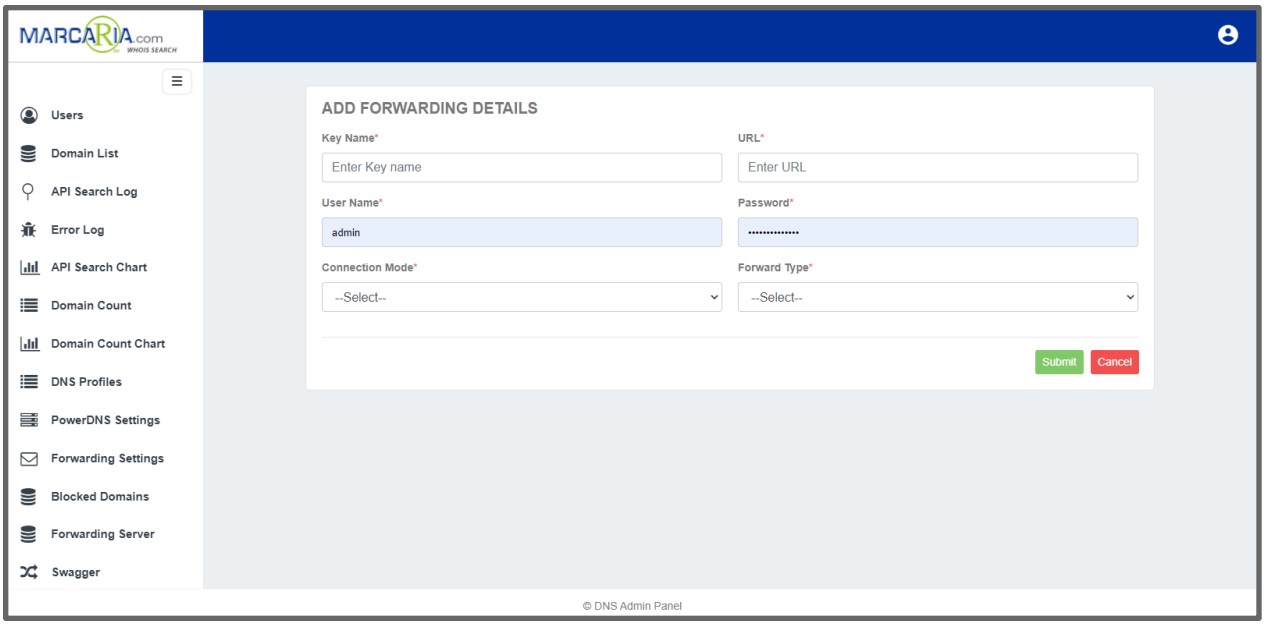


- **Edit Icon:**
    - **Key Name:** User can edit the Key name which is used to map the plesk URL.
    - **URL:** User can edit the plesk URL
    - **Mode:** Users can edit the mode either Live or Test.
    - **User:** User can edit the User name
    - **Password:** password for the user.
    - **Is Disabled:** User can edit the status i.e. Here if the user does not check the checkbox its value is N otherwise its value is Y. Here if users don't want to use the API longer at that time they can do Is Disabled checkbox checked instead of hard delete.
    - **Forward Type:** User can edit the type of forwarding i.e. Email Forwarding, Web Forwarding or both.
    - **Save Button:** Click on the Save button user can save the modification.
    - **Cancel Button:** Click on the Cancel button and the user can cancel the process.

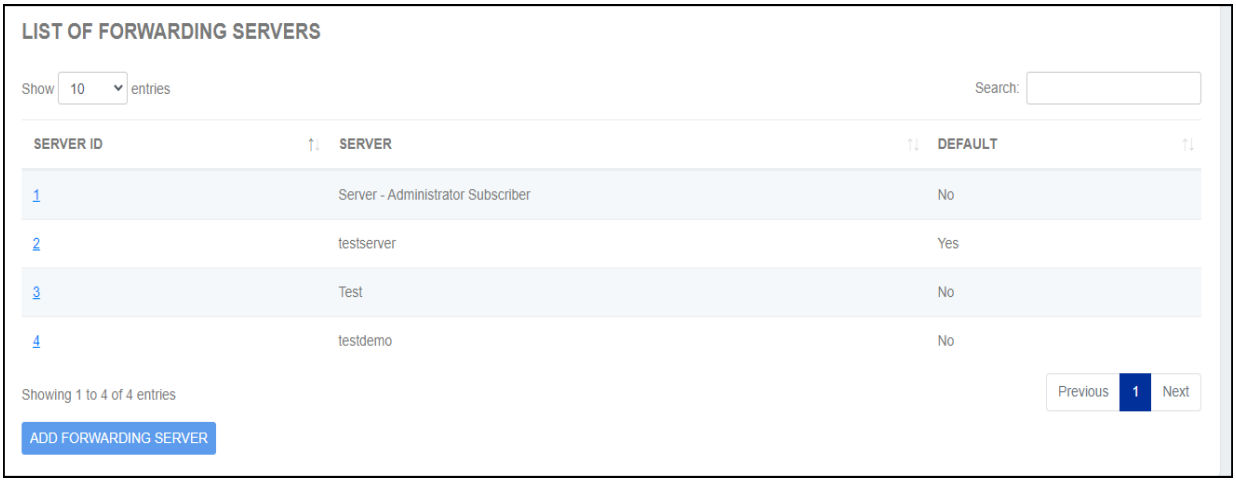


- **Delete Icon:**
  - By clicking on the delete user is able to delete the record.

- **Add Button:**
  - Click on the Add button user will redirect to the Add Forwarding Details Page.
  - Here users are able to view/add following information..
    - **Key Name:** Key Name user for the plesk URL.
    - **URL:** A plesk URL.
    - **User Name:** Display the User Name.
    - **Password :** Password for the User.
    - **Connection Mode:** Select the mode i.e. Live : 1 or Test : 2
    - **Forward Type:** Select the forwarding type i.e.
      1. Mail Forwarding,
      2. Web Forwarding,
      3. Web & Mail Forwarding.
    - **Submit Button:** Click on Submit button it will submit the data.
    - **Cancel Button:** Click on Cancel button it will cancel the process.

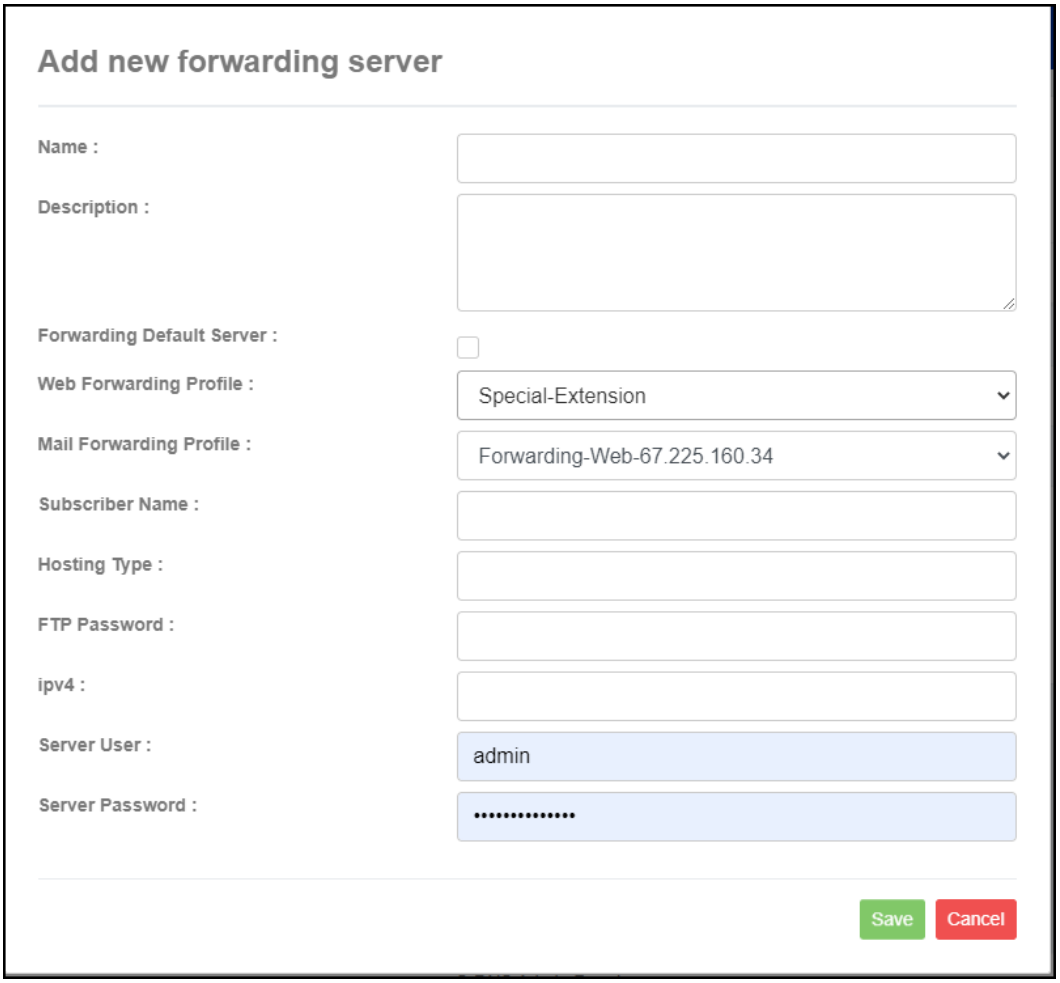


- **<u>Forwarding Server:</u>**
  - Click on the Forwarding Server menu and the user will be able to view List of Forwarding Servers.
  - On this page users are able to view tables with fields Server ID, Server, Default,show entries tab,search tab and Add Forwarding Button.
  - Here users see **Default = Yes** in front of the profile which is “**Active”** and **Default = No** in front of the profile which is **“InActive”**.
  - When creating a new profile at that time the newly added profile will have **Default = Yes**. It means the newly added profile will be considered as “**Active”.**
  - Using sorting functionality users are able to sort the data.

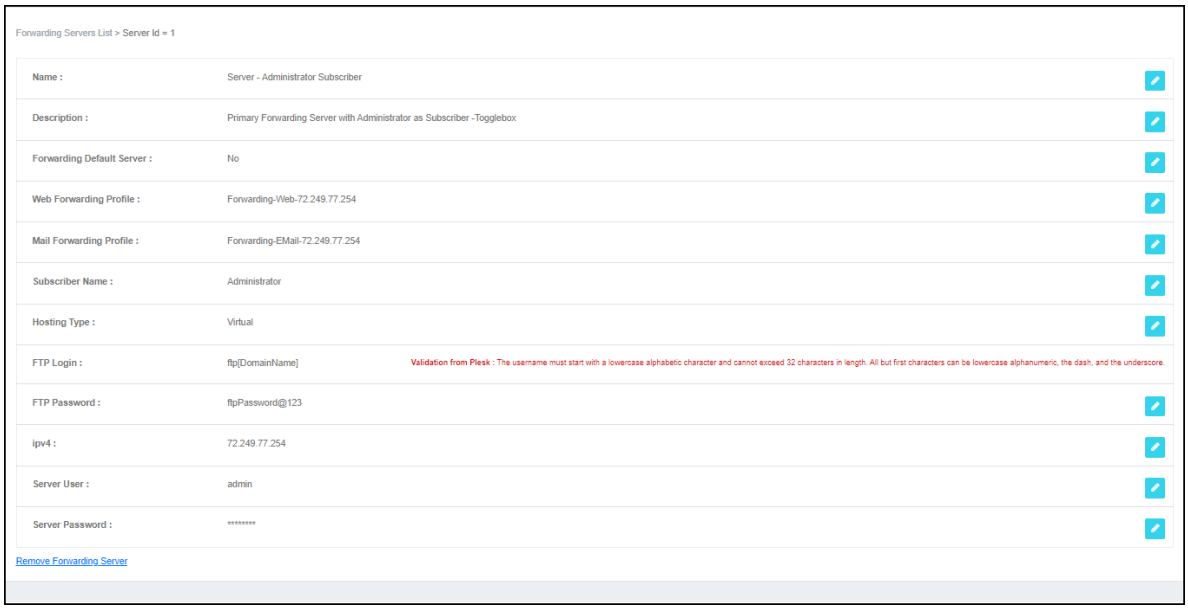


- **Add Forwarding Server:**
    - Click on this button and the user will get a popup with following information..
        - **Name:** Provide name to the forwarding server.
        - **Description:** Add description for the forwarding server.
        - **Forwarding Default Server:** Here checkbox is available if user checks the checkbox it will make profile as Default = Yes and make the profile **"Active".**
          NOTE: Once a user makes the profile Active it will not be able to **"Deactivate the profile"**. Users can change the status by creating the new profile.
        - **Web Forwarding Profile:** Select the profile which is used to add the records. Here users are able to view all the profiles in drop down which are added from the DNS **Profiles menu**.
        - **Mail forwarding Profile:** Here user can able to view all the profile in drop down which are added from **DNS Profile menu**
        - **Subscriber Name:** Add the name of the Subscriber.(Plesk)
        - **Hosting Type:** Add Hosting type.(Plesk)

- **FTP Password:** Add password for the FTP(Plesk)
- **IPV4:** Add the IPV4 address.(Plesk)
- **Server User:** Add name of the server user(Plesk)
- **Server Password:** Add server password.(Plesk)



- **View Details of the Forwarding Server:**
  - When the user clicks on the ServerId it will redirect to the user to the detailed page.

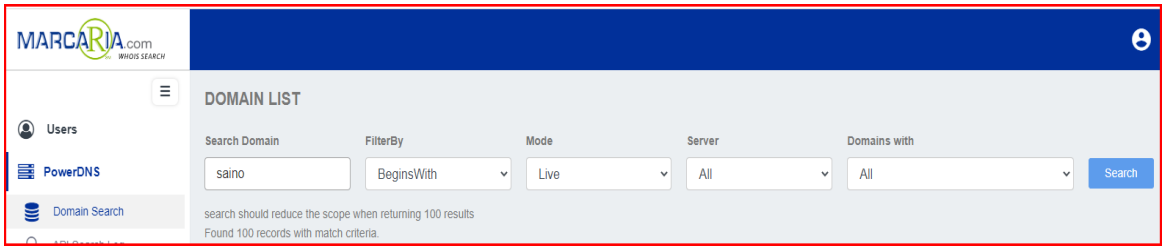


- Here users are able to view details along with the Edit Icon.
- Here users are able to edit any information using the edit icon.
- Here users are able to remove the forwarding server with clicking on **remove forwarding server.**

- **Brand new Domain Search for the “standard” case: (PMS_Userstory_249707) Cloudflare**

  Standard Case: When no Servers and “Domains with” (Web or Email) are picked
  (https://it-tools.marcaria.com/imageserve/images/png/UqSLAaoNV46BkBI.png)
  we should search only “zones”:
  To do this, we will set the parameter **Object_type=zone** and set the query like this:

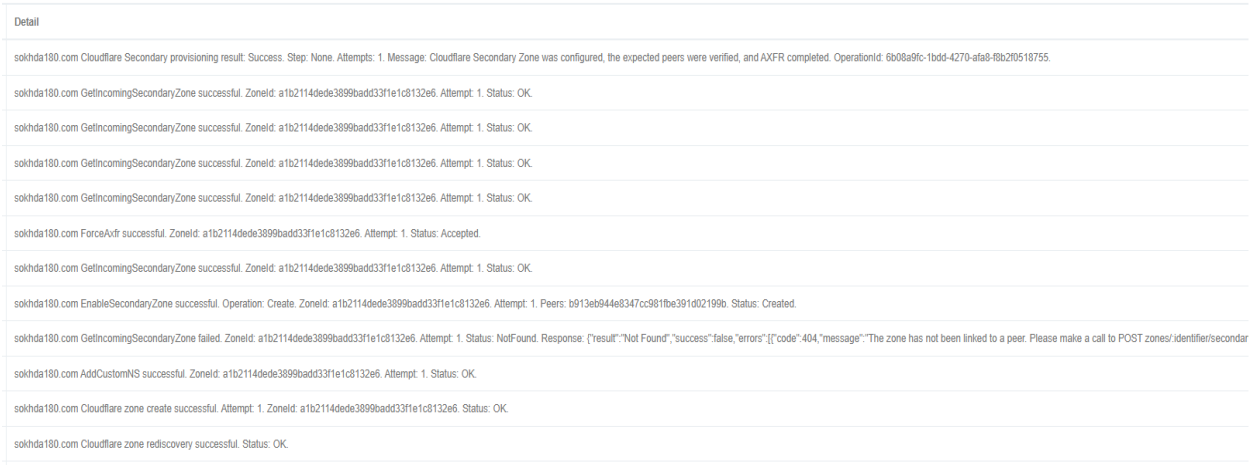
  1.BeginsWith: /servers/localhost/search-data?max=100&object_type=zone&q=mdp15112022*
  2.Contains: : /servers/localhost/search-data?max=100&object_type=zone&q=*mdp15112022*
  3.EndsWith: /servers/localhost/search-data?max=100&object_type=zone&q=*mdp15112022.com



Master Domain Creation Flow with Secondary DNS enables:

- When a domain is created with Secondary DNS **enabled**, the system establishes the domain as a Master zone in PowerDNS.
- The system validates that the PowerDNS zone has been created with the expected Master configuration; the corresponding Cloudflare zone is available, the required Incoming Secondary configuration and Peer information are present.
- The system also validates the SOA serial and ensures that a valid `soa_serial > 0` is available before considering the Master/Secondary relationship successfully established
- Any temporary synchronization or propagation delay between PowerDNS and Cloudflare is handled through the defined retry and verification mechanism.
- The domain should be reported as successfully created only after the required PowerDNS, Cloudflare, zone-transfer, SOA, and database validations have been completed successfully.

- Below is the Expected Log for Domain creation in Cloudflare Secondary zone API Search log:



Race Condition Handling

- The Secondary Zone workflow handles race conditions that can occur when a domain is deleted and immediately recreated. External systems such as Cloudflare and PowerDNS may require additional time to complete deletion or release existing resources. To accommodate this behavior, the system retries operations at the defined intervals of **0, 2, 4, 8, and 10 seconds**. This prevents temporary resource availability issues from being incorrectly treated as permanent failures.

Missing Resource Handling

- The system differentiates between an expected missing resource and an actual operational failure. For example, during cleanup, a `404` response may indicate that the requested resource has already been deleted. Such cases can be treated as successful/idempotent cleanup where appropriate. This prevents repeated cleanup operations from producing misleading failures.

Incorrect Configuration Handling

- The system validates configuration received from Cloudflare and other external services before using it as the basis for a successful result. If an expected configuration is missing, malformed, or inconsistent, the system does not blindly continue as though the setup were complete. Depending on the scenario, the configuration may be retried, repaired through the WebJob, logged as a warning, or reported as a hard failure.

API Validation and Error Handling

- The Secondary Zone APIs validate the result of each required operation and return an overall result based on the complete workflow rather than an individual successful API response. The system handles successful responses, missing resources, invalid configurations, external-service failures, and retry scenarios according to the defined business rules. A successful API response from Cloudflare or PowerDNS alone does not guarantee successful Secondary Zone setup if subsequent mandatory validation or database persistence fails.

- ## Database Synchronization
- The application database maintains the state and relevant information associated with Secondary Zone domains. After successful processing, the corresponding database record is created or updated with the appropriate status and configuration information. Database persistence is therefore treated as an important part of the overall transaction and must remain consistent with the actual state of Cloudflare and PowerDNS
- ## Database Failure Handling
- The system also handles scenarios where the database becomes unavailable while the external APIs continue to return successful responses. For example, if all required Cloudflare and PowerDNS APIs return `200` responses, but the database connection is disrupted, the system must not incorrectly report the entire operation as successful. The database persistence failure must be detected, logged, and reflected in the final operation result according to the implemented error-handling behavior.

- Kindly refer to the Test Cases file for a detailed steps above all related scenarios From TC – 10 to 62

Deferred verification implementation:

- A deferred verification process is performed after domain creation, based on the configured verification interval (60 or 120 seconds). The process verifies whether the domain has been successfully created and whether the expected peer is correctly linked.
- If the expected peer is not linked, the deferred verification process automatically attempts to repair the configuration and logs the details of the repair operation.
- The logs below show the deferred verification process and the corresponding verification/repair activities:
  sokhda180.com Cloudflare Secondary deferred verification. Classification: DeferredVerificationVerified. OperationId: 6b08a9fc-1bdd-4270-afa8-f8b2f0518755. Status: Verified. Attempt: 1. RepairPerformed: False. Message: The expected peers were present, force AXFR completed and the fresh verification succeeded.
- If you want to check the auto repair related scenario performed this Test Case steps: TC: 69

Automated Web Job Verification

- The automated WebJob periodically checks Secondary Zones that have recently been created or are awaiting verification.
- The WebJob validates the required Cloudflare and PowerDNS configurations and identifies zones that have not reached the expected state.

- This provides an additional layer of verification beyond the synchronous domain creation process and ensures that transient external-system delays or configuration issues are detected after the initial operation.
- All the logs related to the Web Job & Safety Sweep will be logged in **tblWebJobCloudflareSecondaryVerification** [Note: `All Log records older than 90 days are deleted by webjob`]

48-hour SafetySweep

- The 48-hour SafetySweep is a backup check. It is not redundant with the every-minute pending verification.
- The difference is:
- **Deferred verification:** Processes a `PendingVerification` operation after a create/recreate. Once it becomes `Verified`, the minute-by-minute processor will not normally check that operation again.
- **48-hour SafetySweep:** Once daily, it creates new `SafetySweep` operations for recently created zones and rechecks them—even if their original Provision operation already became `Verified`.
- Your test currently proves the first path:
  - Create domain → `PendingVerification`.
  - Delete peer before verification completes.
  - Every-minute WebJob claims that pending operation.
  - WebJob discovers the missing peer and repairs it.
  - Operation becomes `Verified`.
- The SafetySweep protects against this different timeline:
  - Create domain.
  - Deferred WebJob verifies it successfully → `Verified`.
  - Cloudflare asynchronously removes the peer later.
  - There is now no pending operation for the minute WebJob to claim.
  - Daily SafetySweep selects the recently created zone.
  - It creates a new `SafetySweep` operation.
  - The normal deferred processor claims that operation and repairs the peer.
- This is important because the original production race can occur after the initial request and potentially after the first successful verification. The sweep provides a second independent opportunity to discover delayed peer loss.

- The limitation is intentional: it covers zones created during the configured lookback—currently 48 hours. It is designed to catch delayed damage associated with provisioning/recreation, not arbitrary peer removal on every historical Cloudflare zone.
- If you want to check the auto repair by Safety Sweep related scenario performed this Test Case steps: TC: 70

Master Domain Creation with Secondary DNS Disabled

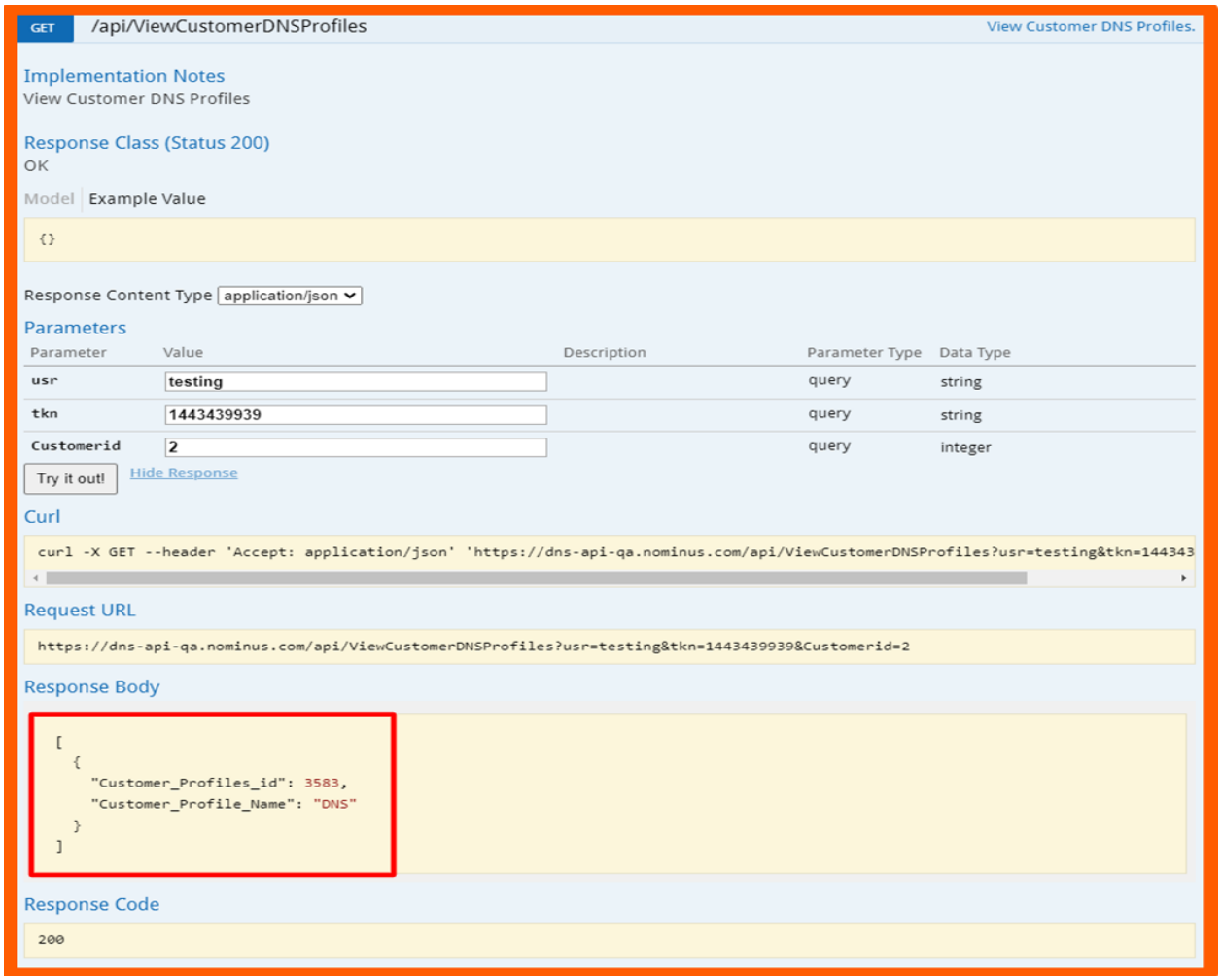
- The system also supports creation of a domain without Secondary DNS functionality. In this scenario, the domain is created as a **Native zone** in PowerDNS and no Cloudflare Secondary DNS configuration is provisioned. The system must ensure that Secondary DNS-specific operations, such as Cloudflare zone creation, Incoming Secondary configuration, peer creation, or AXFR validation, are not triggered when the Secondary option is disabled. This provides a clear separation between standard Native domain creation and Master/Secondary DNS domain creation. If Secondary DNS is enabled later for the existing domain, the system can initiate the required Secondary DNS configuration as a separate operation.

SWAGGER.

Customer Profiles

1. **Get /api/Viewcustomerallprofiles**

- It will list all the domains which are created.
- Following data is required to pass…
    - **usr:** testing
    - **tkn:** 1443439939
    - **customerid:** 2



2. Get /api/ViewCustomerProfileRecords

- Records associated with a given Customer ID and profile id.
- Pass parameters user, token, customer id, profileId
  - **usr:** testing
  - **tkn:** 1443439939
  - **customer id:-** 50
  - **Profile id:-** 3802

CustomerProfiles

Show/Hide | List Operations | Expand Operations

GET /api/ViewCustomerDNSProfiles — View Customer DNS Profiles.

GET /api/ViewCustomerDNSProfileRecords — View Customer DNS Profile Records.

Implementation Notes

View Customer DNS Profile Records

Response Class (Status 200)

OK

Model | Example Value

```
{}
```

Response Content Type application/json

Parameters

Parameter	Value	Description	Parameter Type	Data Type
usr	testing		query	string
tkn	1443439939		query	string
CustomerId	50		query	long
ProfileId	3802		query	integer

Try it out! Hide Response

Curl

```
curl -X GET --header 'Accept: application/json' 'https://dns-api-qa.nominus.com/api/ViewCustomerDNSProfileRecords?usr=testing&tkn=
```

Request URL

```
https://dns-api-qa.nominus.com/api/ViewCustomerDNSProfileRecords?usr=testing&tkn=1443439939&CustomerId=50&ProfileId=3802
```

Response Body

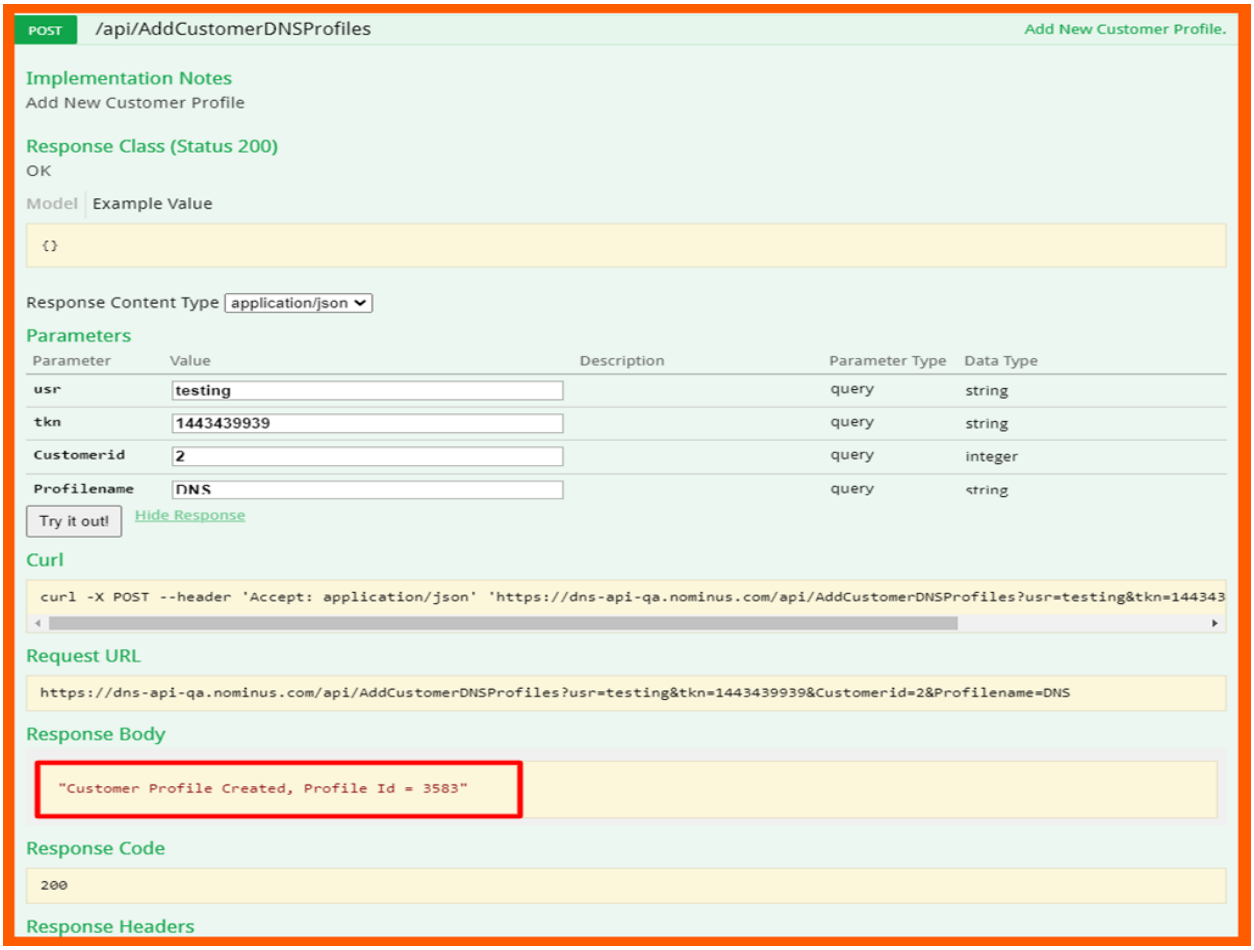

```
[
  {
    "Type": "A",
    "Name": "",
    "Content": "2.2.2.2",
    "TTL": "3600",
    "IsDisabled": true
  },
  {
    "Type": "A",
    "Name": "",
    "Content": "2.23.2.2",
    "TTL": "3600",
    "IsDisabled": true
  },
  {
    "Type": "AAAA",
    "Name": "",
    "Content": "3000:db8:3333:4444:5555:6666:7777:888",
```


Response Code

```
200
```

3. Post/api/AddCustomerDNSProfile

- It will be used to create a new profile.
- Following data is required to pass…
- **profilename:** Name of the profile which you want to create.
- **usr:** testing
- **tkn:** 1443439939
- **customerid:** 2
- **Profile name:** DNS



4. Post/api/AddCustomerDNSProfileRecord

- This API is used to Add a Domain Record
- Domain Name: nikee.com
- Usr: testing
- Tkn: 1443439939
- Customerid:-2
- Profileid:-3583
- Here, users get (204 no content) response code Add a profile record.

POST /api/AddCustomerDNSProfileRecords — Add New Customer Profile Records.

Implementation Notes

Add New Customer Profile Records

Response Class (Status 200)

OK

Model | Example Value

```
{}
```

Response Content Type application/json

Parameters

Parameter	Value	Description	Parameter Type	Data Type
usr	testing		query	string
tkn	1443439939		query	string
Customerid	2		query	integer
profileid	3583		query	integer
model	{ "AddRecord": { "Type": "A", "Name": "A", "Content": "1.1.1.1", "TTL": "8000", "IsDisabled": true } }		body	Model | Example Value

Parameter content type: application/json

Try it out! Hide Response

Curl

```
curl -X POST --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{ \
   "AddRecord": { \
     "Type": "A", \
     "Name": "A", \
     "Content": "1.1.1.1", \
     "TTL": "8000", \
     "IsDisabled": true \
   } \
 }' 'https://dns-api-qa.nominus.com/api/AddCustomerDNSProfileRecords?usr=testing&tkn=1443439939&Customerid=2&profileid=3583'
```

Request URL

```
https://dns-api-qa.nominus.com/api/AddCustomerDNSProfileRecords?usr=testing&tkn=1443439939&Customerid=2&profileid=3583
```

Response Body

```
"Customer Profile Record Added Successfull"
```

Response Code

```
200
```

5. Patch /api/UpdateCustomerDNSProfile

- This API is used to Update Customer DNS Profile records.
- Usr: testing
- Tkn: 1443439939
- Customerid:-2
- Profileid:-3583

PATCH /api/UpdateCustomerDNSProfile — Update Customer DNS Profile Record.

Implementation Notes

Update Customer DNS Profile Record

Response Class (Status 200)

OK

Model | Example Value

```
{}
```

Response Content Type: application/json

Parameters

Parameter	Value	Description	Parameter Type	Data Type
usr	testing		query	string
tkn	1443439939		query	string
Customerid	2		query	integer
profileid	3583		query	integer
model	{ "Newrecord": { "Type": "A", "Name": "A", "Content": "2.2.2.2", "TTL": "2500", "IsDisabled": true }, "Oldrecord": { "Type": "A", "Name": "A", "Content": "1.1.1.1", "TTL": "3600", "IsDisabled": true } }		body	Model | Example Value

Model | Example Value

```
{
  "Newrecord": {
    "Type": "string",
    "Name": "string",
    "Content": "string",
    "TTL": "string",
    "IsDisabled": true
  },
  "Oldrecord": {
    "Type": "string",
    "Name": "string",
```

Parameter content type: application/json

Try it out! Hide Response

Curl

```
curl -X PATCH --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{ \
   "Newrecord": { \
     "Type": "A", \
     "Name": "A", \
     "Content": "2.2.2.2", \
     "TTL": "2500", \
     "IsDisabled": true \
   }, \
   "Oldrecord": { \
     "Type": "A", \
     "Name": "A", \
     "Content": "1.1.1.1", \
     "TTL": "3600", \
     "IsDisabled": true \
   } \
 }' 'https://dns-api-qa.nominus.com/api/UpdateCustomerDNSProfile?usr=testing&tkn=1443439939&Customerid=2&profileid=3583'
```

Request URL

```
https://dns-api-qa.nominus.com/api/UpdateCustomerDNSProfile?usr=testing&tkn=1443439939&Customerid=2&profileid=3583
```

Response Body

```
"Customer Record Updated Succesfully"
```

Response Code

```
200
```

Response Headers

6. patch /api/UpdateCustomerDNSProfileName

- To update the profile Name associated with the given customer id and profile ID.
- Pass the parameters user, token, customer id and profileid and model.
- happy profile name is now changed to read using API as below.

PATCH /api/UpdateCustomerDNSProfileName — Update Customer DNS Profile Name.

Implementation Notes

Update Customer DNS Profile Name

Response Class (Status 200)

OK

Model | Example Value

```
{}
```

Response Content Type: application/json

Parameters

Parameter	Value	Description	Parameter Type	Data Type
usr	testing		query	string
tkn	1443439939		query	string
CustomerId	50		query	long
ProfileId	3802		query	integer
model	{ "OldProfile": { "Name": "happy" }, "NewProfile": { "Name": "read" } }		body	Model | Example Value

Model | Example Value

```
{
  "OldProfile": {
    "Name": "string"
  },
  "NewProfile": {
    "Name": "string"
  }
}
```

Parameter content type: application/json

Try it out! Hide Response

Curl

```
curl -X PATCH --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{ \
   "OldProfile": { \
     "Name": "happy" \
   }, \
   "NewProfile": { \
     "Name": "read" \
   } \
 }' 'https://dns-api-qa.nominus.com/api/UpdateCustomerDNSProfileName?usr=testing&tkn=1443439939&CustomerId=50&ProfileId=3802'
```

Request URL

```
https://dns-api-qa.nominus.com/api/UpdateCustomerDNSProfileName?usr=testing&tkn=1443439939&CustomerId=50&ProfileId=3802
```

Response Body

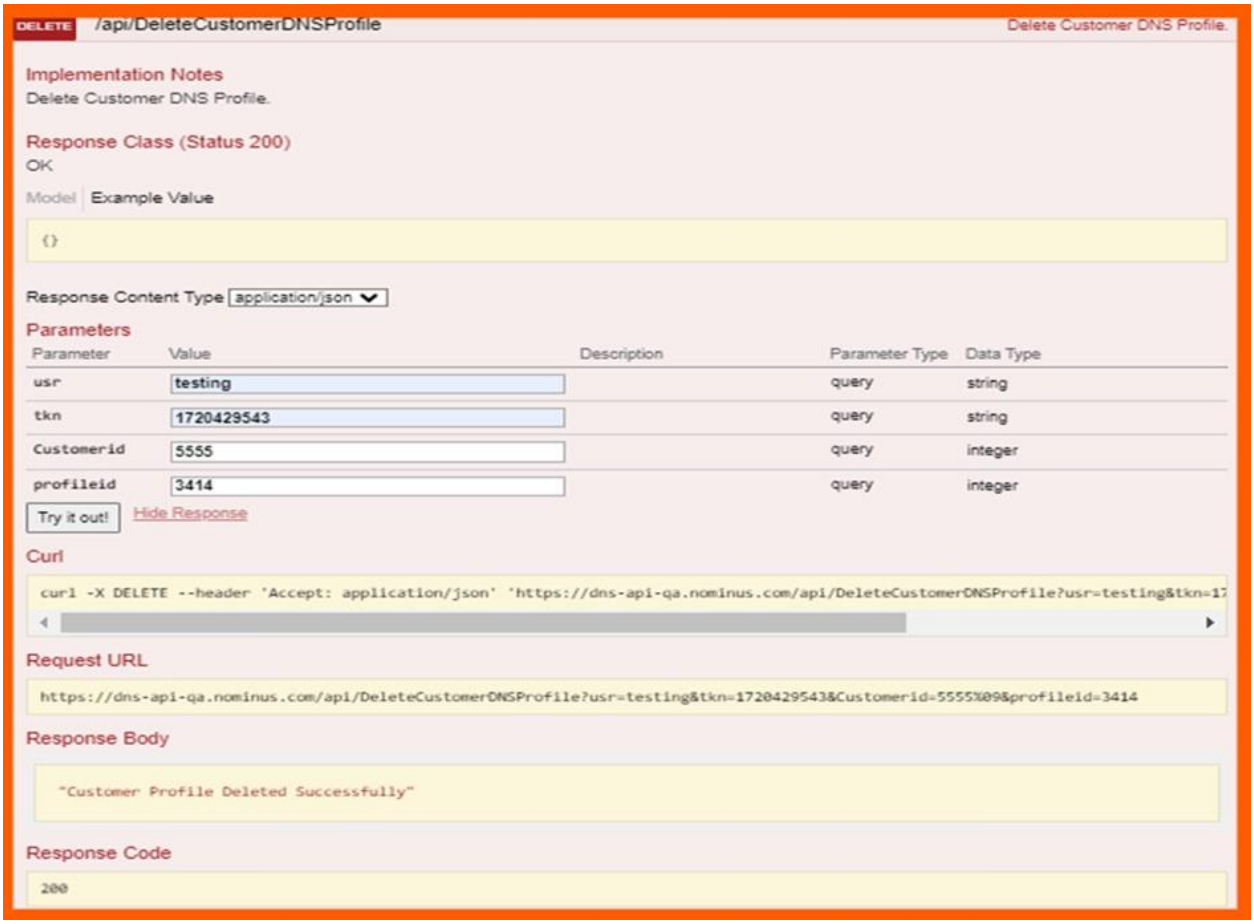
```
"Customer Profile Name Updated To read"
```

Response Code

```
200
```

Delete/api/DeleteCustomerDNSProfile

- This API is used to Delete Customer DNS Profile.
- Pass the parameters like user, token, customer id and profile id
- Case: -1 Correct parameters
- Customerid: -5555
- profiled: -3414



Delete/api/DeleteCustomerDNSProfile

- This API is used to Delete Customer DNS Profile records.
- Pass the parameters like user, token, customer id and profile id
- Case: -1 Correct parameters
- Customerid: -1
- profiled: -3820

DELETE /api/DeleteCustomerDNSProfileRecord — Delete Customer DNS Profile Record.

Implementation Notes

Delete Customer DNS Profile Record

Response Class (Status 200)

OK

Model | Example Value

```
{}
```

Response Content Type: application/json

Parameters

Parameter	Value	Description	Parameter Type	Data Type
usr	testing		query	string
tkn	1443439939		query	string
Customerid	1		query	long
Profileid	3820		query	integer
model	{ "DeleteRecord": { "Type": "AAAA", "Name": "AA", "Content": "2001:db8:3333:4444:5555:6666:7777:8888", "TTL": "1800", "IsDisabled": true } }		body	Model | Example Value

Example Value:

```
{
  "DeleteRecord": {
    "Type": "string",
    "Name": "string",
    "Content": "string",
    "TTL": "string",
    "IsDisabled": true
  }
}
```

Parameter content type: application/json

Try it out! Hide Response

Curl

```
curl -X DELETE --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{ \
   "DeleteRecord": { \
     "Type": "AAAA", \
     "Name": "AA", \
     "Content": "2001:db8:3333:4444:5555:6666:7777:8888", \
     "TTL": "1800", \
     "IsDisabled": true \
   } \
 }' 'https://dns-api-qa.nominus.com/api/DeleteCustomerDNSProfileRecord?usr=testing&tkn=1443439939&Customerid=1&Profileid=3820'
```

Request URL

```
https://dns-api-qa.nominus.com/api/DeleteCustomerDNSProfileRecord?usr=testing&tkn=1443439939&Customerid=1&Profileid=3820
```

Response Body

```
"Customer Profile Record Deleted Successfully"
```

Response Code

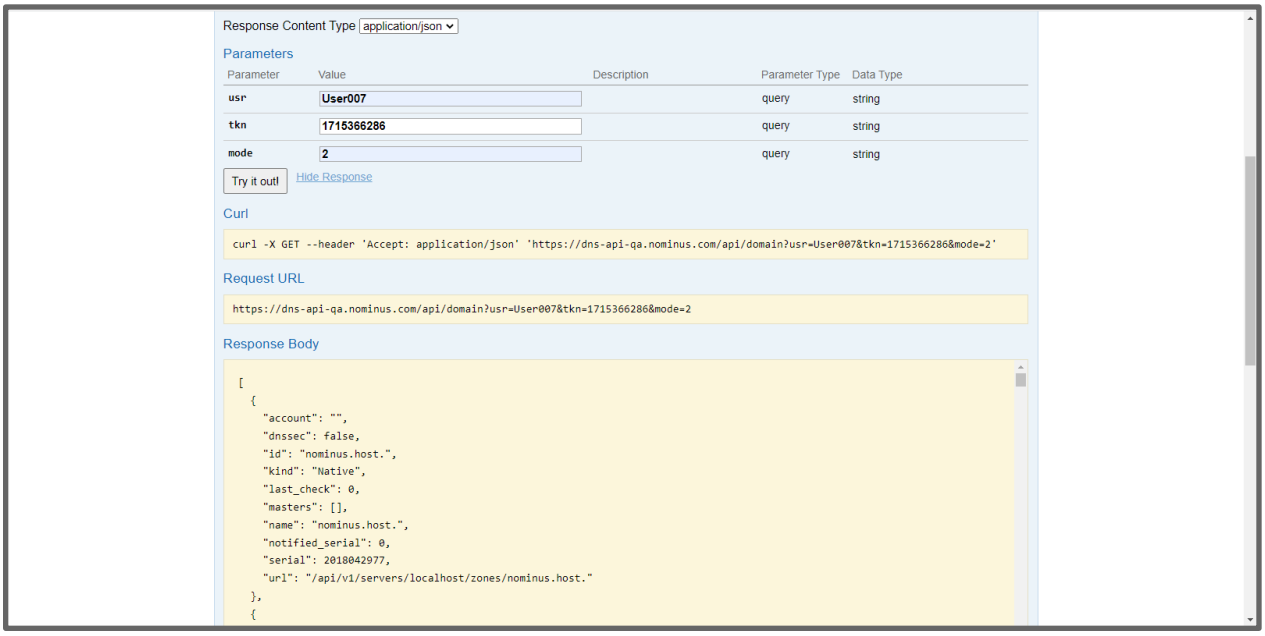
```
200
```

Response Headers

Domain

1) **Get - /api/domain - Get all domains:**

- It will list all the domains which are created.
- Following data is required to pass…
    - **usr:** User007
    - **tkn:** 1715366286
    - **mode:** 2



- Here, if a user gets 200 http response codes it will succeed.

2) **Post - /api/domain - add domain:**

- It will used to create new domain
- Following data is required to pass…
    - **Domain name:** Name of the domain which you want to create. i.e. ax.com
    - **usr:** User007
    - **tkn:** 1715366286
    - **mode:** 2
    - **RequestBody:**
      Native Zone Body example:
      {

```
  "kind": "Native",
   “masters": [],
   "name": "ax.com",
   "nameservers": [
      "ns1.example.net",
      "ns2.example.net"
   ],
   "records": [
      {
         "content": "ns.example.net. hostmaster.example.com. 1 1800 900
604800 86400",
         "disabled": false,
         "name": "example.net",
         "ttl": 86400,
         "type": "SOA"
      },
      {
         "content": "192.168.1.42",
         "disabled": false,
         "name": "www.example.net",
         "ttl": 3600,
         "type": "A"
      }
   ]
}
```

Slave Zone Body example:

```
{
   "kind": "Slave",
   "masters": ["200.224.32.176"],
   "name": "example.net",
   "nameservers": [
      "ns1.example.net",
      "ns2.example.net"
   ]
}
```

Master Zone Body example:

```
{
  "name": "te.com",
```

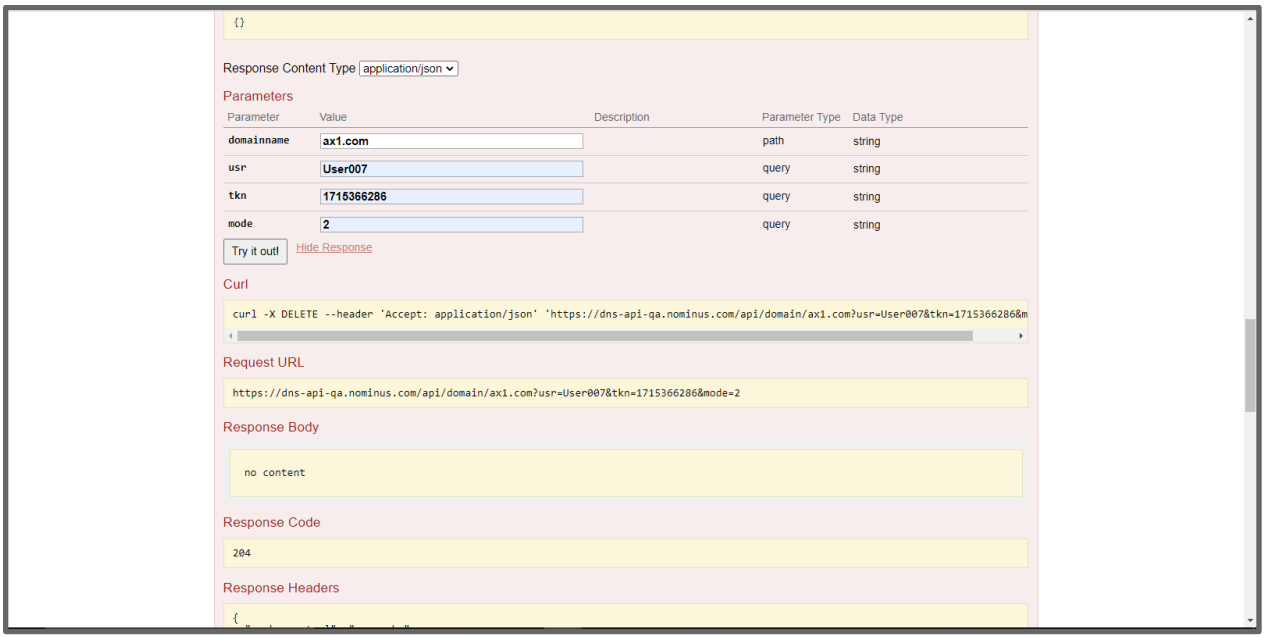
```
 "kind": "Master",
 "masters": [
  "5.5.5.5"
 ],
 "nameservers": [
  "ns02.nominus.host"
 ],
 "rrsets": [
  {
   "name": "abc.te.com.",
   "type": "A",
   "ttl": 3600,
   "changetype": "REPLACE",
   "records": [
    {
     "content": "5.3.6.5.",
     "disabled": false,
     "name": "null",
     "ttl": 0,
     "type": "null",
     "priority": 0
    }
   ]
  }
 ]
}
```


- **IsRequestBodySent:** Pass either true or false. Here, if you pass true then it is required to add proper format data in the RequestBody field and if you pass false then add {} or any data in the Requested Body field.

- Here, users get 201 response code if the domain is successfully created.

3) **Delete- /api/domain/{domain name} - Delete domain:**

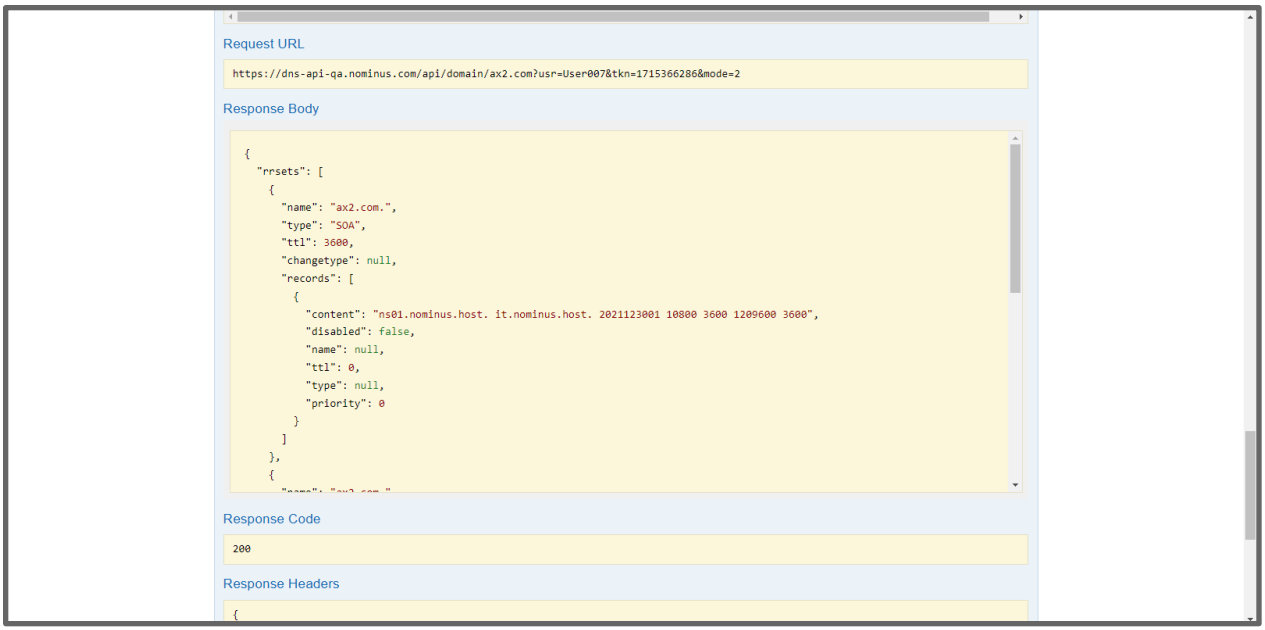
- It will delete the domain by passing the domain name which you want to delete.
    - **Domain name:** Name of the domain which you want to delete. i.e. ax1.com
    - **usr:** User007
    - **tkn:** 1715366286
    - **mode:** 2



Here, the user will get the 204 response code for successful deletion.

4) **Get- /api/domain/{domain name} - Get domain detail by name:**

- Here, User will get the details of the domain by passing the name.
- Here, if the entered domain is not found, the user will get the message like `"{\"error\": \"Not Found\"}"` and http status code 404.
- And if the entered domain is found, the user will get the response code 200.
    - **Domain name:** Name of the domain for which you want to get the data. i.e. test00.com
    - **usr:** User007
    - **tkn:** 1715366286
    - **mode:** 2



5) **Patch- /api/domain/{domain name} - Update/Delete DNS Record:**

- This is the old API now it is replaced with the API of the Delete Domain, Update DNS Record, Add a domain record.
- This API is used to Update/Delete DNS records.
- Domain Name: nike.com
- Usr: shah
- Tkn: 1279000935
- Mode: 2
- RequestBody :

```
{
          "rrsets":
[
 {
  "name": "blackshoes.nike.com.",
  "type": "A",
  "ttl": 1800,
      "changetype": "REPLACE",
  "records":
 [
   {
      "content": "2.2.2.2",
      "disabled": false,
      "name": null,
      "ttl": 0,
      "type": null,
      "priority": 0
   }
```

]

}

]

}

- And if the domain record is updated/delete , the user will get the response code 204.

Curl

```
curl -X PATCH --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{ \
     "rrsets":  \
    [ \
       { \
         "name": "jeenal.nike.com.", \
         "type": "A", \
         "ttl": 1800, \
         "changetype": "REPLACE", \
         "records":  \
        [ \
           { \
             "content": "2.2.2.2", \
             "disabled": false, \
             "name": null, \
             "ttl": 0, \
             "type": null, \
             "priority": 0 \
           } \
        ] \
      } \
   ] \
 } \
 ' 'https://dns-api-dev.nominus.com/api/domain/nike.com?usr=shah&tkn=1279000935&mode=2'
```

Request URL

```
https://dns-api-dev.nominus.com/api/domain/nike.com?usr=shah&tkn=1279000935&mode=2
```

Response Body

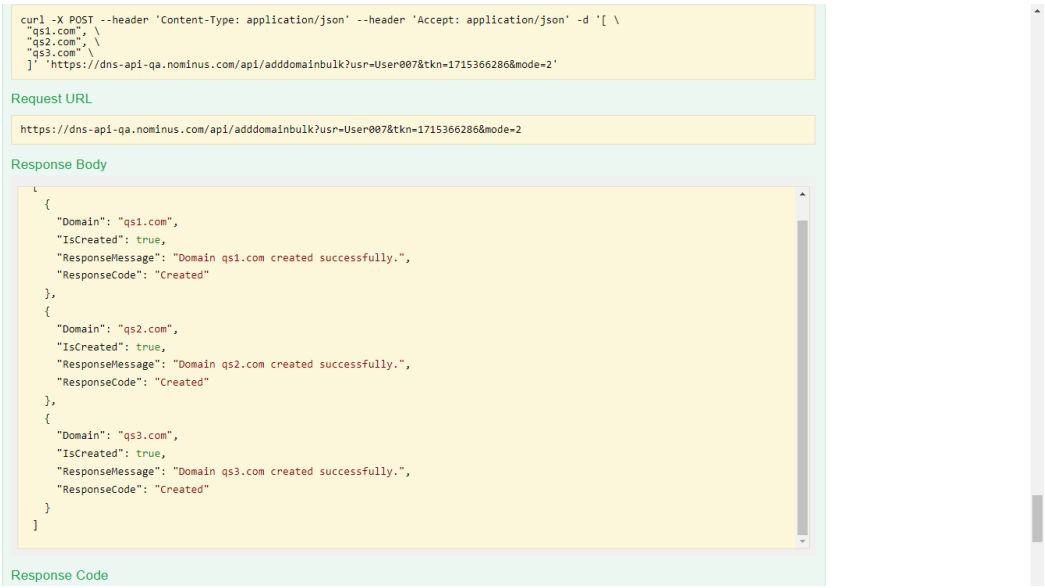
```
no content
```

Response Code

```
204
```

6) **Post- /api/add domain bulk - Add domain in bulk:**

- This API is used to add domains in the bulk.
- Domain list : [
  "Qs1.com",
  "Qs2.com",
  "Qs3.com"]
- User: User007
- tkn : 1715366286
- Mode: 2
- And if the domain entered in bulk, the user will get the response code 201.



```
curl -X POST --header 'Content-Type: application/json' --header 'Accept: application/json' -d '[ \
 "qs1.com", \
 "qs2.com", \
 "qs3.com" \
 ]' 'https://dns-api-qa.nominus.com/api/adddomainbulk?usr=User007&tkn=1715366286&mode=2'
```

Request URL

```
https://dns-api-qa.nominus.com/api/adddomainbulk?usr=User007&tkn=1715366286&mode=2
```

Response Body

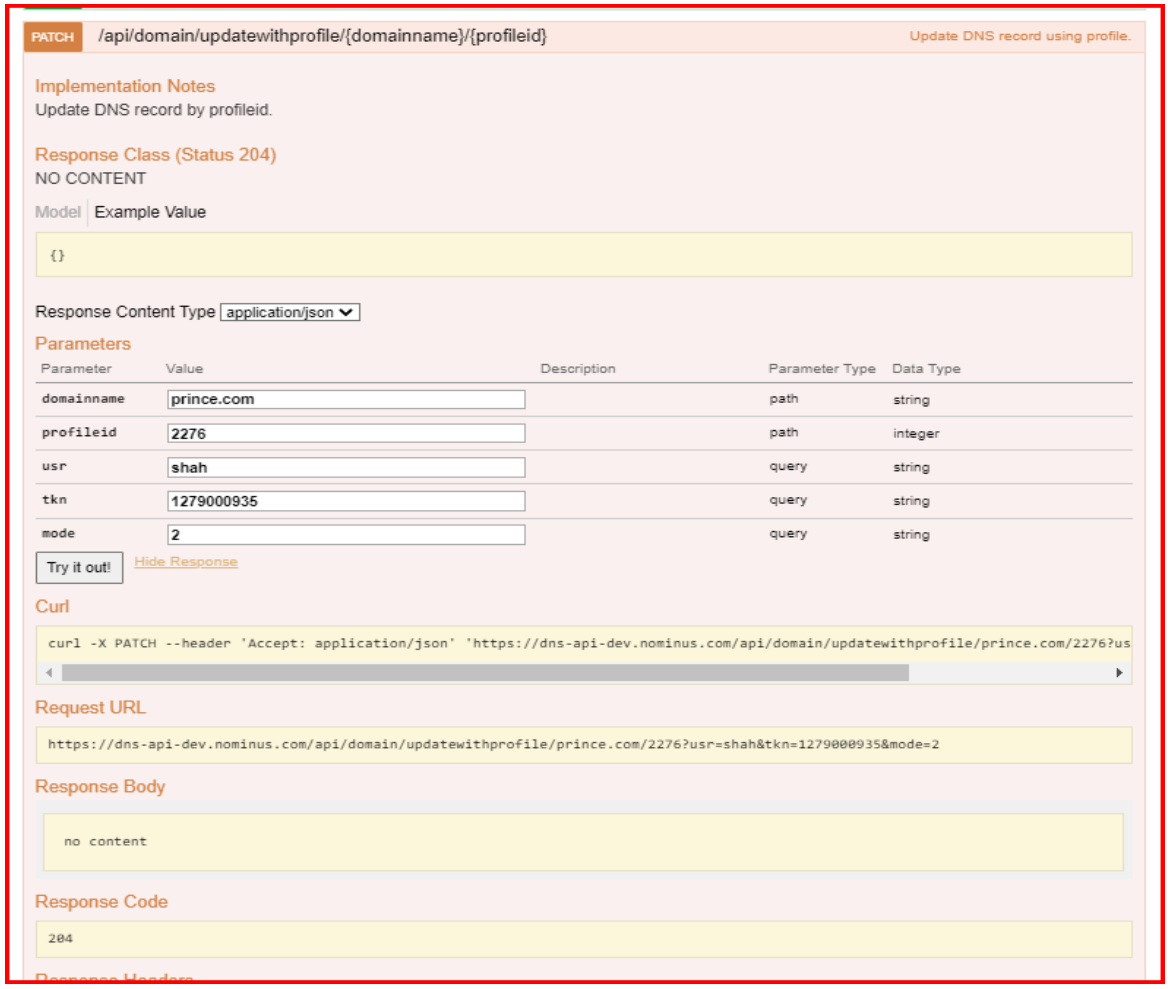

```
[
  {
    "Domain": "qs1.com",
    "IsCreated": true,
    "ResponseMessage": "Domain qs1.com created successfully.",
    "ResponseCode": "Created"
  },
  {
    "Domain": "qs2.com",
    "IsCreated": true,
    "ResponseMessage": "Domain qs2.com created successfully.",
    "ResponseCode": "Created"
  },
  {
    "Domain": "qs3.com",
    "IsCreated": true,
    "ResponseMessage": "Domain qs3.com created successfully.",
    "ResponseCode": "Created"
  }
]
```


Response Code

7) **PATCH/api/domain/updatewithprofile/{domainname}/{profileid}:Update DNS record using profile.**

- This is the old API now it is replaced with the API of the Delete Domain, Update DNS Record, Add a domain record.
- This api is used to Update DNS Record Using profile.
- Domain Name : Prince.com
- Profile Id:2276
- Usr : shah
- Tkn : 1279000935
- Mode : 2

- Here, users get (204 No content) response code Update DNS Record using Profile.



DomainRecords :

1) **Delete - /api/domainrecord/{domainname} - Delete Domain Record**
    - This API is used to Delete Domain record.
    - Domain Name: nikee.com
    - Usr: shah
    - Tkn: 1279000935
    - Mode: 2

- Here, users get (204 No content) response code Delete Domain Record.

Curl

```
curl -X DELETE --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{ \
   "DeleteBody": { \
     "Oldrrset": { \
       "name": "jeenal.nikee.com.", \
       "type": "A", \
       "ttl": 1800, \
       "changetype": "REPLACE", \
       "records": [ \
         { \
           "content": "9.9.9.9", \
           "disabled": false, \
           "name": "null", \
           "ttl": 0, \
           "type": "null", \
           "priority": 0 \
         } \
       ] \
     } \
   } \
 } \
 ' 'https://dns-api-dev.nominus.com/api/domainrecord/nikee.com?usr=shah&tkn=1279000935&mode=2'
```

Request URL

```
https://dns-api-dev.nominus.com/api/domainrecord/nikee.com?usr=shah&tkn=1279000935&mode=2
```

Response Body

```
no content
```

Response Code

```
204
```

2) Patch - /api/domainrecord/{domainname} - Update DNS Record

- This API is used to Update DNS Record
- Domain Name: nikee.com
- Usr: shah
- Tkn: 1279000935
- Mode: 2

- Here, user get (204 no content) response code update DNS record

```
curl -X PATCH --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{
   "EditBody": { \
     "Newrrset": { \
       "name": "blackshoes.nikee.com.", \
       "type": "A", \
       "ttl": 1800, \
       "changetype": "REPLACE", \
       "records": [ \
         { \
           "content": "9.9.9.0", \
           "disabled": false, \
           "name": "null", \
           "ttl": 0, \
           "type": "null", \
           "priority": 0 \
         } \
       ] \
     }, \
     "Oldrrset": { \
       "name": "blackshoes.nikee.com.", \
       "type": "A", \
       "ttl": 1800, \
       "changetype": "REPLACE", \
       "records": [ \
         { \
           "content": "9.9.9.9", \
           "disabled": false, \
           "name": "null", \
           "ttl": 0, \
           "type": "null", \
           "priority": 0 \
         } \
```

Request URL

```
https://dns-api-dev.nominus.com/api/domainrecord/nikee.com?usr=shah&tkn=1279000935&mode=2
```

Response Body

```
no content
```

Response Code

```
204
```

3) **Post - /api/domainrecord/{domainname} - Add a domain record**
   - This API is used to Add a Domain Record
   - Domain Name: nikee.com
   - Usr: shah
   - Tkn: 1279000935
   - Mode: 2

- Here, user get (204 no content) response code Add a domain record.

Curl

```
curl -X POST --header 'Content-Type: application/json' --header 'Accept: application/json' -d '{ \
   "AddBody": { \
     "Newrrset": { \
       "name": "prince.nikee.com.", \
       "type": "A", \
       "ttl": 1800, \
       "changetype": "REPLACE", \
       "records": [ \
         { \
           "content": "1.1.1.1", \
           "disabled": false, \
           "name": "null", \
           "ttl": 0, \
           "type": "null", \
           "priority": 0 \
         } \
       ] \
     } \
   } \
 } \
  \
  \
 ' 'https://dns-api-dev.nominus.com/api/domainrecord/nikee.com?usr=shah&tkn=1279000935&mode=2'
```

Request URL

```
https://dns-api-dev.nominus.com/api/domainrecord/nikee.com?usr=shah&tkn=1279000935&mode=2
```

Response Body

```
no content
```

Response Code

```
204
```

Response Headers

```
{
  "cache-control": "no-cache",
  "content-type": "application/json; charset=utf-8",
  "date": "Fri, 25 Nov 2022 07:15:11 GMT",
  "expires": "-1",
  "pragma": "no-cache",
  "server": "Microsoft-IIS/10.0",
  "x-aspnet-version": "4.0.30319",
  "x-powered-by": "ASP.NET"
}
```

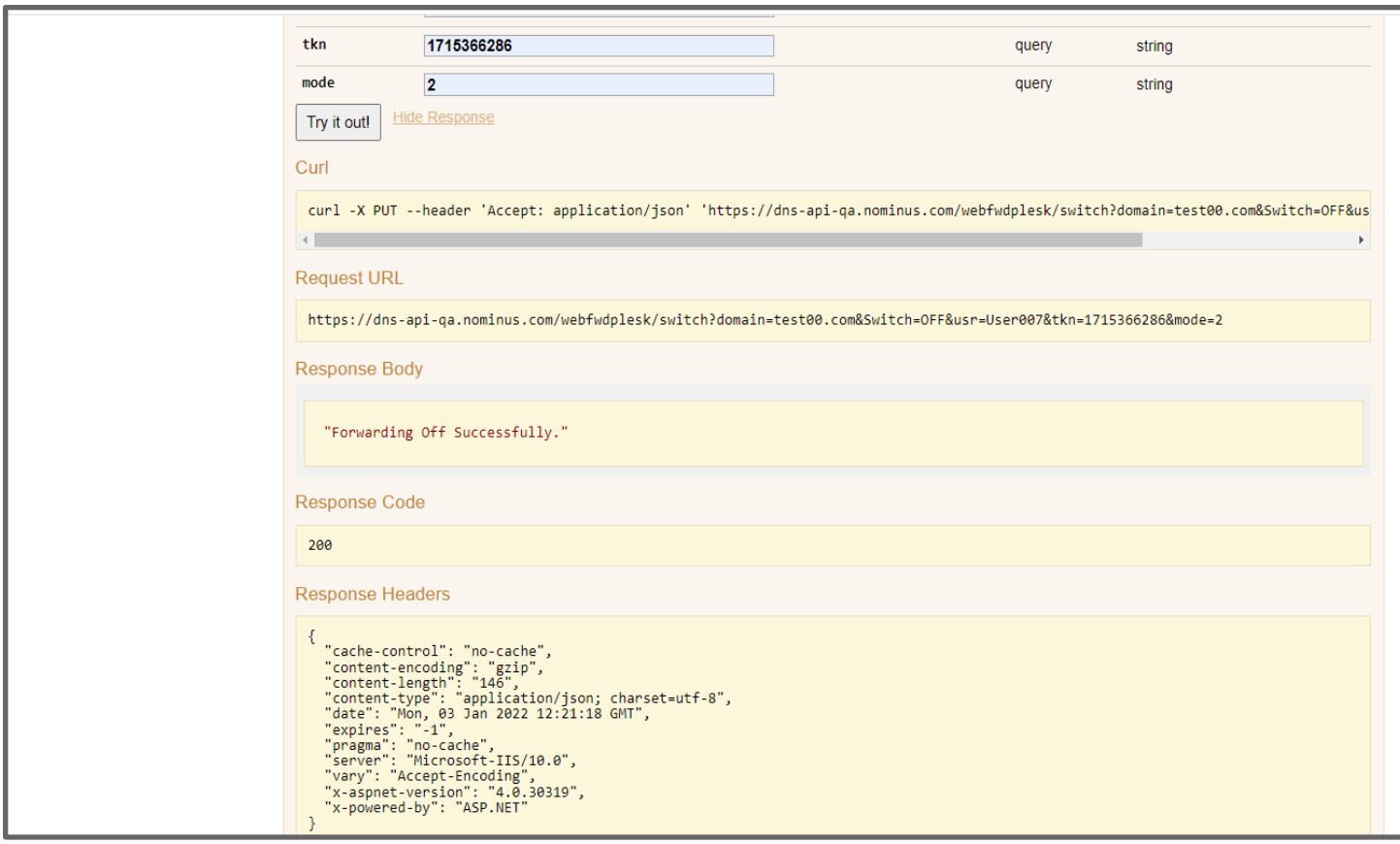
Forwarding :

1) **Put - /webfwdplesk/switch - Enable/Disable web forwarding (PLESK)**

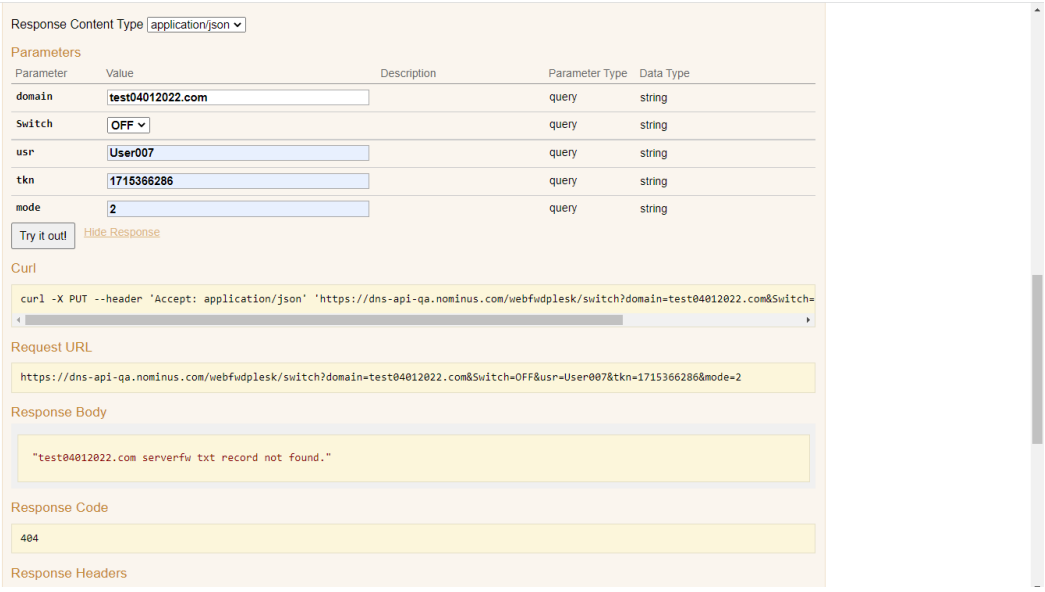
- This API is used to do on or off the web forwarding switch.
    - **Domain name:** Name of the domain for which you want to get the data. i.e. test00.com
    - **Switch:** ON/OFF
    - **usr:** User007
    - **tkn:** 1715366286
    - **mode:** 2
- Here , once doing switch ON user will get the response code 200 and the message `"Forwarding On Successfully."`



- Here , once doing switch OFF user will get the response code 200 and message `"Forwarding Off Successfully."`



- Here, if user do OFF without doing ON any time than usr will get the error for not found txt record, response code 404 and message like `"test04012022.com serverfw txt record not found."`

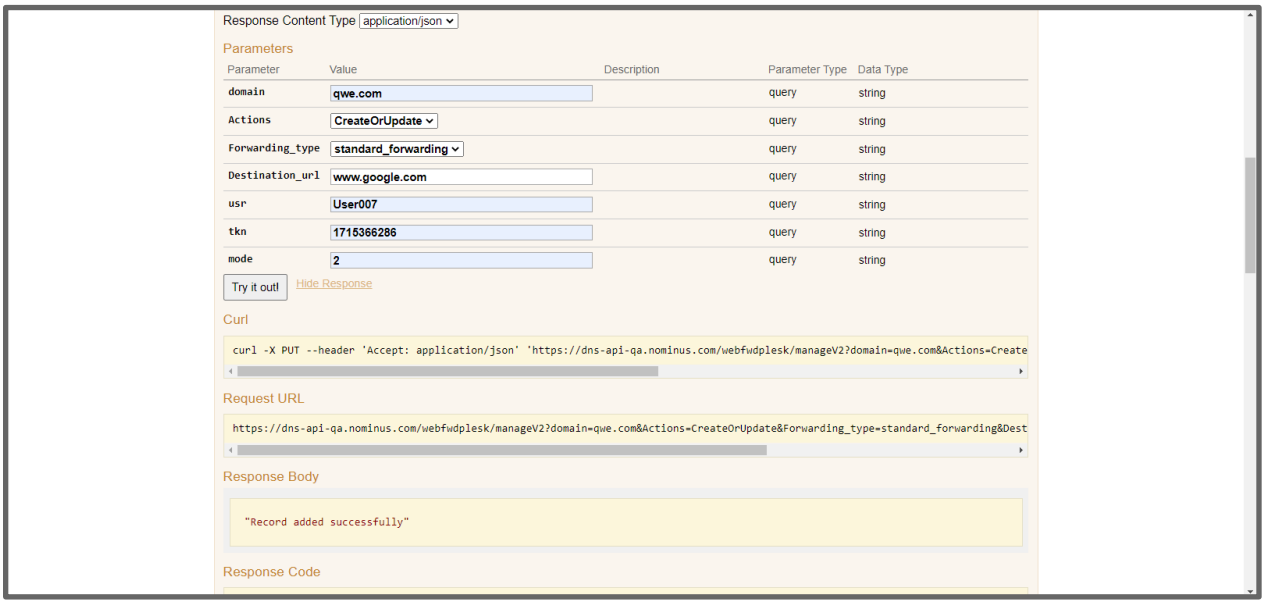


2) Put - /webfwdplesk/create - Create/Update Web forwarding record for domain (PLESK):

- This API is older version API and it is now replaced with the Post - webfwdplesk/manageV2

3) Put - webfwdplesk/manageV2 - Create/Update Web forwarding record for domain (PLESK):

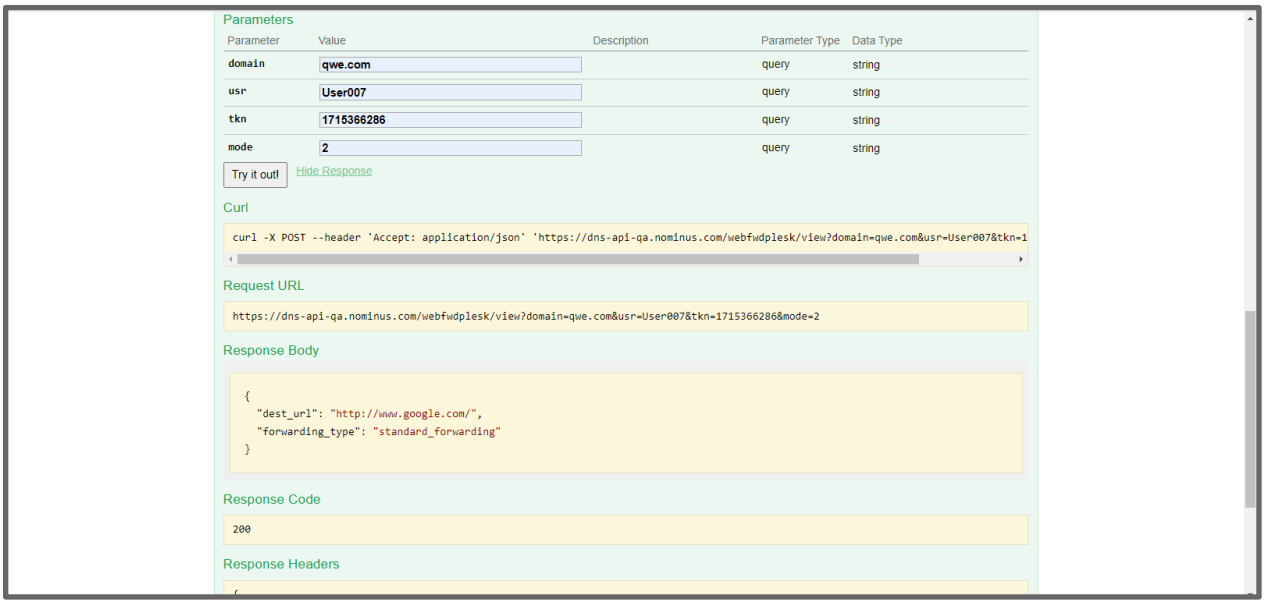
- This API is used to create/update record for web forwarding in plesk
- It also provides facility Delete i.e. None the record in plesk.
    - **Domain :** Name of the domain i.e. qwe.com
    - **Actions :** Select any one option from the drop down i.e. CreateOrUpdate or Delete
    - **Forwarding_type :** standard_forwarding, frame_forwarding, none
    - **Destination_url :** any url on which you want to do forwarding
    - **Usr :** User007
    - **Tkn :** 1715366286
    - **Mode :** 2



4) Post - webfwdplesk/view - View Web forwarding record for domain (PLESK)

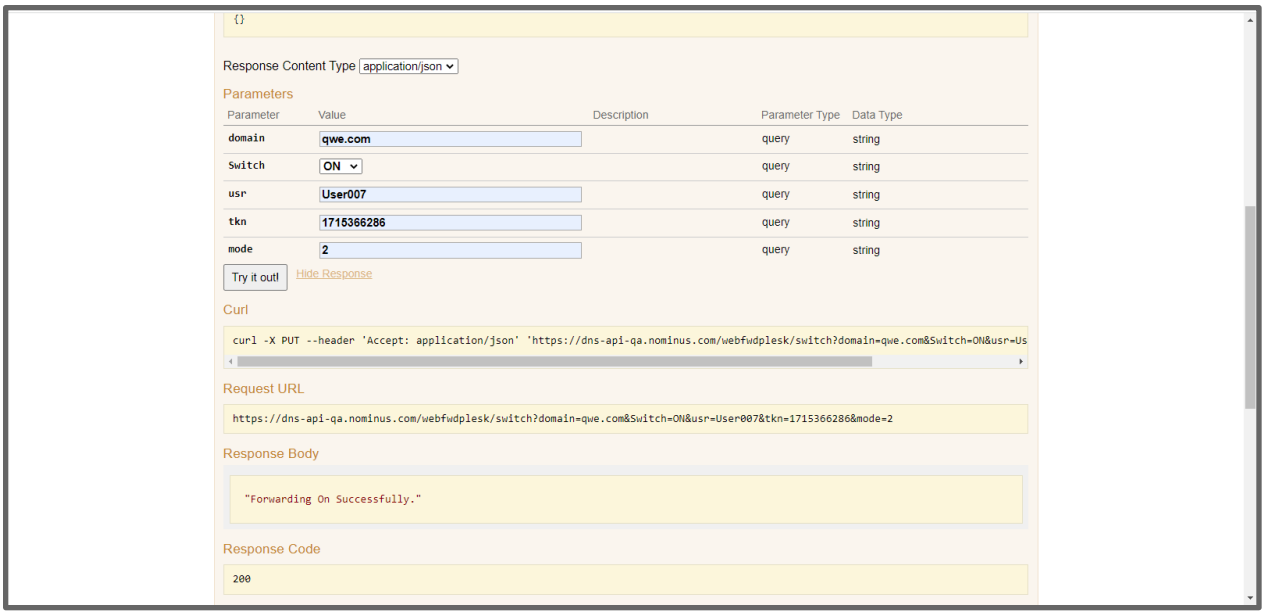
- This API is used to view the details of the web forwarding record.
    - **Domain :** Name of the domain i.e. qwe.com

- **Usr :** User007
- **Tkn :** 1715366286
- **Mode :** 2

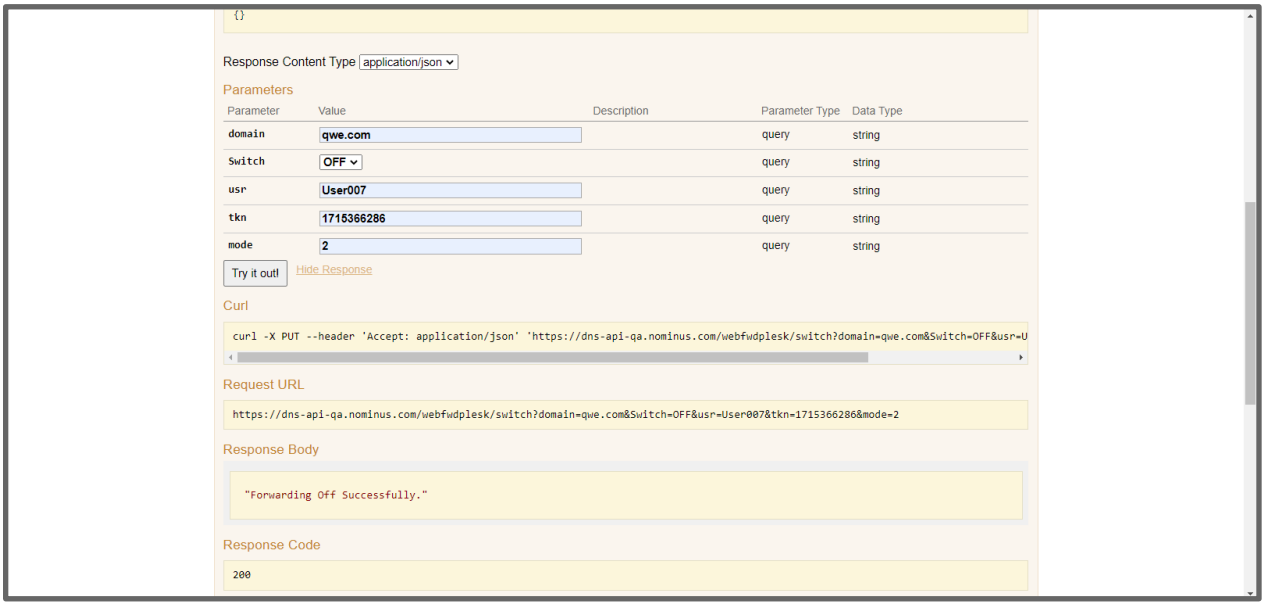


5) Put - emailfwdplesk/switch - Enabled/Disabled Email forwarding (PLESK)

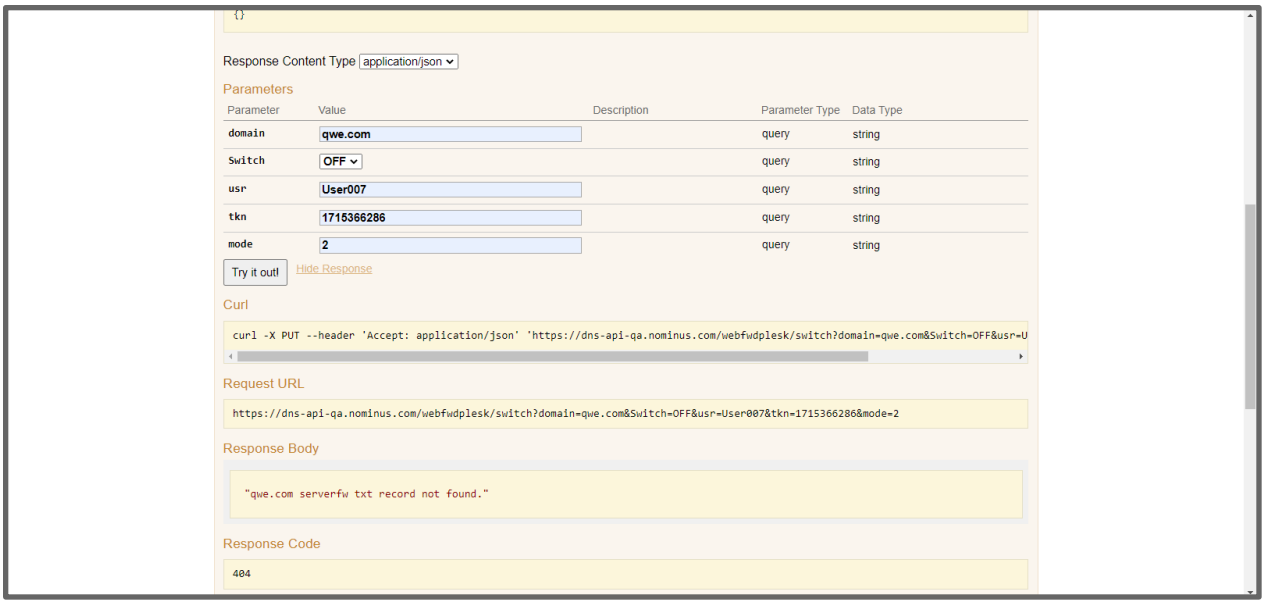
- This API is used to do on or off the email forwarding switch.
  - **Domain name:** Name of the domain for which you want to get the data. i.e. test00.com
  - **Switch:** ON/OFF
  - **usr:** User007
  - **tkn:** 1715366286
  - **mode:** 2
- Here , once doing switch ON user will get the response code 200 and message `"Forwarding On Successfully."`



- Here , once doing switch OFF user will get the response code 200 and message `"Forwarding Off Successfully."`



- Here, if user do OFF without doing ON any time than usr will get the error for not found txt record, response code 404 and message like `"test04012022.com serverfw txt record not found."`

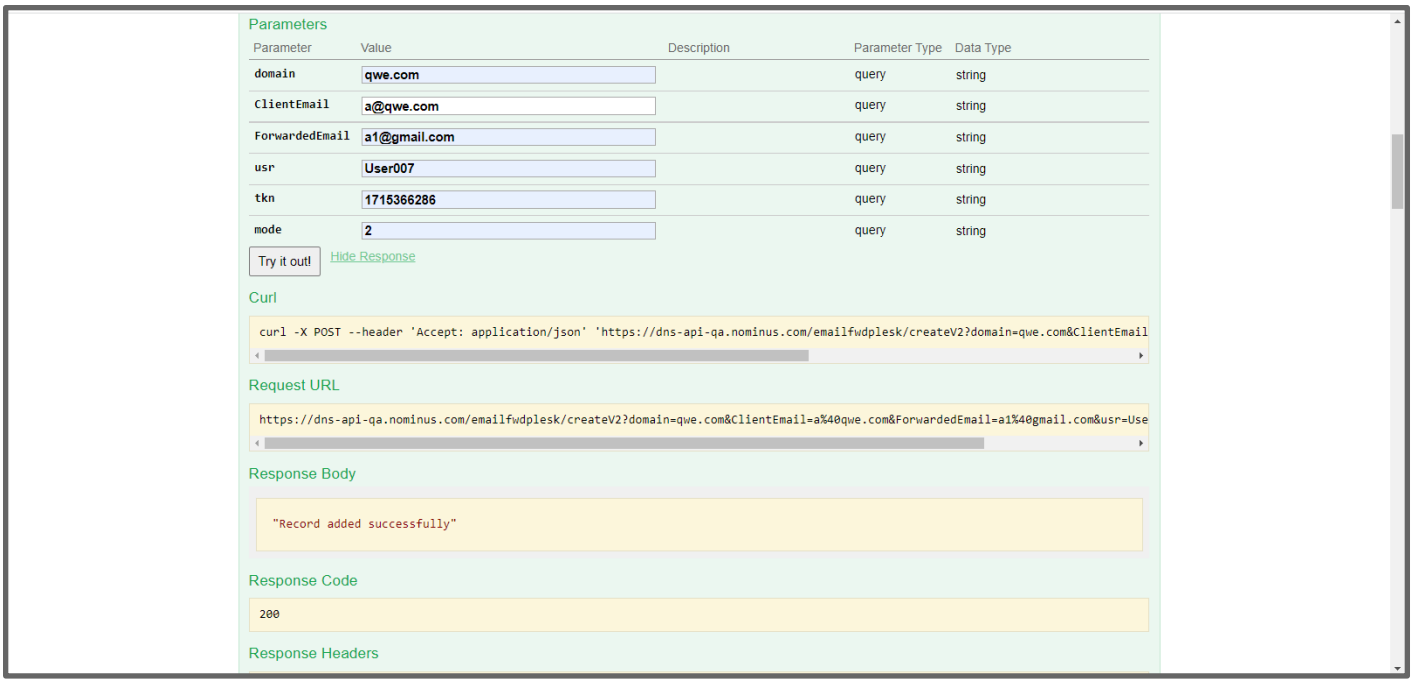


6) Put - emailfwdplesk/create - Create Email forwarding record for domain (PLESK)

- This API is older version API and it is now replaced with the Post - emailfwdplesk/createV2

7) Post - emailfwdplesk/createV2 - Create Email forwarding record for domain (PLESK):

- This API is used to add email address for the forwarding.
    - **Domain :** Name of the domain
    - **Clientemail :** Client mail address
    - **Forwardedemail :** Email address for forwarding the mail.
    - **Usr:** User007
    - **Tkn:** 1715366286
    - **Mode: 2**

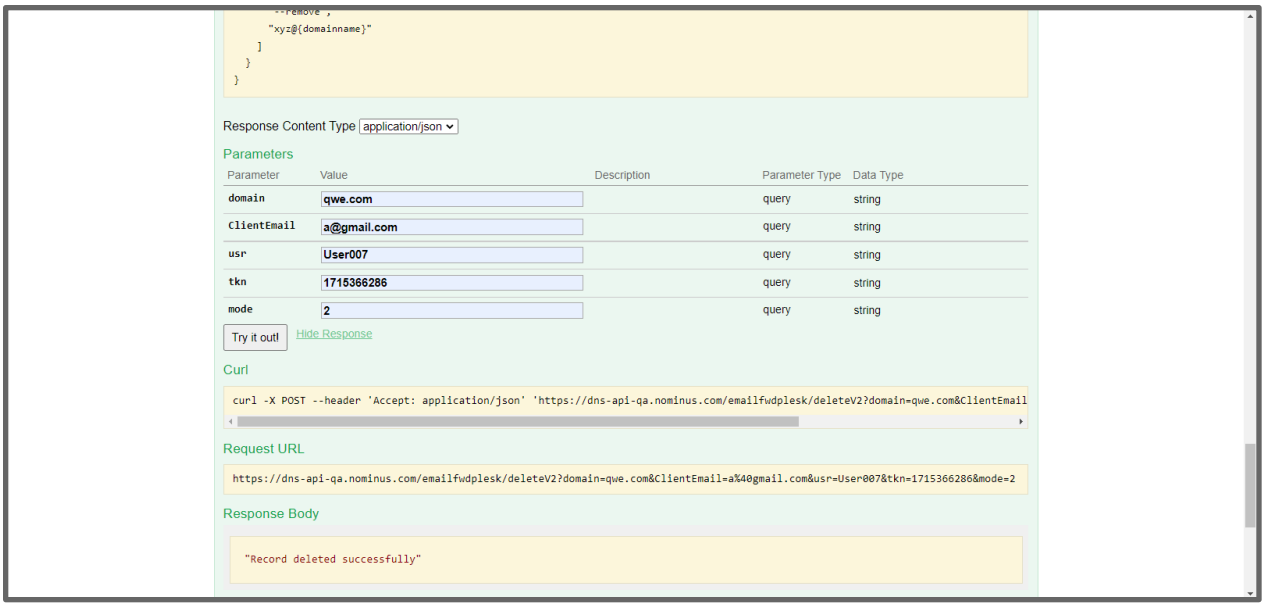


8) **Post - emailfwdplesk/delete - Remove email forwarding record for domain (PLESK):**

- This API is older version API and it is now replaced with the Post - emailfwdplesk/deleteV2

9) **Post - emailfwdplesk/deleteV2 - Remove email forwarding record for domain (PLESK):**

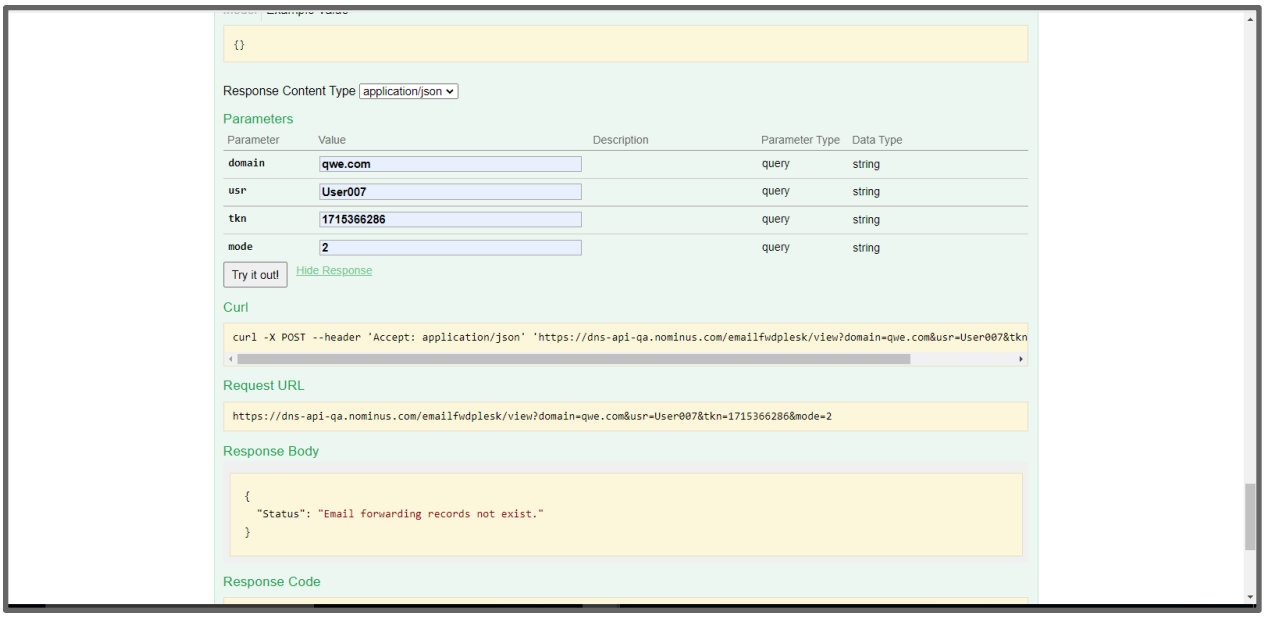
- This API is used to remove email forwarding from the plesk.
    - **Domain :** Name of the domain i.e. qwe.com
    - **Clientemail :** Client mail address i.e. a@gmail.com
    - **Usr:** User007
    - **Tkn:** 1715366286
    - **Mode: 2**



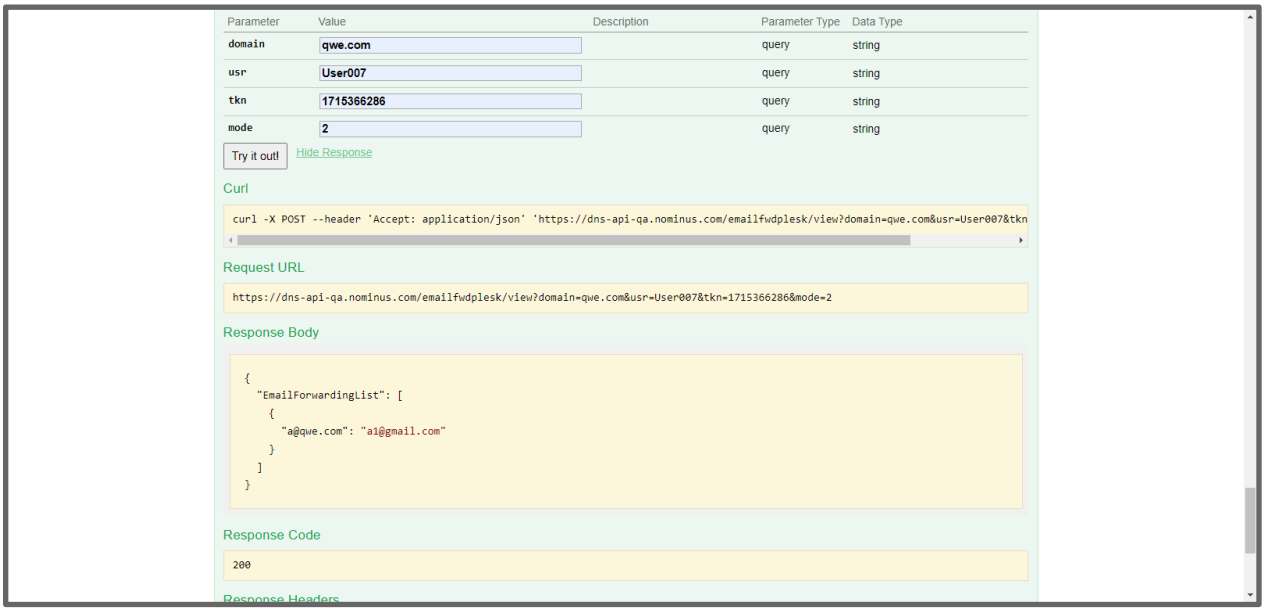
10) **Post - emailfwdplesk/view - View email forwarding record for domain (PLESK):**

- This API is used to view all the records of the email forwarding.
    - **Domain :** Name of the domain i.e. qwe.com
    - **Usr:** User007
    - **Tkn:** 1715366286
    - **Mode:** 2

- If record is not found user will get message like- `{ "Status": "Email forwarding records not exist." }`



- If record is found user will get the forwarding data.



PleskDomain :

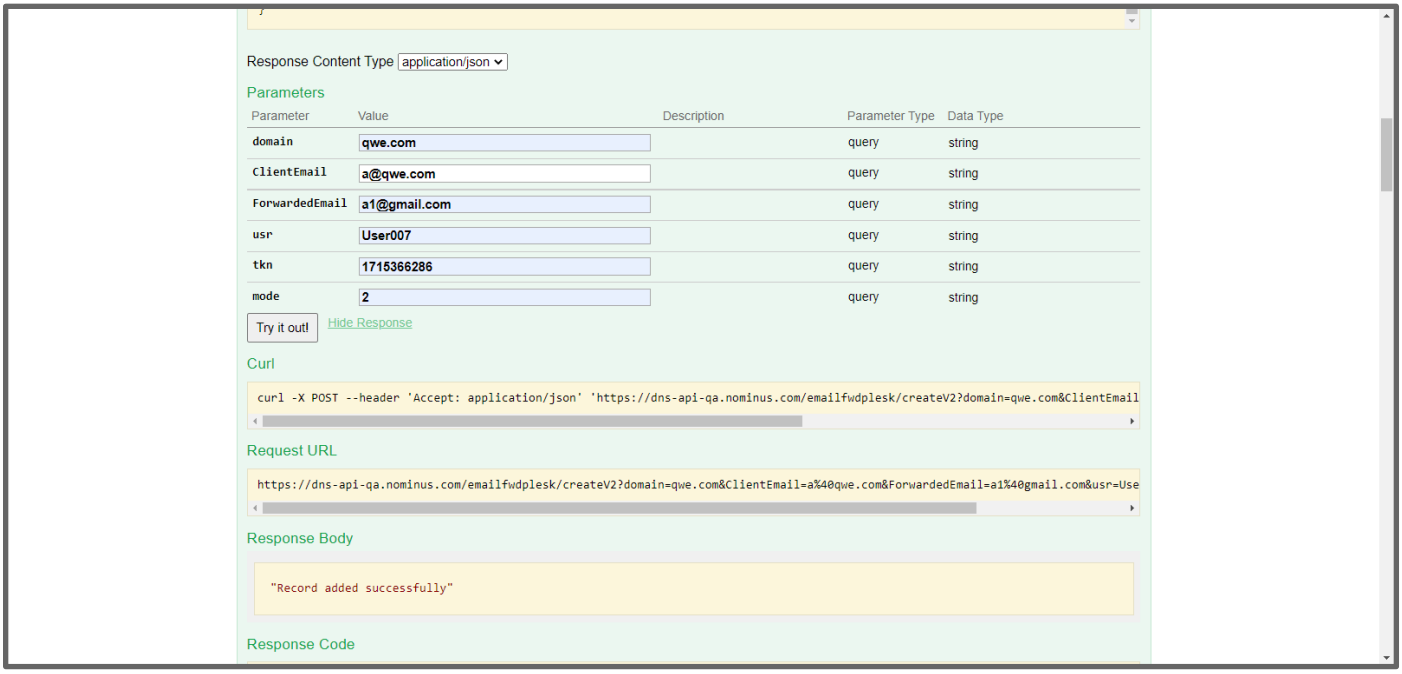
1) **Post - /api/pleskdomaincreate - Create domain using plesk**

- This API is older version API and it is now replaced with the Post - **/api/pleskdomaincreateV2**

2) **Post - /api/pleskdomaincreateV2 - Domain Creation in Plesk**

- This API is used to create domain in plesk. Here, domain created from swagger must be available in the Power DNS else it is not allowed to create domain in plesk.

    - **Domain :** Name of the domain
    - **Clientemail :** Client mail address
    - **Forwardedemail :** Email address for forwarding the mail.
    - **Usr:** User007
    - **Tkn:** 1715366286
    - **Mode: 2**



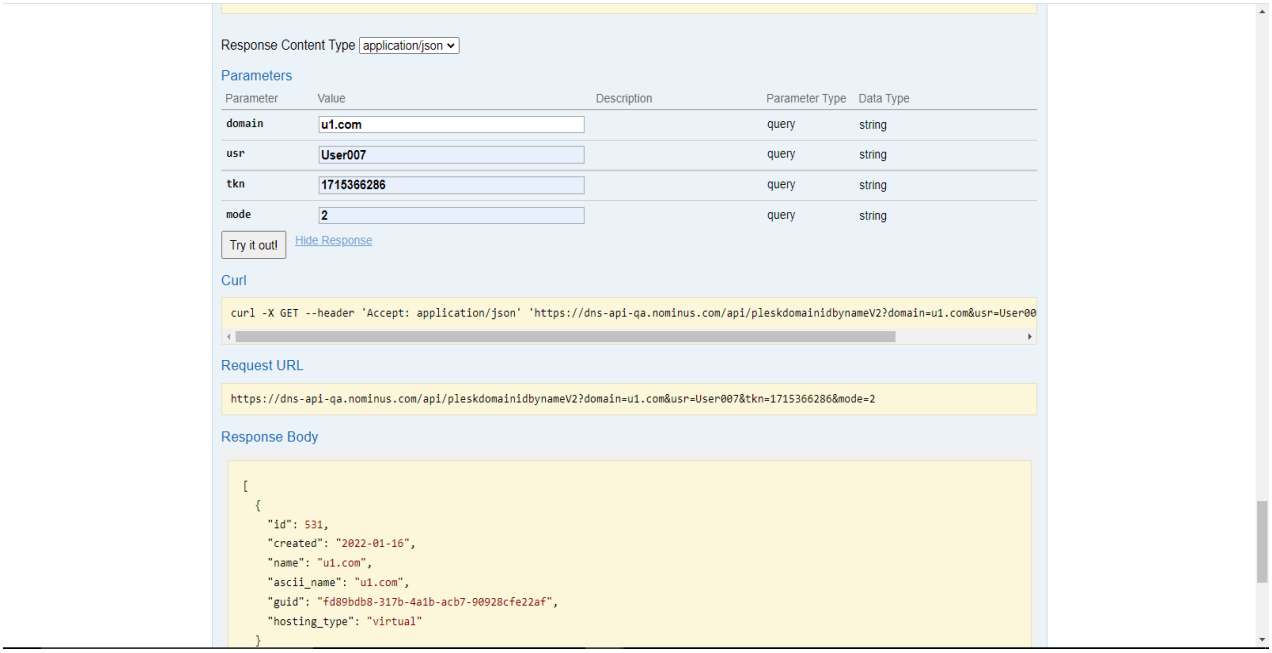
3) **Post - /api/pleskdomainidbyname - Get domain details using plesk**

- This API is older version API and it is now replaced with the Post - **/api/pleskdomainidbynameV2**

4) **Post - /api/pleskdomainidbynameV2 - Get domain details using plesk**

- This API is used to get the domain details along with ID from the plesk.

- **NOTE:** Here, user can only able to get the data if data is available in the plesk i.e. either email or web forwarding should be on.
    - **Domain :** Name of the domain
    - **Usr:** User007
    - **Tkn:** 1715366286
    - **Mode: 2**



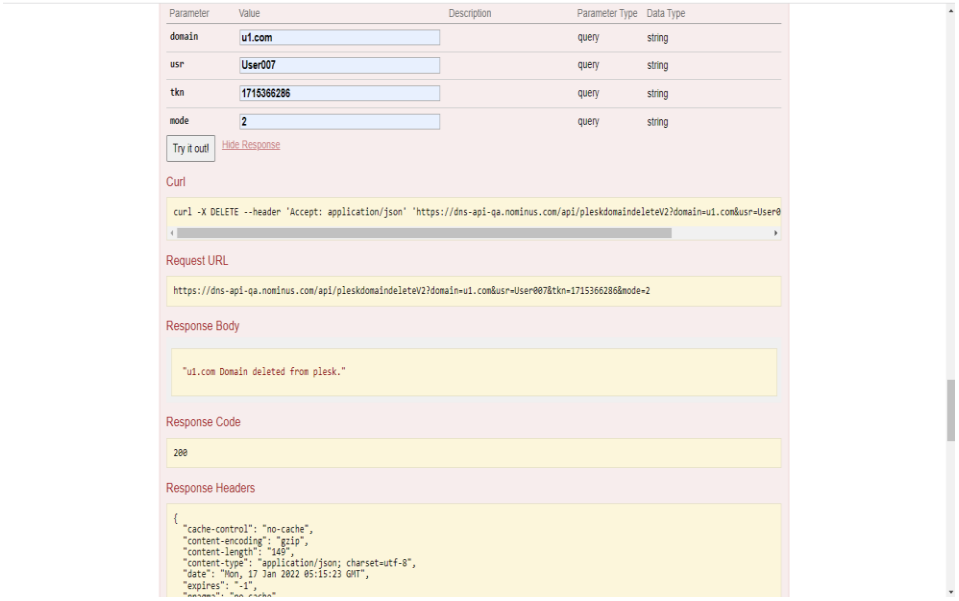
5) **Post - /api/pleskdomaindelete - Delete domain details from plesk**

- This API is older version API and it is now replaced with the Post - **/api/pleskdomaindeleteV2**

6) **Post - /api/pleskdomaindeleteV2 - Delete domain from plesk**

- This API is used to delete the domain from the plesk.

    - **Domain :** Name of the domain
    - **Usr:** User007
    - **Tkn:** 1715366286
    - **Mode: 2**



Users:
Domain List:
API Search Log:
Error Log:
API Search Chart:
Domain Count:
Domain Count Chart:
DNS Profile:
PowerDNS Setting:
Forwarding Setting:
Block Domains:
Forwarding Server:
Swagger:

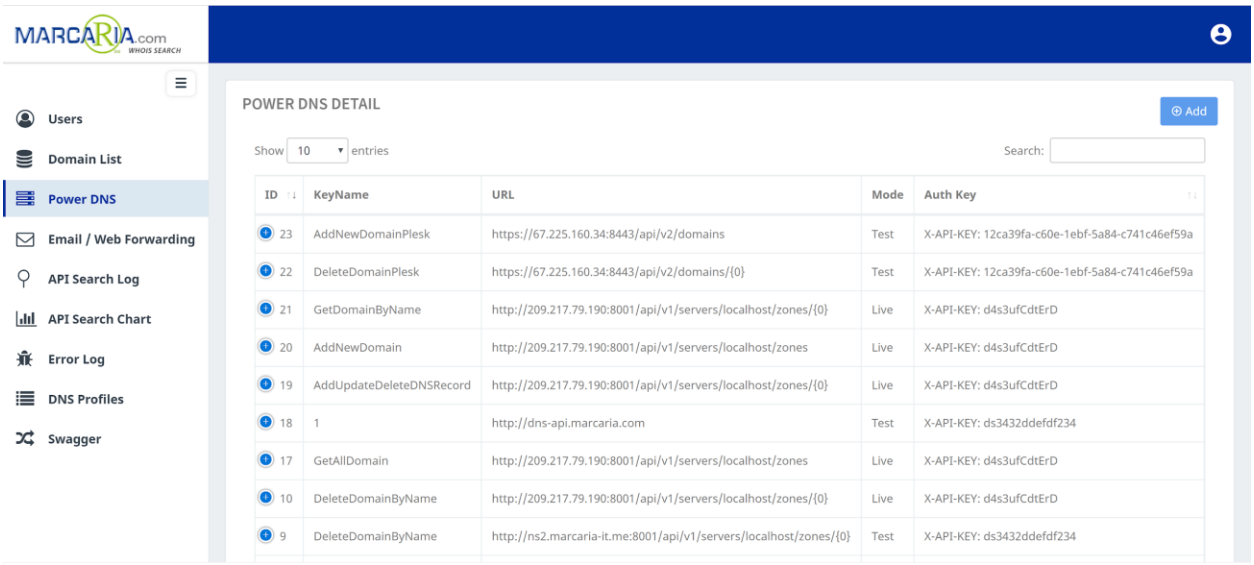
==========

Old content

==========

DNS Admin Panel

It is used by Marcaria DNS API application administrator to configure DNS API. It will allow administrators to configure api url for test and live mode.



DNS-API.Marcaria

This application is basically one layer above the actual API. All marcaria and their client applications will perform DNS operation using this API. DNS-API applications internally manage which API to call for web or email forwarding. It means a user of DNS-API just says to perform Web Forwarding but isn't aware what call internally made to do this operation.

[tblPowerDNSSettings]

	Id	KeyName	URL	Mode	Auth_Key	IsDisabled
1	5	GetAllDomain	http://ns2.marcaria-it.me:8001/api/v1/servers/loca...	2	X-API-KEY: ds3432ddefdf234	0
2	6	GetDomainByName	http://ns2.marcaria-it.me:8001/api/v1/servers/loca...	2	X-API-KEY: ds3432ddefdf234	0
3	7	AddNewDomain	http://ns2.marcaria-it.me:8001/api/v1/servers/loca...	2	X-API-KEY: ds3432ddefdf234	0
4	8	AddUpdateDeleteDNSRecord	http://ns2.marcaria-it.me:8001/api/v1/servers/loca...	2	X-API-KEY: ds3432ddefdf234	0
5	9	DeleteDomainByName	http://ns2.marcaria-it.me:8001/api/v1/servers/loca...	2	X-API-KEY: ds3432ddefdf234	0
6	10	DeleteDomainByName	http://209.217.79.190:8001/api/v1/servers/localh...	1	X-API-KEY: d4s3ufCdtErD	0
7	17	GetAllDomain	http://209.217.79.190:8001/api/v1/servers/localh...	1	X-API-KEY: d4s3ufCdtErD	0
8	18	1	http://dns-api.marcaria.com	2	X-API-KEY: ds3432ddefdf234	0
9	19	AddUpdateDeleteDNSRecord	http://209.217.79.190:8001/api/v1/servers/localh...	1	X-API-KEY: d4s3ufCdtErD	0
10	20	AddNewDomain	http://209.217.79.190:8001/api/v1/servers/localh...	1	X-API-KEY: d4s3ufCdtErD	0
11	21	GetDomainByName	http://209.217.79.190:8001/api/v1/servers/localh...	1	X-API-KEY: d4s3ufCdtErD	0
12	22	DeleteDomainPlesk	https://67.225.160.34:8443/api/v2/domains/{0}	2	X-API-KEY: 12ca39fa-c60e-1ebf-5a84-c741c46ef59a	0
13	23	AddNewDomainPlesk	https://67.225.160.34:8443/api/v2/domains	2	X-API-KEY: 12ca39fa-c60e-1ebf-5a84-c741c46ef59a	0

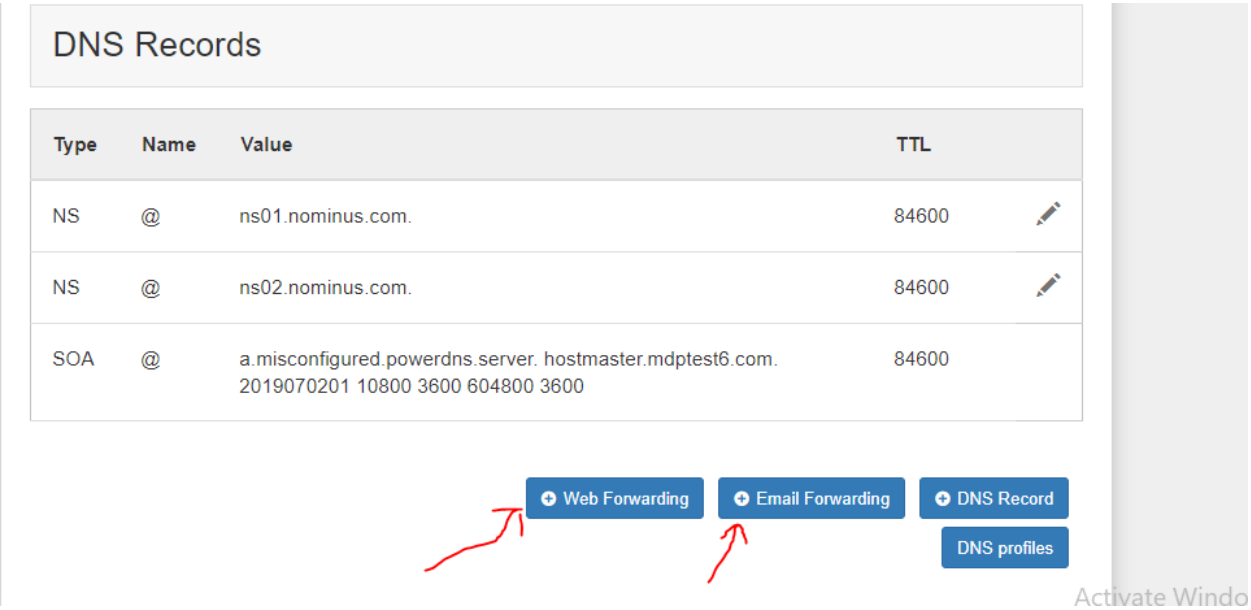
[tblForwardingSettings]

	Id	KeyName	URL	Mode	Auth_Key	IsDisabled	ForwardType
1	8	AddMailFWD	http://mfw.marcaria-it.me/mailfwd/emf2.asp?dominio={0}&localmail={1}&re...	2	X-API-KEY: ds3432ddefdf234	0	1
2	9	DeleteMailFWD	http://mfw.marcaria-it.me/mailfwd/emf3.asp?dominio={0}&localmail={1}&re...	2	X-API-KEY: ds3432ddefdf234	0	1
3	10	AddWebFWD	http://ufw.marcaria-it.me/ForwardCreate.asp?dominio={0}&desturl={1}&jsfr...	2	X-API-KEY: ds3432ddefdf234	0	1
4	11	UpdateWebFWD	http://ufw.marcaria-it.me/ForwardUpdate.asp?dominio={0}&desturl={1}&jsfr...	2	X-API-KEY: ds3432ddefdf234	0	2
5	12	DeleteWebFWD	http://ufw.marcaria-it.me/ForwardDelete.asp?dominio={0}	2	X-API-KEY: ds3432ddefdf234	0	1
6	14	pleskemailfwd	https://67.225.160.34:8443/api/v2/cli/mail/call	2	X-API-KEY: 12ca39fa-c60e-1ebf-5a84-c741c46ef59a	0	1
7	15	PleskGetIdByName	https://67.225.160.34:8443/api/v2/domains?name={0}	2	X-API-KEY: 12ca39fa-c60e-1ebf-5a84-c741c46ef59a	0	2
8	16	pleskwebfwd	https://67.225.160.34:8443/api/v2/domains/{0}	2	X-API-KEY: 12ca39fa-c60e-1ebf-5a84-c741c46ef59a	0	2

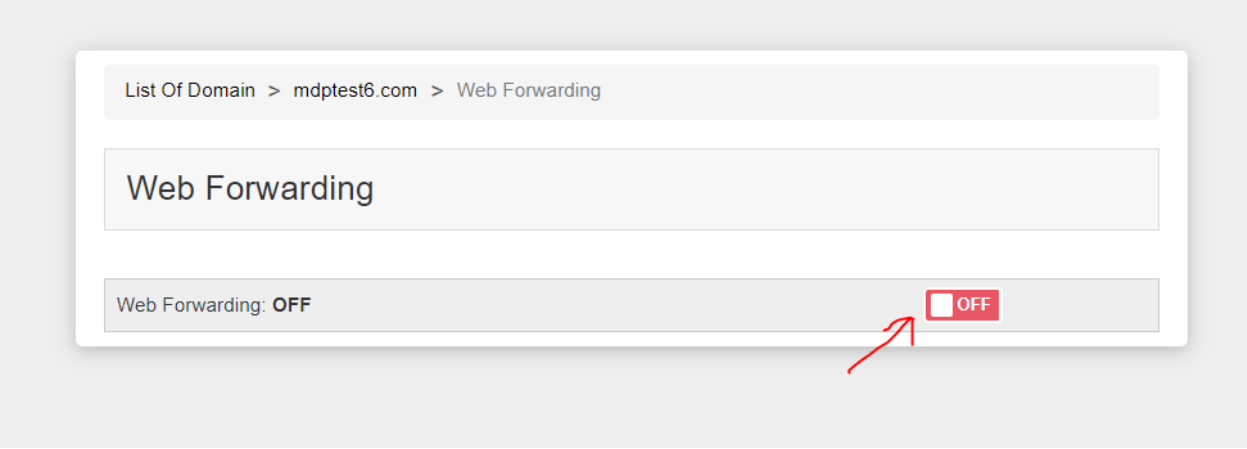
2. MODIFICATION TO DNS TESTING PAGES

Steps to verify
Navigate on http://dns-recods.whois.marcaria.com and click on domain name.

Now click on ‘web forwarding’ or ‘email forwarding’ button.

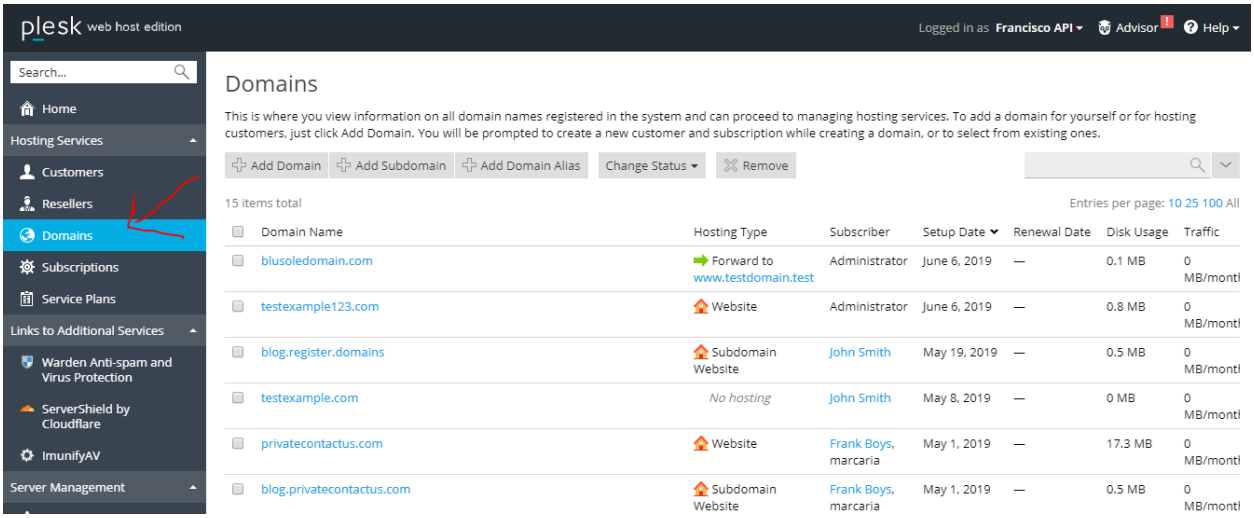


After clicking on this button we get pop up box like below.



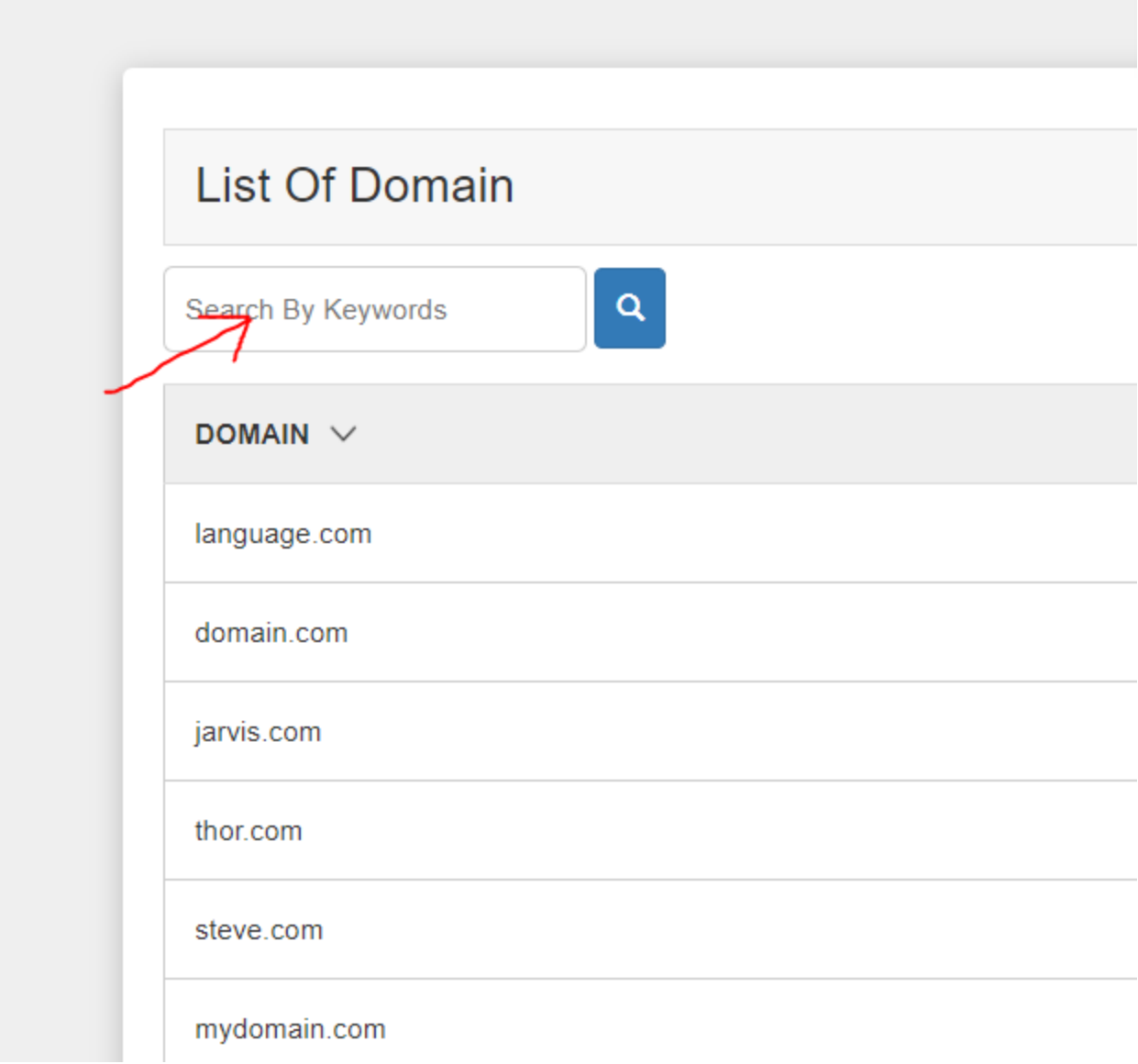
If “web” and “email” forwarding both or any one of them **ON** then that domain available in plesk domain list.and if both are **OFF** that means domain not available in list.
To check that plesk domain list we should login on plesk.
So navigate on https://host-linux1.nominus.host:8443/login_up.php?

After successfully log in click on ‘Domain’ module which is at the left side menu.

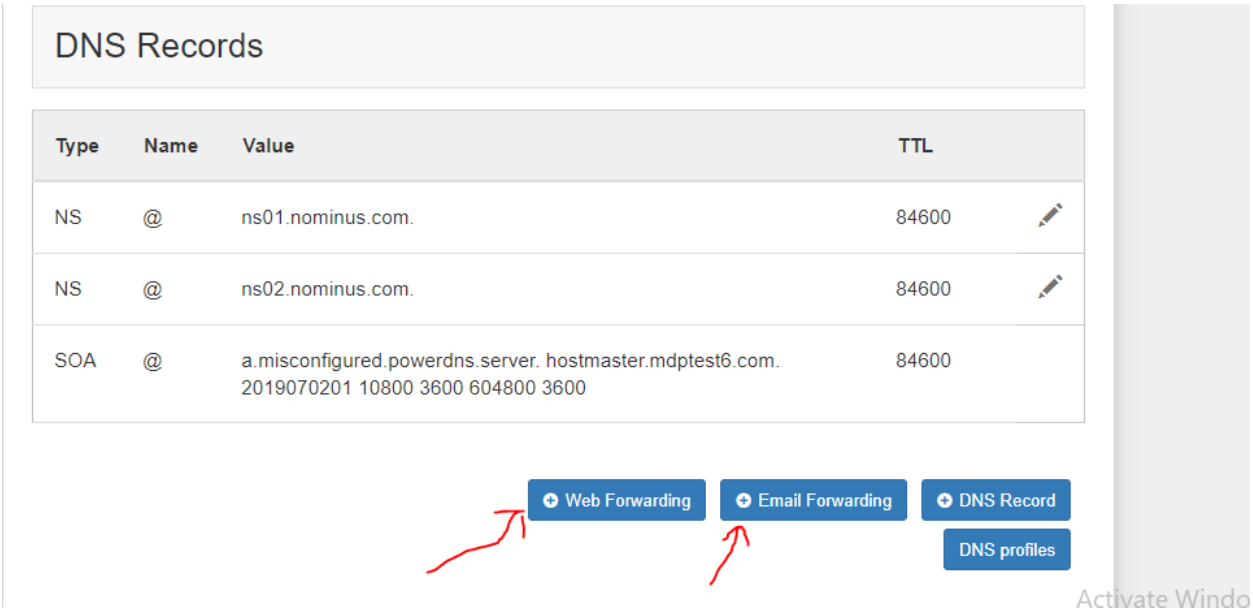


Here we get a list of domains in plesk.

Now if we want to remove any domain which is in plesk domain list then copy that name and paste it on dns record list page search bar and click on the search icon.



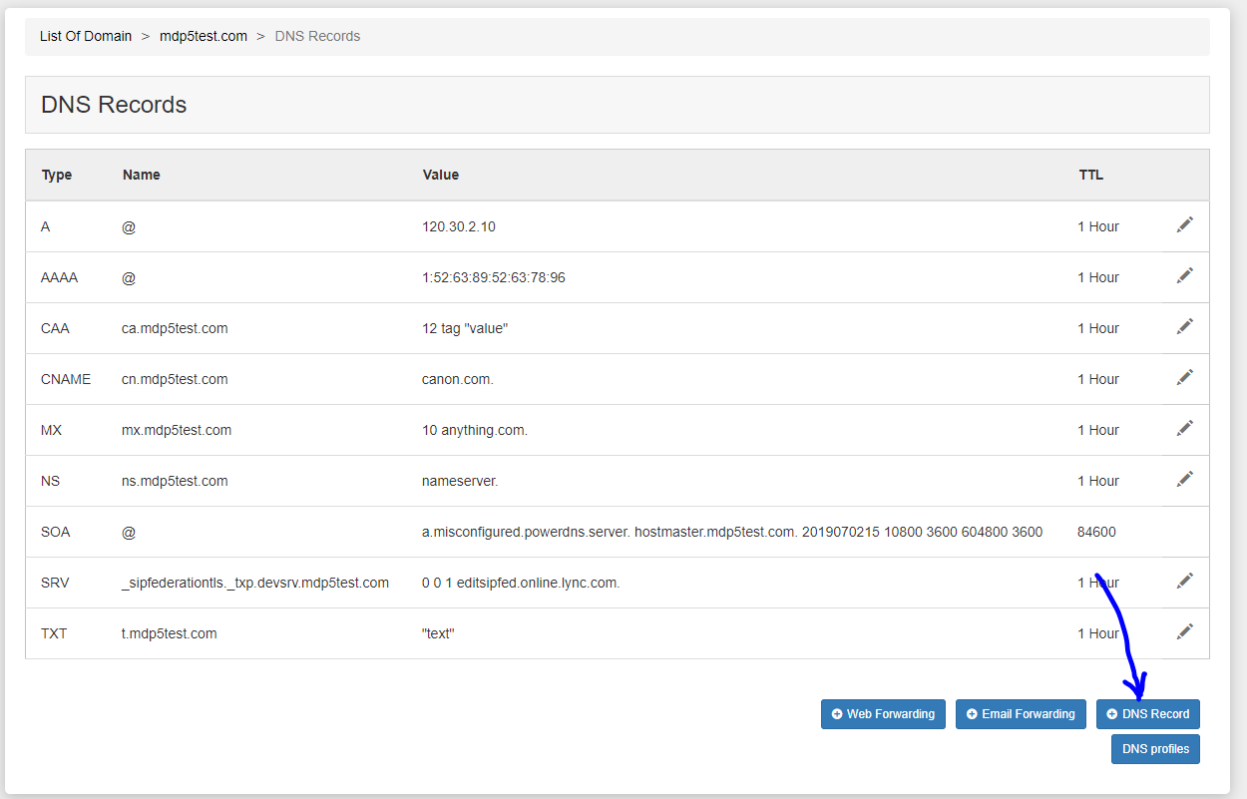
Now as said above to remove that domain we should **‘OFF’** the both forwarding.



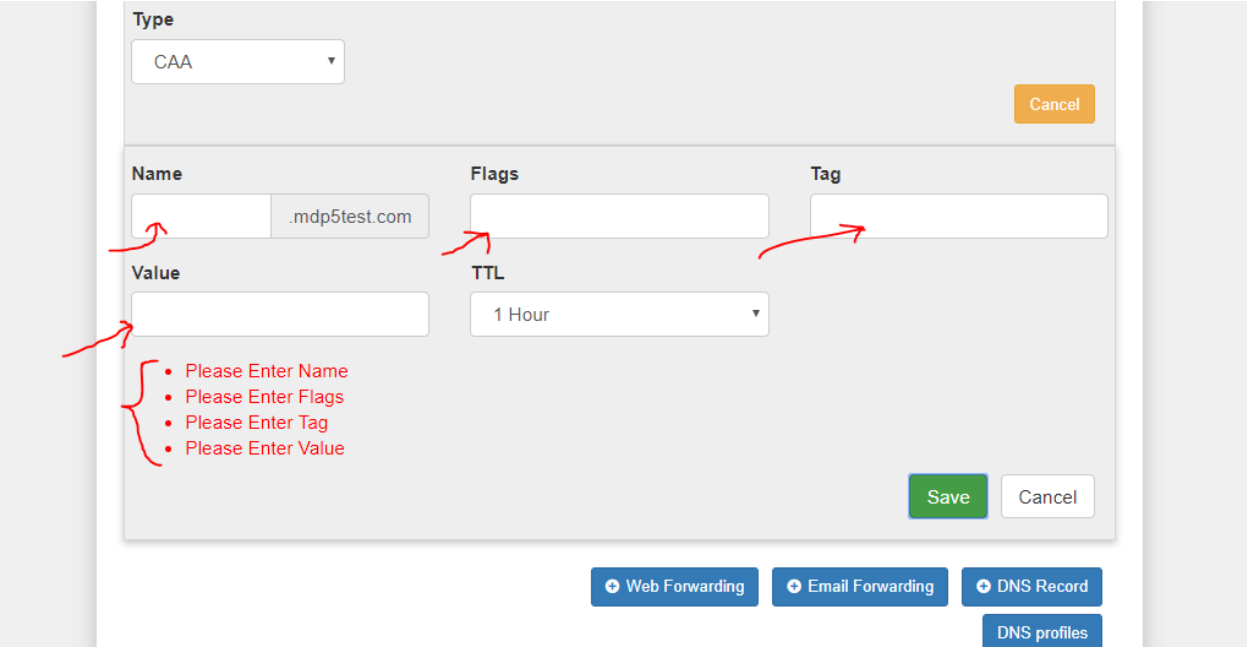
3. ADDITIONAL MODIFICATIONS TO DNS TESTING PAGES

Steps to verify
Browser to http://dns-recods.whois.marcaria.com and click on domain name.

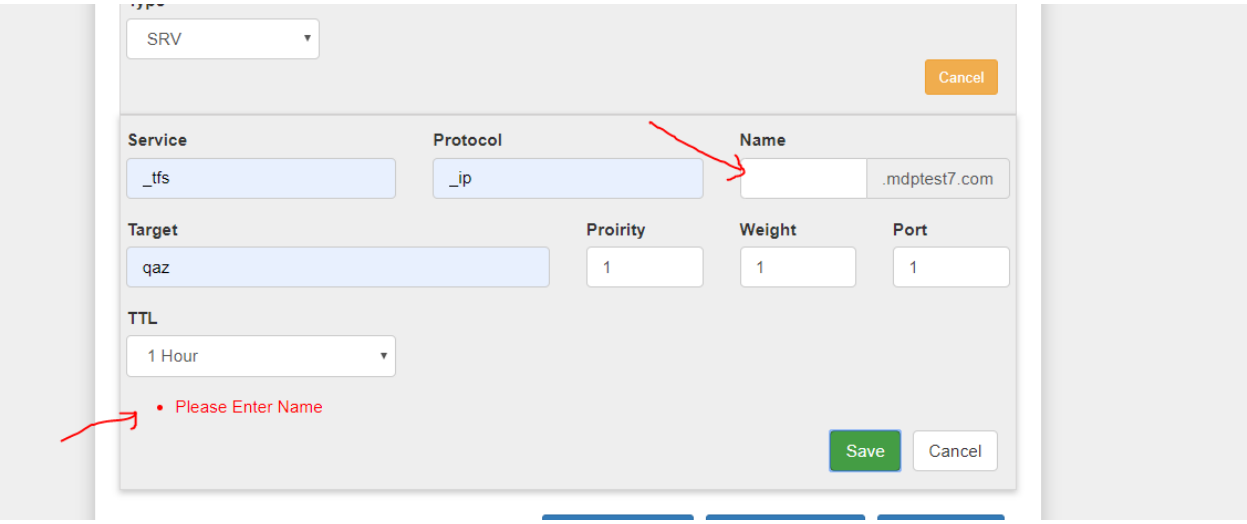
Now click on “DNS Record” Button



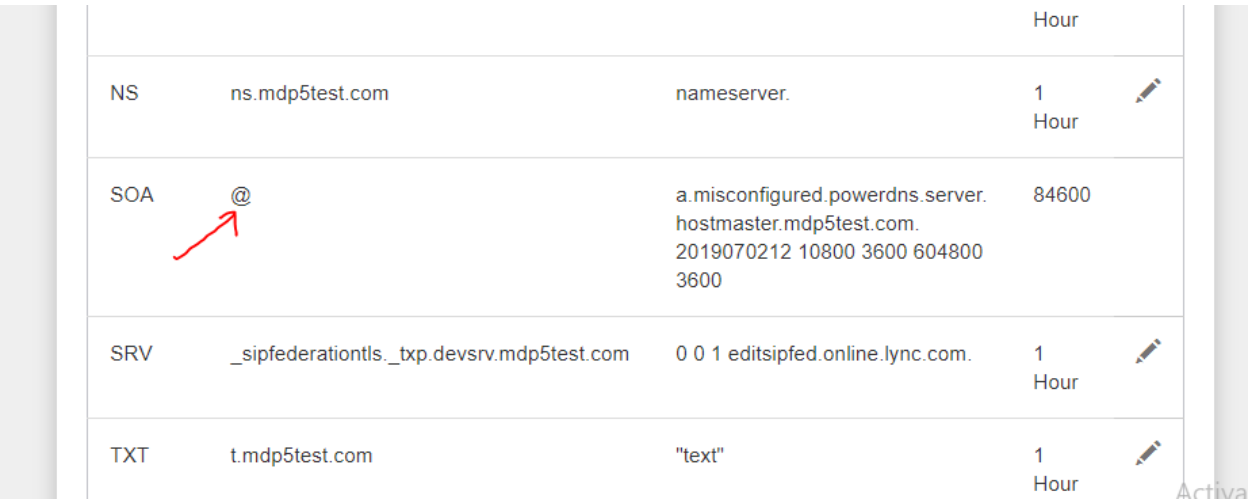
Now select Any Type of DNS record and check validation. Application should Validate and display appropriate messages.



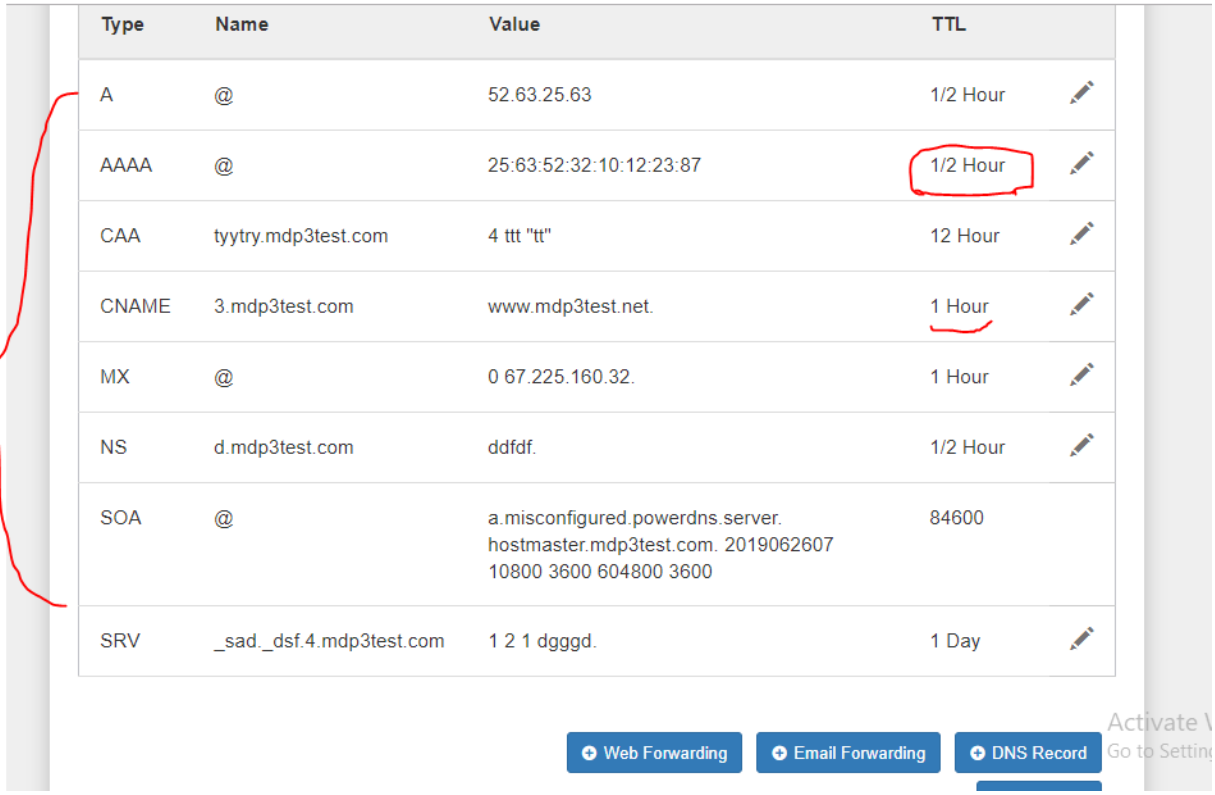
If Name field empty in SRV record it appears a phrase “please enter Name”.



Here Host field accept @ character and domain name has been removed from the “name” column.

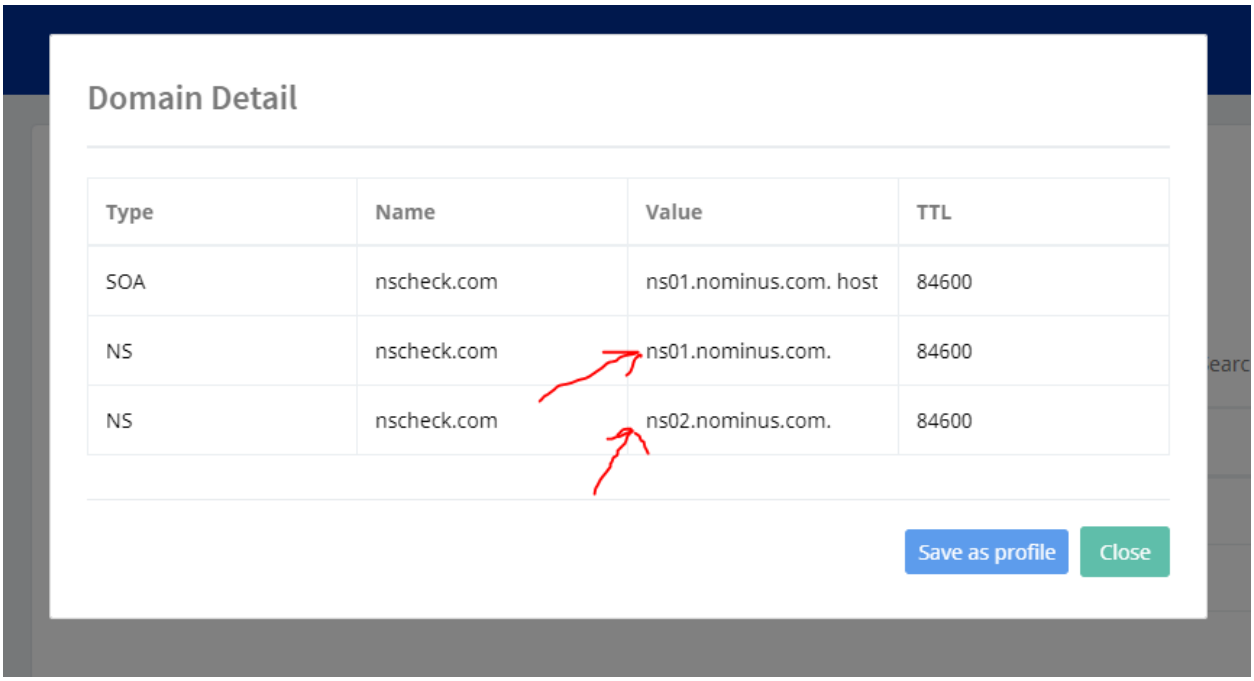


All dns records arranged as an alphabetically and TTL changed seconds to hours



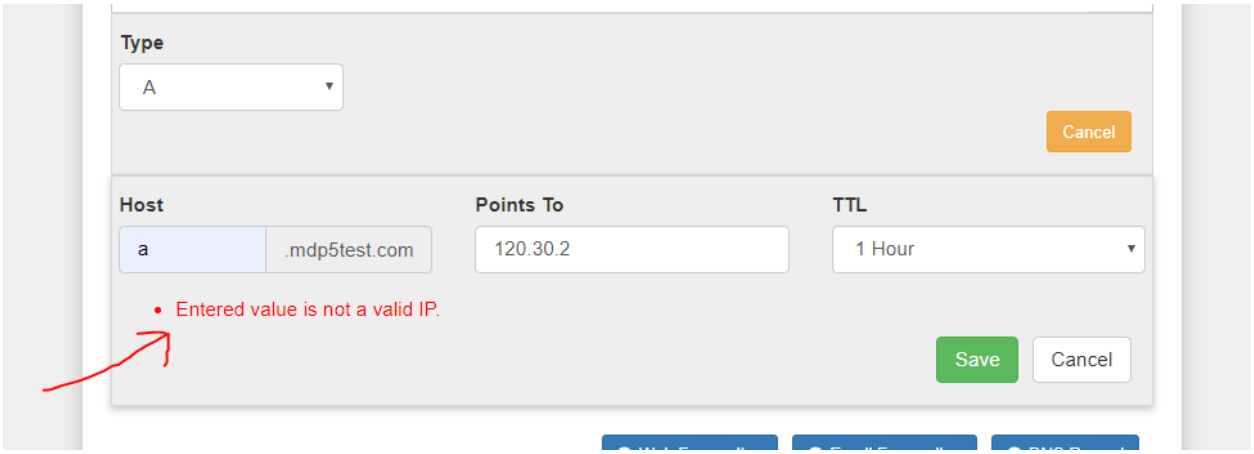
Type	Name	Value	TTL
A	@	52.63.25.63	1/2 Hour
AAAA	@	25:63:52:32:10:12:23:87	1/2 Hour
CAA	tyytry.mdp3test.com	4 ttt "tt"	12 Hour
CNAME	3.mdp3test.com	www.mdp3test.net.	1 Hour
MX	@	0 67.225.160.32.	1 Hour
NS	d.mdp3test.com	ddfdf.	1/2 Hour
SOA	@	a.misconfigured.powerdns.server. hostmaster.mdp3test.com. 2019062607 10800 3600 604800 3600	84600
SRV	_sad._dsf.4.mdp3test.com	1 2 1 dgggd.	1 Day

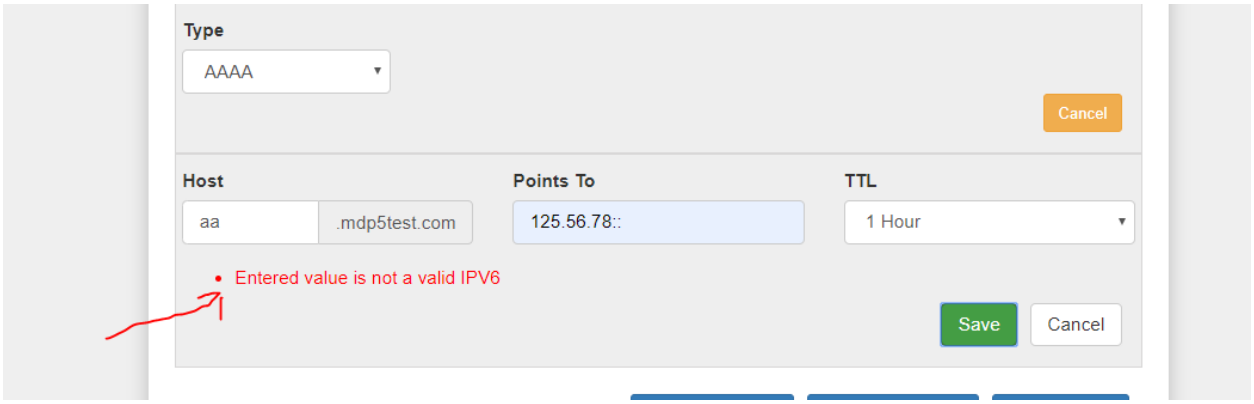
And as per requirement NS record will save in live site like this.



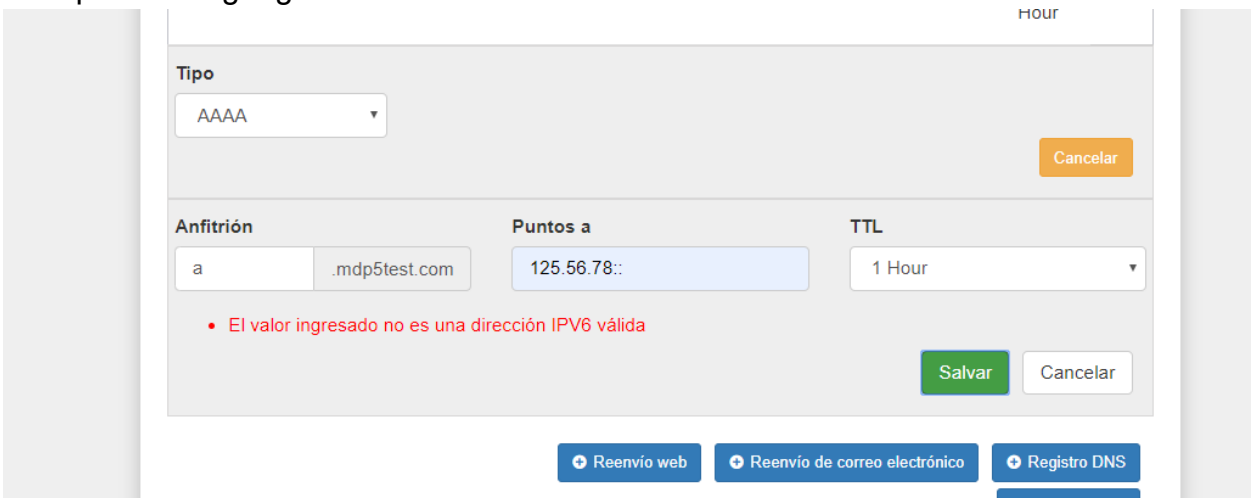
For “A” records, and for “AAA” records in Points To field information does not match then validation appears.

For English



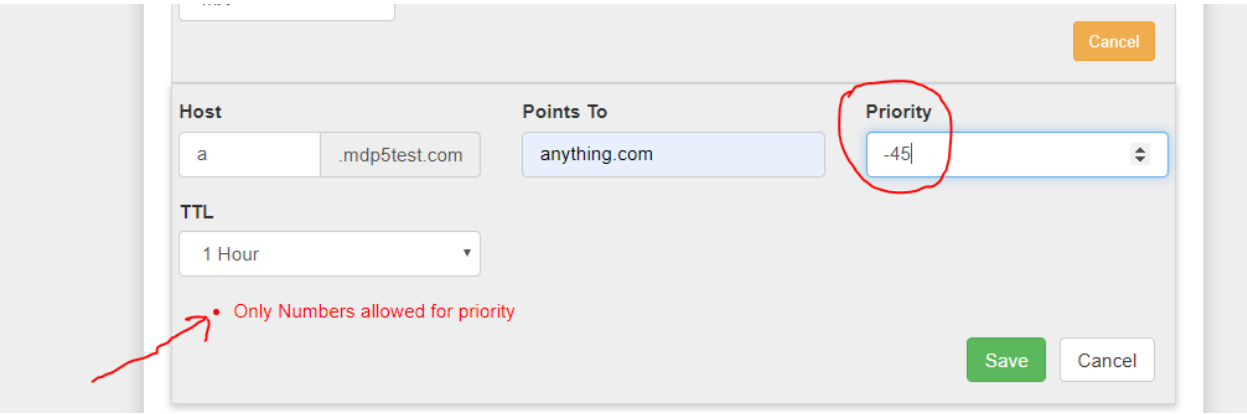


For spanish language

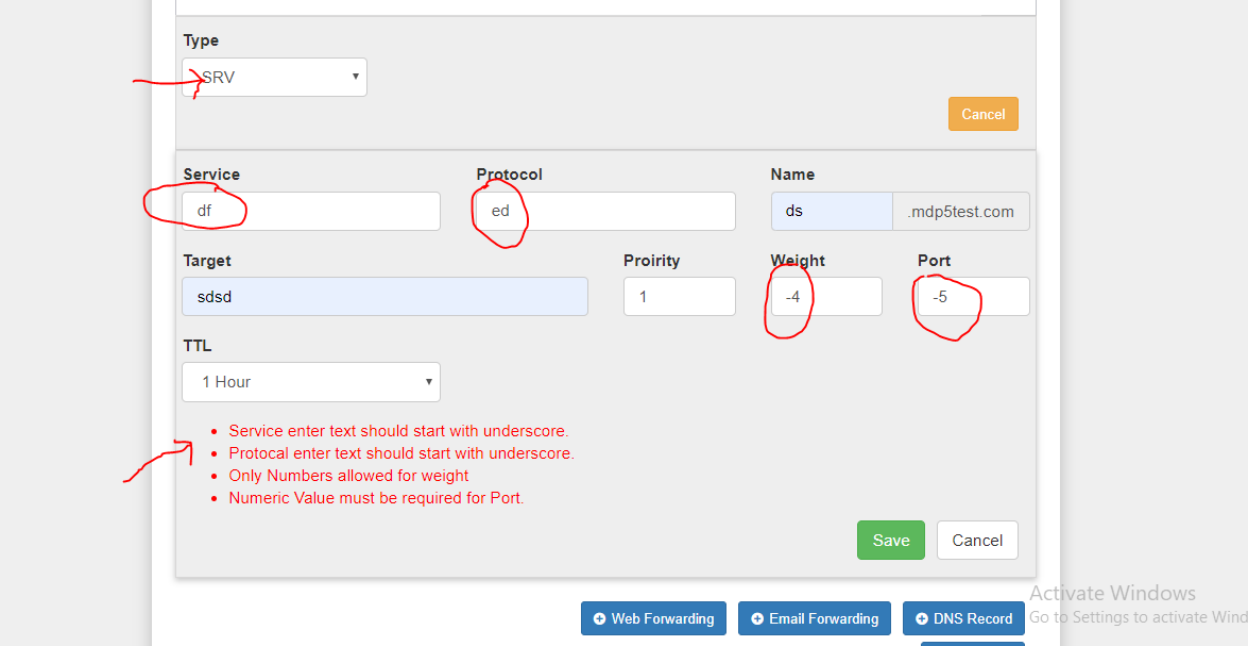




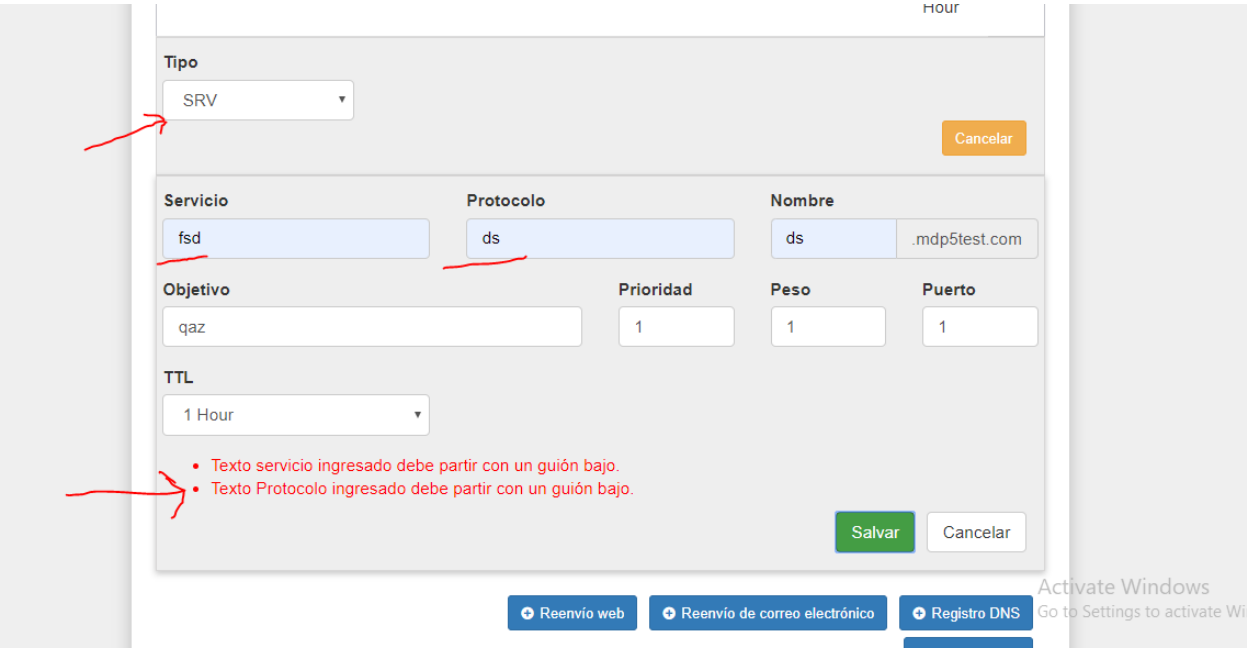
For “Priority” field – Record “MX”: if user enter Negative numbers then it display validation.



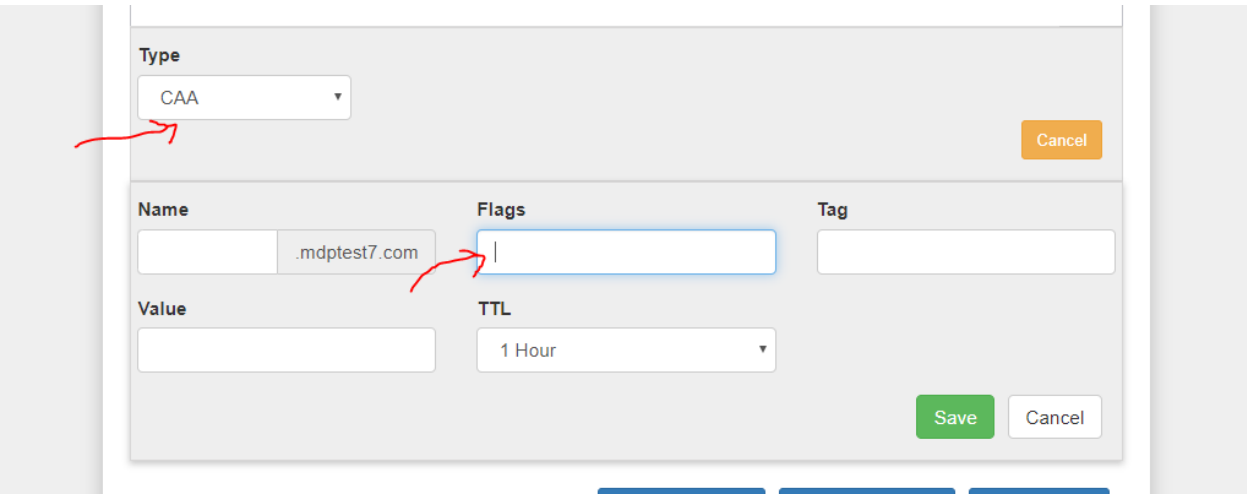
In SRV record if user enter weight and port as negative number then it gives validation.for “protocol” and “service” field value should start with underscore otherwise it display validation.



value should start with underscore in service and protocol field else it gives validation. For spanish language



For “Flags” field – Record “CAA”: dropdown has been erased.



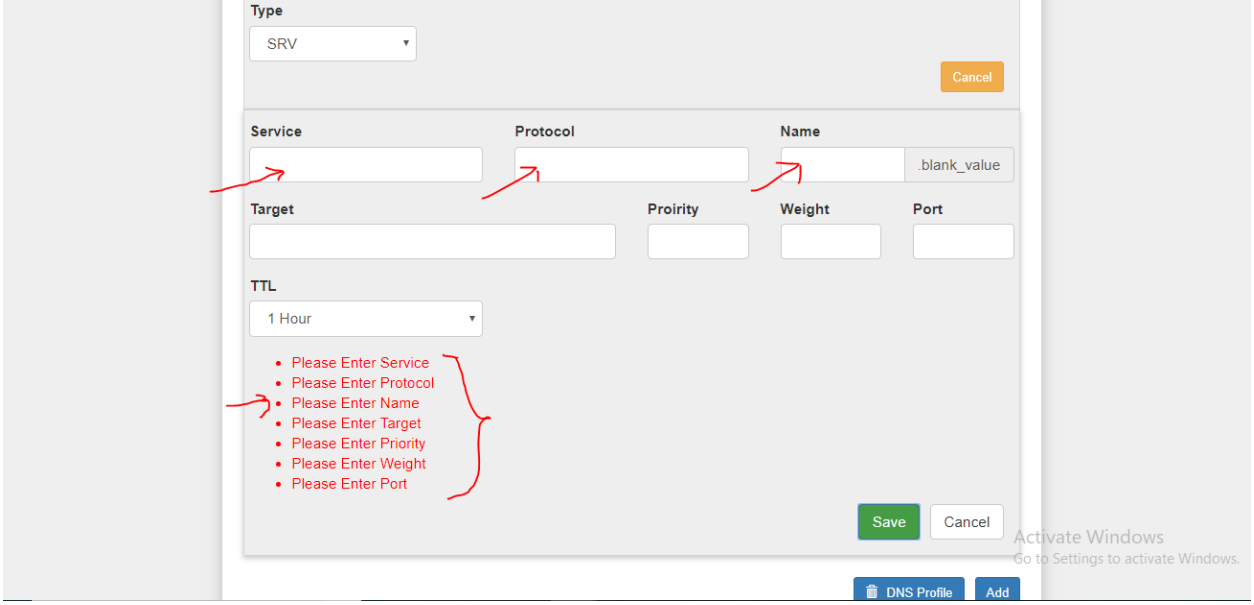
4. DNS RECORD PROFILES

Steps.

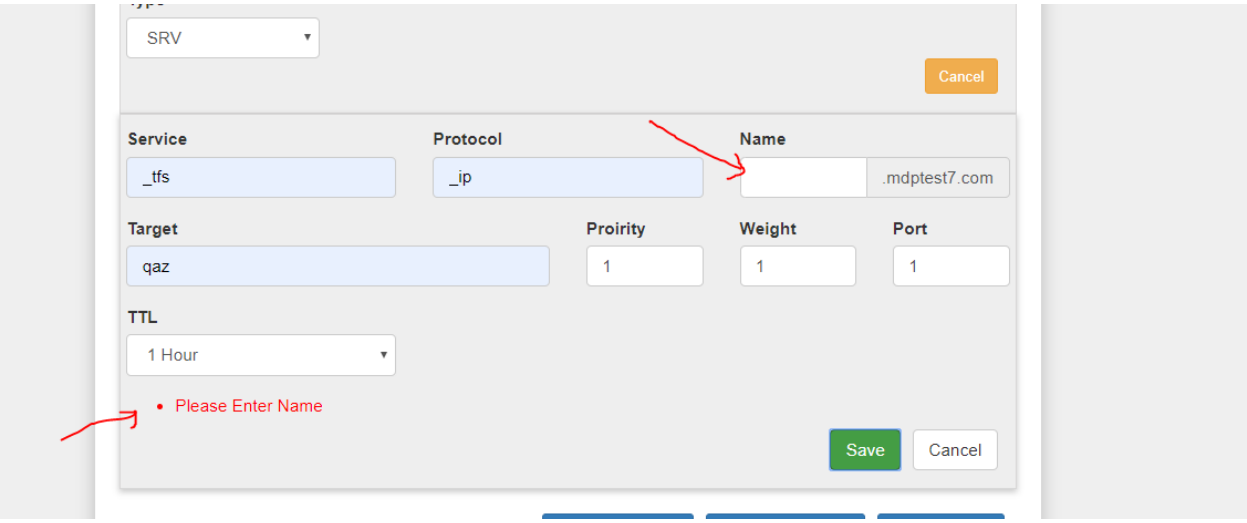
Go on

http://dns-recods.whois.marcaria.com/en/Dns-Profile-Records.aspx?name=checkpro

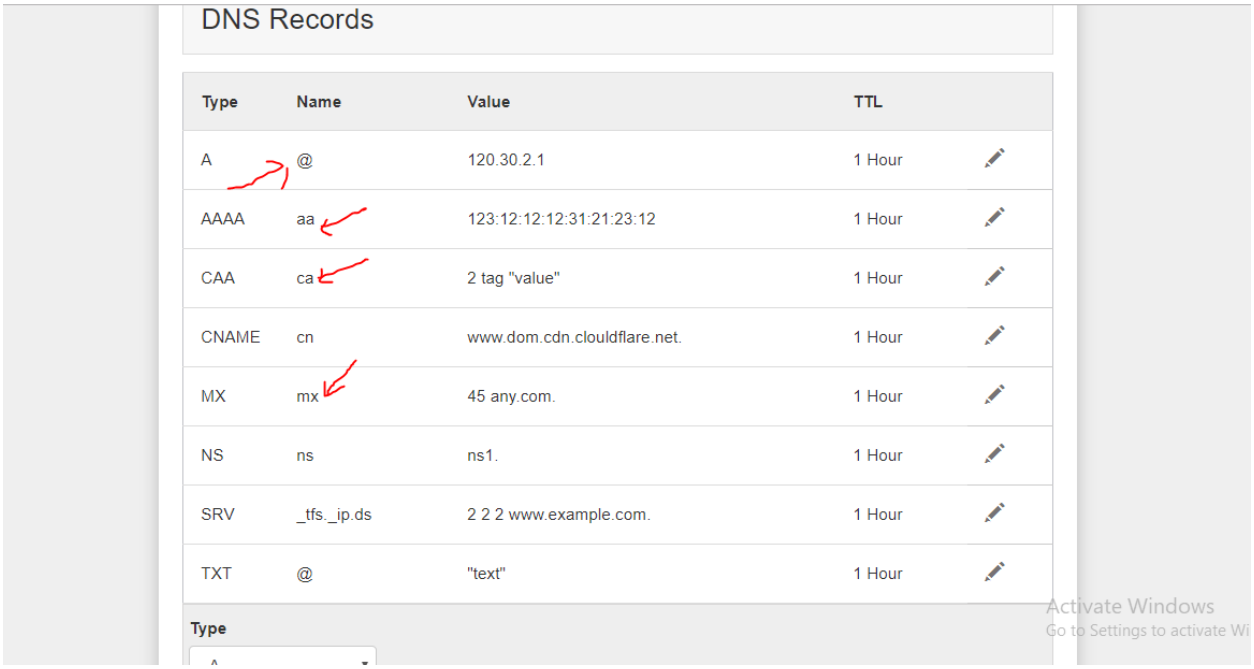
Add any DNS record at the DNS record profile page and Validation displaying if user leave any field blank.



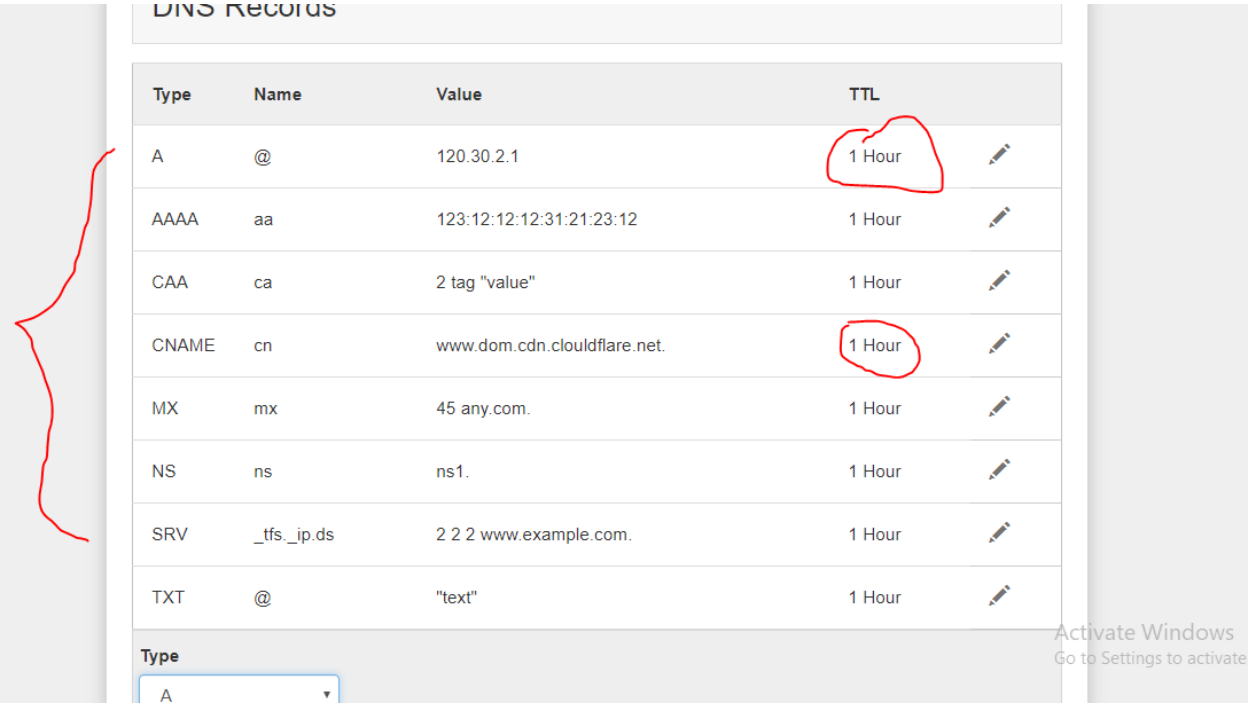
If the Name field is empty in the SRV record it appears a phrase “please enter Name”.



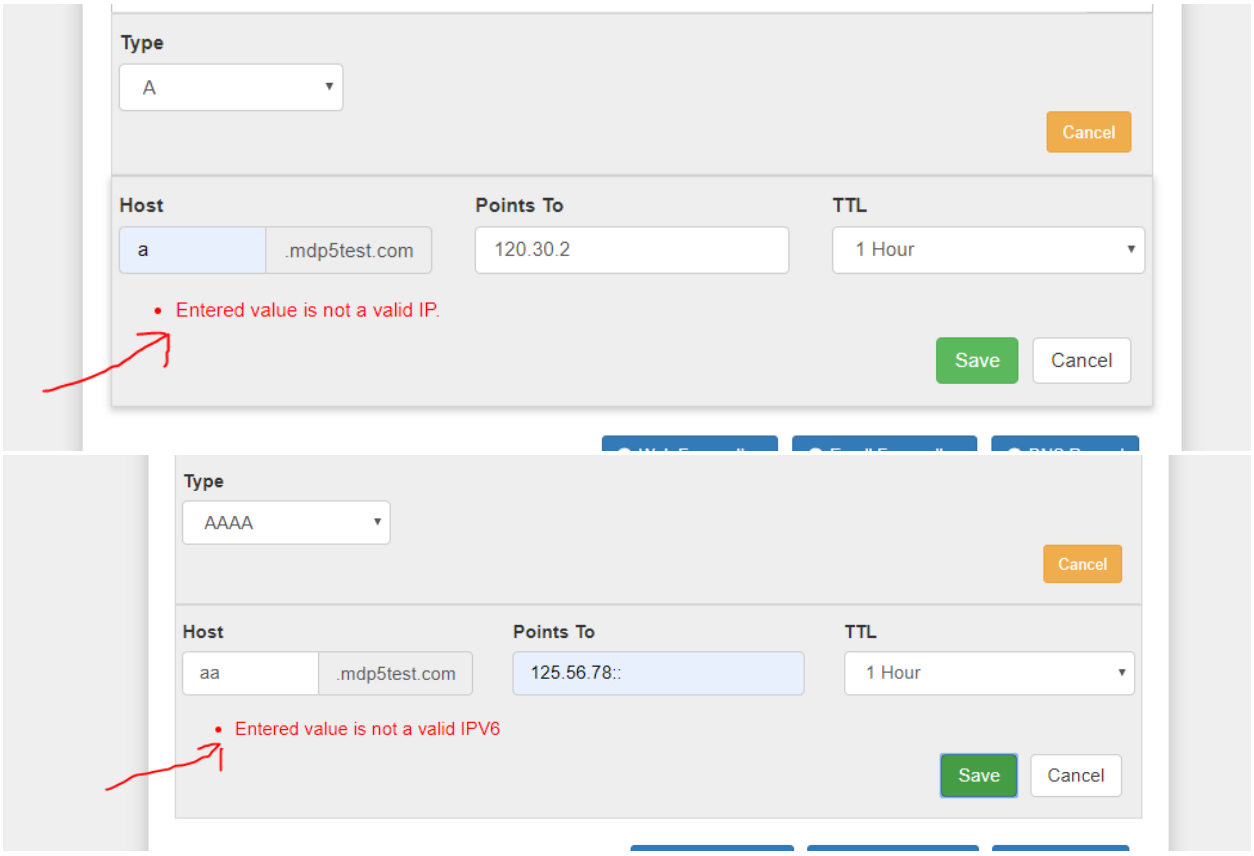
Here Host field accept @ character and domain name has been removed from the “name” column.



All dns records arranged as an alphabetically and TTL changed seconds to hour



For “A” records, and for “AAA” records in Points To field information does not match then validation appear.

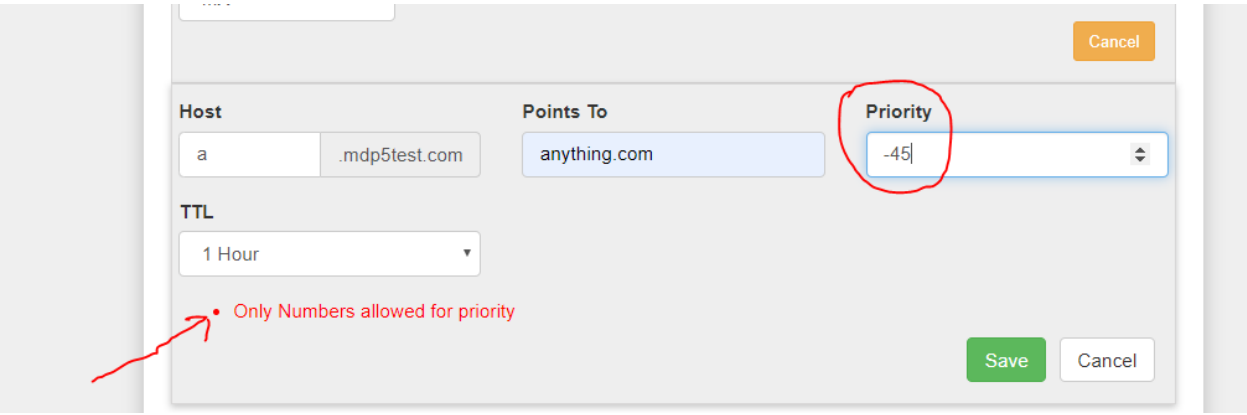


For spanish language

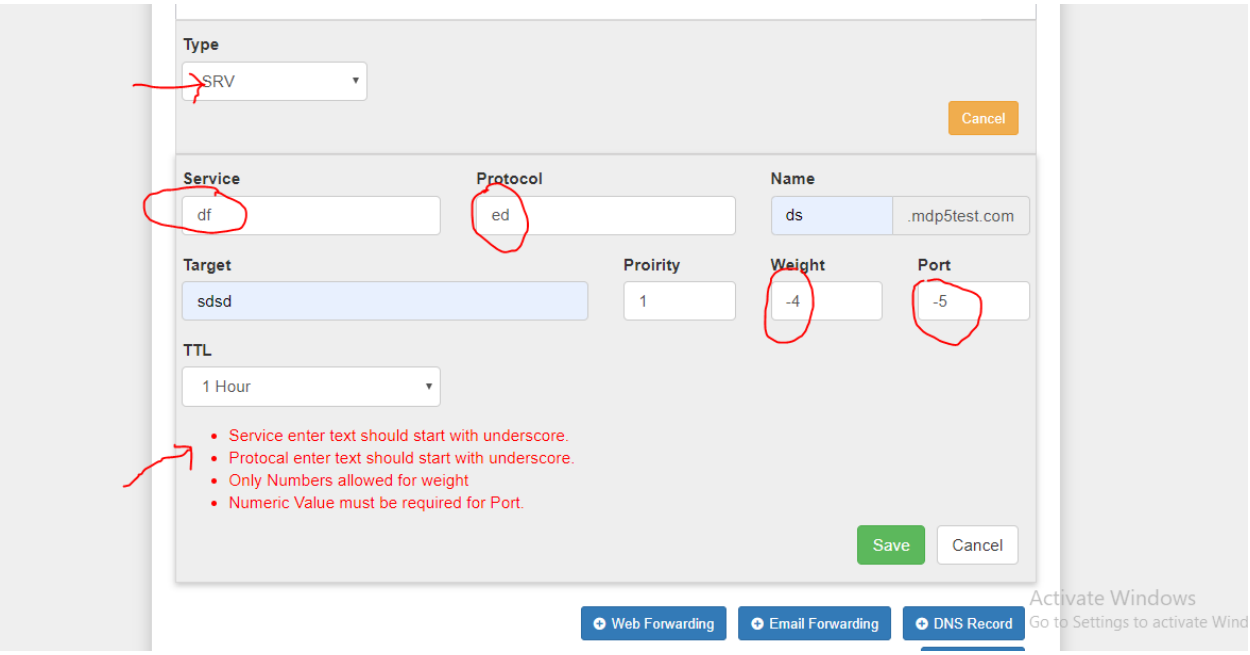




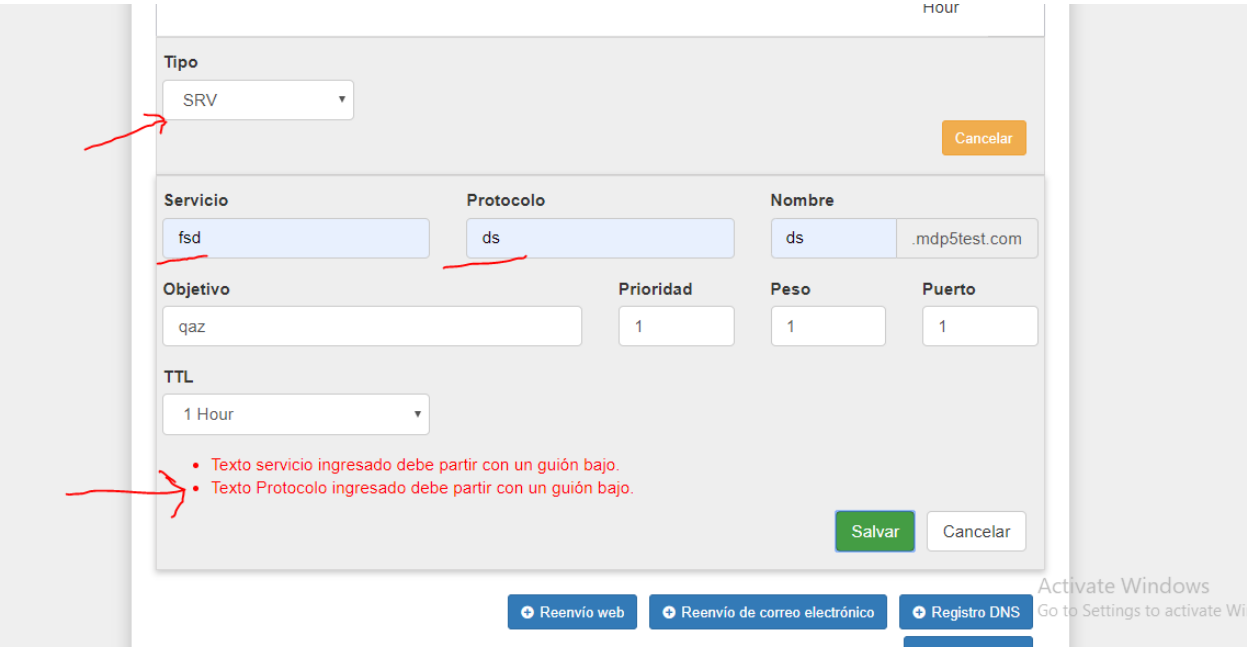
For “Priority” field – Record “MX”: if user enters Negative numbers then appears validation.



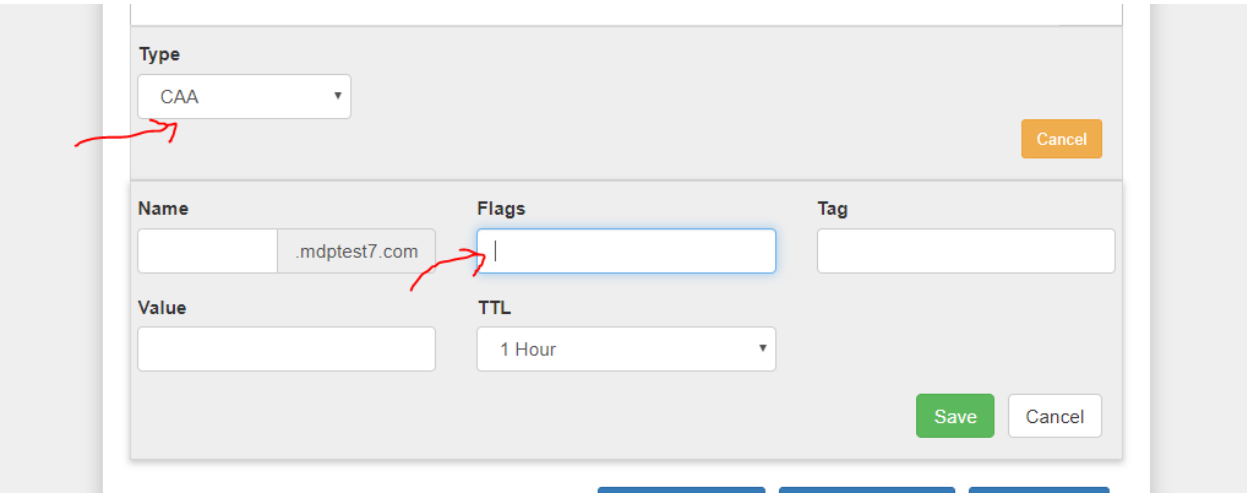
In the SRV record if user enter weight and port as negative number then it gives validation.for “protocol” and “service” field value should start with underscore otherwise it display validation.



value should start with underscore in service and protocol field else it gives validation For spanish language

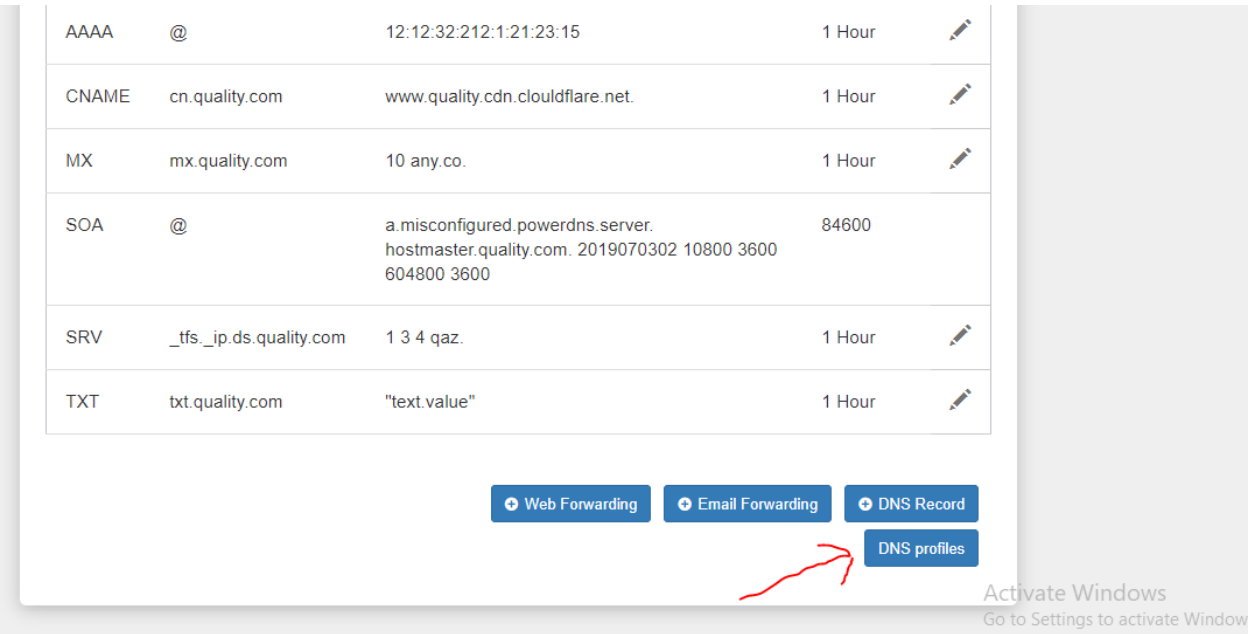


For “Flags” field – Record “CAA”: dropdown has been erased.

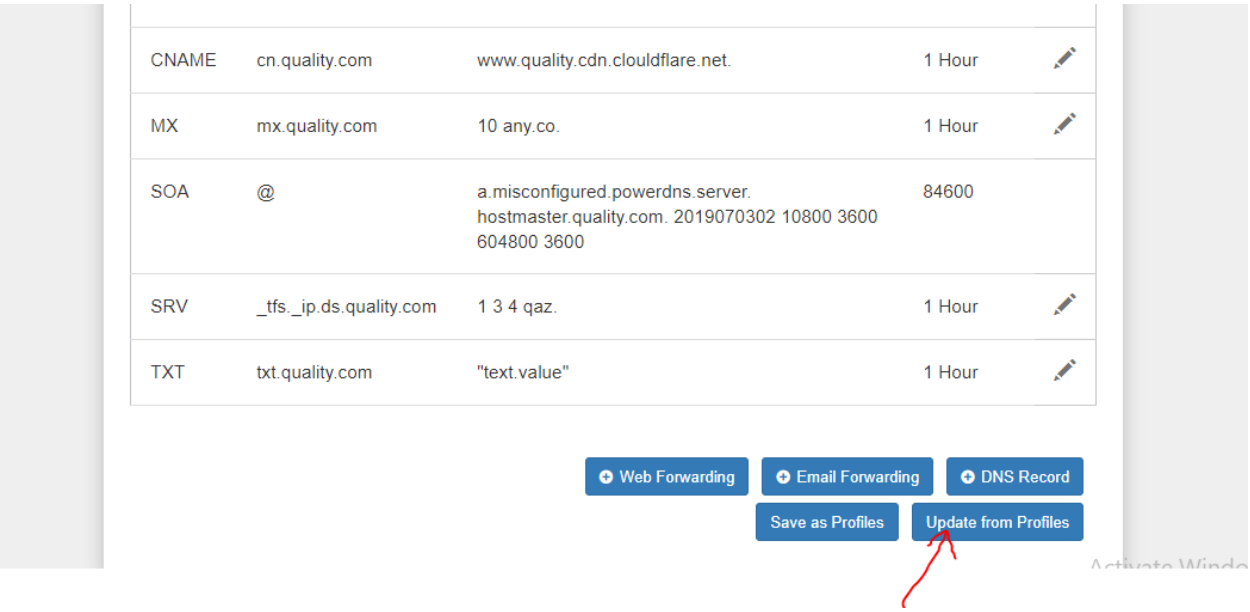


Steps for check domain name update from profile:

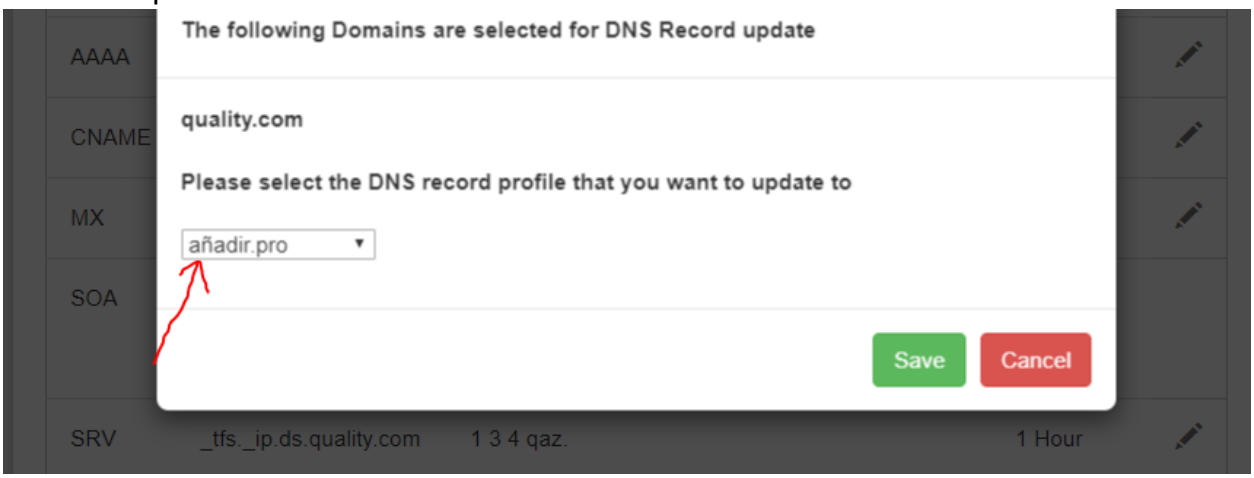
- Go on http://dns-recods.whois.marcaria.com
- And select any domain name
- Now click on DNS Profiles Button



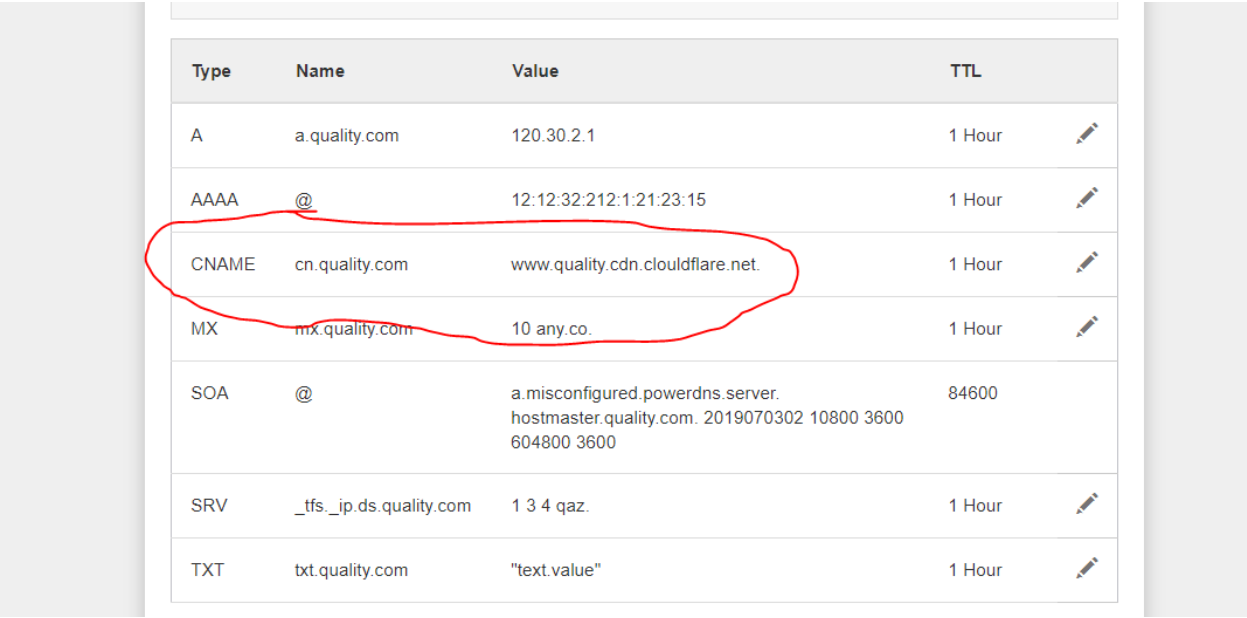
- after that click on ‘update from profile’ button and pop up box will open



- In the pop up box go to the dropdown menu and select any profile which you want to update and click on the save button.



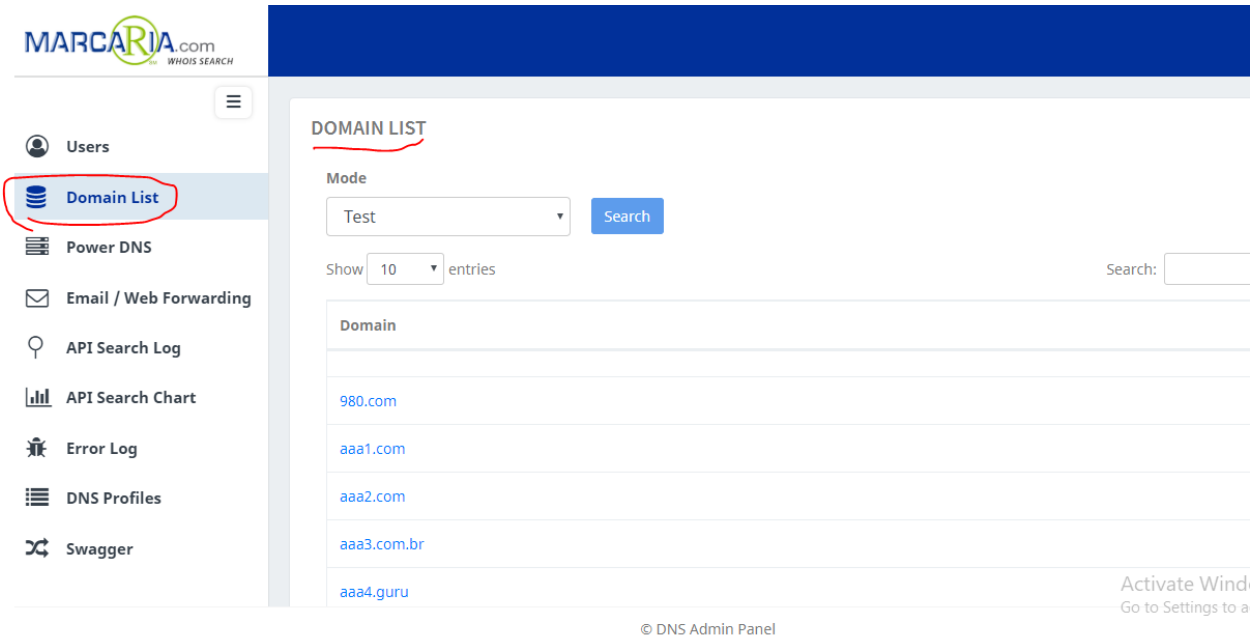
- And as per requirement CNAME record will be updated.



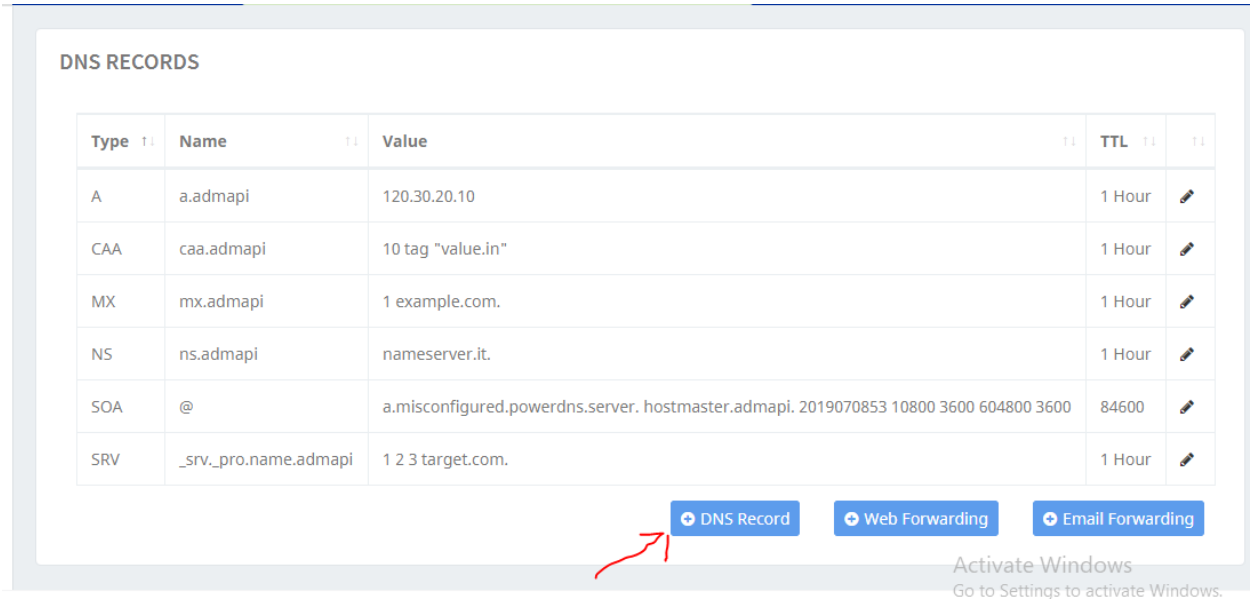
Type	Name	Value	TTL
A	a.quality.com	120.30.2.1	1 Hour
AAAA	@	12:12:32:212:1:21:23:15	1 Hour
CNAME	cn.quality.com	www.quality.cdn.clouldflare.net.	1 Hour
MX	mx.quality.com	10 any.co.	1 Hour
SOA	@	a.misconfigured.powerdns.server. hostmaster.quality.com. 2019070302 10800 3600 604800 3600	84600
SRV	_tfs._ip.ds.quality.com	1 3 4 qaz.	1 Hour
TXT	txt.quality.com	"text.value"	1 Hour

5. ADD THE DNS RECORD AND FORWARDING BUTTONS

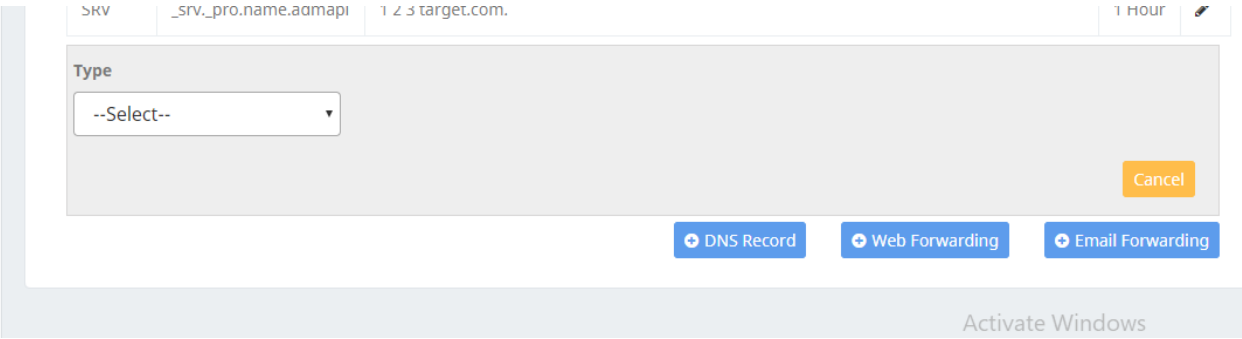
Steps to verify:
-Navigate to http://admin.dns-api.marcaria.com and it will redirect on the login page.
-enter valid credentials and log in to that site.
-successfully login will redirect on the 'Domain list' page which is the home page of the site.



- Now select any domain from the domain list page and it will look like the screenshot below.
- click on the “DNS Record” button.



- Select type from the type dropdown field (i.e., A, AAAA, MX etc)



Now implement all requirement what was described in
“3. ADDITIONAL MODIFICATIONS TO DNS TESTING PAGES.” And also fulfill all the records as per requirement.

Cloudflare Document :

==========

Cloudflare Platform Requirement

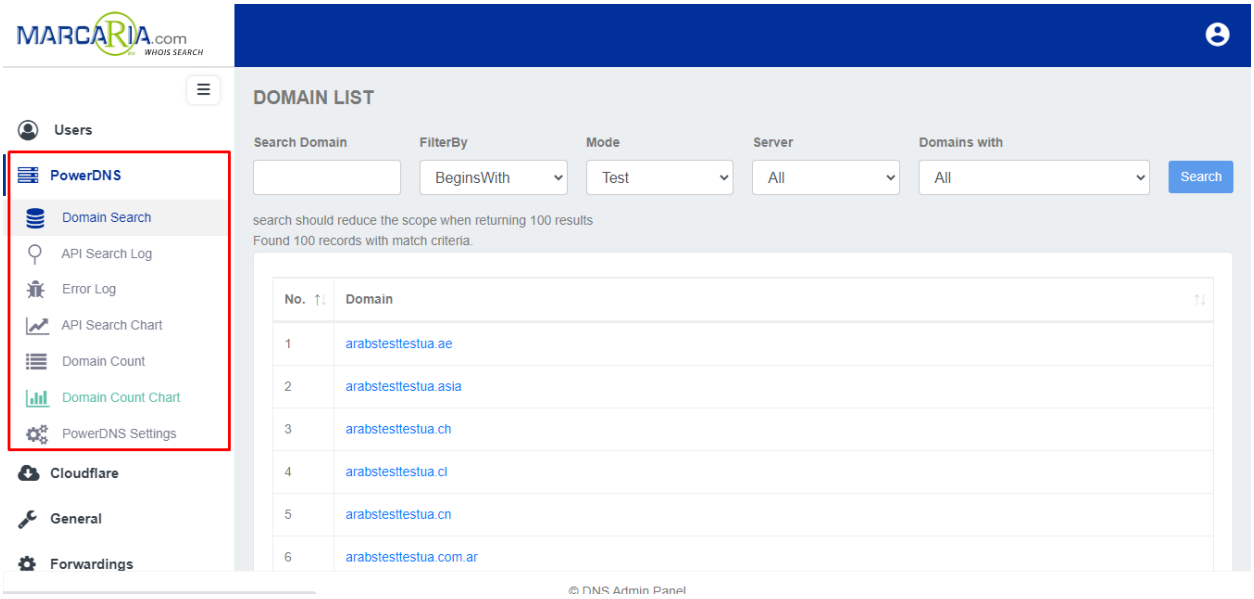
https://www.dropbox.com/s/krkh3nrvddp328d/SpecificationsCloudflareAsDNSplatform.docx

==========

The cloudflare API working is little bit of different then power dns :

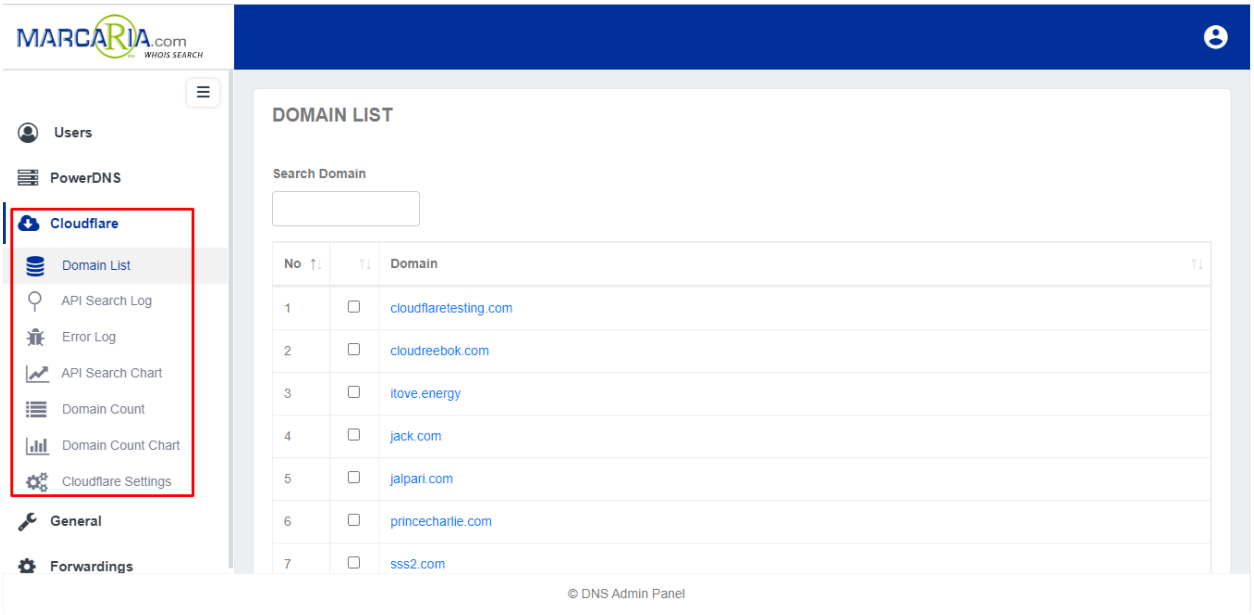
After Improving Cloudflare As DNS Platform the menu will look like as provided document(above link) from client and it looks like below screenshot

1)



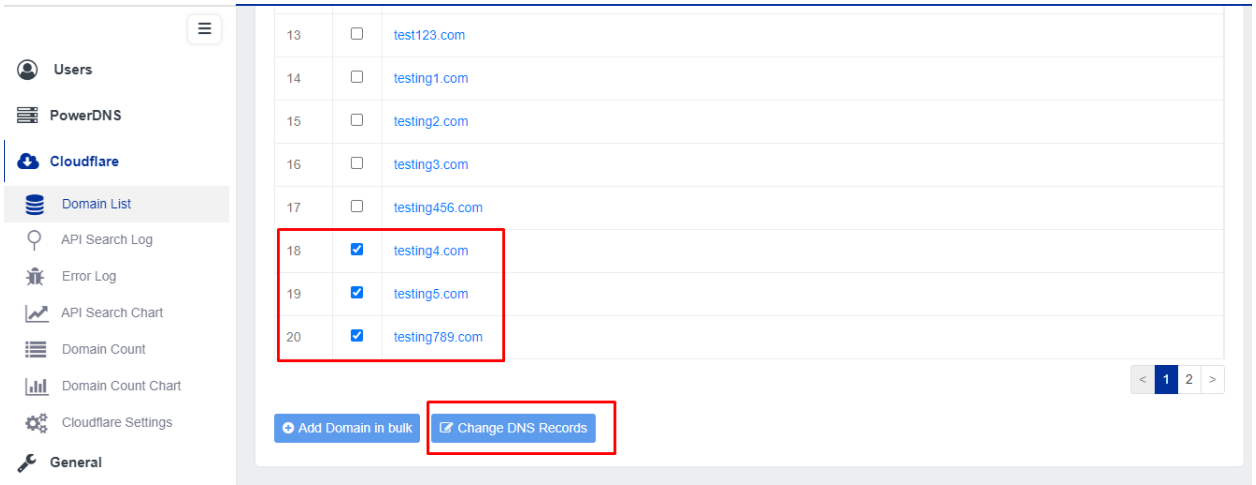
Working of power dns submenu is same as old logic

2)Cloudflare menu uses the cloudflare api to perform the actions like add or remove domain,add or update or delete records.

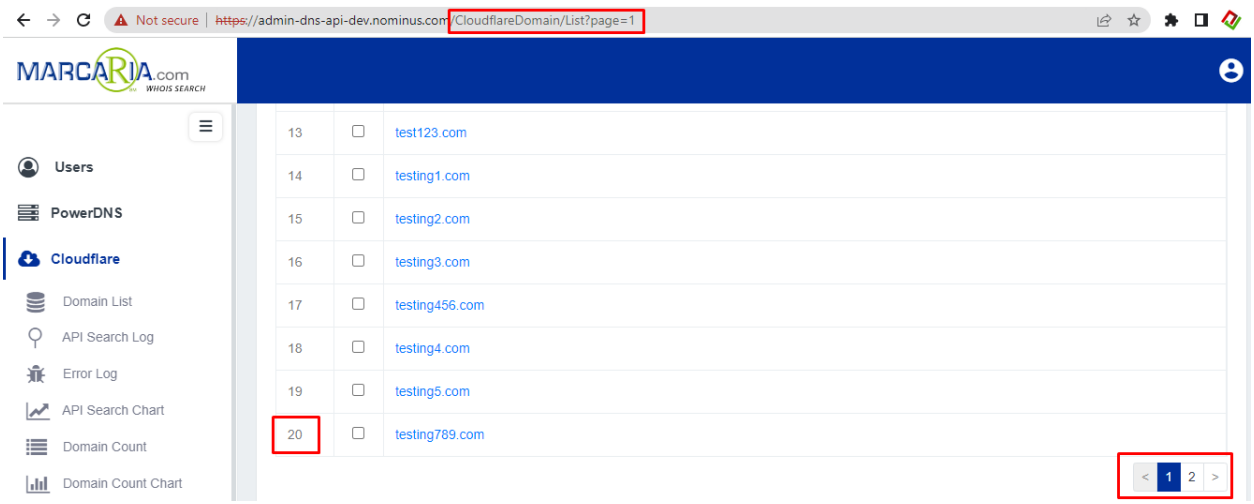


Working of Cloudflare menu is as same as power dns menu the little change is in the Cloudflare>Domainlist sub menu item

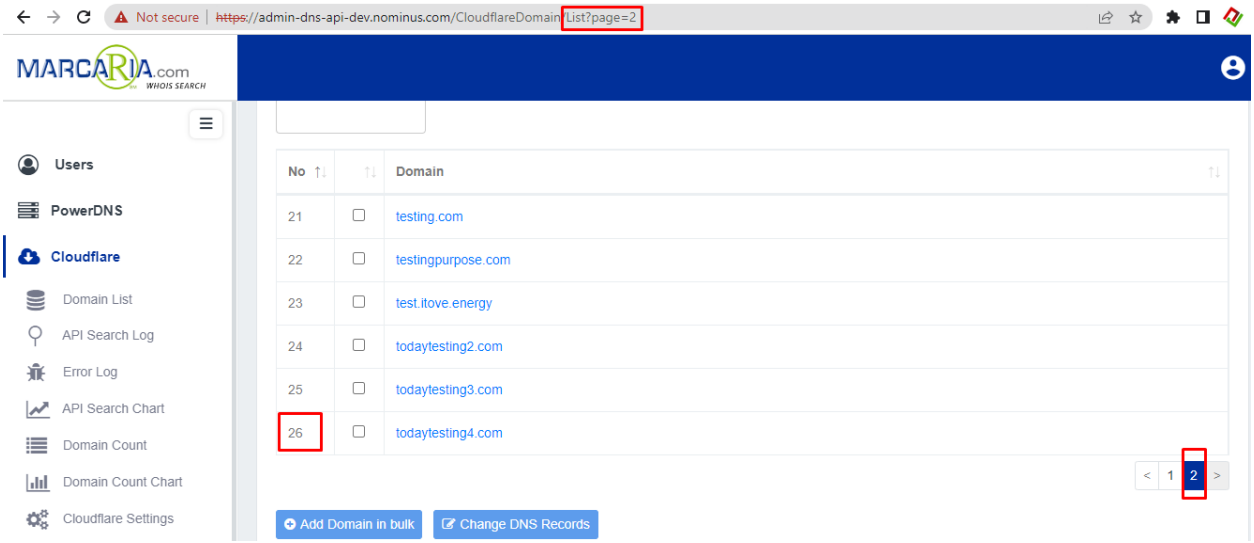
If i want to add records by profile id or clean dns records of domain selected in bulk or remove domain in bulk i have to click on the button ChangeDNSRecords provided at the down side and manually add domain in to the model and hit the button,
but now if a want to perform any of the action highlighted above i just have to select the checkbox of that domains as we can see in the below screenshot



After clicking on the ChangeDNSRecords button it will work the same as the old logic.
The remaining submenu items in the cloudflare are working the same as old logic.



In the above screenshot I can see in the url page =1 the cloudflare is providing 20 domains(limit) per page, i click on the 2 pagination tab the url will page=2 and provide more 20 domains(if available) as per below screenshot.
(Page 1 : 1 to 20 no Domain Display)
(Page 2 : 21 to 40 no Domain Display)



In the swagger cloudflare api`s are created and its details are in the folder common>project>marcaria>dns>Cloudeflare_Swagger_API_Run_List_ClientShare Document.